



NEVER MISS BUS

COMPLETE SOURCE CODE + APP STATUS

SRBS International School · Version 1.3.0

Flutter (Dart) · Firebase · Security Rules · Android + iOS

13 sections · 67 Dart files · 11,000+ lines of code

flutter analyze: 0 issues · 15/15 tests passing

August 2026

Part A — App Status (Current)

Version	1.3.0 (build 5) — latest APK: NeverMissBus-v1.3.0.apk (26.7 MB)
Platforms	Android: APK ready & installed-tested. iOS: code ready, needs Apple Developer account + Mac build.
Backend	Firebase project 'never-miss-bus-c67b5' — LIVE. Firestore + Realtime DB (Singapore) + Auth (Email/Password). FREE Spark plan, no card.
Security rules	Deployed. Role checks server-side; student sees only own bus; tamper-proof audit logs; single-device mirror in RTDB.
Admin account	Working (agrawalvipinlata@gmail.com). Creates all other accounts inside the app.
Login system	Students: Class + Section + Roll + Password (generation system for password resets). Staff: email + password.
Device security	Single-device login (WhatsApp-style kick-out), admin sees LOGGED IN status, Force Logout button.
UI	v1.1.1 redesign: blue gradient headers, drawer, 5-tab nav. v1.3.0: animated splash (bus drive-in), YouTube-style shimmer skeletons, premium login card.
Stability	v1.2.1 hardening: startup-only error screen, 16 streams error-quiet, ref-after-dispose fixed in all logout flows.
Quality	flutter analyze: 0 issues · 15/15 unit tests passing
Maps	OpenStreetMap (flutter_map) — free, no API key, no billing.
Known limits (free plan)	No push notifications when app is closed (in-app inbox works); no SMS OTP; iOS build pending.

Version Journey

v1.0	Base app, Firebase connected, first APK
v1.4–v1.7	Class-wise admin, roll-number login, trip history, password reset, stops reorder, back-button fix, new logo
v1.1.1	Premium UI redesign (mock-based): headers, drawer, 5 tabs
v1.2.0	Single-device login security + full English + login status
v1.2.1	Stability: root-cause fixes for recurring error screens
v1.3.0	Animated splash, shimmer skeleton loading, premium login

Part B — Code Structure (App Architecture)

Layered architecture: UI (features) → State (providers) → Services (Firebase gateways) → Models (pure Dart). UI never touches Firebase directly. Security is enforced by backend rules, not the app.

```

never_miss_bus/
  ■■■ pubspec.yaml # dependencies (Flutter, Firebase, flutter_map...)
  ■■■ lib/
    ■ ■■■ main.dart # startup + crash-safe error handling
    ■ ■■■ app.dart # root widget, single-device login guard
    ■ ■■■ firebase_options.dart # Firebase project config (never-miss-bus-c67b5)
    ■ ■■■ core/
      ■ ■■■ theme/ # colors, typography, ThemeData
      ■ ■■■ constants/ # enums, classes list, StudentLoginId
      ■ ■■■ utils/ # validators, formatters, Result type
      ■ ■■■ widgets/ # NmbCard, StatusPill, dialogs, states,
    ■ ■■■ skeleton.dart (shimmer loading)
    ■ ■■■ models/ # 9 pure-Dart data classes
    ■ ■■■ services/ # 9 services – the ONLY Firebase layer
      ■ ■■■ auth_service.dart # login/session (roll generations)
      ■ ■■■ firestore_service.dart # all Firestore reads/writes
      ■ ■■■ admin_service.dart # provisioning, reset, force-logout
      ■ ■■■ live_location_service.dart # RTDB GPS node per bus
      ■ ■■■ driver_trip_service.dart # trip lifecycle + GPS stream
      ■ ■■■ eta_service.dart # honest ETA calculation
      ■ ■■■ notification_service.dart # FCM tokens + local notifications
      ■ ■■■ connectivity_service.dart # online/offline stream
      ■ ■■■ device_session_service.dart # single-device IDs
    ■ ■■■ providers/ # Riverpod graph (error-quiet streams)
    ■ ■■■ router/ # go_router with role fencing
    ■ ■■■ features/
      ■ ■■■ auth/ # animated splash + premium login
      ■ ■■■ student/ # shell(5 tabs), home, drawer, map,
      ■ ■■■ stops, alerts, profile, settings
      ■ ■■■ driver/ # shell, trip dashboard, route, profile
      ■ ■■■ admin/ # shell(drawer), dashboard, students,
      ■ ■■■ drivers, buses, routes/stops,
      ■ ■■■ monitoring, notifications, trips,
      ■ ■■■ audit logs, settings
      ■ ■■■ shared/ # BusMarkerWidget (painted bus)
    ■ ■■■ firebase/
      ■ ■■■ firestore.rules # role-based access (free mode)
      ■ ■■■ database.rules.json # live-location + access mirror rules
      ■ ■■■ firestore.indexes.json # composite indexes
    ■ ■■■ android/ # manifest, gradle (desugaring), splash
    ■ ■■■ ios/ # Info.plist, AppDelegate
    ■ ■■■ test/ # 15 unit tests

```

Data flow (live tracking): Driver GPS → RTDB /liveLocations/{busId} (server-timestamped) → student stream (rules-fenced to own bus) → map marker + freshness logic (LIVE / stale / unavailable).

Single-device flow: login → device ID saved in settings.activeDevice → other phones watching profile see a different ID → auto sign-out with message. Admin sets id=REVOKED for Force Logout.

Part C — Complete Source Code

Section	Files	Lines
1. Project Configuration	5 files	409 lines
2. Design System & Core	8 files	521 lines
3. Reusable Widgets	6 files	670 lines
4. Data Models	9 files	421 lines
5. Services	9 files	1366 lines
6. State & Navigation	3 files	486 lines
7. Auth Screens	2 files	764 lines
8. Student App	11 files	2608 lines
9. Driver App	4 files	755 lines
10. Admin App	12 files	3203 lines
11. Firebase Backend — Security Rules	4 files	248 lines
12. Platform Config	4 files	157 lines
13. Tests	2 files	163 lines

1. Project Configuration

Dependencies, lint rules, entry point, Firebase config.

pubspec.yaml

55 lines

```
1 ■ name: never_miss_bus
2 ■ description: "Never Miss Bus – secure school bus live tracking for SRBS International School."
3 ■ publish_to: "none"
4 ■ version: 1.3.0+5
5 ■
6 ■ environment:
7 ■   sdk: ">=3.4.0 <4.0.0"
8 ■
9 ■ dependencies:
10 ■   flutter:
11 ■     sdk: flutter
12 ■
13 ■   # State management
14 ■   flutter_riverpod: ^2.5.1
15 ■
16 ■   # Navigation
17 ■   go_router: ^14.2.7
18 ■
19 ■   # Firebase (current stable, official FlutterFire)
20 ■   # Firebase – sab FREE (Spark) plan pe chalta hai; koi card nahi chahiye.
21 ■   firebase_core: ^3.6.0
22 ■   firebase_auth: ^5.3.1
23 ■   cloud_firestore: ^5.4.4
24 ■   firebase_database: ^11.1.4
25 ■   firebase_messaging: ^15.1.3
26 ■
27 ■   # Maps & location – flutter_map uses OpenStreetMap: 100% free,
28 ■   # NO API key, NO billing/card required.
29 ■   flutter_map: ^7.0.2
30 ■   latlong2: ^0.9.1
31 ■   geolocator: ^13.0.1
32 ■
33 ■   # Platform / UX
34 ■   permission_handler: ^11.3.1
35 ■   connectivity_plus: ^6.0.5
36 ■   flutter_local_notifications: ^17.2.3
37 ■   shared_preferences: ^2.3.2
38 ■   device_info_plus: ^10.1.2
39 ■   intl: ^0.19.0
40 ■   cupertino_icons: ^1.0.8
41 ■
42 ■   # Pins for Flutter 3.24.x compatibility: newer geolocator_android builds
43 ■   # require the Gradle plugin loader introduced in Flutter 3.27.
44 ■   dependency_overrides:
45 ■     geolocator_android: 4.6.1
46 ■
47 ■   dev_dependencies:
48 ■     flutter_test:
49 ■       sdk: flutter
50 ■     flutter_lints: ^4.0.0
51 ■
52 ■ flutter:
53 ■   uses-material-design: true
54 ■   assets:
55 ■     - assets/images/
```

analysis_options.yaml

9 lines

```
1 ■ include: package:flutter_lints/flutter.yaml
2 ■
3 ■ linter:
4 ■   rules:
5 ■     prefer_const_constructors: true
6 ■     prefer_final_locals: true
7 ■     avoid_print: true
8 ■     require_trailing_commas: true
9 ■     always_declare_return_types: true
```

lib/main.dart

143 lines

```

1 ■ import 'dart:async';
2 ■
3 ■ import 'package:firebase_core/firebase_core.dart';
4 ■ import 'package:flutter/material.dart';
5 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
6 ■
7 ■ import 'app.dart';
8 ■ import 'firebase_options.dart';
9 ■
10 ■ /// True once the real app UI is up. After this point, stray background
11 ■ /// errors (e.g. a stream closing during logout) must NEVER replace the
12 ■ /// whole app with an error screen – they are only logged.
13 ■ bool _appStarted = false;
14 ■
15 ■ Future<void> main() async {
16 ■   runZonedGuarded<Future<void>>(() async {
17 ■     WidgetsFlutterBinding.ensureInitialized();
18 ■
19 ■     FlutterError.onError = (FlutterErrorDetails details) {
20 ■       // Widget-level errors: log; never tear down the running app.
21 ■       FlutterError.presentError(details);
22 ■     };
23 ■
24 ■     try {
25 ■       await Firebase.initializeApp(
26 ■         options: DefaultFirebaseOptions.currentPlatform,
27 ■       );
28 ■     } on FirebaseException catch (e) {
29 ■       // Android auto-initializes from google-services.json; a second init
30 ■       // throws duplicate-app which simply means "already ready".
31 ■       if (e.code != 'duplicate-app') {
32 ■         runApp(_StartupErrorApp(
33 ■           title: 'Firebase connection problem',
34 ■           details: e.toString(),
35 ■         ));
36 ■         return;
37 ■       }
38 ■     } catch (e) {
39 ■       runApp(_StartupErrorApp(
40 ■         title: 'Firebase connection problem',
41 ■         details: e.toString(),
42 ■       ));
43 ■       return;
44 ■     }
45 ■
46 ■     runApp(const ProviderScope(child: NeverMissBusApp()));
47 ■     _appStarted = true;
48 ■   }, (Object error, StackTrace stack) {
49 ■     if (_appStarted) {
50 ■       // App is already running: background hiccups (stream permission
51 ■       // errors during logout, disposed listeners, network blips) are
52 ■       // logged only – the user keeps their working app.
53 ■       debugPrint('Background error (ignored): $error');
54 ■       return;
55 ■     }
56 ■     // Genuine startup failure → readable screen instead of black screen.
57 ■     runApp(_StartupErrorApp(
58 ■       title: 'App failed to start',

```

```

59 ■     details: '$error',
60 ■   ),);
61 ■ });
62 ■ }
63 ■
64 ■ /// Friendly full-screen error shown ONLY for real startup failures.
65 ■ class _StartupErrorApp extends StatelessWidget {
66 ■   const _StartupErrorApp({required this.title, required this.details});
67 ■
68 ■   final String title;
69 ■   final String details;
70 ■
71 ■   @override
72 ■   Widget build(BuildContext context) {
73 ■     return MaterialApp(
74 ■       debugShowCheckedModeBanner: false,
75 ■       home: Scaffold(
76 ■         backgroundColor: const Color(0xFFF6F8FC),
77 ■         body: SafeArea(
78 ■           child: Center(
79 ■             child: SingleChildScrollView(
80 ■               padding: const EdgeInsets.all(28),
81 ■               child: Column(
82 ■                 mainAxisAlignment: MainAxisAlignment.min,
83 ■                 children: <Widget>[
84 ■                   Container(
85 ■                     width: 84,
86 ■                     height: 84,
87 ■                     decoration: const BoxDecoration(
88 ■                       color: Color(0xFFCE8E7),
89 ■                       shape: BoxShape.circle,
90 ■                     ),
91 ■                     child: const Icon(
92 ■                       Icons.error_outline_rounded,
93 ■                       size: 44,
94 ■                       color: Color(0xFFC5221F),
95 ■                     ),
96 ■                   ),
97 ■                   const SizedBox(height: 20),
98 ■                   Text(
99 ■                     title,
100 ■                     textAlign: TextAlign.center,
101 ■                     style: const TextStyle(
102 ■                       fontSize: 20,
103 ■                       fontWeight: FontWeight.w700,
104 ■                       color: Color(0xFF17233B),
105 ■                     ),
106 ■                   ),
107 ■                   const SizedBox(height: 10),
108 ■                   const Text(
109 ■                     'Please share a screenshot of this screen so the '
110 ■                     'problem can be fixed.',
111 ■                     textAlign: TextAlign.center,
112 ■                     style: TextStyle(
113 ■                       fontSize: 14,
114 ■                       color: Color(0xFF5A6779),
115 ■                     ),
116 ■                   ),

```

```
117 ■      const SizedBox(height: 18),
118 ■      Container(
119 ■        width: double.infinity,
120 ■        padding: const EdgeInsets.all(14),
121 ■        decoration: BoxDecoration(
122 ■          color: Colors.white,
123 ■          borderRadius: BorderRadius.circular(14),
124 ■          border: Border.all(color: const Color(0xFFE4E9F2)),
125 ■        ),
126 ■        child: Text(
127 ■          details,
128 ■          style: const TextStyle(
129 ■            fontSize: 12,
130 ■            fontFamily: 'monospace',
131 ■            color: Color(0xFF5A6779),
132 ■          ),
133 ■        ),
134 ■      ),
135 ■    ],
136 ■  ),
137 ■ ),
138 ■ ),
139 ■ ),
140 ■ ),
141 ■ );
142 ■ }
143 ■ }
```

lib/app.dart

157 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import 'core/constants/nmb_constants.dart';
6 ■ import 'core/theme/nmb_theme.dart';
7 ■ import 'models/app_user.dart';
8 ■ import 'providers/app_providers.dart';
9 ■ import 'providers/data_providers.dart';
10 ■ import 'router/app_router.dart';
11 ■ import 'services/auth_service.dart';
12 ■ import 'services/device_session_service.dart';
13 ■ import 'services/firestore_service.dart';
14 ■ import 'services/notification_service.dart';
15 ■
16 ■ /// Global messenger so the single-device kick-out can show a message from
17 ■ /// outside any screen's context.
18 ■ final GlobalKey<ScaffoldMessengerState> rootMessengerKey =
19 ■   GlobalKey<ScaffoldMessengerState>();
20 ■
21 ■ class NeverMissBusApp extends ConsumerStatefulWidget {
22 ■   const NeverMissBusApp({super.key});
23 ■
24 ■   @override
25 ■   ConsumerState<NeverMissBusApp> createState() => _NeverMissBusAppState();
26 ■ }
27 ■
28 ■ class _NeverMissBusAppState extends ConsumerState<NeverMissBusApp>
29 ■   with WidgetsBindingObserver {
30 ■   String? _registeredForUid;
31 ■   bool _kickInProgress = false;
32 ■
33 ■   @override
34 ■   void initState() {
35 ■     super.initState();
36 ■     WidgetsBinding.instance.addObserver(this);
37 ■     Future<void>.microtask(() async {
38 ■       if (!mounted) return;
39 ■       final NotificationService notif = ref.read(notificationServiceProvider);
40 ■       try {
41 ■         await notif.initialize();
42 ■       } catch (_) {
43 ■         // Notifications failing must never block the app.
44 ■       }
45 ■     });
46 ■   }
47 ■
48 ■   @override
49 ■   void dispose() {
50 ■     WidgetsBinding.instance.removeObserver(this);
51 ■     super.dispose();
52 ■   }
53 ■
54 ■   @override
55 ■   void didChangeAppLifecycleState(AppLifecycleState state) {
56 ■     if (state == AppLifecycleState.resumed && mounted) {
57 ■       ref.read(authServiceProvider).readSession(forceRefresh: true);
58 ■     }

```

```

59 ■ }
60 ■
61 ■ /// SINGLE-DEVICE LOGIN – register this device as the user's active one.
62 ■ /// All ref-reads happen synchronously BEFORE any await.
63 ■ void _registerDevice(AuthSession session) {
64 ■     if (_registeredForUid == session.uid) return;
65 ■     _registeredForUid = session.uid;
66 ■     final DeviceSessionService dev = ref.read(deviceSessionServiceProvider);
67 ■     final FirestoreService fs = ref.read(firestoreServiceProvider);
68 ■     final NotificationService notif = ref.read(notificationServiceProvider);
69 ■
70 ■     dev.loginWriteInProgress = true;
71 ■     () async {
72 ■         try {
73 ■             final String id = await dev.getDeviceId();
74 ■             final String name = await dev.getDeviceName();
75 ■             await fs.registerActiveDevice(
76 ■                 uid: session.uid,
77 ■                 deviceId: id,
78 ■                 deviceName: name,
79 ■             );
80 ■             await notif.registerDevice(session.uid);
81 ■         } catch (_) {
82 ■             // Registration failure must not block login.
83 ■         } finally {
84 ■             dev.loginWriteInProgress = false;
85 ■         }
86 ■     }();
87 ■ }
88 ■
89 ■ /// SINGLE-DEVICE LOGIN – if another device took over, sign out here.
90 ■ /// Ref-reads captured synchronously; async work uses captured objects.
91 ■ void _enforceSingleDevice(AppUser? me) {
92 ■     if (_kickInProgress) return;
93 ■     final AuthSession? session = ref.read(currentSessionProvider);
94 ■     if (session == null || me == null) return;
95 ■     final DeviceSessionService dev = ref.read(deviceSessionServiceProvider);
96 ■     if (dev.loginWriteInProgress) return;
97 ■     final AuthService auth = ref.read(authServiceProvider);
98 ■
99 ■     final Map<String, dynamic>? active = me.activeDevice;
100 ■     if (active == null) return;
101 ■     final String activeId = (active['id'] as String?) ?? '';
102 ■     if (activeId.isEmpty) return;
103 ■
104 ■     _kickInProgress = true;
105 ■     () async {
106 ■         try {
107 ■             final String myId = await dev.getDeviceId();
108 ■             if (activeId == myId) return; // we are the active device – fine
109 ■
110 ■             await auth.signOut();
111 ■             _registeredForUid = null;
112 ■             rootMessengerKey.currentState?.showSnackBar(
113 ■                 SnackBar(
114 ■                     duration: const Duration(seconds: 6),
115 ■                     content: Text(
116 ■                         activeId == DeviceSessionService.revokedId

```

```

117 ■         ? 'You were signed out by the school admin.'
118 ■         : 'This account was signed in on another device. '
119 ■           'Only one device can be logged in at a time.',
120 ■       ),
121 ■     ),
122 ■   );
123 ■   } catch (_) {
124 ■     // Never crash the app from the guard.
125 ■   } finally {
126 ■     _kickInProgress = false;
127 ■   }
128 ■ }();
129 ■ }
130 ■
131 ■ @override
132 ■ Widget build(BuildContext context) {
133 ■   final GoRouter router = ref.watch(appRouterProvider);
134 ■
135 ■   ref.listen(sessionProvider,
136 ■     (Object? prev, AsyncValue<AuthSession?>? next) {
137 ■       final AuthSession? session = next?.valueOrNull;
138 ■       if (session != null) {
139 ■         _registerDevice(session);
140 ■       } else {
141 ■         _registeredForUid = null;
142 ■       }
143 ■     });
144 ■
145 ■   ref.listen(myProfileProvider, (Object? prev, AsyncValue<AppUser?> next) {
146 ■     _enforceSingleDevice(next.valueOrNull);
147 ■   });
148 ■
149 ■   return MaterialApp.router(
150 ■     title: NmbConstants.appName,
151 ■     debugShowCheckedModeBanner: false,
152 ■     scaffoldMessengerKey: rootMessengerKey,
153 ■     theme: NmbTheme.light(),
154 ■     routerConfig: router,
155 ■   );
156 ■ }
157 ■ }

```

lib/firebase_options.dart

45 lines

```

1 ■ // Generated for Firebase project: never-miss-bus-c67b5
2 ■ // (equivalent of `flutterfire configure` output, built from the app's
3 ■ // registered google-services.json)
4 ■ //
5 ■ // NOTE: These values are PUBLIC app identifiers, not secrets. Real access
6 ■ // control lives in Firestore/RTDB security rules, which are already
7 ■ // deployed to this project.
8 ■ //
9 ■ // Platforms configured: Android.
10 ■ // iOS requires registering an iOS app in Firebase Console first
11 ■ // (see docs/SETUP.md) – until then iOS builds will throw below.
12 ■ import 'package:firebase_core/firebase_core.dart' show FirebaseOptions;
13 ■ import 'package:flutter/foundation.dart'
14 ■     show defaultTargetPlatform, TargetPlatform;
15 ■
16 ■ class DefaultFirebaseOptions {
17 ■   static FirebaseOptions get currentPlatform {
18 ■     switch (defaultTargetPlatform) {
19 ■       case TargetPlatform.android:
20 ■         return android;
21 ■       case TargetPlatform.iOS:
22 ■         throw UnsupportedError(
23 ■           'iOS app is not registered in Firebase yet. '
24 ■           'Register an iOS app for project never-miss-bus-c67b5 and '
25 ■           'regenerate firebase_options.dart (see docs/SETUP.md).',
26 ■         );
27 ■       default:
28 ■         throw UnsupportedError(
29 ■           'Never Miss Bus targets Android and iOS only.',
30 ■         );
31 ■     }
32 ■   }
33 ■
34 ■   static const FirebaseOptions android = FirebaseOptions(
35 ■     apiKey: 'AIzaSyAiwEpyTqaAaNu87ytlKwMdF844PFL_cFE',
36 ■     appId: '1:642048321838:android:061125110463d63bf5f2bf',
37 ■     messagingSenderId: '642048321838',
38 ■     projectId: 'never-miss-bus-c67b5',
39 ■     storageBucket: 'never-miss-bus-c67b5.firebasestorage.app',
40 ■     // Realtime Database is in asia-southeast1 (Singapore), so the regional
41 ■     // URL must be set explicitly – the default (us-central1) would fail.
42 ■     databaseURL:
43 ■       'https://never-miss-bus-c67b5-default-rtdb.asia-southeast1.firebaseio.app',
44 ■   );
45 ■ }

```

2. Design System & Core

Colors, typography, theme, constants (classes, student login-IDs), validators, formatters, Result type.

lib/core/theme/nmb_colors.dart

39 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■
3 ■ /// Never Miss Bus design-system palette.
4 ■ ///
5 ■ /// Friendly, school-appropriate, high-contrast. All pairs meet WCAG AA on
6 ■ /// their intended backgrounds.
7 ■ abstract final class NmbColors {
8 ■   // Brand
9 ■   static const Color primary = Color(0xFF2557D6); // trustworthy school blue
10 ■   static const Color primaryDark = Color(0xFF1A3FA0);
11 ■   static const Color primarySoft = Color(0xFFE8EEFC); // tinted card backgrounds
12 ■   static const Color accent = Color(0xFFFFB300); // school-bus amber
13 ■   static const Color accentDark = Color(0xFF8A5B00); // AA-safe on soft amber
14 ■   static const Color accentSoft = Color(0xFFFFF3D6);
15 ■
16 ■   // Semantic
17 ■   static const Color success = Color(0xFF1E8E3E);
18 ■   static const Color successSoft = Color(0xFFE2F3E7);
19 ■   static const Color warning = Color(0xFFB26A00);
20 ■   static const Color warningSoft = Color(0xFFFFF1DC);
21 ■   static const Color danger = Color(0xFFC5221F);
22 ■   static const Color dangerSoft = Color(0xFFFFCE8E7);
23 ■   static const Color info = Color(0xFF0B6CBD);
24 ■   static const Color infoSoft = Color(0xFFE3F0FA);
25 ■
26 ■   // Neutrals
27 ■   static const Color background = Color(0xFFF6F8FC);
28 ■   static const Color surface = Colors.white;
29 ■   static const Color textPrimary = Color(0xFF17233B);
30 ■   static const Color textSecondary = Color(0xFF5A6779);
31 ■   static const Color textTertiary = Color(0xFF8A94A6);
32 ■   static const Color divider = Color(0xFFE4E9F2);
33 ■   static const Color disabled = Color(0xFFB9C2D0);
34 ■
35 ■   // Live-tracking states
36 ■   static const Color live = success;
37 ■   static const Color stale = warning;
38 ■   static const Color offline = textTertiary;
39 ■ }
```

lib/core/theme/nmb_typography.dart

65 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'nmb_colors.dart';
3 ■
4 ■ /// Typography scale. Large, readable, friendly.
5 ■ abstract final class NmbTypography {
6 ■   static const String fontFamily = 'Roboto'; // system default; swap freely
7 ■
8 ■   static const TextStyle displayTitle = TextStyle(
9 ■     fontSize: 28,
10 ■    fontWeight: FontWeight.w800,
11 ■    color: NmbColors.textPrimary,
12 ■    height: 1.2,
13 ■  );
14 ■
15 ■   static const TextStyle screenTitle = TextStyle(
16 ■     fontSize: 22,
17 ■     fontWeight: FontWeight.w700,
18 ■     color: NmbColors.textPrimary,
19 ■     height: 1.25,
20 ■   );
21 ■
22 ■   static const TextStyle sectionTitle = TextStyle(
23 ■     fontSize: 17,
24 ■     fontWeight: FontWeight.w700,
25 ■     color: NmbColors.textPrimary,
26 ■   );
27 ■
28 ■   static const TextStyle cardTitle = TextStyle(
29 ■     fontSize: 16,
30 ■     fontWeight: FontWeight.w600,
31 ■     color: NmbColors.textPrimary,
32 ■   );
33 ■
34 ■   static const TextStyle body = TextStyle(
35 ■     fontSize: 15,
36 ■     fontWeight: FontWeight.w400,
37 ■     color: NmbColors.textPrimary,
38 ■     height: 1.45,
39 ■   );
40 ■
41 ■   static const TextStyle bodySecondary = TextStyle(
42 ■     fontSize: 14,
43 ■     fontWeight: FontWeight.w400,
44 ■     color: NmbColors.textSecondary,
45 ■     height: 1.4,
46 ■   );
47 ■
48 ■   static const TextStyle caption = TextStyle(
49 ■     fontSize: 12.5,
50 ■     fontWeight: FontWeight.w500,
51 ■     color: NmbColors.textTertiary,
52 ■   );
53 ■
54 ■   static const TextStyle button = TextStyle(
55 ■     fontSize: 16,
56 ■     fontWeight: FontWeight.w700,
57 ■     letterSpacing: 0.2,
58 ■   );
59 ■
60 ■   static const TextStyle statLarge = TextStyle(
61 ■     fontSize: 30,
62 ■     fontWeight: FontWeight.w800,
63 ■     color: NmbColors.textPrimary,
64 ■   );
65 ■ }

```

lib/core/theme/nmb_theme.dart

143 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter/services.dart';
3 ■ import 'nmb_colors.dart';
4 ■ import 'nmb_typography.dart';
5 ■
6 ■ /// Single ThemeData used across Student, Driver and Admin experiences so the
7 ■ /// whole product feels like one design system.
8 ■ abstract final class NmbTheme {
9 ■   static const double radiusCard = 20;
10 ■   static const double radiusButton = 16;
11 ■   static const double radiusField = 14;
12 ■
13 ■   static ThemeData light() {
14 ■     final ColorScheme scheme = ColorScheme.fromSeed(
15 ■       seedColor: NmbColors.primary,
16 ■       primary: NmbColors.primary,
17 ■       secondary: NmbColors.accent,
18 ■       surface: NmbColors.surface,
19 ■       error: NmbColors.danger,
20 ■     );
21 ■
22 ■     return ThemeData(
23 ■       useMaterial3: true,
24 ■       colorScheme: scheme,
25 ■       scaffoldBackgroundColor: NmbColors.background,
26 ■       fontFamily: NmbTypography.fontFamily,
27 ■       splashFactory: InkSparkle.splashFactory,
28 ■       appBarTheme: const AppBarTheme(
29 ■         backgroundColor: NmbColors.background,
30 ■         foregroundColor: NmbColors.textPrimary,
31 ■         elevation: 0,
32 ■         scrolledUnderElevation: 0.5,
33 ■         centerTitle: false,
34 ■         titleTextStyle: NmbTypography.screenTitle,
35 ■         systemOverlayStyle: SystemUiOverlayStyle.dark,
36 ■       ),
37 ■       cardTheme: CardTheme(
38 ■         color: NmbColors.surface,
39 ■         elevation: 0,
40 ■         shape: RoundedRectangleBorder(
41 ■           borderRadius: BorderRadius.circular(radiusCard),
42 ■         ),
43 ■         margin: EdgeInsets.zero,
44 ■       ),
45 ■       filledButtonTheme: FilledButtonThemeData(
46 ■         style: FilledButton.styleFrom(
47 ■           minimumSize: const Size.fromHeight(54), // large touch target
48 ■           textStyle: NmbTypography.button,
49 ■           shape: RoundedRectangleBorder(
50 ■             borderRadius: BorderRadius.circular(radiusButton),
51 ■           ),
52 ■         ),
53 ■       ),
54 ■       outlinedButtonTheme: OutlinedButtonThemeData(
55 ■         style: OutlinedButton.styleFrom(
56 ■           minimumSize: const Size.fromHeight(52),
57 ■           textStyle: NmbTypography.button,
58 ■           side: const BorderSide(color: NmbColors.primary, width: 1.4),

```

```

59 ■         shape: RoundedRectangleBorder(
60 ■             borderRadius: BorderRadius.circular(radiusButton),
61 ■         ),
62 ■     ),
63 ■ ),
64 ■ textButtonTheme: TextButtonThemeData(
65 ■     style: TextButton.styleFrom(
66 ■         minimumSize: const Size(48, 44),
67 ■         textStyle: NmbTypography.button.copyWith(fontSize: 15),
68 ■     ),
69 ■ ),
70 ■ inputDecorationTheme: InputDecorationTheme(
71 ■     filled: true,
72 ■     fillColor: NmbColors.surface,
73 ■     contentPadding:
74 ■         const EdgeInsets.symmetric(horizontal: 16, vertical: 16),
75 ■     border: OutlineInputBorder(
76 ■         borderRadius: BorderRadius.circular(radiusField),
77 ■         borderSide: const BorderSide(color: NmbColors.divider),
78 ■     ),
79 ■     enabledBorder: OutlineInputBorder(
80 ■         borderRadius: BorderRadius.circular(radiusField),
81 ■         borderSide: const BorderSide(color: NmbColors.divider),
82 ■     ),
83 ■     focusedBorder: OutlineInputBorder(
84 ■         borderRadius: BorderRadius.circular(radiusField),
85 ■         borderSide: const BorderSide(color: NmbColors.primary, width: 1.6),
86 ■     ),
87 ■     errorBorder: OutlineInputBorder(
88 ■         borderRadius: BorderRadius.circular(radiusField),
89 ■         borderSide: const BorderSide(color: NmbColors.danger),
90 ■     ),
91 ■     hintStyle: NmbTypography.bodySecondary,
92 ■ ),
93 ■ navigationBarTheme: NavigationBarThemeData(
94 ■     backgroundColor: NmbColors.surface,
95 ■     indicatorColor: NmbColors.primarySoft,
96 ■     height: 68,
97 ■     elevation: 3,
98 ■     labelTextStyle: WidgetStatePropertyAll(
99 ■         NmbTypography.caption.copyWith(
100 ■             color: NmbColors.textSecondary,
101 ■             fontWeight: FontWeight.w600,
102 ■         ),
103 ■     ),
104 ■ ),
105 ■ snackBarTheme: SnackBarThemeData(
106 ■     behavior: SnackBarBehavior.floating,
107 ■     backgroundColor: NmbColors.textPrimary,
108 ■     contentTextStyle:
109 ■         NmbTypography.body.copyWith(color: Colors.white),
110 ■     shape: RoundedRectangleBorder(
111 ■         borderRadius: BorderRadius.circular(14),
112 ■     ),
113 ■ ),
114 ■ dialogTheme: DialogTheme(
115 ■     backgroundColor: NmbColors.surface,
116 ■     shape: RoundedRectangleBorder(

```

```
117 ■      borderRadius: BorderRadius.circular(24),
118 ■    ),
119 ■    titleTextStyle: NmbTypography.sectionTitle,
120 ■    contentTextStyle: NmbTypography.bodySecondary,
121 ■  ),
122 ■  dividerTheme: const DividerThemeData(
123 ■    color: NmbColors.divider,
124 ■    thickness: 1,
125 ■    space: 1,
126 ■  ),
127 ■  listTileTheme: const ListTileThemeData(
128 ■    iconColor: NmbColors.textSecondary,
129 ■    titleTextStyle: NmbTypography.cardTitle,
130 ■    subtitleTextStyle: NmbTypography.bodySecondary,
131 ■  ),
132 ■  chipTheme: ChipThemeData(
133 ■    backgroundColor: NmbColors.primarySoft,
134 ■    labelStyle: NmbTypography.caption
135 ■      .copyWith(color: NmbColors.primaryDark),
136 ■    shape: RoundedRectangleBorder(
137 ■      borderRadius: BorderRadius.circular(10),
138 ■    ),
139 ■    side: BorderSide.none,
140 ■  ),
141 ■ );
142 ■ }
143 ■ }
```

lib/core/constants/enums.dart

66 lines

```

1 ■ /// Core enums shared across the app. Names match backend string values.
2 ■ library;
3 ■
4 ■ enum UserRole {
5 ■   student,
6 ■   driver,
7 ■   admin;
8 ■
9 ■   static UserRole? tryParse(String? raw) {
10 ■     for (final UserRole r in UserRole.values) {
11 ■       if (r.name == raw) return r;
12 ■     }
13 ■     return null;
14 ■   }
15 ■ }
16 ■
17 ■ enum TripStatus {
18 ■   active,
19 ■   completed,
20 ■   autoClosed;
21 ■
22 ■   static TripStatus parse(String raw) => TripStatus.values.firstWhere(
23 ■     (TripStatus s) => s.name == raw,
24 ■     orElse: () => TripStatus.completed,
25 ■   );
26 ■ }
27 ■
28 ■ enum TripDirection {
29 ■   pickup,
30 ■   drop;
31 ■
32 ■   static TripDirection parse(String raw) => TripDirection.values.firstWhere(
33 ■     (TripDirection d) => d.name == raw,
34 ■     orElse: () => TripDirection.pickup,
35 ■   );
36 ■
37 ■   String get label => this == TripDirection.pickup ? 'Pickup' : 'Drop-off';
38 ■ }
39 ■
40 ■ /// Freshness of the live bus location – drives honest status UI.
41 ■ enum LocationFreshness {
42 ■   /// Updated within the last 30 seconds. Safe to call "LIVE".
43 ■   live,
44 ■
45 ■   /// Updated 30s-3min ago. Show "Last updated X ago", never "live".
46 ■   stale,
47 ■
48 ■   /// Older than 3 minutes, or no data / no active trip.
49 ■   unavailable,
50 ■ }
51 ■
52 ■ enum NotificationType {
53 ■   tripStarted,
54 ■   busApproaching,
55 ■   busReachedStop,
56 ■   busReachedSchool,
57 ■   busDelayed,
58 ■   trackingUnavailable,
59 ■   announcement;
60 ■
61 ■   static NotificationType parse(String raw) =>
62 ■     NotificationType.values.firstWhere(
63 ■       (NotificationType t) => t.name == raw,
64 ■       orElse: () => NotificationType.announcement,
65 ■     );
66 ■ }

```

```

1 ■ /// App-wide constants and tuning values.
2 ■ abstract final class NmbConstants {
3 ■   static const String appName = 'Never Miss Bus';
4 ■   static const String schoolName = 'SRBS International School';
5 ■
6 ■   // Live location freshness thresholds
7 ■   static const Duration liveThreshold = Duration(seconds: 30);
8 ■   static const Duration staleThreshold = Duration(minutes: 3);
9 ■
10 ■  // Driver GPS stream tuning
11 ■  static const int locationDistanceFilterMeters = 20;
12 ■  static const Duration locationMinInterval = Duration(seconds: 5);
13 ■
14 ■  // ETA guardrails
15 ■  static const double etaMinSpeedKmh = 12; // floor so ETA never explodes
16 ■  static const double etaMaxReliableDistanceKm = 30;
17 ■
18 ■  // Geofence for "Bus Approaching" (also enforced server-side)
19 ■  static const double approachingRadiusMeters = 800;
20 ■
21 ■  // Layout
22 ■  static const double screenPadding = 20;
23 ■  static const double cardGap = 14;
24 ■  static const double maxContentWidth = 560; // keeps tablets/large phones tidy
25 ■ }
26 ■
27 ■ /// School classes & sections – admin dropdowns mein use hote hain
28 ■ /// (typing ki zaroorat nahi, sirf select karna hai).
29 ■ abstract final class SchoolClasses {
30 ■   static const List<String> classes = <String>[
31 ■     'Nursery', 'LKG', 'UKG',
32 ■     '1', '2', '3', '4', '5', '6', '7', '8', '9', '10', '11', '12',
33 ■   ];
34 ■
35 ■   static const List<String> sections = <String>['A', 'B', 'C', 'D'];
36 ■
37 ■   /// "7" + "B" → "7-B" ; "Nursery" + "A" → "Nursery-A"
38 ■   static String label(String klass, String section) => '$klass-$section';
39 ■ }
40 ■
41 ■ /// Student login IDs: bachhon ko email yaad nahi rakhna padta.
42 ■ /// Class + Section + Roll Number se ek internal login-email banta hai jo
43 ■ /// Firebase Auth ke andar use hota hai – user ko kabhi nahi dikhta.
44 ■ abstract final class StudentLoginId {
45 ■   static const String domain = 'student.nmb-srbs.app';
46 ■
47 ■   /// e.g. class "7", section "B", roll "23" → "st.7b.23@student.nmb-srbs.app"
48 ■   /// [gen] > 1 hone par ".g2" jaisa suffix lagta hai – password reset par
49 ■   /// naya internal account banta hai (purana disable ho jaata hai).
50 ■   static String email({
51 ■     required String klass,
52 ■     required String section,
53 ■     required String roll,
54 ■     int gen = 1,
55 ■   }) {
56 ■     final String k = klass.toLowerCase().replaceAll(RegExp(r'^a-z0-9'), '');
57 ■     final String s = section.toLowerCase();
58 ■     final String r = roll.trim();
59 ■
60 ■     final String g = gen <= 1 ? '' : '.g$gen';
61 ■     return 'st.$k$s.$r$g@$domain';
62 ■   }
63 ■
64 ■   /// Login ke waqt kitni generations try karni hain.
65 ■   static const int maxGenerations = 6;
66 ■ }

```

```

1 ■ /// Client-side input validation (server re-validates everything).
2 ■ abstract final class Validators {
3 ■   static final RegExp _email =
4 ■     RegExp(r'^[\w.+-]+@[a-zA-Z\d-]+\.[a-zA-Z\d-]+\.$');
5 ■
6 ■   static String? email(String? value) {
7 ■     final String v = value?.trim() ?? '';
8 ■     if (v.isEmpty) return 'Please enter your email address';
9 ■     if (!_email.hasMatch(v)) return 'That email address doesn\'t look right';
10 ■    return null;
11 ■  }
12 ■
13 ■   static String? password(String? value) {
14 ■     final String v = value ?? '';
15 ■     if (v.isEmpty) return 'Please enter your password';
16 ■     if (v.length < 8) return 'Password must be at least 8 characters';
17 ■     return null;
18 ■   }
19 ■
20 ■   static String? strongPassword(String? value) {
21 ■     final String v = value ?? '';
22 ■     if (v.length < 10) return 'Use at least 10 characters';
23 ■     if (!v.contains(RegExp(r'[A-Z]'))) return 'Add an uppercase letter';
24 ■     if (!v.contains(RegExp(r'[0-9]'))) return 'Add a number';
25 ■     return null;
26 ■   }
27 ■
28 ■   static String? requiredField(String? value, {String label = 'This field'}) {
29 ■     if ((value ?? '').trim().isEmpty) return '$label is required';
30 ■     return null;
31 ■   }
32 ■
33 ■   static String? latitude(String? value) {
34 ■     final double? v = double.tryParse((value ?? '').trim());
35 ■     if (v == null || v < -90 || v > 90) return 'Enter a valid latitude';
36 ■     return null;
37 ■   }
38 ■
39 ■   static String? longitude(String? value) {
40 ■     final double? v = double.tryParse((value ?? '').trim());
41 ■     if (v == null || v < -180 || v > 180) return 'Enter a valid longitude';
42 ■     return null;
43 ■   }
44 ■ }

```

```

1 ■ import 'package:intl/intl.dart';
2 ■
3 ■ /// Shared human-friendly formatting. Keeping ETA wording here guarantees the
4 ■ /// app never presents an estimate as a promise.
5 ■ abstract final class Formatters {
6 ■   static String relativeTime(DateTime time) {
7 ■     final Duration diff = DateTime.now().difference(time);
8 ■     if (diff.inSeconds < 45) return 'just now';
9 ■     if (diff.inMinutes < 1) return 'less than a minute ago';
10 ■    if (diff.inMinutes == 1) return '1 minute ago';
11 ■    if (diff.inMinutes < 60) return '${diff.inMinutes} minutes ago';
12 ■    if (diff.inHours == 1) return '1 hour ago';
13 ■    if (diff.inHours < 24) return '${diff.inHours} hours ago';
14 ■    return DateFormat('d MMM, h:mm a').format(time);
15 ■  }
16 ■
17 ■  /// Always approximate, never exact: "arriving in approximately 8 minutes".
18 ■  static String eta(Duration eta) {
19 ■    final int mins = eta.inMinutes;
20 ■    if (mins <= 1) return 'Bus arriving in approximately 1 minute';
21 ■    return 'Bus arriving in approximately $mins minutes';
22 ■  }
23 ■
24 ■  static String etaShort(Duration eta) {
25 ■    final int mins = eta.inMinutes.clamp(1, 999);
26 ■    return '~$mins min';
27 ■  }
28 ■
29 ■  static String timeOfDay(DateTime t) => DateFormat('h:mm a').format(t);
30 ■
31 ■  static String dateTime(DateTime t) =>
32 ■    DateFormat('EEE, d MMM • h:mm a').format(t);
33 ■
34 ■  static String greetingForNow({required String firstName}) {
35 ■    final int h = DateTime.now().hour;
36 ■    final String part = h < 12
37 ■      ? 'Good morning'
38 ■      : h < 17
39 ■        ? 'Good afternoon'
40 ■        : 'Good evening';
41 ■    return '$part, $firstName!';
42 ■  }
43 ■ }

```

lib/core/utils/result.dart

56 lines

```

1 ■ /// Lightweight Result type so services return typed failures instead of
2 ■ /// throwing raw exceptions into the UI layer.
3 ■ sealed class Result<T> {
4 ■     const Result();
5 ■
6 ■     R when<R>({
7 ■         required R Function(T value) ok,
8 ■         required R Function(AppFailure failure) err,
9 ■     }) {
10 ■         final Result<T> self = this;
11 ■         return switch (self) {
12 ■             Ok<T>(:final T value) => ok(value),
13 ■             Err<T>(:final AppFailure failure) => err(failure),
14 ■         };
15 ■     }
16 ■
17 ■     bool get isOk => this is Ok<T>;
18 ■ }
19 ■
20 ■ final class Ok<T> extends Result<T> {
21 ■     const Ok(this.value);
22 ■     final T value;
23 ■ }
24 ■
25 ■ final class Err<T> extends Result<T> {
26 ■     const Err(this.failure);
27 ■     final AppFailure failure;
28 ■ }
29 ■
30 ■ /// Normalized, user-presentable failure.
31 ■ class AppFailure {
32 ■     const AppFailure(this.code, this.message);
33 ■
34 ■     final String code;
35 ■     final String message;
36 ■
37 ■     static const AppFailure network = AppFailure(
38 ■         'network',
39 ■         'No internet connection. Please check your network and try again.',
40 ■     );
41 ■     static const AppFailure permissionDenied = AppFailure(
42 ■         'permission-denied',
43 ■         'You don\'t have permission to do that.',
44 ■     );
45 ■     static const AppFailure sessionExpired = AppFailure(
46 ■         'session-expired',
47 ■         'Your session has expired. Please sign in again.',
48 ■     );
49 ■     static const AppFailure unknown = AppFailure(
50 ■         'unknown',
51 ■         'Something went wrong. Please try again.',
52 ■     );
53 ■
54 ■     @override
55 ■     String toString() => 'AppFailure($code: $message)';
56 ■ }

```

3. Reusable Widgets

Cards, status pills, dialogs, states + NEW: YouTube-style shimmer skeleton loading.

lib/core/widgets/nmb_card.dart

52 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../theme/nmb_colors.dart';
3 ■ import '../theme/nmb_theme.dart';
4 ■
5 ■ /// Rounded card with a soft shadow – the base surface of the design system.
6 ■ class NmbCard extends StatelessWidget {
7 ■   const NmbCard({
8 ■     super.key,
9 ■     required this.child,
10 ■     this.padding = const EdgeInsets.all(18),
11 ■     this.color = NmbColors.surface,
12 ■     this.onTap,
13 ■     this.borderColor,
14 ■   });
15 ■
16 ■   final Widget child;
17 ■   final EdgeInsetsGeometry padding;
18 ■   final Color color;
19 ■   final VoidCallback? onTap;
20 ■   final Color? borderColor;
21 ■
22 ■   @override
23 ■   Widget build(BuildContext context) {
24 ■     final Widget card = Container(
25 ■       decoration: BoxDecoration(
26 ■         color: color,
27 ■         borderRadius: BorderRadius.circular(NmbTheme.radiusCard),
28 ■         border: borderColor != null
29 ■           ? Border.all(color: borderColor!, width: 1.2)
30 ■           : null,
31 ■         boxShadow: const <BoxShadow>[
32 ■           BoxShadow(
33 ■             color: Color(0x0D17233B),
34 ■             blurRadius: 18,
35 ■             offset: Offset(0, 6),
36 ■           ),
37 ■         ],
38 ■       ),
39 ■       child: Padding(padding: padding, child: child),
40 ■     );
41 ■
42 ■     if (onTap == null) return card;
43 ■     return Material(
44 ■       color: Colors.transparent,
45 ■       child: InkWell(
46 ■         onTap: onTap,
47 ■         borderRadius: BorderRadius.circular(NmbTheme.radiusCard),
48 ■         child: card,
49 ■       ),
50 ■     );
51 ■   }
52 ■ }

```

lib/core/widgets/status_pill.dart

119 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../constants/enums.dart';
3 ■ import '../theme/nmb_colors.dart';
4 ■ import '../theme/nmb_typography.dart';
5 ■
6 ■ /// Small colored capsule used for every status in the app (bus status,
7 ■ /// tracking status, account status ...) so states always look consistent.
8 ■ class StatusPill extends StatelessWidget {
9 ■   const StatusPill({
10 ■     super.key,
11 ■     required this.label,
12 ■     required this.color,
13 ■     required this.background,
14 ■     this.pulse = false,
15 ■   });
16 ■
17 ■   final String label;
18 ■   final Color color;
19 ■   final Color background;
20 ■
21 ■   /// Shows an animated dot for genuinely-live states only.
22 ■   final bool pulse;
23 ■
24 ■   factory StatusPill.freshness(LocationFreshness f) {
25 ■     switch (f) {
26 ■       case LocationFreshness.live:
27 ■         return const StatusPill(
28 ■           label: 'LIVE',
29 ■           color: NmbColors.live,
30 ■           background: NmbColors.successSoft,
31 ■           pulse: true,
32 ■         );
33 ■       case LocationFreshness.stale:
34 ■         return const StatusPill(
35 ■           label: 'DELAYED SIGNAL',
36 ■           color: NmbColors.stale,
37 ■           background: NmbColors.warningSoft,
38 ■         );
39 ■       case LocationFreshness.unavailable:
40 ■         return const StatusPill(
41 ■           label: 'NOT TRACKING',
42 ■           color: NmbColors.offline,
43 ■           background: NmbColors.divider,
44 ■         );
45 ■     }
46 ■   }
47 ■
48 ■   @override
49 ■   Widget build(BuildContext context) {
50 ■     return Container(
51 ■       padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
52 ■       decoration: BoxDecoration(
53 ■         color: background,
54 ■         borderRadius: BorderRadius.circular(999),
55 ■       ),
56 ■       child: Row(
57 ■         mainAxisAlignment: MainAxisAlignment.min,
58 ■         children: <Widget>[

```

```

59 ■         if (pulse) ...<Widget>[
60 ■             _PulsingDot(color: color),
61 ■             const SizedBox(width: 6),
62 ■         ] else ...<Widget>[
63 ■             Container(
64 ■                 width: 7,
65 ■                 height: 7,
66 ■                 decoration: BoxDecoration(color: color, shape: BoxShape.circle),
67 ■             ),
68 ■             const SizedBox(width: 6),
69 ■         ],
70 ■         Text(
71 ■             label,
72 ■             style: NmbTypography.caption.copyWith(
73 ■                 color: color,
74 ■                 fontWeight: FontWeight.w800,
75 ■                 letterSpacing: 0.6,
76 ■             ),
77 ■         ),
78 ■     ],
79 ■ ),
80 ■ );
81 ■ }
82 ■ }
83 ■
84 ■ class _PulsingDot extends StatefulWidget {
85 ■     const _PulsingDot({required this.color});
86 ■     final Color color;
87 ■
88 ■     @override
89 ■     State<_PulsingDot> createState() => _PulsingDotState();
90 ■ }
91 ■
92 ■ class _PulsingDotState extends State<_PulsingDot>
93 ■     with SingleTickerProviderStateMixin {
94 ■     late final AnimationController _controller = AnimationController(
95 ■         vsync: this,
96 ■         duration: const Duration(milliseconds: 1100),
97 ■     )..repeat(reverse: true);
98 ■
99 ■     @override
100 ■     void dispose() {
101 ■         _controller.dispose();
102 ■         super.dispose();
103 ■     }
104 ■
105 ■     @override
106 ■     Widget build(BuildContext context) {
107 ■         return FadeTransition(
108 ■             opacity: Tween<double>(begin: 0.35, end: 1).animate(
109 ■                 CurvedAnimation(parent: _controller, curve: Curves.easeInOut),
110 ■             ),
111 ■             child: Container(
112 ■                 width: 8,
113 ■                 height: 8,
114 ■                 decoration:
115 ■                     BoxDecoration(color: widget.color, shape: BoxShape.circle),
116 ■             ),
117 ■         );
118 ■     }
119 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../theme/nmb_colors.dart';
3 ■ import '../theme/nmb_typography.dart';
4 ■
5 ■ /// Full-area loading indicator.
6 ■ class LoadingView extends StatelessWidget {
7 ■   const LoadingView({super.key, this.message});
8 ■   final String? message;
9 ■
10 ■   @override
11 ■   Widget build(BuildContext context) {
12 ■     return Center(
13 ■       child: Column(
14 ■         mainAxisAlignment: MainAxisAlignment.min,
15 ■         children: <Widget>[
16 ■           const SizedBox(
17 ■             width: 34,
18 ■             height: 34,
19 ■             child: CircularProgressIndicator(strokeWidth: 3),
20 ■           ),
21 ■           if (message != null) ...<Widget>[
22 ■             const SizedBox(height: 14),
23 ■             Text(message!, style: NmbTypography.bodySecondary),
24 ■           ],
25 ■         ],
26 ■     ),
27 ■   );
28 ■ }
29 ■ }
30 ■
31 ■ /// Friendly empty state with icon, title, message and optional action.
32 ■ class EmptyState extends StatelessWidget {
33 ■   const EmptyState({
34 ■     super.key,
35 ■     required this.icon,
36 ■     required this.title,
37 ■     required this.message,
38 ■     this.actionLabel,
39 ■     this.onAction,
40 ■   });
41 ■
42 ■   final IconData icon;
43 ■   final String title;
44 ■   final String message;
45 ■   final String? actionLabel;
46 ■   final VoidCallback? onAction;
47 ■
48 ■   @override
49 ■   Widget build(BuildContext context) {
50 ■     return Center(
51 ■       child: Padding(
52 ■         padding: const EdgeInsets.all(32),
53 ■         child: Column(
54 ■           mainAxisAlignment: MainAxisAlignment.min,
55 ■           children: <Widget>[
56 ■             Container(
57 ■               width: 88,
58 ■               height: 88,

```

```

59 ■         decoration: const BoxDecoration(
60 ■             color: NmbColors.primarySoft,
61 ■             shape: BoxShape.circle,
62 ■         ),
63 ■         child: Icon(icon, size: 42, color: NmbColors.primary),
64 ■     ),
65 ■     const SizedBox(height: 18),
66 ■     Text(title,
67 ■         style: NmbTypography.sectionTitle,
68 ■         textAlign: TextAlign.center,),
69 ■     const SizedBox(height: 8),
70 ■     Text(message,
71 ■         style: NmbTypography.bodySecondary,
72 ■         textAlign: TextAlign.center,),
73 ■     if (actionLabel != null && onAction != null) ...<Widget>[
74 ■         const SizedBox(height: 20),
75 ■         FilledButton(onPressed: onAction, child: Text(actionLabel!)),
76 ■     ],
77 ■ ],
78 ■ ),
79 ■ ),
80 ■ );
81 ■ }
82 ■ }
83 ■
84 ■ /// Error state with retry – used for network/database failures.
85 ■ class ErrorView extends StatelessWidget {
86 ■     const ErrorView({
87 ■         super.key,
88 ■         this.title = 'Something went wrong',
89 ■         required this.message,
90 ■         this.onRetry,
91 ■     });
92 ■
93 ■     final String title;
94 ■     final String message;
95 ■     final VoidCallback? onRetry;
96 ■
97 ■     @override
98 ■     Widget build(BuildContext context) {
99 ■         return EmptyState(
100 ■             icon: Icons.cloud_off_rounded,
101 ■             title: title,
102 ■             message: message,
103 ■             actionLabel: onRetry != null ? 'Try again' : null,
104 ■             onAction: onRetry,
105 ■         );
106 ■     }
107 ■ }
108 ■
109 ■ /// Banner shown when the device is offline.
110 ■ class OfflineBanner extends StatelessWidget {
111 ■     const OfflineBanner({super.key});
112 ■
113 ■     @override
114 ■     Widget build(BuildContext context) {
115 ■         return Material(
116 ■             color: NmbColors.warningSoft,
117 ■
118 ■             child: SafeArea(
119 ■                 bottom: false,
120 ■                 child: Padding(
121 ■                     padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
122 ■                     child: Row(
123 ■                         children: <Widget>[
124 ■                             const Icon(Icons.wifi_off_rounded,
125 ■                                 size: 18, color: NmbColors.warning,),
126 ■                             const SizedBox(width: 8),
127 ■                             Expanded(
128 ■                                 child: Text(
129 ■                                     'You\'re offline. Information may not be up to date.',
130 ■                                     style: NmbTypography.caption
131 ■                                     .copyWith(color: NmbColors.warning),
132 ■                                 ),
133 ■                             ),
134 ■                         ],
135 ■                     ),
136 ■                 ),
137 ■             );
138 ■     }
139 ■ }

```

lib/core/widgets/nmb_dialogs.dart

88 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../theme/nmb_colors.dart';
3 ■ import '../theme/nmb_typography.dart';
4 ■
5 ■ /// Consistent confirmation dialog. Returns true if confirmed.
6 ■ Future<bool> showNmbConfirmDialog(
7 ■   BuildContext context, {
8 ■     required String title,
9 ■     required String message,
10 ■     String confirmLabel = 'Confirm',
11 ■     String cancelLabel = 'Cancel',
12 ■     bool destructive = false,
13 ■     IconData? icon,
14 ■ }) async {
15 ■   final bool? result = await showDialog<bool>(
16 ■     context: context,
17 ■     builder: (BuildContext ctx) => AlertDialog(
18 ■       title: Row(
19 ■         children: <Widget>[
20 ■           if (icon != null) ...<Widget>[
21 ■             Icon(icon,
22 ■               color: destructive ? NmbColors.danger : NmbColors.primary,),
23 ■             const SizedBox(width: 10),
24 ■           ],
25 ■           Expanded(child: Text(title)),
26 ■         ],
27 ■       ),
28 ■       content: Text(message),
29 ■       actions: <Widget>[
30 ■         TextButton(
31 ■           onPressed: () => Navigator.of(ctx).pop(false),
32 ■           child: Text(cancelLabel),
33 ■         ),
34 ■         FilledButton(
35 ■           style: destructive
36 ■             ? FilledButton.styleFrom(
37 ■               backgroundColor: NmbColors.danger,
38 ■               minimumSize: const Size(110, 46),
39 ■             )
40 ■             : FilledButton.styleFrom(minimumSize: const Size(110, 46)),
41 ■           onPressed: () => Navigator.of(ctx).pop(true),
42 ■           child: Text(confirmLabel),
43 ■         ),
44 ■       ],
45 ■     ),
46 ■   );
47 ■   return result ?? false;
48 ■ }
49 ■
50 ■ /// Consistent snackbars.
51 ■ void showNmbSnack(
52 ■   BuildContext context,
53 ■   String message, {
54 ■     bool isError = false,
55 ■     bool isSuccess = false,
56 ■ }) {
57 ■   ScaffoldMessenger.of(context)
58 ■     ..hideCurrentSnackBar()

```

```

59 ■ ..showSnackBar(
60 ■   SnackBar(
61 ■     backgroundColor: isError
62 ■       ? NmbColors.danger
63 ■       : isSuccess
64 ■       ? NmbColors.success
65 ■       : NmbColors.textPrimary,
66 ■     content: Row(
67 ■       children: <Widget>[
68 ■         Icon(
69 ■           isError
70 ■             ? Icons.error_outline_rounded
71 ■             : isSuccess
72 ■             ? Icons.check_circle_outline_rounded
73 ■             : Icons.info_outline_rounded,
74 ■           color: Colors.white,
75 ■           size: 20,
76 ■         ),
77 ■         const SizedBox(width: 10),
78 ■         Expanded(
79 ■           child: Text(
80 ■             message,
81 ■             style: NmbTypography.body.copyWith(color: Colors.white),
82 ■           ),
83 ■         ),
84 ■       ],
85 ■     ),
86 ■   ),
87 ■ );
88 ■ }

```

lib/core/widgets/responsive_scaffold_body.dart

43 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import '../constants/nmb_constants.dart';
3 ■
4 ■ /// Constrains content width on large screens and applies standard padding –
5 ■ /// keeps layouts responsive without hard-coded dimensions in screens.
6 ■ class ResponsiveBody extends StatelessWidget {
7 ■   const ResponsiveBody({
8 ■     super.key,
9 ■     required this.child,
10 ■     this.scrollable = true,
11 ■     this.padding,
12 ■   });
13 ■
14 ■   final Widget child;
15 ■   final bool scrollable;
16 ■   final EdgeInsetsGeometry? padding;
17 ■
18 ■   @override
19 ■   Widget build(BuildContext context) {
20 ■     final Widget constrained = Center(
21 ■       child: ConstrainedBox(
22 ■         constraints:
23 ■           const BoxConstraints(maxWidth: NmbConstants.maxContentWidth),
24 ■         child: Padding(
25 ■           padding: padding ??
26 ■             const EdgeInsets.symmetric(
27 ■               horizontal: NmbConstants.screenPadding,
28 ■               vertical: 12,
29 ■             ),
30 ■         child: child,
31 ■       ),
32 ■     ),
33 ■   );
34 ■
35 ■   if (!scrollable) return SafeArea(child: constrained);
36 ■   return SafeArea(
37 ■     child: SingleChildScrollView(
38 ■       physics: const BouncingScrollPhysics(),
39 ■       child: constrained,
40 ■     ),
41 ■   );
42 ■ }
43 ■ }

```

lib/core/widgets/skeleton.dart

229 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■
3 ■ import '../theme/nmb_colors.dart';
4 ■
5 ■ /// YouTube-style shimmer skeleton loading.
6 ■ ///
7 ■ /// Instead of a blank screen or a spinner, screens show grey placeholder
8 ■ /// shapes that shimmer until real data arrives – the app feels fast and
9 ■ /// alive from the first frame.
10 ■ class Shimmer extends StatefulWidget {
11 ■   const Shimmer({super.key, required this.child});
12 ■
13 ■   final Widget child;
14 ■
15 ■   @override
16 ■   State<Shimmer> createState() => _ShimmerState();
17 ■ }
18 ■
19 ■ class _ShimmerState extends State<Shimmer>
20 ■   with SingleTickerProviderStateMixin {
21 ■   late final AnimationController _controller = AnimationController(
22 ■     vsync: this,
23 ■     duration: const Duration(milliseconds: 1300),
24 ■     ..repeat();
25 ■
26 ■   @override
27 ■   void dispose() {
28 ■     _controller.dispose();
29 ■     super.dispose();
30 ■   }
31 ■
32 ■   @override
33 ■   Widget build(BuildContext context) {
34 ■     return AnimatedBuilder(
35 ■       animation: _controller,
36 ■       builder: (BuildContext context, Widget? child) {
37 ■         return ShaderMask(
38 ■           blendMode: BlendMode.srcATop,
39 ■           shaderCallback: (Rect bounds) {
40 ■             final double slide = _controller.value * 2.4 - 1.2;
41 ■             return LinearGradient(
42 ■               begin: Alignment(-1 + slide, -0.2),
43 ■               end: Alignment(0.2 + slide, 0.2),
44 ■               colors: const <Color>[
45 ■                 Color(0xFFE4E9F2),
46 ■                 Color(0xFFF5F7FB),
47 ■                 Color(0xFFE4E9F2),
48 ■               ],
49 ■               stops: const <double>[0.25, 0.5, 0.75],
50 ■             ).createShader(bounds);
51 ■           },
52 ■           child: child,
53 ■         );
54 ■       },
55 ■       child: widget.child,
56 ■     );
57 ■   }
58 ■ }

```

```

59 ■
60 ■ /// A single grey placeholder block.
61 ■ class Bone extends StatelessWidget {
62 ■   const Bone({
63 ■     super.key,
64 ■     this.width,
65 ■     this.height = 14,
66 ■     this.radius = 8,
67 ■     this.circle = false,
68 ■   });
69 ■
70 ■   final double? width;
71 ■   final double height;
72 ■   final double radius;
73 ■   final bool circle;
74 ■
75 ■   @override
76 ■   Widget build(BuildContext context) {
77 ■     return Container(
78 ■       width: circle ? height : width,
79 ■       height: height,
80 ■       decoration: BoxDecoration(
81 ■         color: const Color(0xFFE4E9F2),
82 ■         shape: circle ? BoxShape.circle : BoxShape.rectangle,
83 ■         borderRadius: circle ? null : BorderRadius.circular(radius),
84 ■       ),
85 ■     );
86 ■   }
87 ■ }
88 ■
89 ■ /// Ready-made skeleton for the student home screen (matches its layout:
90 ■ /// header, bus card, banner, quick actions, list items).
91 ■ class HomeSkeleton extends StatelessWidget {
92 ■   const HomeSkeleton({super.key});
93 ■
94 ■   @override
95 ■   Widget build(BuildContext context) {
96 ■     return Shimmer(
97 ■       child: SingleChildScrollView(
98 ■         physics: const NeverScrollableScrollPhysics(),
99 ■         child: Column(
100 ■           children: <Widget>[
101 ■             // Header block
102 ■             Container(
103 ■               height: 190,
104 ■               decoration: const BoxDecoration(
105 ■                 color: Color(0xFFE4E9F2),
106 ■                 borderRadius: BorderRadius.only(
107 ■                   bottomLeft: Radius.circular(28),
108 ■                   bottomRight: Radius.circular(28),
109 ■                 ),
110 ■             ),
111 ■             padding: const EdgeInsets.fromLTRB(20, 60, 20, 20),
112 ■             child: const Row(
113 ■               children: <Widget>[
114 ■                 Expanded(
115 ■                   child: Column(
116 ■                     crossAxisAlignment: CrossAxisAlignment.start,

```

```

117 ■         children: <Widget>[
118 ■             Bone(width: 120, height: 14),
119 ■             SizedBox(height: 10),
120 ■             Bone(width: 180, height: 24),
121 ■             SizedBox(height: 8),
122 ■             Bone(width: 150, height: 12),
123 ■         ],
124 ■     ),
125 ■ ),
126 ■     Bone(height: 64, circle: true),
127 ■ ],
128 ■ ),
129 ■ ),
130 ■ Padding(
131 ■     padding: const EdgeInsets.all(20),
132 ■     child: Column(
133 ■         children: <Widget>[
134 ■             _cardBone(height: 92),
135 ■             const SizedBox(height: 14),
136 ■             _cardBone(height: 84),
137 ■             const SizedBox(height: 20),
138 ■             const Row(
139 ■                 mainAxisAlignment: MainAxisAlignment.spaceBetween,
140 ■                 children: <Widget>[
141 ■                     Column(children: <Widget>[
142 ■                         Bone(height: 54, circle: true),
143 ■                         SizedBox(height: 8),
144 ■                         Bone(width: 52, height: 10),
145 ■                     ],),
146 ■                     Column(children: <Widget>[
147 ■                         Bone(height: 54, circle: true),
148 ■                         SizedBox(height: 8),
149 ■                         Bone(width: 52, height: 10),
150 ■                     ],),
151 ■                     Column(children: <Widget>[
152 ■                         Bone(height: 54, circle: true),
153 ■                         SizedBox(height: 8),
154 ■                         Bone(width: 52, height: 10),
155 ■                     ],),
156 ■                     Column(children: <Widget>[
157 ■                         Bone(height: 54, circle: true),
158 ■                         SizedBox(height: 8),
159 ■                         Bone(width: 52, height: 10),
160 ■                     ],),
161 ■                 ],),
162 ■             ),
163 ■             const SizedBox(height: 20),
164 ■             _cardBone(height: 110),
165 ■             const SizedBox(height: 14),
166 ■             _cardBone(height: 130),
167 ■         ],
168 ■     ),
169 ■ ),
170 ■ ],
171 ■ ),
172 ■ ),
173 ■ );
174 ■ }

```

```

175 ■
176 ■ Widget _cardBone({required double height}) {
177 ■   return Container(
178 ■     height: height,
179 ■     width: double.infinity,
180 ■     decoration: BoxDecoration(
181 ■       color: const Color(0xFFE4E9F2),
182 ■       borderRadius: BorderRadius.circular(20),
183 ■     ),
184 ■   );
185 ■ }
186 ■ }
187 ■
188 ■ /// Generic list skeleton (alerts, students, trips...).
189 ■ class ListSkeleton extends StatelessWidget {
190 ■   const ListSkeleton({super.key, this.items = 6});
191 ■
192 ■   final int items;
193 ■
194 ■   @override
195 ■   Widget build(BuildContext context) {
196 ■     return Shimmer(
197 ■       child: ListView.separated(
198 ■         physics: const NeverScrollableScrollPhysics(),
199 ■         padding: const EdgeInsets.all(20),
200 ■         itemCount: items,
201 ■         separatorBuilder: (_, __) => const SizedBox(height: 12),
202 ■         itemBuilder: (_, __) => Container(
203 ■           padding: const EdgeInsets.all(14),
204 ■           decoration: BoxDecoration(
205 ■             color: Colors.white,
206 ■             borderRadius: BorderRadius.circular(16),
207 ■             border: Border.all(color: NmbColors.divider),
208 ■           ),
209 ■           child: const Row(
210 ■             children: <Widget>[
211 ■               Bone(height: 44, circle: true),
212 ■               SizedBox(width: 12),
213 ■               Expanded(
214 ■                 child: Column(
215 ■                   crossAxisAlignment: CrossAxisAlignment.start,
216 ■                   children: <Widget>[
217 ■                     Bone(width: 160, height: 14),
218 ■                     SizedBox(height: 8),
219 ■                     Bone(width: 220, height: 11),
220 ■                   ],
221 ■                 ),
222 ■               ),
223 ■             ],
224 ■           ),
225 ■         ),
226 ■       );
227 ■   };
228 ■ }
229 ■ }

```

4. Data Models

Pure-Dart data classes incl. rollNumber & activeDevice (single-device login).

lib/models/geo_point_data.dart

38 lines

```

1 ■ import 'dart:math' as math;
2 ■
3 ■ /// Plain lat/lng value object (keeps models free of Firebase imports).
4 ■ class GeoPointData {
5 ■   const GeoPointData({required this.lat, required this.lng});
6 ■
7 ■   final double lat;
8 ■   final double lng;
9 ■
10 ■   factory GeoPointData.fromMap(Map<String, dynamic> map) => GeoPointData(
11 ■     lat: (map['lat'] as num?).toDouble() ?? 0,
12 ■     lng: (map['lng'] as num?).toDouble() ?? 0,
13 ■   );
14 ■
15 ■   Map<String, dynamic> toMap() => <String, dynamic>{'lat': lat, 'lng': lng};
16 ■
17 ■   /// Haversine distance in meters.
18 ■   double distanceTo(GeoPointData other) {
19 ■     const double r = 6371000;
20 ■     final double dLat = _rad(other.lat - lat);
21 ■     final double dLng = _rad(other.lng - lng);
22 ■     final double a = math.sin(dLat / 2) * math.sin(dLat / 2) +
23 ■       math.cos(_rad(lat)) *
24 ■       math.cos(_rad(other.lat)) *
25 ■       math.sin(dLng / 2) *
26 ■       math.sin(dLng / 2);
27 ■     return 2 * r * math.atan2(math.sqrt(a), math.sqrt(1 - a));
28 ■   }
29 ■
30 ■   static double _rad(double deg) => deg * math.pi / 180;
31 ■
32 ■   @override
33 ■   bool operator ==(Object other) =>
34 ■     other is GeoPointData && other.lat == lat && other.lng == lng;
35 ■
36 ■   @override
37 ■   int get hashCode => Object.hash(lat, lng);
38 ■ }

```

lib/models/app_user.dart

79 lines

```

1 ■ import '../core/constants/enums.dart';
2 ■
3 ■ /// A user profile document (`users/{uid}`).
4 ■ ///
5 ■ /// NOTE: [role] and [busId] here are display mirrors. The *authoritative*
6 ■ /// values live in Firebase Auth custom claims and are enforced by security
7 ■ /// rules – the app never grants access based on this document alone.
8 ■ class AppUser {
9 ■   const AppUser({
10 ■     required this.uid,
11 ■     required this.role,
12 ■     required this.fullName,
13 ■     required this.email,
14 ■     this.phone,
15 ■     this.busId,
16 ■     this.stopId,
17 ■     this.classSection,
18 ■     this.rollNumber,
19 ■     this.isActive = true,
20 ■     this.createdAt,
21 ■     this.activeDevice,
22 ■   });
23 ■
24 ■   final String uid;
25 ■   final UserRole role;
26 ■   final String fullName;
27 ■   final String email;
28 ■   final String? phone;
29 ■   final String? busId;
30 ■   final String? stopId;
31 ■   final String? classSection;
32 ■   final String? rollNumber;
33 ■   final bool isActive;
34 ■   final DateTime? createdAt;
35 ■
36 ■   /// Single-device login: {id, name, at(ms)} from settings.activeDevice.
37 ■   final Map<String, dynamic>? activeDevice;
38 ■
39 ■   String get firstName =>
40 ■     fullName.trim().isEmpty ? 'there' : fullName.trim().split(' ').first;
41 ■
42 ■   factory AppUser.fromMap(String uid, Map<String, dynamic> map) => AppUser(
43 ■     uid: uid,
44 ■     role: UserRole.tryParse(map['role'] as String?) ?? UserRole.student,
45 ■     fullName: (map['fullName'] as String?) ?? '',
46 ■     email: (map['email'] as String?) ?? '',
47 ■     phone: map['phone'] as String?,
48 ■     busId: map['busId'] as String?,
49 ■     stopId: map['stopId'] as String?,
50 ■     classSection: map['classSection'] as String?,
51 ■     rollNumber: map['rollNumber'] as String?,
52 ■     isActive: (map['isActive'] as bool?) ?? true,
53 ■     createdAt: _toDate(map['createdAt']),
54 ■     activeDevice: ((map['settings'] as Map?)?['activeDevice'] as Map?)
55 ■       ?.cast<String, dynamic>(),
56 ■   );
57 ■
58 ■   Map<String, dynamic> toMap() => <String, dynamic>{
59 ■     'role': role.name,
60 ■     'fullName': fullName,
61 ■     'email': email,
62 ■     if (phone != null) 'phone': phone,
63 ■     if (busId != null) 'busId': busId,
64 ■     if (stopId != null) 'stopId': stopId,
65 ■     if (classSection != null) 'classSection': classSection,
66 ■     if (rollNumber != null) 'rollNumber': rollNumber,
67 ■     'isActive': isActive,
68 ■   };
69 ■
70 ■   static DateTime? _toDate(Object? v) {
71 ■     if (v == null) return null;
72 ■     // cloud_firestore Timestamp exposes toDate(); avoid importing it here.
73 ■     try {
74 ■       return (v as dynamic).toDate() as DateTime?;
75 ■     } catch (_) {
76 ■       return null;
77 ■     }
78 ■   }
79 ■ }

```

```
1 ■ class Bus {
2 ■   const Bus({
3 ■     required this.id,
4 ■     required this.busNumber,
5 ■     required this.plateNumber,
6 ■     this.capacity = 0,
7 ■     this.driverId,
8 ■     this.routeId,
9 ■     this.isActive = true,
10 ■   });
11 ■
12 ■   final String id;
13 ■   final String busNumber; // e.g. "Bus 3"
14 ■   final String plateNumber; // e.g. "UK07 PA 1234"
15 ■   final int capacity;
16 ■   final String? driverId;
17 ■   final String? routeId;
18 ■   final bool isActive;
19 ■
20 ■   factory Bus.fromMap(String id, Map<String, dynamic> map) => Bus(
21 ■     id: id,
22 ■     busNumber: (map['busNumber'] as String?) ?? '',
23 ■     plateNumber: (map['plateNumber'] as String?) ?? '',
24 ■     capacity: (map['capacity'] as num?)?.toInt() ?? 0,
25 ■     driverId: map['driverId'] as String?,
26 ■     routeId: map['routeId'] as String?,
27 ■     isActive: (map['isActive'] as bool?) ?? true,
28 ■   );
29 ■
30 ■   Map<String, dynamic> toMap() => <String, dynamic>{
31 ■     'busNumber': busNumber,
32 ■     'plateNumber': plateNumber,
33 ■     'capacity': capacity,
34 ■     'driverId': driverId,
35 ■     'routeId': routeId,
36 ■     'isActive': isActive,
37 ■   };
38 ■ }
```

lib/models/bus_route.dart

48 lines

```

1 ■ import '../core/constants/enums.dart';
2 ■ import 'geo_point_data.dart';
3 ■
4 ■ class BusRoute {
5 ■   const BusRoute({
6 ■     required this.id,
7 ■     required this.name,
8 ■     required this.busId,
9 ■     required this.stopOrder,
10 ■    required this.schoolLocation,
11 ■    this.polyline = '',
12 ■    this.direction = TripDirection.pickup,
13 ■    this.isActive = true,
14 ■  });
15 ■
16 ■   final String id;
17 ■   final String name;
18 ■   final String busId;
19 ■   final List<String> stopOrder; // ordered stop ids
20 ■   final GeoPointData schoolLocation;
21 ■   final String polyline; // encoded polyline; may be empty (fallback: stops)
22 ■   final TripDirection direction;
23 ■   final bool isActive;
24 ■
25 ■   factory BusRoute.fromMap(String id, Map<String, dynamic> map) => BusRoute(
26 ■     id: id,
27 ■     name: (map['name'] as String?) ?? '',
28 ■     busId: (map['busId'] as String?) ?? '',
29 ■     stopOrder: List<String>.from(map['stopOrder'] as List? ?? <String>[]),
30 ■     schoolLocation: GeoPointData.fromMap(
31 ■       Map<String, dynamic>.from(map['schoolLocation'] as Map? ?? {}),
32 ■     ),
33 ■     polyline: (map['polyline'] as String?) ?? '',
34 ■     direction:
35 ■       TripDirection.parse((map['direction'] as String?) ?? 'pickup'),
36 ■     isActive: (map['isActive'] as bool?) ?? true,
37 ■   );
38 ■
39 ■   Map<String, dynamic> toMap() => <String, dynamic>{
40 ■     'name': name,
41 ■     'busId': busId,
42 ■     'stopOrder': stopOrder,
43 ■     'schoolLocation': schoolLocation.toMap(),
44 ■     'polyline': polyline,
45 ■     'direction': direction.name,
46 ■     'isActive': isActive,
47 ■   };
48 ■ }

```

```

1 ■ import 'geo_point_data.dart';
2 ■
3 ■ class BusStop {
4 ■   const BusStop({
5 ■     required this.id,
6 ■     required this.name,
7 ■     required this.routeId,
8 ■     required this.busId,
9 ■     required this.location,
10 ■    required this.order,
11 ■    this.scheduledTime,
12 ■    this.isActive = true,
13 ■  });
14 ■
15 ■   final String id;
16 ■   final String name;
17 ■   final String routeId;
18 ■   final String busId;
19 ■   final GeoPointData location;
20 ■   final int order;
21 ■   final String? scheduledTime; // "07:15"
22 ■   final bool isActive;
23 ■
24 ■   factory BusStop.fromMap(String id, Map<String, dynamic> map) => BusStop(
25 ■     id: id,
26 ■     name: (map['name'] as String?) ?? '',
27 ■     routeId: (map['routeId'] as String?) ?? '',
28 ■     busId: (map['busId'] as String?) ?? '',
29 ■     location: GeoPointData.fromMap(
30 ■       Map<String, dynamic>.from(map['location'] as Map? ?? {}),
31 ■     ),
32 ■     order: (map['order'] as num?)?.toInt() ?? 0,
33 ■     scheduledTime: map['scheduledTime'] as String?,
34 ■     isActive: (map['isActive'] as bool?) ?? true,
35 ■   );
36 ■
37 ■   Map<String, dynamic> toMap() => <String, dynamic>{
38 ■     'name': name,
39 ■     'routeId': routeId,
40 ■     'busId': busId,
41 ■     'location': location.toMap(),
42 ■     'order': order,
43 ■     if (scheduledTime != null) 'scheduledTime': scheduledTime,
44 ■     'isActive': isActive,
45 ■   };
46 ■ }

```

lib/models/trip.dart

46 lines

```

1 ■ import '../core/constants/enums.dart';
2 ■
3 ■ class Trip {
4 ■   const Trip({
5 ■     required this.id,
6 ■     required this.busId,
7 ■     required this.driverId,
8 ■     required this.routeId,
9 ■     required this.direction,
10 ■    required this.status,
11 ■    this.startedAt,
12 ■    this.endedAt,
13 ■   });
14 ■
15 ■   final String id;
16 ■   final String busId;
17 ■   final String driverId;
18 ■   final String routeId;
19 ■   final TripDirection direction;
20 ■   final TripStatus status;
21 ■   final DateTime? startedAt;
22 ■   final DateTime? endedAt;
23 ■
24 ■   bool get isActive => status == TripStatus.active;
25 ■
26 ■   factory Trip.fromMap(String id, Map<String, dynamic> map) => Trip(
27 ■     id: id,
28 ■     busId: (map['busId'] as String?) ?? '',
29 ■     driverId: (map['driverId'] as String?) ?? '',
30 ■     routeId: (map['routeId'] as String?) ?? '',
31 ■     direction:
32 ■       TripDirection.parse((map['direction'] as String?) ?? 'pickup'),
33 ■     status: TripStatus.parse((map['status'] as String?) ?? 'completed'),
34 ■     startedAt: _toDate(map['startedAt']),
35 ■     endedAt: _toDate(map['endedAt']),
36 ■   );
37 ■
38 ■   static DateTime? _toDate(Object? v) {
39 ■     if (v == null) return null;
40 ■     try {
41 ■       return (v as dynamic).toDate() as DateTime?;
42 ■     } catch (_) {
43 ■       return null;
44 ■     }
45 ■   }
46 ■ }

```

lib/models/live_location.dart

45 lines

```

1 ■ import '../core/constants/enums.dart';
2 ■ import '../core/constants/nmb_constants.dart';
3 ■ import 'geo_point_data.dart';
4 ■
5 ■ /// A single live GPS fix from `/liveLocations/{busId}` in Realtime Database.
6 ■ class LiveLocation {
7 ■   const LiveLocation({
8 ■     required this.position,
9 ■     required this.heading,
10 ■    required this.speedKmh,
11 ■    required this.accuracy,
12 ■    required this.tripId,
13 ■    required this.updatedAt,
14 ■   });
15 ■
16 ■   final GeoPointData position;
17 ■   final double heading;
18 ■   final double speedKmh;
19 ■   final double accuracy;
20 ■   final String tripId;
21 ■   final DateTime updatedAt;
22 ■
23 ■   factory LiveLocation.fromMap(Map<Object?, Object?> map) => LiveLocation(
24 ■     position: GeoPointData(
25 ■       lat: (map['lat'] as num?)?.toDouble() ?? 0,
26 ■       lng: (map['lng'] as num?)?.toDouble() ?? 0,
27 ■     ),
28 ■     heading: (map['heading'] as num?)?.toDouble() ?? 0,
29 ■     speedKmh: (map['speedKmh'] as num?)?.toDouble() ?? 0,
30 ■     accuracy: (map['accuracy'] as num?)?.toDouble() ?? 0,
31 ■     tripId: (map['tripId'] as String?) ?? '',
32 ■     updatedAt: DateTime.fromMillisecondsSinceEpoch(
33 ■       (map['updatedAt'] as num?)?.toInt() ?? 0,
34 ■     ),
35 ■   );
36 ■
37 ■   /// Honest freshness classification – the UI must never show a stale fix
38 ■   /// as "live".
39 ■   LocationFreshness freshness({DateTime? now}) {
40 ■     final Duration age = (now ?? DateTime.now()).difference(updatedAt);
41 ■     if (age <= NmbConstants.liveThreshold) return LocationFreshness.live;
42 ■     if (age <= NmbConstants.staleThreshold) return LocationFreshness.stale;
43 ■     return LocationFreshness.unavailable;
44 ■   }
45 ■ }

```

```

1 ■ import '../core/constants/enums.dart';
2 ■
3 ■ /// An item in the user's private inbox (`users/{uid}/inbox/{id}`).
4 ■ class AppNotification {
5 ■   const AppNotification({
6 ■     required this.id,
7 ■     required this.type,
8 ■     required this.title,
9 ■     required this.body,
10 ■    required this.receivedAt,
11 ■    this.read = false,
12 ■  });
13 ■
14 ■   final String id;
15 ■   final NotificationType type;
16 ■   final String title;
17 ■   final String body;
18 ■   final DateTime receivedAt;
19 ■   final bool read;
20 ■
21 ■   factory AppNotification.fromMap(String id, Map<String, dynamic> map) =>
22 ■     AppNotification(
23 ■       id: id,
24 ■       type: NotificationType.parse((map['type'] as String?) ?? ''),
25 ■       title: (map['title'] as String?) ?? '',
26 ■       body: (map['body'] as String?) ?? '',
27 ■       receivedAt: _toDate(map['receivedAt']) ?? DateTime.now(),
28 ■       read: (map['read'] as bool?) ?? false,
29 ■     );
30 ■
31 ■   static DateTime? _toDate(Object? v) {
32 ■     if (v == null) return null;
33 ■     try {
34 ■       return (v as dynamic).toDate() as DateTime?;
35 ■     } catch (_) {
36 ■       return null;
37 ■     }
38 ■   }
39 ■ }

```

lib/models/audit_log.dart

42 lines

```

1 ■ class AuditLog {
2 ■     const AuditLog({
3 ■         required this.id,
4 ■         required this.actorUid,
5 ■         required this.actorRole,
6 ■         required this.action,
7 ■         required this.targetType,
8 ■         required this.targetId,
9 ■         required this.at,
10 ■         this.details = const <String, dynamic>{},
11 ■     });
12 ■
13 ■     final String id;
14 ■     final String actorUid;
15 ■     final String actorRole;
16 ■     final String action;
17 ■     final String targetType;
18 ■     final String targetId;
19 ■     final DateTime at;
20 ■     final Map<String, dynamic> details;
21 ■
22 ■     factory AuditLog.fromMap(String id, Map<String, dynamic> map) => AuditLog(
23 ■         id: id,
24 ■         actorUid: (map['actorUid'] as String?) ?? '',
25 ■         actorRole: (map['actorRole'] as String?) ?? '',
26 ■         action: (map['action'] as String?) ?? '',
27 ■         targetType:
28 ■             ((map['target'] as Map?)?['type'] as String?) ?? '',
29 ■         targetId: ((map['target'] as Map?)?['id'] as String?) ?? '',
30 ■         at: _toDate(map['at']) ?? DateTime.now(),
31 ■         details: Map<String, dynamic>.from(map['details'] as Map? ?? {}),
32 ■     );
33 ■
34 ■     static DateTime? _toDate(Object? v) {
35 ■         if (v == null) return null;
36 ■         try {
37 ■             return (v as dynamic).toDate() as DateTime?;
38 ■         } catch (_) {
39 ■             return null;
40 ■         }
41 ■     }
42 ■ }

```

5. Services

Firebase gateways: auth (roll-number generations), Firestore, admin ops (password reset, force logout), live GPS, ETA, notifications, device sessions.

lib/services/auth_service.dart

135 lines

```

1 ■ import 'package:cloud_firestore/cloud_firestore.dart';
2 ■ import 'package:firebase_auth/firebase_auth.dart';
3 ■
4 ■ import '../core/constants/enums.dart';
5 ■ import '../core/utils/result.dart';
6 ■
7 ■ /// Session snapshot for the signed-in user.
8 ■ ///
9 ■ /// FREE-MODE (Spark plan) design: the role/bus assignment lives in the
10 ■ /// `users/{uid}` Firestore document and is mirrored to RTDB `/access/{uid}`.
11 ■ /// Both databases' security rules read those records server-side, so a
12 ■ /// modified client still cannot widen its own access. (The Blaze-plan
13 ■ /// variant with custom claims + Cloud Functions is documented in
14 ■ /// docs/SECURITY.md for when the school upgrades.)
15 ■ class AuthSession {
16 ■   const AuthSession({
17 ■     required this.uid,
18 ■     required this.email,
19 ■     required this.role,
20 ■     this.busId,
21 ■   });
22 ■
23 ■   final String uid;
24 ■   final String email;
25 ■   final UserRole role;
26 ■   final String? busId;
27 ■ }
28 ■
29 ■ class AuthService {
30 ■   AuthService(this._auth, this._db);
31 ■
32 ■   final FirebaseAuth _auth;
33 ■   final FirebaseFirestore _db;
34 ■
35 ■   Stream<User?> get authStateChanges => _auth.authStateChanges();
36 ■
37 ■   User? get currentUser => _auth.currentUser;
38 ■
39 ■   Future<Result<AuthSession>> signIn({
40 ■     required String email,
41 ■     required String password,
42 ■   }) async {
43 ■     try {
44 ■       await _auth.signInWithEmailAndPassword(
45 ■         email: email.trim(),
46 ■         password: password,
47 ■       );
48 ■       final AuthSession? session = await readSession(forceRefresh: true);
49 ■       if (session == null) {
50 ■         await _auth.signOut();
51 ■         return const Err<AuthSession>(
52 ■           AppFailure(
53 ■             'no-role',
54 ■             'This account is not set up yet or has been disabled. '
55 ■             'Please contact the school office.',
56 ■           ),
57 ■         );
58 ■       }

```

```

59 ■     return Ok<AuthSession>(session);
60 ■   } on FirebaseAuthException catch (e) {
61 ■     return Err<AuthSession>(_mapAuthError(e));
62 ■   } catch (_) {
63 ■     return const Err<AuthSession>(AppFailure.unknown);
64 ■   }
65 ■ }
66 ■
67 ■ /// Reads role/bus from the caller's own `users/{uid}` document.
68 ■ /// Security rules only allow reading one's own doc, and every other
69 ■ /// collection's rules re-verify the role server-side on each request.
70 ■ Future<AuthSession?> readSession({bool forceRefresh = false}) async {
71 ■   final User? user = _auth.currentUser;
72 ■   if (user == null) return null;
73 ■   try {
74 ■     final DocumentSnapshot<Map<String, dynamic>> doc = await _db
75 ■       .collection('users')
76 ■       .doc(user.uid)
77 ■       .get(
78 ■         GetOptions(
79 ■           source: forceRefresh ? Source.server : Source.serverAndCache,
80 ■         ),
81 ■       );
82 ■     final Map<String, dynamic>? data = doc.data();
83 ■     if (data == null) return null;
84 ■     final UserRole? role = UserRole.tryParse(data['role'] as String?);
85 ■     final bool active = (data['isActive'] as bool?) ?? false;
86 ■     if (role == null || !active) return null;
87 ■     return AuthSession(
88 ■       uid: user.uid,
89 ■       email: user.email ?? '',
90 ■       role: role,
91 ■       busId: data['busId'] as String?,
92 ■     );
93 ■   } catch (_) {
94 ■     // Doc unreadable (rules denied / offline with no cache) → no session.
95 ■     return null;
96 ■   }
97 ■ }
98 ■
99 ■ Future<Result<void>> sendPasswordReset(String email) async {
100 ■   try {
101 ■     await _auth.sendPasswordResetEmail(email: email.trim());
102 ■     return const Ok<void>(null);
103 ■   } on FirebaseAuthException catch (e) {
104 ■     return Err<void>(_mapAuthError(e));
105 ■   }
106 ■ }
107 ■
108 ■ Future<void> signOut() => _auth.signOut();
109 ■
110 ■ AppFailure _mapAuthError(FirebaseAuthException e) {
111 ■   switch (e.code) {
112 ■     case 'invalid-credential':
113 ■     case 'wrong-password':
114 ■     case 'user-not-found':
115 ■       return const AppFailure(
116 ■         'invalid-credential',
117 ■
118 ■         'Email or password is incorrect. Please try again.',
119 ■       );
120 ■     case 'user-disabled':
121 ■       return const AppFailure(
122 ■         'user-disabled',
123 ■         'This account has been disabled. Please contact the school office.',
124 ■       );
125 ■     case 'too-many-requests':
126 ■       return const AppFailure(
127 ■         'too-many-requests',
128 ■         'Too many attempts. Please wait a moment and try again.',
129 ■       );
130 ■     case 'network-request-failed':
131 ■       return AppFailure.network;
132 ■     default:
133 ■       return AppFailure(e.code, 'Sign-in failed. Please try again.');
```

[illegible]

[illegible]

```

117 ■     .where('busId', isEqualTo: busId)
118 ■     .where('isActive', isEqualTo: true)
119 ■     .limit(1)
120 ■     .snapshots()
121 ■     .map(
122 ■         (QuerySnapshot<Map<String, dynamic>> q) => q.docs.isEmpty
123 ■             ? null
124 ■             : BusRoute.fromMap(q.docs.first.id, q.docs.first.data()),
125 ■     );
126 ■
127 ■ Stream<List<BusStop>> watchStopsOfBus(String busId) => _db
128 ■     .collection('stops')
129 ■     .where('busId', isEqualTo: busId)
130 ■     .where('isActive', isEqualTo: true)
131 ■     .orderBy('order')
132 ■     .snapshots()
133 ■     .map(
134 ■         (QuerySnapshot<Map<String, dynamic>> q) => q.docs
135 ■             .map((QueryDocumentSnapshot<Map<String, dynamic>> d) =>
136 ■                 BusStop.fromMap(d.id, d.data()),)
137 ■             .toList(),
138 ■     );
139 ■
140 ■ // Admin CRUD (rules restrict to admin claim)
141 ■ Future<String> createBus(Bus bus) async {
142 ■     final DocumentReference<Map<String, dynamic>> ref =
143 ■         await _db.collection('buses').add(<String, dynamic>{
144 ■             ...bus.toMap(),
145 ■             'createdAt': FieldValue.serverTimestamp(),
146 ■             'updatedAt': FieldValue.serverTimestamp(),
147 ■         });
148 ■     return ref.id;
149 ■ }
150 ■
151 ■ Future<void> updateBus(String busId, Map<String, dynamic> data) =>
152 ■     _db.collection('buses').doc(busId).update(<String, dynamic>{
153 ■         ...data,
154 ■         'updatedAt': FieldValue.serverTimestamp(),
155 ■     });
156 ■
157 ■ Future<String> createStop(BusStop stop) async {
158 ■     final DocumentReference<Map<String, dynamic>> ref =
159 ■         await _db.collection('stops').add(<String, dynamic>{
160 ■             ...stop.toMap(),
161 ■             'createdAt': FieldValue.serverTimestamp(),
162 ■         });
163 ■     return ref.id;
164 ■ }
165 ■
166 ■ Future<void> updateStop(String stopId, Map<String, dynamic> data) =>
167 ■     _db.collection('stops').doc(stopId).update(data);
168 ■
169 ■ /// Admin: stop delete (soft – isActive false, taaki purane trips ka data
170 ■ /// kharab na ho).
171 ■ Future<void> deactivateStop(String stopId) =>
172 ■     _db.collection('stops').doc(stopId).update(<String, dynamic>{
173 ■         'isActive': false,
174 ■     });

```

[illegible]

[illegible]

lib/services/admin_service.dart

422 lines

```

1 ■ import 'package:cloud_firestore/cloud_firestore.dart';
2 ■ import 'package:firebase_auth/firebase_auth.dart';
3 ■ import 'package:firebase_core/firebase_core.dart';
4 ■ import 'package:firebase_database/firebase_database.dart'
5 ■     hide Query; // Query clash with cloud_firestore
6 ■
7 ■ import '../core/constants/nmb_constants.dart';
8 ■ import '../core/utils/result.dart';
9 ■ import '../models/app_user.dart';
10 ■
11 ■ /// FREE-MODE admin operations (Spark plan – no Cloud Functions needed).
12 ■ ///
13 ■ /// How security still holds without a server:
14 ■ /// - Every write below is validated by Firestore/RTDB security rules,
15 ■ ///   which look up the CALLER's `users/{uid}` doc and only allow these
16 ■ ///   operations when `role == 'admin'`. A student/driver client calling
17 ■ ///   these methods gets PERMISSION_DENIED from the backend.
18 ■ /// - New Auth accounts are created through a SECONDARY Firebase app
19 ■ ///   instance so the admin's own session is never replaced (the default
20 ■ ///   `createUserWithEmailAndPassword` signs in as the new user – the
21 ■ ///   secondary-app trick avoids that).
22 ■ /// - `/access/{uid}` in RTDB mirrors each user's role+bus so RTDB rules
23 ■ ///   (which cannot read Firestore) can authorize live-location access.
24 ■ ///   That mirror is writable only by admins, per RTDB rules.
25 ■ class AdminService {
26 ■   AdminService(this._db, this._rtdb);
27 ■
28 ■   final FirebaseFirestore _db;
29 ■   final FirebaseDatabase _rtdb;
30 ■
31 ■   Future<Result<String>> provisionUser({
32 ■     required String email,
33 ■     required String fullName,
34 ■     required String role, // 'student' | 'driver' | 'admin'
35 ■     required String temporaryPassword,
36 ■     String? busId,
37 ■     String? stopId,
38 ■     String? classSection,
39 ■     String? rollNumber,
40 ■     String? phone,
41 ■   }) async {
42 ■     FirebaseApp? tempApp;
43 ■     try {
44 ■       // Secondary app: creates the Auth user WITHOUT touching our session.
45 ■       tempApp = await Firebase.initializeApp(
46 ■         name: 'nmb_admin_worker',
47 ■         options: Firebase.app().options,
48 ■       );
49 ■       final UserCredential cred = await FirebaseAuth.instanceFor(app: tempApp)
50 ■         .createUserWithEmailAndPassword(
51 ■           email: email.trim(),
52 ■           password: temporaryPassword,
53 ■         );
54 ■       final String uid = cred.user!.uid;
55 ■       await FirebaseAuth.instanceFor(app: tempApp).signOut();
56 ■
57 ■       // Profile doc – rules verify the CALLER (this admin) is admin.
58 ■       await _db.collection('users').doc(uid).set(<String, dynamic>{

```

```

59 ■      'role': role,
60 ■      'fullName': fullName.trim(),
61 ■      'email': email.trim(),
62 ■      'busId': busId,
63 ■      'stopId': role == 'student' ? stopId : null,
64 ■      'classSection': role == 'student' ? classSection : null,
65 ■      'rollNumber': role == 'student' ? rollNumber : null,
66 ■      'phone': phone,
67 ■      'isActive': true,
68 ■      'fcmTokens': <String, dynamic>{},
69 ■      'settings': <String, dynamic>{},
70 ■      'createdAt': FieldValue.serverTimestamp(),
71 ■      'updatedAt': FieldValue.serverTimestamp(),
72 ■    });
73 ■
74 ■    // RTDB access mirror (for live-location rules).
75 ■    await _writeAccessMirror(uid, role: role, busId: busId, active: true);
76 ■
77 ■    await _audit('USER_PROVISIONED', 'user', uid,
78 ■      <String, dynamic>{'role': role, 'busId': busId},);
79 ■    return Ok<String>(uid);
80 ■  } on FirebaseAuthException catch (e) {
81 ■    if (e.code == 'email-already-in-use') {
82 ■      return const Err<String>(
83 ■        AppFailure('email-in-use', 'This email already has an account.'),
84 ■      );
85 ■    }
86 ■    if (e.code == 'weak-password') {
87 ■      return const Err<String>(
88 ■        AppFailure('weak-password', 'Choose a stronger password.'),
89 ■      );
90 ■    }
91 ■    return Err<String>(AppFailure(e.code, 'Could not create the account.'));
92 ■  } on FirebaseException catch (e) {
93 ■    if (e.code == 'permission-denied') {
94 ■      return const Err<String>(AppFailure.permissionDenied);
95 ■    }
96 ■    return const Err<String>(AppFailure.unknown);
97 ■  } finally {
98 ■    await tempApp?.delete();
99 ■  }
100 ■ }
101 ■
102 ■ Future<Result<void>> assignUserToBus({
103 ■   required String uid,
104 ■   String? busId,
105 ■   String? stopId,
106 ■ }) async {
107 ■   try {
108 ■     await _db.collection('users').doc(uid).update(<String, dynamic>{
109 ■       'busId': busId,
110 ■       'stopId': stopId,
111 ■       'updatedAt': FieldValue.serverTimestamp(),
112 ■     });
113 ■     final DocumentSnapshot<Map<String, dynamic>> doc =
114 ■       await _db.collection('users').doc(uid).get();
115 ■     await _writeAccessMirror(
116 ■       uid,

```

```

117 ■         role: (doc.data()?['role'] as String?) ?? 'student',
118 ■         busId: busId,
119 ■         active: (doc.data()?['isActive'] as bool?) ?? true,
120 ■     );
121 ■     await _audit('STUDENT_BUS_REASSIGNED', 'user', uid,
122 ■         <String, dynamic>{'busId': busId, 'stopId': stopId},);
123 ■     return const Ok<void>(null);
124 ■ } on FirebaseException catch (e) {
125 ■     return Err<void>(e.code == 'permission-denied'
126 ■         ? AppFailure.permissionDenied
127 ■         : AppFailure.unknown,);
128 ■ }
129 ■ }
130 ■
131 ■ Future<Result<void>> assignDriverToBus({
132 ■     required String driverUid,
133 ■     String? busId,
134 ■ }) async {
135 ■     try {
136 ■         final WriteBatch batch = _db.batch();
137 ■         // Clear any bus currently pointing at this driver.
138 ■         final QuerySnapshot<Map<String, dynamic>> previous = await _db
139 ■             .collection('buses')
140 ■             .where('driverId', isEqualTo: driverUid)
141 ■             .get();
142 ■         for (final QueryDocumentSnapshot<Map<String, dynamic>> d
143 ■             in previous.docs) {
144 ■             if (d.id != busId) {
145 ■                 batch.update(d.reference, <String, dynamic>{'driverId': null});
146 ■             }
147 ■         }
148 ■         if (busId != null) {
149 ■             batch.update(_db.collection('buses').doc(busId),
150 ■                 <String, dynamic>{'driverId': driverUid},);
151 ■         }
152 ■         batch.update(_db.collection('users').doc(driverUid),
153 ■             <String, dynamic>{
154 ■                 'busId': busId,
155 ■                 'updatedAt': FieldValue.serverTimestamp(),
156 ■             });
157 ■         await batch.commit();
158 ■
159 ■         await _writeAccessMirror(driverUid,
160 ■             role: 'driver', busId: busId, active: true,);
161 ■         await _audit('DRIVER_BUS_REASSIGNED', 'user', driverUid,
162 ■             <String, dynamic>{'busId': busId},);
163 ■         return const Ok<void>(null);
164 ■     } on FirebaseException catch (e) {
165 ■         return Err<void>(e.code == 'permission-denied'
166 ■             ? AppFailure.permissionDenied
167 ■             : AppFailure.unknown,);
168 ■     }
169 ■ }
170 ■
171 ■ /// Admin: user profile edit (naam, class, roll, phone).
172 ■ Future<Result<void>> updateUserProfile({
173 ■     required String uid,
174 ■     required Map<String, dynamic> data,

```

```

175 ■ }) async {
176 ■   try {
177 ■     await _db.collection('users').doc(uid).update(<String, dynamic>{
178 ■       ...data,
179 ■       'updatedAt': FieldValue.serverTimestamp(),
180 ■     });
181 ■     await _audit('USER_PROFILE_UPDATED', 'user', uid, data);
182 ■     return const Ok<void>(null);
183 ■   } on FirebaseException catch (e) {
184 ■     return Err<void>(e.code == 'permission-denied'
185 ■       ? AppFailure.permissionDenied
186 ■       : AppFailure.unknown,);
187 ■   }
188 ■ }
189 ■
190 ■ /// FREE-MODE student password reset.
191 ■ ///
192 ■ /// Bina Cloud Functions ke kisi doosre user ka Auth password nahi badla
193 ■ /// ja sakta, isliye trick: naya "generation" Auth account banta hai
194 ■ /// (st.7b.23.g2@...) naye password ke saath. Purana account disable +
195 ■ /// uska profile naye account par shift. Student wahi Class+Roll+naya
196 ■ /// password se login karega – usse koi fark nahi dikhta.
197 ■ Future<Result<void>> resetStudentPassword({
198 ■   required AppUser student,
199 ■   required String newPassword,
200 ■ }) async {
201 ■   if (student.classSection == null || student.rollNumber == null) {
202 ■     return const Err<void>{
203 ■       AppFailure('no-roll', 'This student has no class/roll set.'),
204 ■     };
205 ■   }
206 ■   final List<String> parts = student.classSection!.split('-');
207 ■   final String klass = parts.first;
208 ■   final String section = parts.length > 1 ? parts[1] : 'A';
209 ■
210 ■   FirebaseApp? tempApp;
211 ■   try {
212 ■     tempApp = await Firebase.initializeApp(
213 ■       name: 'nmb_pw_worker',
214 ■       options: Firebase.app().options,
215 ■     );
216 ■     final FirebaseAuth tempAuth = FirebaseAuth.instanceFor(app: tempApp);
217 ■
218 ■     // Agli free generation dhundo aur account banao.
219 ■     String? newEmail;
220 ■     String? newUid;
221 ■     for (int gen = 2; gen <= StudentLoginId.maxGenerations; gen++) {
222 ■       final String candidate = StudentLoginId.email(
223 ■         klass: klass,
224 ■         section: section,
225 ■         roll: student.rollNumber!,
226 ■         gen: gen,
227 ■       );
228 ■       try {
229 ■         final UserCredential cred =
230 ■           await tempAuth.createUserWithEmailAndPassword(
231 ■             email: candidate,
232 ■             password: newPassword,

```

```

233 ■         );
234 ■         newEmail = candidate;
235 ■         newUid = cred.user!.uid;
236 ■         await tempAuth.signOut();
237 ■         break;
238 ■     } on FirebaseAuthException catch (e) {
239 ■         if (e.code == 'email-already-in-use') continue; // agla gen try
240 ■         rethrow;
241 ■     }
242 ■ }
243 ■ if (newUid == null || newEmail == null) {
244 ■     return const Err<void>{
245 ■         AppFailure('resets-exhausted',
246 ■             'Too many resets. Please handle via the school office.'),
247 ■     };
248 ■ }
249 ■
250 ■ // Naya profile doc (same data, naya uid) + purana disable.
251 ■ final Map<String, dynamic> profile = <String, dynamic>{
252 ■     'role': 'student',
253 ■     'fullName': student.fullName,
254 ■     'email': newEmail,
255 ■     'busId': student.busId,
256 ■     'stopId': student.stopId,
257 ■     'classSection': student.classSection,
258 ■     'rollNumber': student.rollNumber,
259 ■     'isActive': true,
260 ■     'fcmTokens': <String, dynamic>{},
261 ■     'settings': <String, dynamic>{},
262 ■     'createdAt': FieldValue.serverTimestamp(),
263 ■     'updatedAt': FieldValue.serverTimestamp(),
264 ■ };
265 ■ await _db.collection('users').doc(newUid).set(profile);
266 ■ await _writeAccessMirror(newUid,
267 ■     role: 'student', busId: student.busId, active: true,);
268 ■
269 ■ await _db.collection('users').doc(student.uid).update(
270 ■     <String, dynamic>{'isActive': false, 'replacedBy': newUid},);
271 ■ await _writeAccessMirror(student.uid,
272 ■     role: 'student', busId: student.busId, active: false,);
273 ■
274 ■ await _audit('STUDENT_PASSWORD_RESET', 'user', student.uid,
275 ■     <String, dynamic>{'newUid': newUid},);
276 ■ return const Ok<void>(null);
277 ■ } on FirebaseException catch (e) {
278 ■     return Err<void>(e.code == 'permission-denied'
279 ■         ? AppFailure.permissionDenied
280 ■         : AppFailure.unknown,);
281 ■ } catch (_) {
282 ■     return const Err<void>(AppFailure.unknown);
283 ■ } finally {
284 ■     await tempApp?.delete();
285 ■ }
286 ■ }
287 ■
288 ■ /// Admin: force logout – user ke activeDevice ko REVOKED mark karo.
289 ■ /// User ka phone profile-stream se yeh dekh kar turant sign out ho
290 ■ /// jaata hai ("You were signed out by the school admin").

```

```

291 ■ Future<Result<void>> forceLogout(String uid) async {
292 ■   try {
293 ■     await _db.collection('users').doc(uid).set(
294 ■       <String, dynamic>{
295 ■         'settings': <String, dynamic>{
296 ■           'activeDevice': <String, dynamic>{
297 ■             'id': 'REVOKED',
298 ■             'name': 'Signed out by admin',
299 ■             'at': DateTime.now().millisecondsSinceEpoch,
300 ■           },
301 ■         },
302 ■       },
303 ■       SetOptions(merge: true),
304 ■     );
305 ■     await _audit('FORCE_LOGOUT', 'user', uid, const <String, dynamic>{});
306 ■     return const Ok<void>(null);
307 ■   } on FirebaseException catch (e) {
308 ■     return Err<void>(e.code == 'permission-denied'
309 ■       ? AppFailure.permissionDenied
310 ■       : AppFailure.unknown,);
311 ■   }
312 ■ }
313 ■
314 ■ Future<Result<void>> setAccountActive({
315 ■   required String uid,
316 ■   required bool active,
317 ■ }) async {
318 ■   try {
319 ■     await _db.collection('users').doc(uid).update(<String, dynamic>{
320 ■       'isActive': active,
321 ■       'updatedAt': FieldValue.serverTimestamp(),
322 ■     });
323 ■     final DocumentSnapshot<Map<String, dynamic>> doc =
324 ■       await _db.collection('users').doc(uid).get();
325 ■     await _writeAccessMirror(
326 ■       uid,
327 ■       role: (doc.data()?['role'] as String?) ?? 'student',
328 ■       busId: doc.data()?['busId'] as String?,
329 ■       active: active,
330 ■     );
331 ■     await _audit(active ? 'ACCOUNT_ENABLED' : 'ACCOUNT_DISABLED',
332 ■       'user', uid, const <String, dynamic>{});
333 ■     return const Ok<void>(null);
334 ■   } on FirebaseException catch (e) {
335 ■     return Err<void>(e.code == 'permission-denied'
336 ■       ? AppFailure.permissionDenied
337 ■       : AppFailure.unknown,);
338 ■   }
339 ■ }
340 ■
341 ■ /// Announcement fan-out to each recipient's private inbox.
342 ■ /// Rules allow inbox CREATE only for admins.
343 ■ Future<Result<void>> sendAnnouncement({
344 ■   required String title,
345 ■   required String body,
346 ■   required String scope, // 'all' | 'bus'
347 ■   String? busId,
348 ■ }) async {

```

```

349 ■ try {
350 ■   Query<Map<String, dynamic>> q =
351 ■     _db.collection('users').where('isActive', isEqualTo: true);
352 ■   if (scope == 'bus') {
353 ■     if (busId == null) {
354 ■       return const Err<void>(
355 ■         AppFailure('invalid', 'Choose a bus first.'),
356 ■       );
357 ■     }
358 ■     q = q.where('busId', isEqualTo: busId);
359 ■   }
360 ■   final QuerySnapshot<Map<String, dynamic>> users = await q.get();
361 ■
362 ■   // Firestore batches max 500 ops.
363 ■   WriteBatch batch = _db.batch();
364 ■   int ops = 0;
365 ■   for (final QueryDocumentSnapshot<Map<String, dynamic>> u
366 ■     in users.docs) {
367 ■     final DocumentReference<Map<String, dynamic>> ref =
368 ■       u.reference.collection('inbox').doc();
369 ■     batch.set(ref, <String, dynamic>{
370 ■       'type': 'announcement',
371 ■       'title': title.trim(),
372 ■       'body': body.trim(),
373 ■       'read': false,
374 ■       'receivedAt': FieldValue.serverTimestamp(),
375 ■     });
376 ■     ops++;
377 ■     if (ops == 450) {
378 ■       await batch.commit();
379 ■       batch = _db.batch();
380 ■       ops = 0;
381 ■     }
382 ■   }
383 ■   if (ops > 0) await batch.commit();
384 ■
385 ■   await _audit('ANNOUNCEMENT_SENT', 'notification', scope,
386 ■     <String, dynamic>{'title': title, 'busId': busId},);
387 ■   return const Ok<void>(null);
388 ■ } on FirebaseException catch (e) {
389 ■   return Err<void>(e.code == 'permission-denied'
390 ■     ? AppFailure.permissionDenied
391 ■     : AppFailure.unknown,);
392 ■ }
393 ■ }
394 ■
395 ■ Future<void> _writeAccessMirror(
396 ■   String uid, {
397 ■     required String role,
398 ■     String? busId,
399 ■     required bool active,
400 ■   }) =>
401 ■   _rtddb.ref('access/$uid').set(<String, Object?>{
402 ■     'role': role,
403 ■     'busId': busId,
404 ■     'active': active,
405 ■   });
406 ■
407 ■ Future<void> _audit(String action, String targetType, String targetId,
408 ■   Map<String, dynamic> details,) async {
409 ■   try {
410 ■     await _db.collection('auditLogs').add(<String, dynamic>{
411 ■       'actorUid': FirebaseAuth.instance.currentUser?.uid ?? '',
412 ■       'actorRole': 'admin',
413 ■       'action': action,
414 ■       'target': <String, String>{'type': targetType, 'id': targetId},
415 ■       'details': details,
416 ■       'at': FieldValue.serverTimestamp(),
417 ■     });
418 ■   } catch (_) {
419 ■     // Audit failures must never block the operation itself.
420 ■   }
421 ■ }
422 ■ }

```

```

1 ■ import 'package:firebase_database/firebase_database.dart';
2 ■ import '../models/live_location.dart';
3 ■
4 ■ /// Reads/writes the single live-location node per bus in Realtime Database.
5 ■ ///
6 ■ /// RTDB security rules guarantee:
7 ■ /// - students can subscribe ONLY to `liveLocations/{their claim busId}`
8 ■ /// - drivers can write ONLY to their own bus node
9 ■ /// so no client can browse other buses regardless of what this code does.
10 ■ class LiveLocationService {
11 ■   LiveLocationService(this._rtdb);
12 ■
13 ■   final FirebaseDatabase _rtdb;
14 ■
15 ■   DatabaseReference _busRef(String busId) =>
16 ■     _rtdb.ref('liveLocations/$busId');
17 ■
18 ■   /// Live stream of the bus location; emits null when the node is absent
19 ■   /// (no active trip) – an explicit, honest "not tracking" state.
20 ■   Stream<LiveLocation?> watchBusLocation(String busId) =>
21 ■     _busRef(busId).onValue.map((DatabaseEvent event) {
22 ■       final Object? value = event.snapshot.value;
23 ■       if (value is! Map) return null;
24 ■       return LiveLocation.fromMap(Map<Object?, Object?>.from(value));
25 ■     });
26 ■
27 ■   /// Driver-side write of one GPS fix. `updatedAt` uses the SERVER timestamp
28 ■   /// so freshness checks can't be spoofed by a wrong device clock.
29 ■   Future<void> publishFix({
30 ■     required String busId,
31 ■     required String tripId,
32 ■     required double lat,
33 ■     required double lng,
34 ■     required double heading,
35 ■     required double speedKmh,
36 ■     required double accuracy,
37 ■   }) =>
38 ■     _busRef(busId).set(<String, Object?>{
39 ■       'lat': lat,
40 ■       'lng': lng,
41 ■       'heading': heading,
42 ■       'speedKmh': speedKmh,
43 ■       'accuracy': accuracy,
44 ■       'tripId': tripId,
45 ■       'updatedAt': ServerValue.timestamp,
46 ■     });
47 ■
48 ■   /// Removes the node when the trip ends so students see "not tracking"
49 ■   /// instead of a frozen old position.
50 ■   Future<void> clear(String busId) => _busRef(busId).remove();
51 ■ }

```

```

1 ■ import 'dart:async';
2 ■
3 ■ import 'package:geolocator/geolocator.dart';
4 ■
5 ■ import '../core/constants/nmb_constants.dart';
6 ■ import '../core/utils/result.dart';
7 ■ import 'firestore_service.dart';
8 ■ import 'live_location_service.dart';
9 ■
10 ■ /// Orchestrates a driver trip:
11 ■ ///   Start Trip → create trips doc → stream GPS → publish to RTDB
12 ■ ///   End Trip   → stop stream → complete trips doc → clear RTDB node
13 ■ ///
14 ■ /// Uses a foreground-service-backed stream on Android (see
15 ■ /// AndroidSettings.foregroundNotificationConfig) so tracking survives the
16 ■ /// screen turning off – the driver should not need to touch the phone while
17 ■ /// driving.
18 ■ class DriverTripService {
19 ■   DriverTripService(this._firestore, this._liveLocation);
20 ■
21 ■   final FirestoreService _firestore;
22 ■   final LiveLocationService _liveLocation;
23 ■
24 ■   StreamSubscription<Position>? _positionSub;
25 ■   String? _activeTripId;
26 ■   String? _activeBusId;
27 ■
28 ■   bool get hasLocalActiveTrip => _activeTripId != null;
29 ■
30 ■   /// Checks GPS service + permission. Returns a failure describing exactly
31 ■   /// what's wrong so the UI can guide the driver.
32 ■   Future<Result<void>> ensureLocationReady() async {
33 ■     if (!await Geolocator.isLocationServiceEnabled()) {
34 ■       return const Err<void>(
35 ■         AppFailure('gps-off', 'GPS is turned off. Please enable Location.'),
36 ■       );
37 ■     }
38 ■     LocationPermission permission = await Geolocator.checkPermission();
39 ■     if (permission == LocationPermission.denied) {
40 ■       permission = await Geolocator.requestPermission();
41 ■     }
42 ■     if (permission == LocationPermission.denied ||
43 ■         permission == LocationPermission.deniedForever) {
44 ■       return const Err<void>(
45 ■         AppFailure(
46 ■           'location-denied',
47 ■           'Location permission is required to share the bus position. '
48 ■           'Please allow it in Settings.',
49 ■         ),
50 ■       );
51 ■     }
52 ■     return const Ok<void>(null);
53 ■   }
54 ■
55 ■   Future<Result<String>> startTrip({
56 ■     required String driverId,
57 ■     required String busId,
58 ■     required String routeId,

```

```

59 ■     required String direction,
60 ■ } ) async {
61 ■     final Result<void> ready = await ensureLocationReady();
62 ■     if (ready case Err<void>(:final AppFailure failure)) {
63 ■         return Err<String>(failure);
64 ■     }
65 ■
66 ■     try {
67 ■         final String tripId = await _firestore.createTrip(
68 ■             busId: busId,
69 ■             driverId: driverId,
70 ■             routeId: routeId,
71 ■             direction: direction,
72 ■         );
73 ■         _activeTripId = tripId;
74 ■         _activeBusId = busId;
75 ■         await _startPublishing(busId: busId, tripId: tripId);
76 ■         return Ok<String>(tripId);
77 ■     } catch (_) {
78 ■         return const Err<String>(
79 ■             AppFailure('trip-start', 'Could not start the trip. Please retry.'),
80 ■         );
81 ■     }
82 ■ }
83 ■
84 ■ Future<void> _startPublishing({
85 ■     required String busId,
86 ■     required String tripId,
87 ■ }) async {
88 ■     await _positionSub?.cancel();
89 ■
90 ■     final LocationSettings settings = AndroidSettings(
91 ■         accuracy: LocationAccuracy.high,
92 ■         distanceFilter: NmbConstants.locationDistanceFilterMeters,
93 ■         intervalDuration: NmbConstants.locationMinInterval,
94 ■         foregroundNotificationConfig: const ForegroundNotificationConfig(
95 ■             notificationTitle: 'Never Miss Bus – trip active',
96 ■             notificationText: 'Sharing bus location with assigned students.',
97 ■             enableWakeLock: true,
98 ■         ),
99 ■     );
100 ■
101 ■     _positionSub =
102 ■         Geolocator.getPositionStream(locationSettings: settings).listen(
103 ■             (Position pos) {
104 ■                 // Fire-and-forget: a missed packet is fine, freshness logic on the
105 ■                 // student side handles gaps honestly.
106 ■                 _liveLocation.publishFix(
107 ■                     busId: busId,
108 ■                     tripId: tripId,
109 ■                     lat: pos.latitude,
110 ■                     lng: pos.longitude,
111 ■                     heading: pos.heading,
112 ■                     speedKmh: pos.speed * 3.6,
113 ■                     accuracy: pos.accuracy,
114 ■                 );
115 ■             },
116 ■             onError: (_) {

```

```

117 ■      // GPS dropped – keep the subscription; geolocator resumes when the
118 ■      // signal returns. Students see "last updated X ago" meanwhile.
119 ■    },
120 ■  );
121 ■ }
122 ■
123 ■ Future<Result<void>> endTrip() async {
124 ■   final String? tripId = _activeTripId;
125 ■   final String? busId = _activeBusId;
126 ■   await _positionSub?.cancel();
127 ■   _positionSub = null;
128 ■   _activeTripId = null;
129 ■   _activeBusId = null;
130 ■
131 ■   if (tripId == null || busId == null) return const Ok<void>(null);
132 ■   try {
133 ■     await _firestore.endTrip(tripId);
134 ■     await _liveLocation.clear(busId);
135 ■     return const Ok<void>(null);
136 ■   } catch (_) {
137 ■     return const Err<void>(
138 ■       AppFailure(
139 ■         'trip-end',
140 ■         'Trip end could not be confirmed with the server. '
141 ■         'It will be closed automatically if you stay offline.',
142 ■       ),
143 ■     );
144 ■   }
145 ■ }
146 ■
147 ■ /// Re-attach publishing after app restart while a trip is still active
148 ■ /// (e.g. driver phone rebooted mid-route).
149 ■ Future<void> resumePublishing({
150 ■   required String busId,
151 ■   required String tripId,
152 ■ }) async {
153 ■   _activeTripId = tripId;
154 ■   _activeBusId = busId;
155 ■   await _startPublishing(busId: busId, tripId: tripId);
156 ■ }
157 ■
158 ■ Future<void> dispose() async => _positionSub?.cancel();
159 ■ }

```

```

1 ■ import '../core/constants/nmb_constants.dart';
2 ■ import '../models/bus_stop.dart';
3 ■ import '../models/geo_point_data.dart';
4 ■ import '../models/live_location.dart';
5 ■
6 ■ /// Result of an ETA computation. Either a usable approximation, or an
7 ■ explicit "unavailable" – the UI never invents a number.
8 ■ sealed class EtaResult {
9 ■   const EtaResult();
10 ■ }
11 ■
12 ■ class EtaAvailable extends EtaResult {
13 ■   const EtaAvailable(this.duration, this.distanceMeters);
14 ■   final Duration duration;
15 ■   final double distanceMeters;
16 ■ }
17 ■
18 ■ class EtaUnavailable extends EtaResult {
19 ■   const EtaUnavailable(this.reason);
20 ■   final String reason;
21 ■ }
22 ■
23 ■ /// Conservative ETA: distance along the remaining stop sequence divided by a
24 ■ rolling average speed with a sane floor. Deliberately simple and honest –
25 ■ no traffic model, so wording elsewhere is always "approximately".
26 ■ class EtaService {
27 ■   final List<double> _recentSpeeds = <double>[];
28 ■
29 ■   void recordSpeed(double speedKmh) {
30 ■     if (speedKmh.isNaN || speedKmh < 0 || speedKmh > 120) return;
31 ■     _recentSpeeds.add(speedKmh);
32 ■     if (_recentSpeeds.length > 12) _recentSpeeds.removeAt(0);
33 ■   }
34 ■
35 ■   EtaResult etaToStop({
36 ■     required LiveLocation? live,
37 ■     required BusStop targetStop,
38 ■     required List<BusStop> orderedStops,
39 ■   }) {
40 ■     if (live == null) {
41 ■       return const EtaUnavailable('Bus location not available');
42 ■     }
43 ■
44 ■     // Never estimate from stale data.
45 ■     final Duration age = DateTime.now().difference(live.updatedAt);
46 ■     if (age > NmbConstants.staleThreshold) {
47 ■       return const EtaUnavailable('Location data too old for an estimate');
48 ■     }
49 ■
50 ■     final double distance = _remainingDistanceMeters(
51 ■       from: live.position,
52 ■       target: targetStop,
53 ■       orderedStops: orderedStops,
54 ■     );
55 ■
56 ■     if (distance > NmbConstants.etaMaxReliableDistanceKm * 1000) {
57 ■       return const EtaUnavailable('Bus too far for a reliable estimate');
58 ■     }

```

```

59 ■ // Bus already at/past the stop.
60 ■ if (distance < 60) {
61 ■   return EtaAvailable(const Duration(minutes: 1), distance);
62 ■ }
63 ■
64 ■ final double avgSpeed = _recentSpeeds.isEmpty
65 ■   ? NmbConstants.etaMinSpeedKmh
66 ■   : _recentSpeeds.reduce((double a, double b) => a + b) /
67 ■     _recentSpeeds.length;
68 ■ final double effectiveKmh =
69 ■   avgSpeed < NmbConstants.etaMinSpeedKmh
70 ■     ? NmbConstants.etaMinSpeedKmh
71 ■     : avgSpeed;
72 ■
73 ■ final double hours = (distance / 1000) / effectiveKmh;
74 ■ final Duration eta = Duration(seconds: (hours * 3600).round());
75 ■
76 ■ if (eta > const Duration(hours: 2)) {
77 ■   return const EtaUnavailable('Estimate not reliable right now');
78 ■ }
79 ■ return EtaAvailable(eta, distance);
80 ■ }
81 ■
82 ■ /// Distance: bus → next stop → ... → target stop (follows the route order,
83 ■ /// not a straight line to the target).
84 ■ double _remainingDistanceMeters({
85 ■   required GeoPointData from,
86 ■   required BusStop target,
87 ■   required List<BusStop> orderedStops,
88 ■ }) {
89 ■   final int targetIndex =
90 ■     orderedStops.indexWhere((BusStop s) => s.id == target.id);
91 ■   if (targetIndex < 0) return from.distanceTo(target.location);
92 ■
93 ■   // Find the upcoming stop = nearest stop at/before the target.
94 ■   int nearestIndex = 0;
95 ■   double nearest = double.infinity;
96 ■   for (int i = 0; i <= targetIndex; i++) {
97 ■     final double d = from.distanceTo(orderedStops[i].location);
98 ■     if (d < nearest) {
99 ■       nearest = d;
100 ■       nearestIndex = i;
101 ■     }
102 ■   }
103 ■
104 ■   double total = from.distanceTo(orderedStops[nearestIndex].location);
105 ■   for (int i = nearestIndex; i < targetIndex; i++) {
106 ■     total += orderedStops[i]
107 ■       .location
108 ■       .distanceTo(orderedStops[i + 1].location);
109 ■   }
110 ■   return total;
111 ■ }
112 ■ }

```

```

1 ■ import 'package:firebase_messaging/firebase_messaging.dart';
2 ■ import 'package:flutter_local_notifications/flutter_local_notifications.dart';
3 ■
4 ■ import 'firestore_service.dart';
5 ■
6 ■ /// FCM wiring:
7 ■ /// - requests notification permission (Android 13+ / iOS),
8 ■ /// - registers the device token under users/{uid}.fcmTokens (rules allow
9 ■ ///   only the owner to write this field),
10 ■ /// - shows foreground pushes via a local notification channel.
11 ■ ///
12 ■ /// Targeting is done entirely by Cloud Functions against explicit tokens –
13 ■ /// no FCM topics, because any client can subscribe to a topic and that would
14 ■ /// leak bus-scoped notifications.
15 ■ class NotificationService {
16 ■   NotificationService(this._messaging, this._firestore);
17 ■
18 ■   final FirebaseMessaging _messaging;
19 ■   final FirestoreService _firestore;
20 ■   final FlutterLocalNotificationsPlugin _local =
21 ■     FlutterLocalNotificationsPlugin();
22 ■
23 ■   static const AndroidNotificationChannel _channel =
24 ■     AndroidNotificationChannel(
25 ■       'nmb_alerts',
26 ■       'Bus Alerts',
27 ■       description: 'Trip updates, arrival alerts and school announcements.',
28 ■       importance: Importance.high,
29 ■     );
30 ■
31 ■   bool _permissionGranted = false;
32 ■   bool get permissionGranted => _permissionGranted;
33 ■
34 ■   Future<void> initialize() async {
35 ■     const InitializationSettings initSettings = InitializationSettings(
36 ■       android: AndroidInitializationSettings('@mipmap/ic_launcher'),
37 ■       iOS: DarwinInitializationSettings(),
38 ■     );
39 ■     await _local.initialize(initSettings);
40 ■     await _local
41 ■       .resolvePlatformSpecificImplementation<
42 ■         AndroidFlutterLocalNotificationsPlugin>()
43 ■       ?.createNotificationChannel(_channel);
44 ■
45 ■     final NotificationSettings settings = await _messaging.requestPermission(
46 ■       alert: true,
47 ■       badge: true,
48 ■       sound: true,
49 ■     );
50 ■     _permissionGranted =
51 ■       settings.authorizationStatus == AuthorizationStatus.authorized ||
52 ■       settings.authorizationStatus == AuthorizationStatus.provisional;
53 ■
54 ■     // Foreground messages → local notification (FCM shows nothing in
55 ■     // foreground by default).
56 ■     FirebaseMessaging.onMessage.listen((RemoteMessage message) {
57 ■       final RemoteNotification? n = message.notification;
58 ■       if (n == null) return;

```

```

59 ■      _local.show(
60 ■          n.hashCode,
61 ■          n.title,
62 ■          n.body,
63 ■          const NotificationDetails(
64 ■              android: AndroidNotificationDetails(
65 ■                  'nmb_alerts',
66 ■                  'Bus Alerts',
67 ■                  importance: Importance.high,
68 ■                  priority: Priority.high,
69 ■              ),
70 ■              iOS: DarwinNotificationDetails(),
71 ■          ),
72 ■      );
73 ■  });
74 ■ }
75 ■
76 ■  /// Call after login; keeps the token registered for this user.
77 ■  Future<void> registerDevice(String uid) async {
78 ■      try {
79 ■          final String? token = await _messaging.getToken();
80 ■          if (token != null) await _firestore.saveFcmToken(uid, token);
81 ■          _messaging.onTokenRefresh
82 ■              .listen((String t) => _firestore.saveFcmToken(uid, t));
83 ■      } catch (_) {
84 ■          // Non-fatal: push simply won't arrive on this device.
85 ■      }
86 ■  }
87 ■
88 ■  /// Call on logout so the signed-out device stops receiving user pushes.
89 ■  Future<void> unregisterDevice(String uid) async {
90 ■      try {
91 ■          final String? token = await _messaging.getToken();
92 ■          if (token != null) await _firestore.removeFcmToken(uid, token);
93 ■          await _messaging.deleteToken();
94 ■      } catch (_) {/* best effort */}
95 ■  }
96 ■ }

```

lib/services/connectivity_service.dart

20 lines

```

1 ■ import 'package:connectivity_plus/connectivity_plus.dart';
2 ■
3 ■ /// Simple online/offline stream used for offline banners and driver
4 ■ /// connection status.
5 ■ class ConnectivityService {
6 ■     ConnectivityService(this._connectivity);
7 ■
8 ■     final Connectivity _connectivity;
9 ■
10 ■     Stream<bool> get isOnline =>
11 ■         _connectivity.onConnectivityChanged.map(_hasConnection);
12 ■
13 ■     Future<bool> checkOnline() async =>
14 ■         _hasConnection(await _connectivity.checkConnectivity());
15 ■
16 ■     bool _hasConnection(List<ConnectivityResult> results) => results.any(
17 ■         (ConnectivityResult r) =>
18 ■             r != ConnectivityResult.none && r != ConnectivityResult.bluetooth,
19 ■     );
20 ■ }

```

lib/services/device_session_service.dart

58 lines

```

1 ■ import 'dart:math';
2 ■
3 ■ import 'package:device_info_plus/device_info_plus.dart';
4 ■ import 'package:flutter/foundation.dart'
5 ■     show defaultTargetPlatform, TargetPlatform;
6 ■ import 'package:shared_preferences/shared_preferences.dart';
7 ■
8 ■ /// Single-device login support.
9 ■ ///
10 ■ /// Each install gets a persistent random device ID. On login the ID is
11 ■ /// written to the user's profile (settings.activeDevice). If the profile
12 ■ /// later shows a DIFFERENT device ID, this device was replaced by a newer
13 ■ /// login and must sign out ("last login wins" – WhatsApp style).
14 ■ class DeviceSessionService {
15 ■   static const String _prefsKey = 'nmb_device_id';
16 ■
17 ■   /// Special marker used by admin "Force logout".
18 ■   static const String revokedId = 'REVOKED';
19 ■
20 ■   String? _cachedId;
21 ■
22 ■   /// True while a fresh login is still writing its device registration –
23 ■   /// the kick-listener must not act on stale data during this window.
24 ■   bool loginWriteInProgress = false;
25 ■
26 ■   Future<String> getDeviceId() async {
27 ■     if (_cachedId != null) return _cachedId!;
28 ■     final SharedPreferences prefs = await SharedPreferences.getInstance();
29 ■     String? id = prefs.getString(_prefsKey);
30 ■     if (id == null || id.isEmpty) {
31 ■       final Random rng = Random.secure();
32 ■       const String chars =
33 ■         'ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789';
34 ■       id = List<String>.generate(
35 ■         24,
36 ■         (_) => chars[rng.nextInt(chars.length)],
37 ■       ).join();
38 ■       await prefs.setString(_prefsKey, id);
39 ■     }
40 ■     _cachedId = id;
41 ■     return id;
42 ■   }
43 ■
44 ■   Future<String> getDeviceName() async {
45 ■     try {
46 ■       final DeviceInfoPlugin plugin = DeviceInfoPlugin();
47 ■       if (defaultTargetPlatform == TargetPlatform.android) {
48 ■         final AndroidDeviceInfo a = await plugin.androidInfo;
49 ■         return '${a.brand} ${a.model}'.trim();
50 ■       }
51 ■       if (defaultTargetPlatform == TargetPlatform.iOS) {
52 ■         final IosDeviceInfo i = await plugin.iosInfo;
53 ■         return i.utsname.machine;
54 ■       }
55 ■     } catch (_) {/* fall through */}
56 ■     return 'Device';
57 ■   }
58 ■ }

```

6. State & Navigation

Riverpod providers (error-quiet streams) + role-fenced go_router.

lib/providers/app_providers.dart 93 lines

lib/providers/app_providers.dart 93 lines

[illegible]

188 lines

```

1 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
2 ■
3 ■ import '../core/constants/enums.dart';
4 ■ import '../models/app_notification.dart';
5 ■ import '../models/app_user.dart';
6 ■ import '../models/audit_log.dart';
7 ■ import '../models/bus.dart';
8 ■ import '../models/bus_route.dart';
9 ■ import '../models/bus_stop.dart';
10 ■ import '../models/live_location.dart';
11 ■ import '../models/trip.dart';
12 ■ import '../services/auth_service.dart';
13 ■ import 'app_providers.dart';
14 ■
15 ■
16 ■ /// During logout, active Firestore streams briefly lose permission and
17 ■ /// emit permission-denied. That is EXPECTED – swallow stream errors and
18 ■ /// end the stream quietly instead of crashing/error-screening the app.
19 ■ Stream<T> _quiet<T>(Stream<T> source, T fallback) async* {
20 ■   try {
21 ■     await for (final T value in source) {
22 ■       yield value;
23 ■     }
24 ■   } catch (_) {
25 ■     yield fallback;
26 ■   }
27 ■ }
28 ■
29 ■ /// Profile of the signed-in user.
30 ■ final StreamProvider<AppUser?> myProfileProvider =
31 ■   StreamProvider<AppUser?>((Ref ref) {
32 ■     final AuthSession? session = ref.watch(currentSessionProvider);
33 ■     if (session == null) return Stream<AppUser?>.value(null);
34 ■     return _quiet(ref.watch(firestoreServiceProvider).watchUser(session.uid), null);
35 ■   });
36 ■
37 ■ /// The signed-in user's assigned bus (students & drivers).
38 ■ /// Access is enforced by rules: the query only succeeds for the claim busId.
39 ■ final StreamProvider<Bus?> myBusProvider = StreamProvider<Bus?>((Ref ref) {
40 ■   final AuthSession? session = ref.watch(currentSessionProvider);
41 ■   final String? busId = session?.busId;
42 ■   if (busId == null || busId.isEmpty) return Stream<Bus?>.value(null);
43 ■   return _quiet(ref.watch(firestoreServiceProvider).watchBus(busId), null);
44 ■ });
45 ■
46 ■ final StreamProvider<BusRoute?> myRouteProvider =
47 ■   StreamProvider<BusRoute?>((Ref ref) {
48 ■     final String? busId = ref.watch(currentSessionProvider)?.busId;
49 ■     if (busId == null || busId.isEmpty) return Stream<BusRoute?>.value(null);
50 ■     return _quiet(ref.watch(firestoreServiceProvider).watchRouteOfBus(busId), null);
51 ■   });
52 ■
53 ■ final StreamProvider<List<BusStop>> myStopsProvider =
54 ■   StreamProvider<List<BusStop>>((Ref ref) {
55 ■     final String? busId = ref.watch(currentSessionProvider)?.busId;
56 ■     if (busId == null || busId.isEmpty) {
57 ■       return Stream<List<BusStop>>.value(const <BusStop>[]);
58 ■     }

```

```

59 ■ return _quiet(ref.watch(firestoreServiceProvider).watchStopsOfBus(busId), const <BusStop>[]);
60 ■ });
61 ■
62 ■ /// The student's own assigned stop.
63 ■ final Provider<BusStop?> myAssignedStopProvider = Provider<BusStop?>((Ref ref) {
64 ■   final AppUser? me = ref.watch(myProfileProvider).valueOrNull;
65 ■   final List<BusStop> stops =
66 ■     ref.watch(myStopsProvider).valueOrNull ?? const <BusStop>[];
67 ■   if (me?.stopId == null) return null;
68 ■   for (final BusStop s in stops) {
69 ■     if (s.id == me!.stopId) return s;
70 ■   }
71 ■   return null;
72 ■ });
73 ■
74 ■ /// Active trip of my bus (null → "Bus has no active trip").
75 ■ final StreamProvider<Trip?> myBusActiveTripProvider =
76 ■   StreamProvider<Trip?>((Ref ref) {
77 ■     final String? busId = ref.watch(currentSessionProvider)?.busId;
78 ■     if (busId == null || busId.isEmpty) return Stream<Trip?>.value(null);
79 ■     return _quiet(ref.watch(firestoreServiceProvider).watchActiveTripOfBus(busId), null);
80 ■ });
81 ■
82 ■ /// Live location of my bus. Rules restrict this to the assigned bus.
83 ■ final StreamProvider<LiveLocation?> myBusLiveLocationProvider =
84 ■   StreamProvider<LiveLocation?>((Ref ref) {
85 ■     final String? busId = ref.watch(currentSessionProvider)?.busId;
86 ■     if (busId == null || busId.isEmpty) {
87 ■       return Stream<LiveLocation?>.value(null);
88 ■     }
89 ■     return _quiet(ref.watch(liveLocationServiceProvider).watchBusLocation(busId), null);
90 ■ });
91 ■
92 ■ /// Freshness ticker: re-evaluates freshness every 10s even without new data,
93 ■ /// so a silent GPS flips LIVE → stale → unavailable honestly.
94 ■ final StreamProvider<DateTime> freshnessTickProvider =
95 ■   StreamProvider<DateTime>((Ref ref) => Stream<DateTime>.periodic(
96 ■     const Duration(seconds: 10),
97 ■     (_) => DateTime.now(),
98 ■   ),
99 ■ );
100 ■
101 ■
102 ■ final Provider<LocationFreshness> myBusFreshnessProvider =
103 ■   Provider<LocationFreshness>((Ref ref) {
104 ■     ref.watch(freshnessTickProvider); // tick
105 ■     final LiveLocation? live = ref.watch(myBusLiveLocationProvider).valueOrNull;
106 ■     final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
107 ■     if (live == null || trip == null) return LocationFreshness.unavailable;
108 ■     return live.freshness();
109 ■ });
110 ■
111 ■ /// Notification inbox.
112 ■ final StreamProvider<List<AppNotification>> inboxProvider =
113 ■   StreamProvider<List<AppNotification>>((Ref ref) {
114 ■     final AuthSession? session = ref.watch(currentSessionProvider);
115 ■     if (session == null) {
116 ■       return Stream<List<AppNotification>>.value(const <AppNotification>[]);

```

```

174 ■
175 ■ /// Live location of an arbitrary bus (admin monitoring only; rules permit
176 ■ /// admin reads on all buses).
177 ■ final StreamProviderFamily<LiveLocation?, String> busLiveLocationProvider =
178 ■     StreamProvider.family<LiveLocation?, String>(
179 ■         (Ref ref, String busId) => _quiet(
180 ■             ref.watch(liveLocationServiceProvider).watchBusLocation(busId), null,),
181 ■     );
182 ■
183 ■ /// Admin: recent trip history (rules: admin-only read of all trips).
184 ■ final StreamProvider<List<Trip>> recentTripsProvider =
185 ■     StreamProvider<List<Trip>>((
186 ■         (Ref ref) => _quiet(
187 ■             ref.watch(firestoreServiceProvider).watchRecentTrips(), const <Trip>[],),
188 ■     );

```

```

1 ■
2 ■ import 'package:flutter/material.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■ import 'package:go_router/go_router.dart';
5 ■
6 ■ import '../core/constants/enums.dart';
7 ■ import '../features/admin/admin_shell.dart';
8 ■ import '../features/admin/screens/admin_audit_logs_screen.dart';
9 ■ import '../features/admin/screens/admin_buses_screen.dart';
10 ■ import '../features/admin/screens/admin_dashboard_screen.dart';
11 ■ import '../features/admin/screens/admin_drivers_screen.dart';
12 ■ import '../features/admin/screens/admin_monitoring_screen.dart';
13 ■ import '../features/admin/screens/admin_notifications_screen.dart';
14 ■ import '../features/admin/screens/admin_routes_stops_screen.dart';
15 ■ import '../features/admin/screens/admin_settings_screen.dart';
16 ■ import '../features/admin/screens/admin_students_screen.dart';
17 ■ import '../features/admin/screens/admin_trip_history_screen.dart';
18 ■ import '../features/auth/login_screen.dart';
19 ■ import '../features/auth/splash_screen.dart';
20 ■ import '../features/driver/driver_shell.dart';
21 ■ import '../features/driver/screens/driver_home_screen.dart';
22 ■ import '../features/driver/screens/driver_profile_screen.dart';
23 ■ import '../features/driver/screens/driver_route_screen.dart';
24 ■ import '../features/student/student_shell.dart';
25 ■ import '../features/student/screens/student_alerts_screen.dart';
26 ■ import '../features/student/screens/student_bus_details_screen.dart';
27 ■ import '../features/student/screens/student_home_screen.dart';
28 ■ import '../features/student/screens/student_map_screen.dart';
29 ■ import '../features/student/screens/student_profile_screen.dart';
30 ■ import '../features/student/screens/student_settings_screen.dart';
31 ■ import '../features/student/screens/student_stops_screen.dart';
32 ■ import '../providers/app_providers.dart';
33 ■ import '../services/auth_service.dart';
34 ■
35 ■ /// Role-aware router. The redirect runs on EVERY navigation and every auth
36 ■ /// change, so:
37 ■ /// - signed-out users can only reach /login,
38 ■ /// - each role is fenced into its own subtree (a student typing an /admin
39 ■ ///   URL is bounced home),
40 ■ /// - expired/revoked sessions bounce to /login automatically.
41 ■ /// This is UX-level fencing only – real enforcement lives in backend rules.
42 ■ final Provider<GoRouter> appRouterProvider = Provider<GoRouter>((Ref ref) {
43 ■   final ValueNotifier<int> refresh = ValueNotifier<int>(0);
44 ■   ref.listen(sessionProvider, (Object? _, Object? __) => refresh.value++);
45 ■   ref.onDispose(refresh.dispose);
46 ■
47 ■   return GoRouter(
48 ■     initialLocation: '/splash',
49 ■     refreshListenable: refresh,
50 ■     redirect: (BuildContext context, GoRouterState state) {
51 ■       final AsyncValue<AuthSession?> async = ref.read(sessionProvider);
52 ■       if (async.isLoading) {
53 ■         return state.matchedLocation == '/splash' ? null : '/splash';
54 ■       }
55 ■       final AuthSession? session = async.valueOrNull;
56 ■       final String loc = state.matchedLocation;
57 ■       final bool onAuthScreen = loc == '/login' || loc == '/splash';
58 ■

```

[illegible]

```

116 StatefulShellBranch(routes: <RouteBase>[
117     GoRoute(
118         path: '/student/profile',
119         builder: (_, __) => const StudentProfileScreen(),
120         routes: <RouteBase>[
121             GoRoute(
122                 path: 'settings',
123                 builder: (_, __) => const StudentSettingsScreen(),
124             ),
125         ],
126     ),
127 ],),
128 ],
129 ),
130
131 // Driver shell
132 StatefulShellRoute.indexedStack(
133     builder: (_, __, StatefulNavigationShell shell) =>
134         DriverShell(shell: shell),
135     branches: <StatefulShellBranch>[
136         StatefulShellBranch(routes: <RouteBase>[
137             GoRoute(
138                 path: '/driver/home',
139                 builder: (_, __) => const DriverHomeScreen(),
140             ),
141         ],),
142         StatefulShellBranch(routes: <RouteBase>[
143             GoRoute(
144                 path: '/driver/route',
145                 builder: (_, __) => const DriverRouteScreen(),
146             ),
147         ],),
148         StatefulShellBranch(routes: <RouteBase>[
149             GoRoute(
150                 path: '/driver/profile',
151                 builder: (_, __) => const DriverProfileScreen(),
152             ),
153         ],),
154     ],
155 ),
156
157 // Admin shell
158 ShellRoute(
159     builder: (_, __, Widget child) => AdminShell(child: child),
160     routes: <RouteBase>[
161         GoRoute(
162             path: '/admin/dashboard',
163             builder: (_, __) => const AdminDashboardScreen(),
164         ),
165         GoRoute(
166             path: '/admin/students',
167             builder: (_, __) => const AdminStudentsScreen(),
168         ),
169         GoRoute(
170             path: '/admin/drivers',
171             builder: (_, __) => const AdminDriversScreen(),
172         ),
173         GoRoute(

```

```
174 ■      path: '/admin/buses',
175 ■      builder: (_, __) => const AdminBusesScreen(),
176 ■    ),
177 ■    GoRoute(
178 ■      path: '/admin/routes',
179 ■      builder: (_, __) => const AdminRoutesStopsScreen(),
180 ■    ),
181 ■    GoRoute(
182 ■      path: '/admin/monitoring',
183 ■      builder: (_, __) => const AdminMonitoringScreen(),
184 ■    ),
185 ■    GoRoute(
186 ■      path: '/admin/notifications',
187 ■      builder: (_, __) => const AdminNotificationsScreen(),
188 ■    ),
189 ■    GoRoute(
190 ■      path: '/admin/trips',
191 ■      builder: (_, __) => const AdminTripHistoryScreen(),
192 ■    ),
193 ■    GoRoute(
194 ■      path: '/admin/audit',
195 ■      builder: (_, __) => const AdminAuditLogsScreen(),
196 ■    ),
197 ■    GoRoute(
198 ■      path: '/admin/settings',
199 ■      builder: (_, __) => const AdminSettingsScreen(),
200 ■    ),
201 ■  ],
202 ■ ),
203 ■ ],
204 ■ );
205 ■ };
```

7. Auth Screens

Animated splash (bus drive-in) + premium login (curved header, floating card, animated toggle).

lib/features/auth/splash_screen.dart

245 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■
3 ■ import '../core/constants/nmb_constants.dart';
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■
7 ■ /// Animated splash: the bus drives in from the left along a road, the
8 ■ /// app name fades up, loading dots pulse. Router redirects away as soon
9 ■ /// as the session resolves – so this stays smooth and short.
10 ■ class SplashScreen extends StatefulWidget {
11 ■   const SplashScreen({super.key});
12 ■
13 ■   @override
14 ■   State<SplashScreen> createState() => _SplashScreenState();
15 ■ }
16 ■
17 ■ class _SplashScreenState extends State<SplashScreen>
18 ■   with TickerProviderStateMixin {
19 ■   late final AnimationController _drive = AnimationController(
20 ■     vsync: this,
21 ■     duration: const Duration(milliseconds: 1100),
22 ■   )..forward();
23 ■
24 ■   late final AnimationController _pulse = AnimationController(
25 ■     vsync: this,
26 ■     duration: const Duration(milliseconds: 900),
27 ■   )..repeat(reverse: true);
28 ■
29 ■   late final Animation<double> _busX = CurvedAnimation(
30 ■     parent: _drive,
31 ■     curve: Curves.easeOutCubic,
32 ■   );
33 ■
34 ■   late final Animation<double> _fade = CurvedAnimation(
35 ■     parent: _drive,
36 ■     curve: const Interval(0.35, 1, curve: Curves.easeOut),
37 ■   );
38 ■
39 ■   @override
40 ■   void dispose() {
41 ■     _drive.dispose();
42 ■     _pulse.dispose();
43 ■     super.dispose();
44 ■   }
45 ■
46 ■   @override
47 ■   Widget build(BuildContext context) {
48 ■     final double w = MediaQuery.of(context).size.width;
49 ■
50 ■     return Scaffold(
51 ■       body: Container(
52 ■         decoration: const BoxDecoration(
53 ■           gradient: LinearGradient(
54 ■             begin: Alignment.topCenter,
55 ■             end: Alignment.bottomCenter,
56 ■             colors: <Color>[NmbColors.primaryDark, NmbColors.primary],
57 ■           ),
58 ■       ),

```

```

59 ■      child: SafeArea(
60 ■        child: Column(
61 ■          children: <Widget>[
62 ■            const Spacer(flex: 3),
63 ■
64 ■            // ■■ Bus drives in on a road ■■
65 ■            SizedBox(
66 ■              height: 130,
67 ■              child: AnimatedBuilder(
68 ■                animation: _busX,
69 ■                builder: (BuildContext context, Widget? child) {
70 ■                  return Stack(
71 ■                    children: <Widget>[
72 ■                      // Road
73 ■                      Positioned(
74 ■                        left: 0,
75 ■                        right: 0,
76 ■                        bottom: 18,
77 ■                        child: Container(
78 ■                          height: 6,
79 ■                          color: Colors.white24,
80 ■                        ),
81 ■                      ),
82 ■                      // Dashed centre line
83 ■                      Positioned(
84 ■                        left: 0,
85 ■                        right: 0,
86 ■                        bottom: 20,
87 ■                        child: Row(
88 ■                          mainAxisAlignment:
89 ■                            MainAxisAlignment.spaceEvenly,
90 ■                          children: List<Widget>.generate(
91 ■                            8,
92 ■                            (_) => Container(
93 ■                              width: 22,
94 ■                              height: 2.5,
95 ■                              color: NmbColors.accent,
96 ■                            ),
97 ■                          ),
98 ■                        ),
99 ■                      ),
100 ■                    // Bus
101 ■                    Positioned(
102 ■                      left: -120 + (_busX.value * (w / 2 - 40 + 120)),
103 ■                      bottom: 24,
104 ■                      child: const _MiniBus(size: 96),
105 ■                    ),
106 ■                  ],
107 ■                );
108 ■              },
109 ■            ),
110 ■          ),
111 ■        const SizedBox(height: 34),
112 ■
113 ■        // ■■ Name + tagline fade in ■■
114 ■        FadeTransition(
115 ■          opacity: _fade,
116 ■          child: Column(

```

```

117 ■      children: <Widget>[
118 ■        Text(
119 ■          'NEVER MISS BUS',
120 ■          style: NmbTypography.displayTitle.copyWith(
121 ■            color: Colors.white,
122 ■            letterSpacing: 2,
123 ■          ),
124 ■        ),
125 ■        const SizedBox(height: 6),
126 ■        Text(
127 ■          'Ab bus miss nahi hogi!',
128 ■          style: NmbTypography.body
129 ■            .copyWith(color: NmbColors.accent),
130 ■        ),
131 ■        const SizedBox(height: 4),
132 ■        Text(
133 ■          NmbConstants.schoolName,
134 ■          style: NmbTypography.bodySecondary
135 ■            .copyWith(color: Colors.white70),
136 ■        ),
137 ■      ],
138 ■    ),
139 ■  ),
140 ■  const Spacer(flex: 2),
141 ■
142 ■  // ■■ Pulsing loading dots ■■
143 ■  AnimatedBuilder(
144 ■    animation: _pulse,
145 ■    builder: (BuildContext context, Widget? child) {
146 ■      return Row(
147 ■        mainAxisAlignment: MainAxisAlignment.center,
148 ■        children: List<Widget>.generate(3, (int i) {
149 ■          final double t =
150 ■            (_pulse.value + i * 0.33) % 1.0;
151 ■          return Container(
152 ■            margin:
153 ■              const EdgeInsets.symmetric(horizontal: 5),
154 ■            width: 9 + 4 * (1 - (t - 0.5).abs() * 2),
155 ■            height: 9 + 4 * (1 - (t - 0.5).abs() * 2),
156 ■            decoration: BoxDecoration(
157 ■              color: Colors.white
158 ■                .withOpacity(0.5 + 0.5 * (1 - t)),
159 ■              shape: BoxShape.circle,
160 ■            ),
161 ■          );
162 ■        })),
163 ■      );
164 ■    },
165 ■  ),
166 ■  const SizedBox(height: 14),
167 ■  Text(
168 ■    'Getting things ready...',
169 ■    style: NmbTypography.caption
170 ■      .copyWith(color: Colors.white60),
171 ■  ),
172 ■  const SizedBox(height: 36),
173 ■ ],
174 ■ ),

```

```

175 ■     ),
176 ■     ),
177 ■   );
178 ■ }
179 ■ }
180 ■
181 ■ /// Small painted school bus (side view) used on the splash road.
182 ■ class _MiniBus extends StatelessWidget {
183 ■   const _MiniBus({required this.size});
184 ■
185 ■   final double size;
186 ■
187 ■   @override
188 ■   Widget build(BuildContext context) {
189 ■     return SizedBox(
190 ■       width: size,
191 ■       height: size * 0.55,
192 ■       child: CustomPaint(painter: _MiniBusPainter()),
193 ■     );
194 ■   }
195 ■ }
196 ■
197 ■ class _MiniBusPainter extends CustomPainter {
198 ■   @override
199 ■   void paint(Canvas canvas, Size s) {
200 ■     final double w = s.width;
201 ■     final double h = s.height;
202 ■
203 ■     // Body
204 ■     final RRect body = RRect.fromRectAndRadius(
205 ■       Rect.fromLTWH(0, h * 0.05, w, h * 0.66),
206 ■       Radius.circular(h * 0.16),
207 ■     );
208 ■     canvas.drawRRect(body, Paint()..color = NmbColors.accent);
209 ■
210 ■     // Windows
211 ■     final Paint win = Paint()..color = const Color(0xFFE3F2FD);
212 ■     final double winW = w * 0.16;
213 ■     for (int i = 0; i < 4; i++) {
214 ■       canvas.drawRRect(
215 ■         RRect.fromRectAndRadius(
216 ■           Rect.fromLTWH(
217 ■             w * 0.06 + i * (winW + w * 0.045),
218 ■             h * 0.14,
219 ■             winW,
220 ■             h * 0.26,
221 ■           ),
222 ■           Radius.circular(h * 0.05),
223 ■         ),
224 ■         win,
225 ■       );
226 ■     }
227 ■
228 ■     // Stripe
229 ■     canvas.drawRect(
230 ■       Rect.fromLTWH(0, h * 0.5, w, h * 0.06),
231 ■       Paint()..color = Colors.white,
232 ■     );
233 ■
234 ■     // Wheels
235 ■     final Paint wheel = Paint()..color = const Color(0xFF263238);
236 ■     final Paint hub = Paint()..color = Colors.white;
237 ■     for (final double cx in <double>[w * 0.22, w * 0.78]) {
238 ■       canvas.drawCircle(Offset(cx, h * 0.8), h * 0.17, wheel);
239 ■       canvas.drawCircle(Offset(cx, h * 0.8), h * 0.07, hub);
240 ■     }
241 ■   }
242 ■
243 ■   @override
244 ■   bool shouldRepaint(covariant CustomPainter oldDelegate) => false;
245 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/constants/nmb_constants.dart';
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/utils/result.dart';
8 ■ import '../core/utils/validators.dart';
9 ■ import '../core/widgets/nmb_dialogs.dart';
10 ■ import '../providers/app_providers.dart';
11 ■ import '../services/auth_service.dart';
12 ■ import '../shared/bus_marker_widget.dart';
13 ■
14 ■ /// v1.3 Login – premium look:
15 ■ /// blue curved header with the real bus logo, floating white card with
16 ■ /// Student / Staff toggle, clean fields, loading state on the button.
17 ■ class LoginScreen extends ConsumerStatefulWidget {
18 ■   const LoginScreen({super.key});
19 ■
20 ■   @override
21 ■   ConsumerState<LoginScreen> createState() => _LoginScreenState();
22 ■ }
23 ■
24 ■ class _LoginScreenState extends ConsumerState<LoginScreen> {
25 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
26 ■
27 ■   bool _studentMode = true;
28 ■
29 ■   String? _klass;
30 ■   String? _section;
31 ■   final TextEditingController _roll = TextEditingController();
32 ■   final TextEditingController _emailController = TextEditingController();
33 ■   final TextEditingController _passwordController = TextEditingController();
34 ■   bool _obscure = true;
35 ■   bool _loading = false;
36 ■
37 ■   @override
38 ■   void dispose() {
39 ■     _roll.dispose();
40 ■     _emailController.dispose();
41 ■     _passwordController.dispose();
42 ■     super.dispose();
43 ■   }
44 ■
45 ■   Future<void> _signIn() async {
46 ■     if (!(_formKey.currentState?.validate() ?? false)) return;
47 ■     setState(() => _loading = true);
48 ■     final AuthService authSvc = ref.read(authServiceProvider);
49 ■
50 ■     Result<AuthSession> result;
51 ■     if (_studentMode) {
52 ■       result = const Err<AuthSession>(AppFailure.unknown);
53 ■       for (int gen = 1; gen <= StudentLoginId.maxGenerations; gen++) {
54 ■         result = await authSvc.signIn(
55 ■           email: StudentLoginId.email(
56 ■             klass: _klass ?? '',
57 ■             section: _section ?? '',
58 ■             roll: _roll.text,

```

```

59 ■         gen: gen,
60 ■     ),
61 ■     password: _passwordController.text,
62 ■ );
63 ■     final bool disabled = result is Err<AuthSession> &&
64 ■         (result).failure.code == 'user-disabled';
65 ■     if (!disabled) break;
66 ■ }
67 ■ } else {
68 ■     result = await authSvc.signIn(
69 ■         email: _emailController.text.trim(),
70 ■         password: _passwordController.text,
71 ■     );
72 ■ }
73 ■
74 ■ if (!mounted) return;
75 ■ setState(() => _loading = false);
76 ■ result.when(
77 ■     ok: (_) {},
78 ■     err: (AppFailure f) => showNmbSnack(
79 ■         context,
80 ■         _studentMode &&
81 ■             (f.code == 'invalid-credential' || f.code == 'user-disabled')
82 ■             ? 'Incorrect class, roll number or password. Please try again.'
83 ■             : f.message,
84 ■         isError: true,
85 ■     ),
86 ■ );
87 ■ }
88 ■
89 ■ Future<void> _forgotPassword() async {
90 ■     if (_studentMode) {
91 ■         showNmbSnack(
92 ■             context,
93 ■             'Forgot your password? Please ask your class teacher or the '
94 ■             'school office for a new one.',
95 ■         );
96 ■         return;
97 ■     }
98 ■     final AuthService authSvc = ref.read(authServiceProvider);
99 ■     final String email = _emailController.text.trim();
100 ■     if (Validators.email(email) != null) {
101 ■         showNmbSnack(
102 ■             context,
103 ■             'Enter your email above first, then tap "Forgot password".',
104 ■         );
105 ■         return;
106 ■     }
107 ■     final Result<void> result = await authSvc.sendPasswordReset(email);
108 ■     if (!mounted) return;
109 ■     result.when(
110 ■         ok: (_) => showNmbSnack(
111 ■             context,
112 ■             'Password reset email sent to $email',
113 ■             isSuccess: true,
114 ■         ),
115 ■         err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
116 ■     );

```

```

117 ■ }
118 ■
119 ■ InputDecoration _dec({
120 ■   required String hint,
121 ■   required IconData icon,
122 ■   Widget? suffix,
123 ■ }) =>
124 ■   InputDecoration(
125 ■     hintText: hint,
126 ■     prefixIcon: Icon(icon, size: 21),
127 ■     suffixIcon: suffix,
128 ■     filled: true,
129 ■     fillColor: NmbColors.background,
130 ■     border: OutlineInputBorder(
131 ■       borderRadius: BorderRadius.circular(14),
132 ■       borderSide: BorderSide.none,
133 ■     ),
134 ■     enabledBorder: OutlineInputBorder(
135 ■       borderRadius: BorderRadius.circular(14),
136 ■       borderSide: BorderSide.none,
137 ■     ),
138 ■     focusedBorder: OutlineInputBorder(
139 ■       borderRadius: BorderRadius.circular(14),
140 ■       borderSide: const BorderSide(color: NmbColors.primary, width: 1.6),
141 ■     ),
142 ■   );
143 ■
144 ■ @override
145 ■ Widget build(BuildContext context) {
146 ■   return Scaffold(
147 ■     backgroundColor: NmbColors.background,
148 ■     body: CustomScrollView(
149 ■       physics: const BouncingScrollPhysics(),
150 ■       slivers: <Widget>[
151 ■         SliverToBoxAdapter(
152 ■           child: Stack(
153 ■             clipBehavior: Clip.none,
154 ■             children: <Widget>[
155 ■               // ■■ Curved blue header ■■
156 ■               Container(
157 ■                 height: 300,
158 ■                 width: double.infinity,
159 ■                 decoration: const BoxDecoration(
160 ■                   gradient: LinearGradient(
161 ■                     begin: Alignment.topLeft,
162 ■                     end: Alignment.bottomRight,
163 ■                     colors: <Color>[
164 ■                       NmbColors.primaryDark,
165 ■                       NmbColors.primary,
166 ■                     ],
167 ■                 ),
168 ■                 borderRadius: BorderRadius.only(
169 ■                   bottomLeft: Radius.circular(44),
170 ■                   bottomRight: Radius.circular(44),
171 ■                 ),
172 ■               ),
173 ■               child: SafeArea(
174 ■                 bottom: false,

```

```

175 ■         child: Column(
176 ■           mainAxisAlignment: MainAxisAlignment.center,
177 ■           children: <Widget>[
178 ■             const SizedBox(height: 6),
179 ■             // Real bus logo on white badge
180 ■             const BusMarkerWidget(size: 92),
181 ■             const SizedBox(height: 14),
182 ■             Text(
183 ■               'Welcome Back! ■',
184 ■               style: NmbTypography.displayTitle
185 ■                 .copyWith(color: Colors.white),
186 ■             ),
187 ■             const SizedBox(height: 4),
188 ■             Text(
189 ■               NmbConstants.schoolName,
190 ■               style: NmbTypography.bodySecondary
191 ■                 .copyWith(color: Colors.white70),
192 ■             ),
193 ■             const SizedBox(height: 30),
194 ■           ],
195 ■         ),
196 ■       ),
197 ■     ),
198 ■
199 ■     // ■■ Floating login card ■■
200 ■     Padding(
201 ■       padding: const EdgeInsets.fromLTRB(20, 262, 20, 0),
202 ■       child: Container(
203 ■         padding: const EdgeInsets.all(20),
204 ■         decoration: BoxDecoration(
205 ■           color: Colors.white,
206 ■           borderRadius: BorderRadius.circular(24),
207 ■           boxShadow: const <BoxShadow>[
208 ■             BoxShadow(
209 ■               color: Color(0x1A17233B),
210 ■               blurRadius: 24,
211 ■               offset: Offset(0, 8),
212 ■             ),
213 ■           ],
214 ■         ),
215 ■         child: Form(
216 ■           key: _formKey,
217 ■           child: Column(
218 ■             crossAxisAlignment: CrossAxisAlignment.stretch,
219 ■             children: <Widget>[
220 ■               // ■■ Role toggle ■■
221 ■               Container(
222 ■                 padding: const EdgeInsets.all(4),
223 ■                 decoration: BoxDecoration(
224 ■                   color: NmbColors.background,
225 ■                   borderRadius: BorderRadius.circular(14),
226 ■                 ),
227 ■                 child: Row(
228 ■                   children: <Widget>[
229 ■                     _ToggleTab(
230 ■                       label: 'Student',
231 ■                       icon: Icons.school_rounded,
232 ■                       selected: _studentMode,

```

```

233 ■         onTap: () =>
234 ■             setState(() => _studentMode = true),
235 ■     ),
236 ■     _ToggleTab(
237 ■         label: 'Driver / Admin',
238 ■         icon: Icons.badge_rounded,
239 ■         selected: !_studentMode,
240 ■         onTap: () =>
241 ■             setState(() => _studentMode = false),
242 ■     ),
243 ■ ],
244 ■ ),
245 ■ ),
246 ■ const SizedBox(height: 18),
247 ■
248 ■ // ■■ Fields ■■
249 ■ AnimatedSwitcher(
250 ■     duration: const Duration(milliseconds: 250),
251 ■     child: _studentMode
252 ■         ? Column(
253 ■             key: const ValueKey<String>('student'),
254 ■             children: <Widget>[
255 ■                 Row(
256 ■                     children: <Widget>[
257 ■                         Expanded(
258 ■                             flex: 3,
259 ■                             child: DropdownButtonFormField<
260 ■                                 String>(
261 ■                                     value: _klass,
262 ■                                     decoration: _dec(
263 ■                                         hint: 'Class',
264 ■                                         icon: Icons.class_outlined,
265 ■                                     ),
266 ■                                     items: <DropdownMenuItem<
267 ■                                         String>>[
268 ■                                         for (final String c
269 ■                                             in SchoolClasses
270 ■                                                 .classes)
271 ■                                             DropdownMenuItem<String>(
272 ■                                                 value: c,
273 ■                                                 child: Text('Class $c'),
274 ■                                             ),
275 ■                                         ],
276 ■                                     onChanged: (String? v) =>
277 ■                                         setState(
278 ■                                             () => _klass = v,),
279 ■                                     validator: (String? v) =>
280 ■                                         v == null
281 ■                                             ? 'Select class'
282 ■                                             : null,
283 ■                                     ),
284 ■                                 ],
285 ■                                 const SizedBox(width: 10),
286 ■                                 Expanded(
287 ■                                     flex: 2,
288 ■                                     child: DropdownButtonFormField<
289 ■                                         String>(
290 ■                                             value: _section,

```

```

291 ■         decoration: _dec(
292 ■             hint: 'Sec',
293 ■             icon: Icons
294 ■                 .segment_rounded,
295 ■         ),
296 ■         items: <DropdownMenuItem<
297 ■             String>>[
298 ■             for (final String s
299 ■                 in SchoolClasses
300 ■                     .sections)
301 ■                 DropdownMenuItem<String>(
302 ■                     value: s,
303 ■                     child: Text(s),
304 ■                 ),
305 ■         ],
306 ■         onChanged: (String? v) =>
307 ■             setState(
308 ■                 () => _section = v, ),
309 ■         validator: (String? v) =>
310 ■             v == null
311 ■                 ? 'Select'
312 ■                 : null,
313 ■     ),
314 ■ ),
315 ■ ],
316 ■ ),
317 ■ const SizedBox(height: 12),
318 ■ TextFormField(
319 ■     controller: _roll,
320 ■     keyboardType:
321 ■         TextInputType.number,
322 ■     decoration: _dec(
323 ■         hint: 'Roll number',
324 ■         icon: Icons.tag_rounded,
325 ■     ),
326 ■     validator: (String? v) {
327 ■         final String r =
328 ■             (v ?? '').trim();
329 ■         if (r.isEmpty) {
330 ■             return 'Enter roll number';
331 ■         }
332 ■         if (int.tryParse(r) == null) {
333 ■             return 'Numbers only';
334 ■         }
335 ■         return null;
336 ■     },
337 ■ ),
338 ■ ],
339 ■ )
340 ■ : TextFormField(
341 ■     key: const ValueKey<String>('staff'),
342 ■     controller: _emailController,
343 ■     keyboardType:
344 ■         TextInputType.emailAddress,
345 ■     autofillHints: const <String>[
346 ■         AutofillHints.username,
347 ■     ],
348 ■     decoration: _dec(

```

```

349 ■             hint: 'Email',
350 ■             icon: Icons.mail_outline_rounded,
351 ■           ),
352 ■           validator: Validators.email,
353 ■         ),
354 ■       ),
355 ■       const SizedBox(height: 12),
356 ■       TextFormField(
357 ■         controller: _passwordController,
358 ■         obscureText: _obscure,
359 ■         autofillHints: const <String>[
360 ■           AutofillHints.password,
361 ■         ],
362 ■         onFieldSubmitted: (_) => _signIn(),
363 ■         decoration: _dec(
364 ■           hint: 'Password',
365 ■           icon: Icons.lock_outline_rounded,
366 ■           suffix: IconButton(
367 ■             icon: Icon(
368 ■               _obscure
369 ■                 ? Icons.visibility_outlined
370 ■                 : Icons.visibility_off_outlined,
371 ■             size: 21,
372 ■           ),
373 ■           onPressed: () =>
374 ■             setState(() => _obscure = !_obscure),
375 ■         ),
376 ■       ),
377 ■       validator: Validators.password,
378 ■     ),
379 ■     const SizedBox(height: 4),
380 ■     Align(
381 ■       alignment: Alignment.centerRight,
382 ■       child: TextButton(
383 ■         onPressed:
384 ■           _loading ? null : _forgotPassword,
385 ■         child: const Text('Forgot password?'),
386 ■       ),
387 ■     ),
388 ■     const SizedBox(height: 6),
389 ■
390 ■     // ■■ Sign in button with loading state ■■
391 ■     SizedBox(
392 ■       height: 54,
393 ■       child: FilledButton(
394 ■         onPressed: _loading ? null : _signIn,
395 ■         style: FilledButton.styleFrom(
396 ■           backgroundColor: NmbColors.primary,
397 ■           shape: RoundedRectangleBorder(
398 ■             borderRadius:
399 ■               BorderRadius.circular(15),
400 ■           ),
401 ■         ),
402 ■         child: _loading
403 ■           ? const Row(
404 ■             mainAxisAlignment:
405 ■               MainAxisAlignment.center,
406 ■             children: <Widget>[

```

```

407 ■                                     SizedBox(
408 ■                                     width: 20,
409 ■                                     height: 20,
410 ■                                     child:
411 ■                                         CircularProgressIndicator(
412 ■                                             strokeWidth: 2.4,
413 ■                                             color: Colors.white,
414 ■                                         ),
415 ■                                     ),
416 ■                                     SizedBox(width: 12),
417 ■                                     Text('Signing in...'),
418 ■                                 ],
419 ■                             )
420 ■                         : const Text('Sign In'),
421 ■                     ),
422 ■                 ),
423 ■             ],
424 ■         ),
425 ■     ),
426 ■ ),
427 ■ ),
428 ■ ],
429 ■ ),
430 ■ ),
431 ■
432 ■ // ■■ Help note ■■
433 ■ SliverToBoxAdapter(
434 ■   child: Padding(
435 ■     padding: const EdgeInsets.fromLTRB(20, 16, 20, 30),
436 ■     child: Container(
437 ■       padding: const EdgeInsets.all(14),
438 ■       decoration: BoxDecoration(
439 ■         color: NmbColors.infoSoft,
440 ■         borderRadius: BorderRadius.circular(14),
441 ■       ),
442 ■       child: Row(
443 ■         children: <Widget>[
444 ■           const Icon(Icons.school_outlined,
445 ■             color: NmbColors.info, size: 22, ),
446 ■           const SizedBox(width: 10),
447 ■           Expanded(
448 ■             child: Text(
449 ■               _studentMode
450 ■                 ? 'Your roll number and password are provided '
451 ■                   'by the school. Contact your class teacher '
452 ■                     'for help.'
453 ■                 : 'Accounts are created by the school. If you '
454 ■                   'don\'t have one, contact the transport '
455 ■                     'office.',
456 ■               style: NmbTypography.caption
457 ■                 .copyWith(color: NmbColors.info),
458 ■             ),
459 ■           ),
460 ■         ],
461 ■       ),
462 ■     ),
463 ■   ),
464 ■ ),

```

```

465 ■     ],
466 ■     ),
467 ■   );
468 ■ }
469 ■ }
470 ■
471 ■ class _ToggleTab extends StatelessWidget {
472 ■   const _ToggleTab({
473 ■     required this.label,
474 ■     required this.icon,
475 ■     required this.selected,
476 ■     required this.onTap,
477 ■   });
478 ■
479 ■   final String label;
480 ■   final IconData icon;
481 ■   final bool selected;
482 ■   final VoidCallback onTap;
483 ■
484 ■   @override
485 ■   Widget build(BuildContext context) {
486 ■     return Expanded(
487 ■       child: GestureDetector(
488 ■         onTap: onTap,
489 ■         child: AnimatedContainer(
490 ■           duration: const Duration(milliseconds: 200),
491 ■           padding: const EdgeInsets.symmetric(vertical: 11),
492 ■           decoration: BoxDecoration(
493 ■             color: selected ? NmbColors.primary : Colors.transparent,
494 ■             borderRadius: BorderRadius.circular(11),
495 ■           ),
496 ■           child: Row(
497 ■             mainAxisAlignment: MainAxisAlignment.center,
498 ■             children: <Widget>[
499 ■               Icon(
500 ■                 icon,
501 ■                 size: 18,
502 ■                 color: selected ? Colors.white : NmbColors.textTertiary,
503 ■               ),
504 ■               const SizedBox(width: 7),
505 ■               Text(
506 ■                 label,
507 ■                 style: NmbTypography.caption.copyWith(
508 ■                   fontWeight: FontWeight.w700,
509 ■                   color:
510 ■                     selected ? Colors.white : NmbColors.textSecondary,
511 ■                 ),
512 ■             ),
513 ■           ],
514 ■         ),
515 ■       ),
516 ■     );
517 ■   };
518 ■ }
519 ■ }

```

8. Student App

5-tab shell, rich home (blue header, quick actions, route progress), drawer, map, stops, alerts, profile, settings.

lib/features/student/student_shell.dart

93 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter/services.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■ import 'package:go_router/go_router.dart';
5 ■
6 ■ import '../core/widgets/state_views.dart';
7 ■ import '../providers/app_providers.dart';
8 ■ import '../providers/data_providers.dart';
9 ■
10 ■ /// v1.1.1 Student shell – 5 tabs like the design mock:
11 ■ /// Home · Track Bus · My Stop · Notifications · Profile.
12 ■ class StudentShell extends ConsumerWidget {
13 ■   const StudentShell({super.key, required this.shell});
14 ■
15 ■   final StatefulNavigationShell shell;
16 ■
17 ■   void _handleBack(BuildContext context) {
18 ■     final GoRouter router = GoRouter.of(context);
19 ■     if (router.canPop()) {
20 ■       router.pop();
21 ■       return;
22 ■     }
23 ■     if (shell.currentIndex != 0) {
24 ■       shell.goBranch(0);
25 ■       return;
26 ■     }
27 ■     SystemNavigator.pop();
28 ■   }
29 ■
30 ■   @override
31 ■   Widget build(BuildContext context, WidgetRef ref) {
32 ■     final bool online = ref.watch(isOnlineProvider).valueOrNull ?? true;
33 ■     final int unread = ref.watch(unreadCountProvider);
34 ■
35 ■     return PopScope(
36 ■       canPop: false,
37 ■       onPopInvokedWithResult: (bool didPop, Object? result) {
38 ■         if (!didPop) _handleBack(context);
39 ■       },
40 ■       child: Scaffold(
41 ■         body: Column(
42 ■           children: <Widget>[
43 ■             if (!online) const OfflineBanner(),
44 ■             Expanded(child: shell),
45 ■           ],
46 ■         ),
47 ■         bottomNavigationBar: NavigationBar(
48 ■           selectedIndex: shell.currentIndex,
49 ■           labelBehavior: NavigationDestinationLabelBehavior.alwaysShow,
50 ■           onDestinationSelected: (int index) => shell.goBranch(
51 ■             index,
52 ■             initialLocation: index == shell.currentIndex,
53 ■           ),
54 ■           destinations: <Widget>[
55 ■             const NavigationDestination(
56 ■               icon: Icon(Icons.home_outlined),
57 ■               selectedIcon: Icon(Icons.home_rounded),
58 ■               label: 'Home',

```

```

59 ■      ),
60 ■      const NavigationDestination(
61 ■        icon: Icon(Icons.map_outlined),
62 ■        selectedIcon: Icon(Icons.map_rounded),
63 ■        label: 'Track Bus',
64 ■      ),
65 ■      const NavigationDestination(
66 ■        icon: Icon(Icons.pin_drop_outlined),
67 ■        selectedIcon: Icon(Icons.pin_drop_rounded),
68 ■        label: 'My Stop',
69 ■      ),
70 ■      NavigationDestination(
71 ■        icon: Badge(
72 ■          isLabelVisible: unread > 0,
73 ■          label: Text('$unread'),
74 ■          child: const Icon(Icons.notifications_outlined),
75 ■        ),
76 ■        selectedIcon: Badge(
77 ■          isLabelVisible: unread > 0,
78 ■          label: Text('$unread'),
79 ■          child: const Icon(Icons.notifications_rounded),
80 ■        ),
81 ■        label: 'Notifications',
82 ■      ),
83 ■      const NavigationDestination(
84 ■        icon: Icon(Icons.person_outline_rounded),
85 ■        selectedIcon: Icon(Icons.person_rounded),
86 ■        label: 'Profile',
87 ■      ),
88 ■    ],
89 ■  ),
90 ■ ),
91 ■ );
92 ■ }
93 ■ }

```

lib/features/student/screens/student_home_screen.dart

707 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../core/constants/enums.dart';
6 ■ import '../core/theme/nmb_colors.dart';
7 ■ import '../core/theme/nmb_typography.dart';
8 ■ import '../core/utils/formatters.dart';
9 ■ import '../core/widgets/nmb_card.dart';
10 ■ import '../core/widgets/skeleton.dart';
11 ■ import '../core/widgets/state_views.dart';
12 ■ import '../models/app_notification.dart';
13 ■ import '../models/app_user.dart';
14 ■ import '../models/bus.dart';
15 ■ import '../models/bus_stop.dart';
16 ■ import '../models/live_location.dart';
17 ■ import '../models/trip.dart';
18 ■ import '../providers/app_providers.dart';
19 ■ import '../providers/data_providers.dart';
20 ■ import '../services/eta_service.dart';
21 ■ import '../widgets/student_drawer.dart';
22 ■
23 ■ /// v1.1.1 Student Home – rich blue header with avatar, bus card,
24 ■ /// Track My Bus banner, quick actions, next-stop progress, recent alerts.
25 ■ class StudentHomeScreen extends ConsumerWidget {
26 ■   const StudentHomeScreen({super.key});
27 ■
28 ■   @override
29 ■   Widget build(BuildContext context, WidgetRef ref) {
30 ■     final AsyncValue<AppUser?> profileAsync = ref.watch(myProfileProvider);
31 ■
32 ■     return Scaffold(
33 ■       drawer: const StudentDrawer(),
34 ■       body: profileAsync.when(
35 ■         loading: () => const HomeSkeleton(),
36 ■         error: (Object e, _) => ErrorView(
37 ■           message: 'Couldn\'t load your dashboard. Please check your internet.',
38 ■           onRetry: () => ref.invalidate(myProfileProvider),
39 ■         ),
40 ■         data: (AppUser? me) {
41 ■           if (me == null) {
42 ■             return const ErrorView(
43 ■               title: 'Account not ready',
44 ■               message: 'Please contact the school office.',
45 ■             );
46 ■           }
47 ■           return _HomeContent(me: me);
48 ■         },
49 ■       ),
50 ■     );
51 ■   }
52 ■ }
53 ■
54 ■ class _HomeContent extends ConsumerWidget {
55 ■   const _HomeContent({required this.me});
56 ■
57 ■   final AppUser me;
58 ■

```

```

59 ■ @override
60 ■ Widget build(BuildContext context, WidgetRef ref) {
61 ■   final Bus? bus = ref.watch(myBusProvider).valueOrNull;
62 ■   final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
63 ■   final LiveLocation? live =
64 ■     ref.watch(myBusLiveLocationProvider).valueOrNull;
65 ■   final LocationFreshness freshness = ref.watch(myBusFreshnessProvider);
66 ■   final BusStop? myStop = ref.watch(myAssignedStopProvider);
67 ■   final List<BusStop> stops =
68 ■     ref.watch(myStopsProvider).valueOrNull ?? const <BusStop>[];
69 ■   final List<AppNotification> inbox =
70 ■     ref.watch(inboxProvider).valueOrNull ?? const <AppNotification>[];
71 ■   final int unread = ref.watch(unreadCountProvider);
72 ■   final bool onTrip = trip != null && trip.isActive;
73 ■
74 ■   // ETA
75 ■   String? etaText;
76 ■   if (myStop != null && freshness == LocationFreshness.live) {
77 ■     final EtaService etaService = ref.watch(etaServiceProvider);
78 ■     if (live != null) etaService.recordSpeed(live.speedKmh);
79 ■     final EtaResult eta = etaService.etaToStop(
80 ■       live: live,
81 ■       targetStop: myStop,
82 ■       orderedStops: stops,
83 ■     );
84 ■     if (eta is EtaAvailable) {
85 ■       etaText = Formatters.etaShort(eta.duration);
86 ■     }
87 ■   }
88 ■
89 ■   // Bus progress along stops (kaunsa stop sabse paas hai).
90 ■   int progressIndex = -1;
91 ■   if (live != null &&
92 ■     freshness != LocationFreshness.unavailable &&
93 ■     stops.isNotEmpty) {
94 ■     double best = double.infinity;
95 ■     for (int i = 0; i < stops.length; i++) {
96 ■       final double d = live.position.distanceTo(stops[i].location);
97 ■       if (d < best) {
98 ■         best = d;
99 ■         progressIndex = i;
100 ■       }
101 ■     }
102 ■   }
103 ■
104 ■   return CustomScrollView(
105 ■     physics: const BouncingScrollPhysics(),
106 ■     slivers: <Widget>[
107 ■       // ■■ Blue header ■■
108 ■       SliverToBoxAdapter(
109 ■         child: Container(
110 ■           decoration: const BoxDecoration(
111 ■             gradient: LinearGradient(
112 ■               begin: Alignment.topLeft,
113 ■               end: Alignment.bottomRight,
114 ■               colors: <Color>[NmbColors.primaryDark, NmbColors.primary],
115 ■             ),
116 ■           borderRadius: BorderRadius.only(

```

```

117 ■         bottomLeft: Radius.circular(28),
118 ■         bottomRight: Radius.circular(28),
119 ■       ),
120 ■     ),
121 ■     padding: const EdgeInsets.fromLTRB(20, 8, 20, 24),
122 ■     child: SafeArea(
123 ■       bottom: false,
124 ■       child: Column(
125 ■         children: <Widget>[
126 ■           Row(
127 ■             children: <Widget>[
128 ■               Builder(
129 ■                 builder: (BuildContext ctx) => IconButton(
130 ■                   onPressed: () => Scaffold.of(ctx).openDrawer(),
131 ■                   icon: const Icon(Icons.menu_rounded,
132 ■                     color: Colors.white, size: 28,)),
133 ■               ),
134 ■             ],
135 ■             const Spacer(),
136 ■             Stack(
137 ■               children: <Widget>[
138 ■                 IconButton(
139 ■                   onPressed: () => context.go('/student/alerts'),
140 ■                   icon: const Icon(Icons.notifications_outlined,
141 ■                     color: Colors.white, size: 28,)),
142 ■                 ),
143 ■                 if (unread > 0)
144 ■                   Positioned(
145 ■                     right: 6,
146 ■                     top: 6,
147 ■                     child: Container(
148 ■                       padding: const EdgeInsets.all(4),
149 ■                       decoration: const BoxDecoration(
150 ■                         color: NmbColors.danger,
151 ■                         shape: BoxShape.circle,
152 ■                       ),
153 ■                       child: Text(
154 ■                         '$unread',
155 ■                         style: const TextStyle(
156 ■                           color: Colors.white,
157 ■                           fontSize: 10,
158 ■                           fontWeight: FontWeight.w800,
159 ■                         ),
160 ■                       ),
161 ■                     ),
162 ■                   ],
163 ■                 ],
164 ■             ),
165 ■           ],
166 ■         ),
167 ■         const SizedBox(height: 4),
168 ■         Row(
169 ■           children: <Widget>[
170 ■             Expanded(
171 ■               child: Column(
172 ■                 crossAxisAlignment: CrossAxisAlignment.start,
173 ■                 children: <Widget>[
174 ■                   Text(

```

```

175 ■         '$_greetPart()),',
176 ■         style: NmbTypography.body
177 ■         .copyWith(color: Colors.white70),
178 ■     ),
179 ■     Text(
180 ■         '${me.firstName} ■',
181 ■         style: NmbTypography.displayTitle
182 ■         .copyWith(color: Colors.white),
183 ■     ),
184 ■     const SizedBox(height: 4),
185 ■     Text(
186 ■         'Have a great day at school!',
187 ■         style: NmbTypography.bodySecondary
188 ■         .copyWith(color: Colors.white70),
189 ■     ),
190 ■     ],
191 ■ ),
192 ■ ),
193 ■ // Avatar with online dot
194 ■ Stack(
195 ■     children: <Widget>[
196 ■         CircleAvatar(
197 ■             radius: 34,
198 ■             backgroundColor: Colors.white,
199 ■             child: CircleAvatar(
200 ■                 radius: 31,
201 ■                 backgroundColor: NmbColors.primarySoft,
202 ■                 child: Text(
203 ■                     me.firstName.isNotEmpty
204 ■                     ? me.firstName[0].toUpperCase()
205 ■                     : '?',
206 ■                     style: NmbTypography.displayTitle
207 ■                     .copyWith(color: NmbColors.primary),
208 ■                 ),
209 ■             ),
210 ■         ],
211 ■     Positioned(
212 ■         bottom: 4,
213 ■         right: 4,
214 ■         child: Container(
215 ■             width: 14,
216 ■             height: 14,
217 ■             decoration: BoxDecoration(
218 ■                 color: onTrip
219 ■                 ? NmbColors.success
220 ■                 : NmbColors.textTertiary,
221 ■                 shape: BoxShape.circle,
222 ■                 border: Border.all(
223 ■                     color: Colors.white, width: 2,),
224 ■             ),
225 ■         ),
226 ■     ),
227 ■ ],
228 ■ ),
229 ■ ],
230 ■ ),
231 ■ ],
232 ■ ),

```

```

233 ■      ),
234 ■    ),
235 ■  ),
236 ■
237 ■  SliverPadding(
238 ■    padding: const EdgeInsets.fromLTRB(20, 16, 20, 24),
239 ■    sliver: SliverList(
240 ■      delegate: SliverChildListDelegate(<Widget>[
241 ■        // ■■ My Bus card ■■
242 ■        if (me.busId == null)
243 ■          const NmbCard(
244 ■            color: NmbColors.warningSoft,
245 ■            child: Row(
246 ■              children: <Widget>[
247 ■                Icon(Icons.no_transfer_rounded,
248 ■                  color: NmbColors.warning,),
249 ■                SizedBox(width: 12),
250 ■                Expanded(
251 ■                  child: Text(
252 ■                    'No bus has been assigned yet. Please contact the '
253 ■                    'school office.',
254 ■                    style: NmbTypography.bodySecondary,
255 ■                  ),
256 ■                ),
257 ■              ],
258 ■            ),
259 ■          )
260 ■        else ...<Widget>[
261 ■          NmbCard(
262 ■            child: Row(
263 ■              children: <Widget>[
264 ■                Container(
265 ■                  width: 58,
266 ■                  height: 58,
267 ■                  decoration: BoxDecoration(
268 ■                    color: NmbColors.accentSoft,
269 ■                    borderRadius: BorderRadius.circular(16),
270 ■                  ),
271 ■                  child: const Icon(Icons.directions_bus_rounded,
272 ■                    color: NmbColors.accentDark, size: 32,),
273 ■                ),
274 ■                const SizedBox(width: 14),
275 ■                Expanded(
276 ■                  child: Column(
277 ■                    crossAxisAlignment: CrossAxisAlignment.start,
278 ■                    children: <Widget>[
279 ■                      Text('My Bus',
280 ■                        style: NmbTypography.caption
281 ■                          .copyWith(color: NmbColors.primary)),
282 ■                      Text(bus?.busNumber ?? 'Loading...',
283 ■                        style: NmbTypography.screenTitle),
284 ■                      if (bus != null)
285 ■                        Container(
286 ■                          margin: const EdgeInsets.only(top: 2),
287 ■                          padding: const EdgeInsets.symmetric(
288 ■                            horizontal: 8, vertical: 2,),
289 ■                          decoration: BoxDecoration(
290 ■                            color: NmbColors.background,

```

```

291 ■         borderRadius: BorderRadius.circular(6),
292 ■     ),
293 ■     child: Text(bus.plateNumber,
294 ■         style: NmbTypography.caption,),
295 ■ ),
296 ■ ],
297 ■ ),
298 ■ ),
299 ■ Column(
300 ■     crossAxisAlignment: CrossAxisAlignment.end,
301 ■     children: <Widget>[
302 ■         Container(
303 ■             padding: const EdgeInsets.symmetric(
304 ■                 horizontal: 10, vertical: 5,),
305 ■             decoration: BoxDecoration(
306 ■                 color: onTrip
307 ■                     ? NmbColors.successSoft
308 ■                     : NmbColors.divider,
309 ■                 borderRadius: BorderRadius.circular(999),
310 ■             ),
311 ■             child: Text(
312 ■                 onTrip ? 'ON TRIP' : 'IDLE',
313 ■                 style: NmbTypography.caption.copyWith(
314 ■                     color: onTrip
315 ■                         ? NmbColors.success
316 ■                         : NmbColors.textTertiary,
317 ■                     fontWeight: FontWeight.w800,
318 ■                 ),
319 ■             ),
320 ■         ),
321 ■         const SizedBox(height: 6),
322 ■         Row(
323 ■             mainAxisAlignment: MainAxisAlignment.min,
324 ■             children: <Widget>[
325 ■                 Container(
326 ■                     width: 8,
327 ■                     height: 8,
328 ■                     decoration: BoxDecoration(
329 ■                         color: freshness == LocationFreshness.live
330 ■                             ? NmbColors.success
331 ■                             : NmbColors.textTertiary,
332 ■                         shape: BoxShape.circle,
333 ■                     ),
334 ■                 ),
335 ■                 const SizedBox(width: 5),
336 ■                 Text(
337 ■                     freshness == LocationFreshness.live
338 ■                         ? 'Live Tracking'
339 ■                         : freshness == LocationFreshness.stale
340 ■                             ? 'Signal weak'
341 ■                             : 'Not tracking',
342 ■                     style: NmbTypography.caption,
343 ■                 ),
344 ■             ],
345 ■         ),
346 ■     ],
347 ■ ),
348 ■ ],

```

```

349 ■      ),
350 ■    ),
351 ■    const SizedBox(height: 14),
352 ■
353 ■    // ■■ Track My Bus banner ■■
354 ■    Material(
355 ■      color: Colors.transparent,
356 ■      child: InkWell(
357 ■        borderRadius: BorderRadius.circular(20),
358 ■        onTap: () => context.go('/student/map'),
359 ■        child: Container(
360 ■          padding: const EdgeInsets.all(18),
361 ■          decoration: BoxDecoration(
362 ■            gradient: const LinearGradient(
363 ■              colors: <Color>[
364 ■                NmbColors.primary,
365 ■                NmbColors.primaryDark,
366 ■              ],
367 ■            ),
368 ■            borderRadius: BorderRadius.circular(20),
369 ■          ),
370 ■          child: Row(
371 ■            children: <Widget>[
372 ■              Container(
373 ■                width: 52,
374 ■                height: 52,
375 ■                decoration: const BoxDecoration(
376 ■                  color: Colors.white,
377 ■                  shape: BoxShape.circle,
378 ■                ),
379 ■                child: const Icon(Icons.location_on_rounded,
380 ■                  color: NmbColors.danger, size: 30,),
381 ■              ),
382 ■              const SizedBox(width: 14),
383 ■              Expanded(
384 ■                child: Column(
385 ■                  crossAxisAlignment: CrossAxisAlignment.start,
386 ■                  children: <Widget>[
387 ■                    Text('Track My Bus',
388 ■                      style: NmbTypography.sectionTitle
389 ■                        .copyWith(color: Colors.white)),
390 ■                    Text('See your bus live on map',
391 ■                      style: NmbTypography.bodySecondary
392 ■                        .copyWith(color: Colors.white70)),
393 ■                  ],
394 ■                ),
395 ■              ),
396 ■              Container(
397 ■                width: 36,
398 ■                height: 36,
399 ■                decoration: BoxDecoration(
400 ■                  color: Colors.white24,
401 ■                  borderRadius: BorderRadius.circular(999),
402 ■                ),
403 ■                child: const Icon(Icons.chevron_right_rounded,
404 ■                  color: Colors.white),
405 ■              ),
406 ■            ],

```

```

407 ■         ),
408 ■       ),
409 ■     ),
410 ■   ),
411 ■   const SizedBox(height: 20),
412 ■
413 ■   // ■■ Quick Actions ■■
414 ■   const Text('Quick Actions',
415 ■     style: NmbTypography.sectionTitle,),
416 ■   const SizedBox(height: 12),
417 ■   Row(
418 ■     children: <Widget>[
419 ■       _QuickAction(
420 ■         icon: Icons.pin_drop_rounded,
421 ■         color: const Color(0xFF7B61FF),
422 ■         bg: const Color(0xFFEAEAFF),
423 ■         label: 'My Stop',
424 ■         onTap: () => context.go('/student/home/stops'),
425 ■       ),
426 ■       _QuickAction(
427 ■         icon: Icons.schedule_rounded,
428 ■         color: NmbColors.primary,
429 ■         bg: NmbColors.primarySoft,
430 ■         label: 'ETA & Schedule',
431 ■         onTap: () => context.go('/student/home/bus'),
432 ■       ),
433 ■       _QuickAction(
434 ■         icon: Icons.notifications_rounded,
435 ■         color: NmbColors.accentDark,
436 ■         bg: NmbColors.accentSoft,
437 ■         label: 'Notifications',
438 ■         onTap: () => context.go('/student/alerts'),
439 ■       ),
440 ■       _QuickAction(
441 ■         icon: Icons.call_rounded,
442 ■         color: NmbColors.success,
443 ■         bg: NmbColors.successSoft,
444 ■         label: 'Contact School',
445 ■         onTap: () => context.go('/student/profile'),
446 ■       ),
447 ■     ],
448 ■   ),
449 ■   const SizedBox(height: 20),
450 ■
451 ■   // ■■ Next Stop + progress ■■
452 ■   if (myStop != null) ...<Widget>[
453 ■     NmbCard(
454 ■       child: Column(
455 ■         crossAxisAlignment: CrossAxisAlignment.start,
456 ■         children: <Widget>[
457 ■           Row(
458 ■             children: <Widget>[
459 ■               const Icon(Icons.star_rounded,
460 ■                 color: NmbColors.accent, size: 20,),
461 ■               const SizedBox(width: 6),
462 ■               Text('My Stop',
463 ■                 style: NmbTypography.caption.copyWith(
464 ■                   fontWeight: FontWeight.w700,),),

```

```

465 ■         ],
466 ■     ),
467 ■     const SizedBox(height: 4),
468 ■     Text(myStop.name,
469 ■         style: NmbTypography.sectionTitle,),
470 ■     Text(
471 ■         etaText != null
472 ■         ? '$etaText to reach'
473 ■         : onTrip
474 ■         ? (myStop.scheduledTime != null
475 ■             ? 'Scheduled: ${myStop.scheduledTime}'
476 ■             : 'Bus on the way')
477 ■         : 'No trip in progress',
478 ■     style: NmbTypography.bodySecondary.copyWith(
479 ■         color: etaText != null
480 ■         ? NmbColors.success
481 ■         : NmbColors.textSecondary,
482 ■         fontWeight: FontWeight.w600,
483 ■     ),
484 ■ ),
485 ■ if (stops.length >= 2) ...<Widget>[
486 ■     const SizedBox(height: 14),
487 ■     _RouteProgress(
488 ■         stops: stops,
489 ■         busAt: progressIndex,
490 ■         myStopId: myStop.id,
491 ■     ),
492 ■     const SizedBox(height: 4),
493 ■     const Row(
494 ■         mainAxisAlignment:
495 ■             MainAxisAlignment.spaceBetween,
496 ■         children: <Widget>[
497 ■             Text('Start', style: NmbTypography.caption),
498 ■             Text('School', style: NmbTypography.caption),
499 ■         ],
500 ■     ),
501 ■ ],
502 ■ ],
503 ■ ),
504 ■ ),
505 ■     const SizedBox(height: 20),
506 ■ ],
507 ■ ],
508 ■
509 ■ // ■■ Recent Alerts ■■
510 ■ Row(
511 ■     mainAxisAlignment: MainAxisAlignment.spaceBetween,
512 ■     children: <Widget>[
513 ■         const Text('Recent Alerts',
514 ■             style: NmbTypography.sectionTitle,),
515 ■         TextButton(
516 ■             onPressed: () => context.go('/student/alerts'),
517 ■             child: const Text('View All'),
518 ■         ),
519 ■     ],
520 ■ ),
521 ■ if (inbox.isEmpty)
522 ■     const NmbCard(

```

```

523 ■         child: Text(
524 ■             'No alerts yet. Bus updates will appear here.',
525 ■             style: NmbTypography.bodySecondary,
526 ■         ),
527 ■     ),
528 ■     else
529 ■     NmbCard(
530 ■         padding: const EdgeInsets.symmetric(
531 ■             horizontal: 14, vertical: 6,),
532 ■         child: Column(
533 ■             children: <Widget>[
534 ■                 for (final AppNotification n in inbox.take(3))
535 ■                     ListTile(
536 ■                         contentPadding: EdgeInsets.zero,
537 ■                         leading: Container(
538 ■                             width: 40,
539 ■                             height: 40,
540 ■                             decoration: BoxDecoration(
541 ■                                 color: _alertBg(n.type),
542 ■                                 borderRadius: BorderRadius.circular(12),
543 ■                             ),
544 ■                             child: Icon(_alertIcon(n.type),
545 ■                                 color: _alertColor(n.type), size: 20,),
546 ■                         ),
547 ■                         title: Text(n.title,
548 ■                             style: NmbTypography.cardTitle,),
549 ■                         subtitle: Text(
550 ■                             n.body,
551 ■                             maxLines: 1,
552 ■                             overflow: TextOverflow.ellipsis,
553 ■                             style: NmbTypography.bodySecondary,
554 ■                         ),
555 ■                         trailing: Text(
556 ■                             Formatters.timeOfDay(n.receivedAt),
557 ■                             style: NmbTypography.caption,
558 ■                         ),
559 ■                     ),
560 ■                 ],
561 ■             ),
562 ■         ),
563 ■         const SizedBox(height: 20),
564 ■     ]),
565 ■ ),
566 ■ ),
567 ■ ],
568 ■ );
569 ■ }
570 ■
571 ■ String _greetPart() {
572 ■     final int h = DateTime.now().hour;
573 ■     if (h < 12) return 'Good Morning';
574 ■     if (h < 17) return 'Good Afternoon';
575 ■     return 'Good Evening';
576 ■ }
577 ■
578 ■ IconData _alertIcon(NotificationType t) => switch (t) {
579 ■     NotificationType.tripStarted => Icons.play_circle_rounded,
580 ■     NotificationType.busApproaching => Icons.near_me_rounded,

```

```

581 ■      NotificationType.busReachedStop => Icons.pin_drop_rounded,
582 ■      NotificationType.busReachedSchool => Icons.school_rounded,
583 ■      NotificationType.busDelayed => Icons.schedule_rounded,
584 ■      NotificationType.trackingUnavailable => Icons.location_off_rounded,
585 ■      NotificationType.announcement => Icons.campaign_rounded,
586 ■    };
587 ■
588 ■    Color _alertColor(NotificationType t) => switch (t) {
589 ■      NotificationType.tripStarted => NmbColors.success,
590 ■      NotificationType.busApproaching => NmbColors.accentDark,
591 ■      NotificationType.busDelayed => NmbColors.warning,
592 ■      _ => NmbColors.primary,
593 ■    };
594 ■
595 ■    Color _alertBg(NotificationType t) => switch (t) {
596 ■      NotificationType.tripStarted => NmbColors.successSoft,
597 ■      NotificationType.busApproaching => NmbColors.accentSoft,
598 ■      NotificationType.busDelayed => NmbColors.warningSoft,
599 ■      _ => NmbColors.primarySoft,
600 ■    };
601 ■ }
602 ■
603 ■ class _QuickAction extends StatelessWidget {
604 ■   const _QuickAction({
605 ■     required this.icon,
606 ■     required this.color,
607 ■     required this.bg,
608 ■     required this.label,
609 ■     required this.onTap,
610 ■   });
611 ■
612 ■   final IconData icon;
613 ■   final Color color;
614 ■   final Color bg;
615 ■   final String label;
616 ■   final VoidCallback onTap;
617 ■
618 ■   @override
619 ■   Widget build(BuildContext context) {
620 ■     return Expanded(
621 ■       child: InkWell(
622 ■         borderRadius: BorderRadius.circular(16),
623 ■         onTap: onTap,
624 ■         child: Padding(
625 ■           padding: const EdgeInsets.symmetric(vertical: 4),
626 ■           child: Column(
627 ■             children: <Widget>[
628 ■               Container(
629 ■                 width: 54,
630 ■                 height: 54,
631 ■                 decoration: BoxDecoration(
632 ■                   color: bg,
633 ■                   borderRadius: BorderRadius.circular(16),
634 ■                 ),
635 ■               child: Icon(icon, color: color, size: 26),
636 ■             ],
637 ■             const SizedBox(height: 8),
638 ■             Text(

```

```

639 ■         label,
640 ■         textAlign: TextAlign.center,
641 ■         style: NmbTypography.caption
642 ■         .copyWith(color: NmbColors.textSecondary),
643 ■     ),
644 ■ ],
645 ■ ),
646 ■ ),
647 ■ ),
648 ■ );
649 ■ }
650 ■ }
651 ■
652 ■ /// Bus-on-route progress line: dots for stops, bus emoji at current pos,
653 ■ /// star dot = my stop, green = covered.
654 ■ class _RouteProgress extends StatelessWidget {
655 ■   const _RouteProgress({
656 ■     required this.stops,
657 ■     required this.busAt,
658 ■     required this.myStopId,
659 ■   });
660 ■
661 ■   final List<BusStop> stops;
662 ■   final int busAt; // -1 = unknown
663 ■   final String myStopId;
664 ■
665 ■   @override
666 ■   Widget build(BuildContext context) {
667 ■     return SizedBox(
668 ■       height: 34,
669 ■       child: Row(
670 ■         children: <Widget>[
671 ■           for (int i = 0; i < stops.length; i++) ...<Widget>[
672 ■             if (i > 0)
673 ■               Expanded(
674 ■                 child: Container(
675 ■                   height: 3,
676 ■                   color: busAt >= 0 && i <= busAt
677 ■                     ? NmbColors.success
678 ■                     : NmbColors.divider,
679 ■                 ),
680 ■             ),
681 ■             if (i == busAt)
682 ■               const Text('■', style: TextStyle(fontSize: 22))
683 ■             else
684 ■               Container(
685 ■                 width: stops[i].id == myStopId ? 14 : 10,
686 ■                 height: stops[i].id == myStopId ? 14 : 10,
687 ■                 decoration: BoxDecoration(
688 ■                   color: stops[i].id == myStopId
689 ■                     ? NmbColors.accent
690 ■                     : busAt >= 0 && i <= busAt
691 ■                       ? NmbColors.success
692 ■                       : Colors.white,
693 ■                   shape: BoxShape.circle,
694 ■                   border: Border.all(
695 ■                     color: busAt >= 0 && i <= busAt
696 ■                       ? NmbColors.success
697 ■                       : NmbColors.divider,
698 ■                     width: 2,
699 ■                   ),
700 ■                 ),
701 ■             ),
702 ■           ],
703 ■         ],
704 ■       ),
705 ■     );
706 ■   }
707 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../core/constants/nmb_constants.dart';
6 ■ import '../core/theme/nmb_colors.dart';
7 ■ import '../core/theme/nmb_typography.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../models/app_user.dart';
10 ■ import '../providers/app_providers.dart';
11 ■ import '../providers/data_providers.dart';
12 ■ import '../services/auth_service.dart';
13 ■
14 ■ /// v1.1.1 Student side drawer – profile header (avatar, name, class) +
15 ■ /// navigation items + logout, matching the design mock.
16 ■ class StudentDrawer extends ConsumerWidget {
17 ■   const StudentDrawer({super.key});
18 ■
19 ■   Future<void> _logout(BuildContext context, WidgetRef ref) async {
20 ■     final AuthSession? session = ref.read(currentSessionProvider);
21 ■     // Capture EVERYTHING from ref BEFORE any await/pop – after the widget
22 ■     // is disposed, ref must not be touched ("ref after disposed" fix).
23 ■     final notifSvc = ref.read(notificationServiceProvider);
24 ■     final devSvc = ref.read(deviceSessionServiceProvider);
25 ■     final fsSvc = ref.read(firestoreServiceProvider);
26 ■     final authSvc = ref.read(authServiceProvider);
27 ■     final bool ok = await showNmbConfirmDialog(
28 ■       context,
29 ■       title: 'Log out?',
30 ■       message: 'You will need your Class + Roll number + password to sign back in.',
31 ■       confirmLabel: 'Log out',
32 ■       destructive: true,
33 ■       icon: Icons.logout_rounded,
34 ■     );
35 ■     if (!ok) return;
36 ■     if (session != null) {
37 ■       try {
38 ■         await notifSvc.unregisterDevice(session.uid);
39 ■       } catch (_) {/* best effort */}
40 ■       try {
41 ■         final String devId = await devSvc.getDeviceId();
42 ■         await fsSvc.clearActiveDevice(
43 ■           uid: session.uid,
44 ■           deviceId: devId,
45 ■         );
46 ■       } catch (_) {/* best effort */}
47 ■     }
48 ■     await authSvc.signOut();
49 ■   }
50 ■
51 ■   @override
52 ■   Widget build(BuildContext context, WidgetRef ref) {
53 ■     final AppUser? me = ref.watch(myProfileProvider).valueOrNull;
54 ■     final int unread = ref.watch(unreadCountProvider);
55 ■     final String location = GoRouterState.of(context).matchedLocation;
56 ■
57 ■     void go(String path) {
58 ■       Navigator.of(context).pop();

```

```

59 ■      context.go(path);
60 ■    }
61 ■
62 ■    Widget item({
63 ■      required IconData icon,
64 ■      required String label,
65 ■      required String path,
66 ■      int badge = 0,
67 ■    }) {
68 ■      final bool selected = location == path;
69 ■      return Padding(
70 ■        padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 2),
71 ■        child: ListTile(
72 ■          shape: RoundedRectangleBorder(
73 ■            borderRadius: BorderRadius.circular(14),
74 ■          ),
75 ■          selected: selected,
76 ■          selectedTileColor: NmbColors.primarySoft,
77 ■          leading: Icon(icon,
78 ■            color: selected ? NmbColors.primary : NmbColors.textSecondary, ),
79 ■          title: Text(
80 ■            label,
81 ■            style: NmbTypography.body.copyWith(
82 ■              fontWeight: selected ? FontWeight.w700 : FontWeight.w500,
83 ■              color: selected ? NmbColors.primary : NmbColors.textPrimary,
84 ■            ),
85 ■          ),
86 ■          trailing: badge > 0
87 ■            ? Container(
88 ■              padding: const EdgeInsets.symmetric(
89 ■                horizontal: 8, vertical: 3, ),
90 ■              decoration: const BoxDecoration(
91 ■                color: NmbColors.danger,
92 ■                shape: BoxShape.circle,
93 ■              ),
94 ■              child: Text(
95 ■                '$badge',
96 ■                style: const TextStyle(
97 ■                  color: Colors.white,
98 ■                  fontSize: 11,
99 ■                  fontWeight: FontWeight.w800,
100 ■                ),
101 ■            ),
102 ■          )
103 ■          : null,
104 ■          onTap: () => go(path),
105 ■        ),
106 ■      );
107 ■    }
108 ■
109 ■    return Drawer(
110 ■      child: Column(
111 ■        children: <Widget>[
112 ■          // ■■ Profile header ■■
113 ■          Container(
114 ■            width: double.infinity,
115 ■            decoration: const BoxDecoration(
116 ■              gradient: LinearGradient(

```

```

117 ■         begin: Alignment.topLeft,
118 ■         end: Alignment.bottomRight,
119 ■         colors: <Color>[NmbColors.primaryDark, NmbColors.primary],
120 ■     ),
121 ■ ),
122 ■ padding: const EdgeInsets.fromLTRB(20, 0, 20, 24),
123 ■ child: SafeArea(
124 ■     bottom: false,
125 ■     child: Column(
126 ■         children: <Widget>[
127 ■             const SizedBox(height: 16),
128 ■             Stack(
129 ■                 children: <Widget>[
130 ■                     CircleAvatar(
131 ■                         radius: 42,
132 ■                         backgroundColor: Colors.white,
133 ■                         child: CircleAvatar(
134 ■                             radius: 38,
135 ■                             backgroundColor: NmbColors.primarySoft,
136 ■                             child: Text(
137 ■                                 (me?.firstName.isNotEmpty ?? false)
138 ■                                   ? me!.firstName[0].toUpperCase()
139 ■                                   : '?',
140 ■                                 style: NmbTypography.displayTitle
141 ■                                   .copyWith(color: NmbColors.primary),
142 ■                             ),
143 ■                         ),
144 ■                     ),
145 ■                     Positioned(
146 ■                         bottom: 4,
147 ■                         right: 4,
148 ■                         child: Container(
149 ■                             width: 16,
150 ■                             height: 16,
151 ■                             decoration: BoxDecoration(
152 ■                                 color: NmbColors.success,
153 ■                                 shape: BoxShape.circle,
154 ■                                 border:
155 ■                                   Border.all(color: Colors.white, width: 2),
156 ■                             ),
157 ■                         ),
158 ■                     ),
159 ■                 ],
160 ■             ),
161 ■             const SizedBox(height: 12),
162 ■             Text(
163 ■                 me?.fullName ?? '...',
164 ■                 style: NmbTypography.sectionTitle
165 ■                   .copyWith(color: Colors.white),
166 ■             ),
167 ■             if (me?.classSection != null)
168 ■               Text(
169 ■                 'Class ${me!.classSection}'
170 ■                 '$${me.rollNumber != null ? ' • Roll ' : ''} ${me.rollNumber}' : '',
171 ■                 style: NmbTypography.bodySecondary
172 ■                   .copyWith(color: Colors.white70),
173 ■             ),
174 ■             Text(

```

```

175 ■          NmbConstants.schoolName,
176 ■          style: NmbTypography.caption
177 ■          .copyWith(color: Colors.white60),
178 ■        ),
179 ■      ],
180 ■    ),
181 ■  ),
182 ■ ),
183 ■
184 ■ // ■■ Menu ■■
185 ■ Expanded(
186 ■   child: ListView(
187 ■     padding: const EdgeInsets.symmetric(vertical: 10),
188 ■     children: <Widget>[
189 ■       item(
190 ■         icon: Icons.home_rounded,
191 ■         label: 'Home',
192 ■         path: '/student/home',
193 ■       ),
194 ■       item(
195 ■         icon: Icons.map_rounded,
196 ■         label: 'Track My Bus',
197 ■         path: '/student/map',
198 ■       ),
199 ■       item(
200 ■         icon: Icons.pin_drop_rounded,
201 ■         label: 'My Stop',
202 ■         path: '/student/home/stops',
203 ■       ),
204 ■       item(
205 ■         icon: Icons.schedule_rounded,
206 ■         label: 'ETA & Schedule',
207 ■         path: '/student/home/bus',
208 ■       ),
209 ■       item(
210 ■         icon: Icons.notifications_rounded,
211 ■         label: 'Notifications',
212 ■         path: '/student/alerts',
213 ■         badge: unread,
214 ■       ),
215 ■       const Padding(
216 ■         padding: EdgeInsets.symmetric(horizontal: 20, vertical: 8),
217 ■         child: Divider(),
218 ■       ),
219 ■       item(
220 ■         icon: Icons.person_rounded,
221 ■         label: 'Profile',
222 ■         path: '/student/profile',
223 ■       ),
224 ■       item(
225 ■         icon: Icons.settings_rounded,
226 ■         label: 'Settings',
227 ■         path: '/student/profile/settings',
228 ■       ),
229 ■     ],
230 ■   ),
231 ■ ),
232 ■

```

```

233 ■      // ■■ Logout ■■
234 ■      const Divider(height: 1),
235 ■      Padding(
236 ■        padding: const EdgeInsets.all(10),
237 ■        child: ListTile(
238 ■          shape: RoundedRectangleBorder(
239 ■            borderRadius: BorderRadius.circular(14),
240 ■          ),
241 ■          leading:
242 ■            const Icon(Icons.logout_rounded, color: NmbColors.danger),
243 ■          title: Text(
244 ■            'Logout',
245 ■            style: NmbTypography.body.copyWith(
246 ■              color: NmbColors.danger,
247 ■              fontWeight: FontWeight.w600,
248 ■            ),
249 ■          ),
250 ■          onTap: () {
251 ■            Navigator.of(context).pop();
252 ■            _logout(context, ref);
253 ■          },
254 ■        ),
255 ■      ),
256 ■      const SafeArea(top: false, child: SizedBox(height: 4)),
257 ■    ],
258 ■  ),
259 ■ );
260 ■ }
261 ■ }

```

lib/features/student/widgets/tracking_status_card.dart

85 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■
3 ■ import '../core/constants/enums.dart';
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/formatters.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/status_pill.dart';
9 ■ import '../models/live_location.dart';
10 ■ import '../models/trip.dart';
11 ■
12 ■ /// Honest tracking status: LIVE / last-updated / unavailable.
13 ■ /// Never shows an old fix as live.
14 ■ class TrackingStatusCard extends StatelessWidget {
15 ■   const TrackingStatusCard({
16 ■     super.key,
17 ■     required this.freshness,
18 ■     required this.live,
19 ■     required this.trip,
20 ■     this.etaText,
21 ■   });
22 ■
23 ■   final LocationFreshness freshness;
24 ■   final LiveLocation? live;
25 ■   final Trip? trip;
26 ■   final String? etaText;
27 ■
28 ■   @override
29 ■   Widget build(BuildContext context) {
30 ■     final (String title, String subtitle, IconData icon, Color color) info =
31 ■       switch (freshness) {
32 ■         LocationFreshness.live => (
33 ■           'Bus is being tracked live',
34 ■           etaText ?? 'Follow it on the Live Map',
35 ■           Icons.gps_fixed_rounded,
36 ■           NmbColors.live,
37 ■         ),
38 ■         LocationFreshness.stale => (
39 ■           'Location temporarily unavailable',
40 ■           live != null
41 ■             ? 'Last updated ${Formatters.relativeTime(live!.updatedAt)}'
42 ■             : 'Waiting for the bus signal...',
43 ■           Icons.gps_not_fixed_rounded,
44 ■           NmbColors.stale,
45 ■         ),
46 ■         LocationFreshness.unavailable => (
47 ■           trip == null ? 'No trip in progress' : 'Location unavailable',
48 ■           trip == null
49 ■             ? 'Tracking starts when the driver begins the trip.'
50 ■             : 'We\'ll show the bus as soon as the signal returns.',
51 ■           Icons.location_off_rounded,
52 ■           NmbColors.offline,
53 ■         ),
54 ■       };
55 ■
56 ■     return NmbCard(
57 ■       child: Row(
58 ■         children: <Widget>[

```

```
59 ■      Container(  
60 ■        width: 48,  
61 ■        height: 48,  
62 ■        decoration: BoxDecoration(  
63 ■          color: info.$4.withOpacity(0.12),  
64 ■          borderRadius: BorderRadius.circular(14),  
65 ■        ),  
66 ■        child: Icon(info.$3, color: info.$4),  
67 ■      ),  
68 ■      const SizedBox(width: 14),  
69 ■      Expanded(  
70 ■        child: Column(  
71 ■          crossAxisAlignment: CrossAxisAlignment.start,  
72 ■          children: <Widget>[  
73 ■            Text(info.$1, style: NmbTypography.cardTitle),  
74 ■            const SizedBox(height: 3),  
75 ■            Text(info.$2, style: NmbTypography.bodySecondary),  
76 ■          ],  
77 ■        ),  
78 ■      ),  
79 ■      const SizedBox(width: 8),  
80 ■      StatusPill.freshness(freshness),  
81 ■    ],  
82 ■  ),  
83 ■ );  
84 ■ }  
85 ■ }
```

lib/features/student/screens/student_map_screen.dart

329 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_map/flutter_map.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■ import 'package:latlong2/latlong.dart';
5 ■
6 ■ import '../core/constants/enums.dart';
7 ■ import '../core/theme/nmb_colors.dart';
8 ■ import '../core/theme/nmb_typography.dart';
9 ■ import '../core/utils/formatters.dart';
10 ■ import '../core/widgets/status_pill.dart';
11 ■ import '../models/bus_route.dart';
12 ■ import '../models/bus_stop.dart';
13 ■ import '../models/live_location.dart';
14 ■ import '../models/trip.dart';
15 ■ import '../providers/data_providers.dart';
16 ■ import '../shared/bus_marker_widget.dart';
17 ■
18 ■ /// Full live map on OpenStreetMap (flutter_map).
19 ■ ///
20 ■ /// 100% free: no API key, no billing account. Tiles come from the OSM
21 ■ /// public tile server with a proper attribution line (required by OSM).
22 ■ ///
23 ■ /// The map only ever receives THIS student's bus location – the backend
24 ■ /// rules make any other subscription impossible.
25 ■ class StudentMapScreen extends ConsumerStatefulWidget {
26 ■   const StudentMapScreen({super.key});
27 ■
28 ■   @override
29 ■   ConsumerState<StudentMapScreen> createState() => _StudentMapScreenState();
30 ■ }
31 ■
32 ■ class _StudentMapScreenState extends ConsumerState<StudentMapScreen> {
33 ■   final MapController _controller = MapController();
34 ■   bool _followBus = true;
35 ■   bool _mapReady = false;
36 ■
37 ■   void _centerOnBus(LiveLocation live) {
38 ■     if (!_mapReady) return;
39 ■     _controller.move(
40 ■       LatLng(live.position.lat, live.position.lng),
41 ■       _controller.camera.zoom,
42 ■     );
43 ■   }
44 ■
45 ■   @override
46 ■   Widget build(BuildContext context) {
47 ■     final LiveLocation? live =
48 ■       ref.watch(myBusLiveLocationProvider).valueOrNull;
49 ■     final LocationFreshness freshness = ref.watch(myBusFreshnessProvider);
50 ■     final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
51 ■     final BusRoute? route = ref.watch(myRouteProvider).valueOrNull;
52 ■     final List<BusStop> stops =
53 ■       ref.watch(myStopsProvider).valueOrNull ?? const <BusStop>[];
54 ■     final BusStop? myStop = ref.watch(myAssignedStopProvider);
55 ■
56 ■     // Auto-follow the bus while fresh.
57 ■     ref.listen(myBusLiveLocationProvider,
58 ■       (Object? _, AsyncValue<LiveLocation?> next) {

```

```

59 ■         final LiveLocation? l = next.valueOrNull;
60 ■         if (l != null &&
61 ■             _followBus &&
62 ■             freshness != LocationFreshness.unavailable) {
63 ■             _centerOnBus(l);
64 ■         }
65 ■     });
66 ■
67 ■     // Even with no route/stops published yet, ALWAYS show the map
68 ■     // (centered on the school or the region) with a friendly banner –
69 ■     // never a blank screen.
70 ■     final bool routeMissing = route == null && stops.isEmpty;
71 ■
72 ■     final List<Marker> markers = <Marker>[];
73 ■
74 ■     // School marker.
75 ■     if (route != null) {
76 ■         markers.add(
77 ■             Marker(
78 ■                 point: LatLng(route.schoolLocation.lat, route.schoolLocation.lng),
79 ■                 width: 46,
80 ■                 height: 46,
81 ■                 child: const _PinMarker(
82 ■                     icon: Icons.school_rounded,
83 ■                     color: NmbColors.info,
84 ■                     tooltip: 'SRBS International School',
85 ■                 ),
86 ■             ),
87 ■         );
88 ■     }
89 ■
90 ■     // Stop markers; student's own stop highlighted.
91 ■     for (final BusStop stop in stops) {
92 ■         final bool isMine = stop.id == myStop?.id;
93 ■         markers.add(
94 ■             Marker(
95 ■                 point: LatLng(stop.location.lat, stop.location.lng),
96 ■                 width: isMine ? 52 : 40,
97 ■                 height: isMine ? 52 : 40,
98 ■                 child: _PinMarker(
99 ■                     icon: isMine ? Icons.star_rounded : Icons.pin_drop_rounded,
100 ■                     color: isMine ? NmbColors.accent : const Color(0xFF7B61FF),
101 ■                     tooltip: isMine ? '${stop.name} (My stop)' : stop.name,
102 ■                     large: isMine,
103 ■                 ),
104 ■             ),
105 ■         );
106 ■     }
107 ■
108 ■     // Bus marker – only when the fix is live or stale; NEVER when
109 ■     // unavailable (an old point must not pretend to be the bus).
110 ■     if (live != null && freshness != LocationFreshness.unavailable) {
111 ■         markers.add(
112 ■             Marker(
113 ■                 point: LatLng(live.position.lat, live.position.lng),
114 ■                 width: 56,
115 ■                 height: 56,
116 ■                 child: Opacity(

```

```

117 ■         opacity: freshness == LocationFreshness.live ? 1.0 : 0.5,
118 ■         child: const BusMarkerWidget(),
119 ■       ),
120 ■     ),
121 ■   );
122 ■ }
123 ■
124 ■ // Route polyline: stop-to-stop (+ school at the end).
125 ■ final List<LatLng> routePoints = <LatLng>[
126 ■   for (final BusStop s in stops) LatLng(s.location.lat, s.location.lng),
127 ■   if (route != null)
128 ■     LatLng(route.schoolLocation.lat, route.schoolLocation.lng),
129 ■ ];
130 ■
131 ■ // Fallback chain for the initial camera: live bus → first stop →
132 ■ // school → Meerut, India (city default so the map is never blank).
133 ■ const LatLng indiaFallback = LatLng(28.9845, 77.7064); // Meerut, UP
134 ■ final LatLng initialCenter = live != null
135 ■   ? LatLng(live.position.lat, live.position.lng)
136 ■   : stops.isNotEmpty
137 ■     ? LatLng(stops.first.location.lat, stops.first.location.lng)
138 ■     : (route != null &&
139 ■       (route.schoolLocation.lat != 0 ||
140 ■         route.schoolLocation.lng != 0))
141 ■       ? LatLng(route.schoolLocation.lat, route.schoolLocation.lng)
142 ■       : indiaFallback;
143 ■
144 ■ return Scaffold(
145 ■   appBar: AppBar(
146 ■     title: const Text('Live Map'),
147 ■     actions: <Widget>[
148 ■       Padding(
149 ■         padding: const EdgeInsets.only(right: 16),
150 ■         child: Center(child: StatusPill.freshness(freshness)),
151 ■       ),
152 ■     ],
153 ■   ),
154 ■   body: Stack(
155 ■     children: <Widget>[
156 ■       FlutterMap(
157 ■         mapController: _controller,
158 ■         options: MapOptions(
159 ■           initialCenter: initialCenter,
160 ■           initialZoom: 14,
161 ■           onMapReady: () => _mapReady = true,
162 ■           // User dragging disables auto-follow until re-centered.
163 ■           onPositionChanged: (MapCamera camera, bool hasGesture) {
164 ■             if (hasGesture) _followBus = false;
165 ■           },
166 ■         ),
167 ■       children: <Widget>[
168 ■         TileLayer(
169 ■           urlTemplate: 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
170 ■           userAgentPackageName: 'com.srbs.never_miss_bus',
171 ■         ),
172 ■         if (routePoints.length >= 2)
173 ■           PolylineLayer(
174 ■             polylines: <Polyline>[

```

```

175 ■         Polyline(
176 ■             points: routePoints,
177 ■             strokeWidth: 4,
178 ■             color: NmbColors.primary.withOpacity(0.75),
179 ■         ),
180 ■     ],
181 ■ ),
182 ■ MarkerLayer(markers: markers),
183 ■ // OSM attribution (required by OpenStreetMap's usage policy).
184 ■ const Align(
185 ■     alignment: Alignment.bottomLeft,
186 ■     child: Padding(
187 ■         padding: EdgeInsets.all(4),
188 ■         child: Text(
189 ■             '@ OpenStreetMap contributors',
190 ■             style: TextStyle(fontSize: 10, color: NmbColors.textSecondary),
191 ■         ),
192 ■     ),
193 ■ ),
194 ■ ],
195 ■ ),
196 ■
197 ■ // Route-not-published banner (map still visible behind it).
198 ■ if (routeMissing)
199 ■     Positioned(
200 ■         top: 12,
201 ■         left: 16,
202 ■         right: 16,
203 ■         child: Material(
204 ■             borderRadius: BorderRadius.circular(14),
205 ■             color: NmbColors.infoSoft,
206 ■             elevation: 3,
207 ■             child: Padding(
208 ■                 padding: EdgeInsets.all(12),
209 ■                 child: Row(
210 ■                     children: <Widget>[
211 ■                         const Icon(Icons.route_outlined,
212 ■                             color: NmbColors.info, size: 20,),
213 ■                         const SizedBox(width: 10),
214 ■                         Expanded(
215 ■                             child: Text(
216 ■                                 'Route and stops haven\'t been published yet. '
217 ■                                 'The map will show your bus once the school sets '
218 ■                                 'up the route.',
219 ■                                 style: NmbTypography.caption
220 ■                                     .copyWith(color: NmbColors.textSecondary),
221 ■                             ),
222 ■                         ),
223 ■                     ],
224 ■                 ),
225 ■             ),
226 ■         ),
227 ■     ),
228 ■
229 ■ // Honest status banner overlaying the map.
230 ■ if (!routeMissing && freshness != LocationFreshness.live)
231 ■     Positioned(
232 ■         top: 12,

```

```

233 ■         left: 16,
234 ■         right: 16,
235 ■         child: Material(
236 ■             borderRadius: BorderRadius.circular(14),
237 ■             color: freshness == LocationFreshness.stale
238 ■                 ? NmbColors.warningSoft
239 ■                 : NmbColors.surface,
240 ■             elevation: 3,
241 ■             child: Padding(
242 ■                 padding: const EdgeInsets.all(12),
243 ■                 child: Row(
244 ■                     children: <Widget>[
245 ■                         Icon(
246 ■                             freshness == LocationFreshness.stale
247 ■                                 ? Icons.history_rounded
248 ■                                 : Icons.location_off_rounded,
249 ■                             color: freshness == LocationFreshness.stale
250 ■                                 ? NmbColors.warning
251 ■                                 : NmbColors.textTertiary,
252 ■                             size: 20,
253 ■                         ),
254 ■                         const SizedBox(width: 10),
255 ■                         Expanded(
256 ■                             child: Text(
257 ■                                 freshness == LocationFreshness.stale && live != null
258 ■                                     ? 'Location temporarily unavailable – last '
259 ■                                     : 'updated ${Formatters.relativeTime(live.updatedAt)}'
260 ■                                 : trip == null
261 ■                                     ? 'No trip in progress. Tracking starts '
262 ■                                     : 'when the driver begins the trip.'
263 ■                                 : 'Location temporarily unavailable.',
264 ■                                 style: NmbTypography.caption.copyWith(
265 ■                                     color: NmbColors.textSecondary,
266 ■                                 ),
267 ■                             ),
268 ■                         ),
269 ■                     ],
270 ■                 ),
271 ■             ),
272 ■         ),
273 ■     ],
274 ■ ],
275 ■ ),
276 ■ floatingActionButton:
277 ■     live != null && freshness != LocationFreshness.unavailable
278 ■         ? FloatingActionButton.extended(
279 ■             onPressed: () {
280 ■                 _followBus = true;
281 ■                 _centerOnBus(live);
282 ■             },
283 ■             backgroundColor: NmbColors.primary,
284 ■             foregroundColor: Colors.white,
285 ■             icon: const Icon(Icons.center_focus_strong_rounded),
286 ■             label: const Text('Center on Bus'),
287 ■         )
288 ■         : null,
289 ■ );
290 ■ }

```

```
291 ■ }
292 ■
293 ■ /// Small circular pin used for school and stop markers.
294 ■ class _PinMarker extends StatelessWidget {
295 ■   const _PinMarker({
296 ■     required this.icon,
297 ■     required this.color,
298 ■     required this.tooltip,
299 ■     this.large = false,
300 ■   });
301 ■
302 ■   final IconData icon;
303 ■   final Color color;
304 ■   final String tooltip;
305 ■   final bool large;
306 ■
307 ■   @override
308 ■   Widget build(BuildContext context) {
309 ■     return Tooltip(
310 ■       message: tooltip,
311 ■       triggerMode: TooltipTriggerMode.tap,
312 ■       child: Container(
313 ■         decoration: BoxDecoration(
314 ■           color: color,
315 ■           shape: BoxShape.circle,
316 ■           border: Border.all(color: Colors.white, width: 2.5),
317 ■           boxShadow: const <BoxShadow>[
318 ■             BoxShadow(
319 ■               color: Color(0x33000000),
320 ■               blurRadius: 6,
321 ■               offset: Offset(0, 2),
322 ■             ),
323 ■           ],
324 ■         ),
325 ■         child: Icon(icon, color: Colors.white, size: large ? 26 : 20),
326 ■       ),
327 ■     );
328 ■   }
329 ■ }
```

lib/features/shared/bus_marker_widget.dart

88 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■
3 ■ import '../core/theme/nmb_colors.dart';
4 ■
5 ■ /// School-bus map marker as a plain Flutter widget (flutter_map renders
6 ■ widgets directly – no bitmap pipeline needed).
7 ■ ///
8 ■ /// Amber rounded bus with windows and wheels on a white circular badge,
9 ■ matching the app's design system.
10 ■ class BusMarkerWidget extends StatelessWidget {
11 ■   const BusMarkerWidget({super.key, this.size = 56});
12 ■
13 ■   final double size;
14 ■
15 ■   @override
16 ■   Widget build(BuildContext context) {
17 ■     return Container(
18 ■       width: size,
19 ■       height: size,
20 ■       decoration: const BoxDecoration(
21 ■         color: Colors.white,
22 ■         shape: BoxShape.circle,
23 ■         boxShadow: <BoxShadow>[
24 ■           BoxShadow(
25 ■             color: Color(0x40000000),
26 ■             blurRadius: 8,
27 ■             offset: Offset(0, 3),
28 ■           ),
29 ■         ],
30 ■       ),
31 ■       child: Center(
32 ■         child: SizedBox(
33 ■           width: size * 0.62,
34 ■           height: size * 0.52,
35 ■           child: CustomPaint(painter: _BusPainter()),
36 ■         ),
37 ■       ),
38 ■     );
39 ■   }
40 ■ }
41 ■
42 ■ class _BusPainter extends CustomPainter {
43 ■   @override
44 ■   void paint(Canvas canvas, Size s) {
45 ■     final double w = s.width;
46 ■     final double h = s.height;
47 ■
48 ■     // Body
49 ■     final RRect body = RRect.fromRectAndRadius(
50 ■       Rect.fromLTWH(0, h * 0.08, w, h * 0.64),
51 ■       Radius.circular(w * 0.14),
52 ■     );
53 ■     canvas.drawRRect(body, Paint()..color = NmbColors.accent);
54 ■
55 ■     // Windows
56 ■     final Paint windowPaint = Paint()..color = const Color(0xFFE3F2FD);
57 ■     final double winW = w * 0.20;
58 ■     final double winH = h * 0.22;

```

```
59 ■ for (int i = 0; i < 3; i++) {
60 ■   canvas.drawRRect(
61 ■     RRect.fromRectAndRadius(
62 ■       Rect.fromLTWH(
63 ■         w * 0.08 + i * (winW + w * 0.06),
64 ■         h * 0.18,
65 ■         winW,
66 ■         winH,
67 ■       ),
68 ■       Radius.circular(w * 0.04),
69 ■     ),
70 ■     windowPaint,
71 ■   );
72 ■ }
73 ■
74 ■ // Stripe
75 ■ canvas.drawRect(
76 ■   Rect.fromLTWH(0, h * 0.50, w, h * 0.05),
77 ■   Paint()..color = Colors.white,
78 ■ );
79 ■
80 ■ // Wheels
81 ■ final Paint wheelPaint = Paint()..color = const Color(0xFF37474F);
82 ■ canvas.drawCircle(Offset(w * 0.24, h * 0.82), h * 0.14, wheelPaint);
83 ■ canvas.drawCircle(Offset(w * 0.76, h * 0.82), h * 0.14, wheelPaint);
84 ■ }
85 ■
86 ■ @override
87 ■ bool shouldRepaint(covariant CustomPainter oldDelegate) => false;
88 ■ }
```

lib/features/student/screens/student_bus_details_screen.dart

159 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../models/bus.dart';
10 ■ import '../models/bus_route.dart';
11 ■ import '../models/bus_stop.dart';
12 ■ import '../models/trip.dart';
13 ■ import '../providers/data_providers.dart';
14 ■
15 ■ /// Bus & route details for the student's assigned bus. Read-only:
16 ■ /// students cannot change anything here by design.
17 ■ class StudentBusDetailsScreen extends ConsumerWidget {
18 ■   const StudentBusDetailsScreen({super.key});
19 ■
20 ■   @override
21 ■   Widget build(BuildContext context, WidgetRef ref) {
22 ■     final AsyncValue<Bus?> busAsync = ref.watch(myBusProvider);
23 ■     final BusRoute? route = ref.watch(myRouteProvider).valueOrNull;
24 ■     final List<BusStop> stops =
25 ■       ref.watch(myStopsProvider).valueOrNull ?? const <BusStop>[];
26 ■     final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
27 ■
28 ■     return Scaffold(
29 ■       appBar: AppBar(title: const Text('My Bus & Route')),
30 ■       body: busAsync.when(
31 ■         loading: () => const LoadingView(),
32 ■         error: (Object e, _) => ErrorView(
33 ■           message: 'Couldn\'t load bus details.',
34 ■           onRetry: () => ref.invalidate(myBusProvider),
35 ■         ),
36 ■         data: (Bus? bus) {
37 ■           if (bus == null) {
38 ■             return const EmptyState(
39 ■               icon: Icons.no_transfer_rounded,
40 ■               title: 'No bus assigned',
41 ■               message:
42 ■                 'A bus hasn\'t been assigned to your account yet. Please '
43 ■                 'contact the school office.',
44 ■             );
45 ■           }
46 ■           return ResponsiveBody(
47 ■             child: Column(
48 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
49 ■               children: <Widget>[
50 ■                 NmbCard(
51 ■                   child: Column(
52 ■                     children: <Widget>[
53 ■                       _InfoRow(
54 ■                         icon: Icons.directions_bus_rounded,
55 ■                         label: 'Bus number',
56 ■                         value: bus.busNumber,
57 ■                       ),
58 ■                       const Divider(height: 22),

```

```

59 ■         _InfoRow(
60 ■             icon: Icons.pin_outlined,
61 ■             label: 'Plate number',
62 ■             value: bus.plateNumber,
63 ■         ),
64 ■         const Divider(height: 22),
65 ■         _InfoRow(
66 ■             icon: Icons.event_seat_outlined,
67 ■             label: 'Capacity',
68 ■             value: '${bus.capacity} seats',
69 ■         ),
70 ■         const Divider(height: 22),
71 ■         _InfoRow(
72 ■             icon: Icons.play_circle_outline_rounded,
73 ■             label: 'Current status',
74 ■             value: trip != null
75 ■                 ? 'On a ${trip.direction.label.toLowerCase()} trip'
76 ■                 : 'No trip in progress',
77 ■             valueColor: trip != null
78 ■                 ? NmbColors.success
79 ■                 : NmbColors.textSecondary,
80 ■         ),
81 ■     ],
82 ■ ),
83 ■ ),
84 ■ const SizedBox(height: 16),
85 ■ if (route != null) ...<Widget>[
86 ■     const Text('Route', style: NmbTypography.sectionTitle),
87 ■     const SizedBox(height: 10),
88 ■     NmbCard(
89 ■         child: Column(
90 ■             children: <Widget>[
91 ■                 _InfoRow(
92 ■                     icon: Icons.route_rounded,
93 ■                     label: 'Route name',
94 ■                     value: route.name,
95 ■                 ),
96 ■                 const Divider(height: 22),
97 ■                 _InfoRow(
98 ■                     icon: Icons.pin_drop_outlined,
99 ■                     label: 'Stops on route',
100 ■                     value: '${stops.length}',
101 ■                 ),
102 ■                 const Divider(height: 22),
103 ■                 _InfoRow(
104 ■                     icon: Icons.swap_vert_rounded,
105 ■                     label: 'Direction',
106 ■                     value: route.direction.label,
107 ■                 ),
108 ■             ],
109 ■         ),
110 ■     ),
111 ■ ] else
112 ■     const NmbCard(
113 ■         child: Text(
114 ■             'Route details haven\'t been published yet.',
115 ■             style: NmbTypography.bodySecondary,
116 ■         ),

```

```

117 ■         ),
118 ■         const SizedBox(height: 24),
119 ■       ],
120 ■     ),
121 ■   );
122 ■ },
123 ■ ),
124 ■ );
125 ■ }
126 ■ }
127 ■
128 ■ class _InfoRow extends StatelessWidget {
129 ■   const _InfoRow({
130 ■     required this.icon,
131 ■     required this.label,
132 ■     required this.value,
133 ■     this.valueColor,
134 ■   });
135 ■
136 ■   final IconData icon;
137 ■   final String label;
138 ■   final String value;
139 ■   final Color? valueColor;
140 ■
141 ■   @override
142 ■   Widget build(BuildContext context) {
143 ■     return Row(
144 ■       children: <Widget>[
145 ■         Icon(icon, size: 22, color: NmbColors.primary),
146 ■         const SizedBox(width: 12),
147 ■         Expanded(
148 ■           child: Text(label, style: NmbTypography.bodySecondary),
149 ■         ),
150 ■         Text(
151 ■           value,
152 ■           style: NmbTypography.cardTitle.copyWith(
153 ■             color: valueColor ?? NmbColors.textPrimary,
154 ■           ),
155 ■         ),
156 ■       ],
157 ■     );
158 ■   }
159 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../models/bus_stop.dart';
10 ■ import '../providers/data_providers.dart';
11 ■
12 ■ /// Ordered list of the route's stops as a friendly timeline; the student's
13 ■ /// assigned stop is highlighted with a star.
14 ■ class StudentStopsScreen extends ConsumerWidget {
15 ■   const StudentStopsScreen({super.key});
16 ■
17 ■   @override
18 ■   Widget build(BuildContext context, WidgetRef ref) {
19 ■     final AsyncValue<List<BusStop>> stopsAsync = ref.watch(myStopsProvider);
20 ■     final BusStop? myStop = ref.watch(myAssignedStopProvider);
21 ■
22 ■     return Scaffold(
23 ■       appBar: AppBar(title: const Text('Bus Stops')),
24 ■       body: stopsAsync.when(
25 ■         loading: () => const LoadingView(),
26 ■         error: (Object e, _) => ErrorView(
27 ■           message: 'Couldn\'t load the stop list.',
28 ■           onRetry: () => ref.invalidate(myStopsProvider),
29 ■         ),
30 ■         data: (List<BusStop> stops) {
31 ■           if (stops.isEmpty) {
32 ■             return const EmptyState(
33 ■               icon: Icons.pin_drop_outlined,
34 ■               title: 'No stops yet',
35 ■               message: 'The stop list for your route hasn\'t been '
36 ■                 'published yet.',
37 ■             );
38 ■           }
39 ■           return ResponsiveBody(
40 ■             child: Column(
41 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
42 ■               children: <Widget>[
43 ■                 for (int i = 0; i < stops.length; i++) ...<Widget>[
44 ■                   _StopTile(
45 ■                     stop: stops[i],
46 ■                     index: i,
47 ■                     isLast: i == stops.length - 1,
48 ■                     isMine: stops[i].id == myStop?.id,
49 ■                   ),
50 ■                 ],
51 ■                 const SizedBox(height: 16),
52 ■                 const NmbCard(
53 ■                   color: NmbColors.infoSoft,
54 ■                   child: Row(
55 ■                     children: <Widget>[
56 ■                       Icon(Icons.info_outline_rounded,
57 ■                         color: NmbColors.info,),
58 ■                       SizedBox(width: 10),

```

```

59 ■         Expanded(
60 ■             child: Text(
61 ■                 'Your pickup stop is set by the school. To change '
62 ■                 'it, please contact the school office.',
63 ■                 style: NmbTypography.bodySecondary,
64 ■             ),
65 ■         ),
66 ■     ],
67 ■ ),
68 ■ ),
69 ■     const SizedBox(height: 24),
70 ■ ],
71 ■ ),
72 ■ );
73 ■ },
74 ■ ),
75 ■ );
76 ■ }
77 ■ }
78 ■
79 ■ class _StopTile extends StatelessWidget {
80 ■     const _StopTile({
81 ■         required this.stop,
82 ■         required this.index,
83 ■         required this.isLast,
84 ■         required this.isMine,
85 ■     });
86 ■
87 ■     final BusStop stop;
88 ■     final int index;
89 ■     final bool isLast;
90 ■     final bool isMine;
91 ■
92 ■     @override
93 ■     Widget build(BuildContext context) {
94 ■         return IntrinsicHeight(
95 ■             child: Row(
96 ■                 crossAxisAlignment: CrossAxisAlignment.stretch,
97 ■                 children: <Widget>[
98 ■                     // Timeline rail
99 ■                     SizedBox(
100 ■                         width: 36,
101 ■                         child: Column(
102 ■                             children: <Widget>[
103 ■                                 Container(
104 ■                                     width: 26,
105 ■                                     height: 26,
106 ■                                     decoration: BoxDecoration(
107 ■                                         color: isMine ? NmbColors.accent : NmbColors.primarySoft,
108 ■                                         shape: BoxShape.circle,
109 ■                                     ),
110 ■                                     child: Center(
111 ■                                         child: isMine
112 ■                                             ? const Icon(Icons.star_rounded,
113 ■                                                 size: 16, color: Colors.white,)
114 ■                                             : Text(
115 ■                                                 '${index + 1}',
116 ■                                                 style: NmbTypography.caption.copyWith(

```

```

117 ■                                     color: NmbColors.primaryDark,
118 ■                                     fontWeight: FontWeight.w800,
119 ■                                   ),
120 ■                                 ),
121 ■                               ),
122 ■                             ),
123 ■                           if (!isLast)
124 ■                             Expanded(
125 ■                               child: Container(
126 ■                                 width: 2.4,
127 ■                                 color: NmbColors.divider,
128 ■                               ),
129 ■                             ),
130 ■                           ],
131 ■                         ),
132 ■                       ),
133 ■                     const SizedBox(width: 10),
134 ■                   Expanded(
135 ■                     child: Padding(
136 ■                       padding: const EdgeInsets.only(bottom: 12),
137 ■                       child: NmbCard(
138 ■                         color: isMine ? NmbColors.accentSoft : NmbColors.surface,
139 ■                         borderColor: isMine ? NmbColors.accent : null,
140 ■                         padding: const EdgeInsets.all(14),
141 ■                         child: Row(
142 ■                           children: <Widget>[
143 ■                             Expanded(
144 ■                               child: Column(
145 ■                                 crossAxisAlignment: CrossAxisAlignment.start,
146 ■                                 children: <Widget>[
147 ■                                   Text(stop.name, style: NmbTypography.cardTitle),
148 ■                                   if (isMine)
149 ■                                     Text(
150 ■                                       'My assigned stop',
151 ■                                       style: NmbTypography.caption.copyWith(
152 ■                                         color: NmbColors.accentDark,
153 ■                                         fontWeight: FontWeight.w700,
154 ■                                       ),
155 ■                                     ),
156 ■                                 ],
157 ■                               ),
158 ■                             ),
159 ■                           if (stop.scheduledTime != null)
160 ■                             Container(
161 ■                               padding: const EdgeInsets.symmetric(
162 ■                                 horizontal: 10,
163 ■                                 vertical: 5,
164 ■                               ),
165 ■                               decoration: BoxDecoration(
166 ■                                 color: NmbColors.surface,
167 ■                                 borderRadius: BorderRadius.circular(10),
168 ■                                 border: Border.all(color: NmbColors.divider),
169 ■                               ),
170 ■                               child: Text(
171 ■                                 stop.scheduledTime!,
172 ■                                 style: NmbTypography.caption.copyWith(
173 ■                                   color: NmbColors.textSecondary,
174 ■                                   fontWeight: FontWeight.w700,
175 ■                                 ),
176 ■                               ),
177 ■                             ),
178 ■                           ],
179 ■                         ),
180 ■                       ),
181 ■                     ),
182 ■                   ),
183 ■                 ],
184 ■               ),
185 ■             );
186 ■   }
187 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/constants/enums.dart';
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/utils/formatters.dart';
8 ■ import '../core/widgets/nmb_card.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/widgets/skeleton.dart';
11 ■ import '../core/widgets/state_views.dart';
12 ■ import '../models/app_notification.dart';
13 ■ import '../providers/app_providers.dart';
14 ■ import '../providers/data_providers.dart';
15 ■ import '../services/auth_service.dart';
16 ■
17 ■ /// Notification inbox. Items are fanned out server-side per user, so this
18 ■ /// list can only ever contain notifications addressed to this student.
19 ■ class StudentAlertsScreen extends ConsumerWidget {
20 ■   const StudentAlertsScreen({super.key});
21 ■
22 ■   @override
23 ■   Widget build(BuildContext context, WidgetRef ref) {
24 ■     final AsyncValue<List<AppNotification>> inboxAsync =
25 ■       ref.watch(inboxProvider);
26 ■
27 ■     return Scaffold(
28 ■       appBar: AppBar(title: const Text('Alerts')),
29 ■       body: inboxAsync.when(
30 ■         loading: () => const ListSkeleton(),
31 ■         error: (Object e, _) => ErrorView(
32 ■           message: 'Couldn\'t load alerts.',
33 ■           onRetry: () => ref.invalidate(inboxProvider),
34 ■         ),
35 ■         data: (List<AppNotification> items) {
36 ■           if (items.isEmpty) {
37 ■             return const EmptyState(
38 ■               icon: Icons.notifications_none_rounded,
39 ■               title: 'No alerts yet',
40 ■               message: 'Bus updates and school announcements will '
41 ■                 'appear here.',
42 ■             );
43 ■           }
44 ■           return ResponsiveBody(
45 ■             child: Column(
46 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
47 ■               children: <Widget>[
48 ■                 for (final AppNotification n in items) ...<Widget>[
49 ■                   _AlertTile(notification: n),
50 ■                   const SizedBox(height: 10),
51 ■                 ],
52 ■                 const SizedBox(height: 16),
53 ■               ],
54 ■             ),
55 ■           );
56 ■         },
57 ■       ),
58 ■     );

```

```

59 ■ }
60 ■ }
61 ■
62 ■ class _AlertTile extends ConsumerWidget {
63 ■   const _AlertTile({required this.notification});
64 ■
65 ■   final AppNotification notification;
66 ■
67 ■   (IconData, Color, Color) get _style => switch (notification.type) {
68 ■     NotificationType.tripStarted => (
69 ■       Icons.play_circle_rounded,
70 ■       NmbColors.success,
71 ■       NmbColors.successSoft
72 ■     ),
73 ■     NotificationType.busApproaching => (
74 ■       Icons.near_me_rounded,
75 ■       NmbColors.primary,
76 ■       NmbColors.primarySoft
77 ■     ),
78 ■     NotificationType.busReachedStop => (
79 ■       Icons.pin_drop_rounded,
80 ■       NmbColors.accentDark,
81 ■       NmbColors.accentSoft
82 ■     ),
83 ■     NotificationType.busReachedSchool => (
84 ■       Icons.school_rounded,
85 ■       NmbColors.info,
86 ■       NmbColors.infoSoft
87 ■     ),
88 ■     NotificationType.busDelayed => (
89 ■       Icons.schedule_rounded,
90 ■       NmbColors.warning,
91 ■       NmbColors.warningSoft
92 ■     ),
93 ■     NotificationType.trackingUnavailable => (
94 ■       Icons.location_off_rounded,
95 ■       NmbColors.textTertiary,
96 ■       NmbColors.divider
97 ■     ),
98 ■     NotificationType.announcement => (
99 ■       Icons.campaign_rounded,
100 ■       NmbColors.primary,
101 ■       NmbColors.primarySoft
102 ■     ),
103 ■   };
104 ■
105 ■ @override
106 ■ Widget build(BuildContext context, WidgetRef ref) {
107 ■   final (IconData icon, Color color, Color bg) = _style;
108 ■   final AuthSession? session = ref.watch(currentSessionProvider);
109 ■
110 ■   return NmbCard(
111 ■     onTap: notification.read || session == null
112 ■       ? null
113 ■       : () => ref
114 ■         .read(firestoreServiceProvider)
115 ■         .markInboxRead(session.uid, notification.id),
116 ■     borderColor:

```

```

117 ■      notification.read ? null : NmbColors.primary.withOpacity(0.35),
118 ■    child: Row(
119 ■      crossAxisAlignment: CrossAxisAlignment.start,
120 ■      children: <Widget>[
121 ■        Container(
122 ■          width: 42,
123 ■          height: 42,
124 ■          decoration: BoxDecoration(
125 ■            color: bg,
126 ■            borderRadius: BorderRadius.circular(12),
127 ■          ),
128 ■          child: Icon(icon, color: color, size: 22),
129 ■        ),
130 ■        const SizedBox(width: 12),
131 ■        Expanded(
132 ■          child: Column(
133 ■            crossAxisAlignment: CrossAxisAlignment.start,
134 ■            children: <Widget>[
135 ■              Row(
136 ■                children: <Widget>[
137 ■                  Expanded(
138 ■                    child: Text(
139 ■                      notification.title,
140 ■                      style: NmbTypography.cardTitle.copyWith(
141 ■                        fontWeight: notification.read
142 ■                          ? FontWeight.w500
143 ■                          : FontWeight.w700,
144 ■                    ),
145 ■                  ),
146 ■                ],
147 ■                if (!notification.read)
148 ■                  Container(
149 ■                    width: 9,
150 ■                    height: 9,
151 ■                    decoration: const BoxDecoration(
152 ■                      color: NmbColors.primary,
153 ■                      shape: BoxShape.circle,
154 ■                    ),
155 ■                  ),
156 ■              ],
157 ■            ),
158 ■            const SizedBox(height: 3),
159 ■            Text(notification.body, style: NmbTypography.bodySecondary),
160 ■            const SizedBox(height: 6),
161 ■            Text(
162 ■              Formatters.relativeTime(notification.receivedAt),
163 ■              style: NmbTypography.caption,
164 ■            ),
165 ■          ],
166 ■        ),
167 ■      ],
168 ■    ),
169 ■  ),
170 ■ );
171 ■ }
172 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■
5 ■ import '../core/constants/nmb_constants.dart';
6 ■ import '../core/theme/nmb_colors.dart';
7 ■ import '../core/theme/nmb_typography.dart';
8 ■ import '../core/widgets/nmb_card.dart';
9 ■ import '../core/widgets/nmb_dialogs.dart';
10 ■ import '../core/widgets/state_views.dart';
11 ■ import '../models/app_user.dart';
12 ■ import '../models/bus.dart';
13 ■ import '../models/bus_stop.dart';
14 ■ import '../providers/app_providers.dart';
15 ■ import '../providers/data_providers.dart';
16 ■ import '../services/auth_service.dart';
17 ■
18 ■ /// v1.1.1 Profile – blue gradient header with big avatar + name + class,
19 ■ /// then info cards (assignment, school contact) and actions.
20 ■ class StudentProfileScreen extends ConsumerWidget {
21 ■   const StudentProfileScreen({super.key});
22 ■
23 ■   Future<void> _logout(BuildContext context, WidgetRef ref) async {
24 ■     final AuthSession? session = ref.read(currentSessionProvider);
25 ■     // Capture EVERYTHING from ref BEFORE any await/pop – after the widget
26 ■     // is disposed, ref must not be touched ("ref after disposed" fix).
27 ■     final notifSvc = ref.read(notificationServiceProvider);
28 ■     final devSvc = ref.read(deviceSessionServiceProvider);
29 ■     final fsSvc = ref.read(firestoreServiceProvider);
30 ■     final authSvc = ref.read(authServiceProvider);
31 ■     final bool confirmed = await showNmbConfirmDialog(
32 ■       context,
33 ■       title: 'Log out?',
34 ■       message: 'You will need your Class + Roll number + password to sign back in.',
35 ■       confirmLabel: 'Log out',
36 ■       destructive: true,
37 ■       icon: Icons.logout_rounded,
38 ■     );
39 ■     if (!confirmed) return;
40 ■     if (session != null) {
41 ■       try {
42 ■         await notifSvc.unregisterDevice(session.uid);
43 ■       } catch (_) {/* best effort */}
44 ■       try {
45 ■         final String devId = await devSvc.getDeviceId();
46 ■         await fsSvc.clearActiveDevice(
47 ■           uid: session.uid,
48 ■           deviceId: devId,
49 ■         );
50 ■       } catch (_) {/* best effort */}
51 ■     }
52 ■     await authSvc.signOut();
53 ■   }
54 ■
55 ■   @override
56 ■   Widget build(BuildContext context, WidgetRef ref) {
57 ■     final AsyncValue<AppUser?> profileAsync = ref.watch(myProfileProvider);
58 ■     final Bus? bus = ref.watch(myBusProvider).valueOrNull;

```

```

59 ■ final BusStop? stop = ref.watch(myAssignedStopProvider);
60 ■ final Map<String, dynamic>? school =
61 ■     ref.watch(schoolConfigProvider).valueOrNull;
62 ■
63 ■ return Scaffold(
64 ■     body: profileAsync.when(
65 ■         loading: () => const LoadingView(),
66 ■         error: (Object e, _) => ErrorView(
67 ■             message: 'Couldn\'t load your profile.',
68 ■             onRetry: () => ref.invalidate(myProfileProvider),
69 ■         ),
70 ■     data: (AppUser? me) {
71 ■         if (me == null) {
72 ■             return const ErrorView(
73 ■                 title: 'Profile unavailable',
74 ■                 message: 'Please contact the school office.',
75 ■             );
76 ■         }
77 ■         return CustomScrollView(
78 ■             physics: const BouncingScrollPhysics(),
79 ■             slivers: <Widget>[
80 ■                 // ■■ Header ■■
81 ■                 SliverToBoxAdapter(
82 ■                     child: Container(
83 ■                         width: double.infinity,
84 ■                         decoration: const BoxDecoration(
85 ■                             gradient: LinearGradient(
86 ■                                 begin: Alignment.topLeft,
87 ■                                 end: Alignment.bottomRight,
88 ■                                 colors: <Color>[
89 ■                                     NmbColors.primaryDark,
90 ■                                     NmbColors.primary,
91 ■                                 ],
92 ■                             ),
93 ■                         borderRadius: BorderRadius.only(
94 ■                             bottomLeft: Radius.circular(28),
95 ■                             bottomRight: Radius.circular(28),
96 ■                         ),
97 ■                     ),
98 ■                     padding: const EdgeInsets.fromLTRB(20, 8, 20, 28),
99 ■                     child: SafeArea(
100 ■                         bottom: false,
101 ■                         child: Column(
102 ■                             children: <Widget>[
103 ■                                 Row(
104 ■                                     children: <Widget>[
105 ■                                         Text('Profile',
106 ■                                             style: NmbTypography.screenTitle
107 ■                                                 .copyWith(color: Colors.white),),
108 ■                                         const Spacer(),
109 ■                                         IconButton(
110 ■                                             tooltip: 'Settings',
111 ■                                             onPressed: () =>
112 ■                                                 context.go('/student/profile/settings'),
113 ■                                             icon: const Icon(Icons.settings_outlined,
114 ■                                                 color: Colors.white,),
115 ■                                         ),
116 ■                                     ],

```

```

117 ■      ),
118 ■      const SizedBox(height: 8),
119 ■      Stack(
120 ■        children: <Widget>[
121 ■          CircleAvatar(
122 ■            radius: 46,
123 ■            backgroundColor: Colors.white,
124 ■            child: CircleAvatar(
125 ■              radius: 42,
126 ■              backgroundColor: NmbColors.primarySoft,
127 ■              child: Text(
128 ■                me.firstName.isNotEmpty
129 ■                  ? me.firstName[0].toUpperCase()
130 ■                  : '?',
131 ■                style: NmbTypography.displayTitle
132 ■                  .copyWith(color: NmbColors.primary),
133 ■              ),
134 ■            ),
135 ■          ),
136 ■          Positioned(
137 ■            bottom: 5,
138 ■            right: 5,
139 ■            child: Container(
140 ■              width: 16,
141 ■              height: 16,
142 ■              decoration: BoxDecoration(
143 ■                color: NmbColors.success,
144 ■                shape: BoxShape.circle,
145 ■                border: Border.all(
146 ■                  color: Colors.white, width: 2,),
147 ■              ),
148 ■            ),
149 ■          ),
150 ■        ],
151 ■      ),
152 ■      const SizedBox(height: 12),
153 ■      Text(
154 ■        me.fullName,
155 ■        style: NmbTypography.screenTitle
156 ■          .copyWith(color: Colors.white),
157 ■      ),
158 ■      if (me.classSection != null)
159 ■        Text(
160 ■          'Class ${me.classSection}'
161 ■          '${me.rollNumber != null ? ' • Roll ${me.rollNumber}' : ''}',
162 ■          style: NmbTypography.bodySecondary
163 ■            .copyWith(color: Colors.white70),
164 ■        ),
165 ■        Text(
166 ■          NmbConstants.schoolName,
167 ■          style: NmbTypography.caption
168 ■            .copyWith(color: Colors.white60),
169 ■        ),
170 ■      ],
171 ■    ),
172 ■  ),
173 ■ ),
174 ■

```

```

175 ■
176 ■ SliverPadding(
177 ■   padding: const EdgeInsets.fromLTRB(20, 16, 20, 24),
178 ■   sliver: SliverList(
179 ■     delegate: SliverChildListDelegate(<Widget>[
180 ■       // ■■ Assignment ■■
181 ■       NmbCard(
182 ■         child: Column(
183 ■           children: <Widget>[
184 ■             ListTile(
185 ■               contentPadding: EdgeInsets.zero,
186 ■               leading: Container(
187 ■                 width: 42,
188 ■                 height: 42,
189 ■                 decoration: BoxDecoration(
190 ■                   color: NmbColors.accentSoft,
191 ■                   borderRadius: BorderRadius.circular(12),
192 ■                 ),
193 ■               child: const Icon(
194 ■                 Icons.directions_bus_rounded,
195 ■                 color: NmbColors.accentDark,),
196 ■             ),
197 ■             title: const Text('Assigned bus'),
198 ■             trailing: Text(
199 ■               bus?.busNumber ?? 'Not assigned',
200 ■               style: NmbTypography.cardTitle,
201 ■             ),
202 ■           ],
203 ■         const Divider(),
204 ■         ListTile(
205 ■           contentPadding: EdgeInsets.zero,
206 ■           leading: Container(
207 ■             width: 42,
208 ■             height: 42,
209 ■             decoration: BoxDecoration(
210 ■               color: const Color(0xFFE9E9E9),
211 ■               borderRadius: BorderRadius.circular(12),
212 ■             ),
213 ■             child: const Icon(Icons.pin_drop_rounded,
214 ■               color: Color(0xFF7B61FF)),
215 ■           ),
216 ■           title: const Text('Assigned stop'),
217 ■           trailing: Text(
218 ■             stop?.name ?? 'Not assigned',
219 ■             style: NmbTypography.cardTitle,
220 ■           ),
221 ■         ),
222 ■         const SizedBox(height: 6),
223 ■         const Text(
224 ■           'Bus and stop assignments are managed by the school.',
225 ■           style: NmbTypography.caption,
226 ■         ),
227 ■       ],
228 ■     ),
229 ■   ),
230 ■   const SizedBox(height: 14),
231 ■
232 ■   // ■■ School contact ■■

```

```

233 ■      NmbCard(
234 ■        child: Column(
235 ■          crossAxisAlignment: CrossAxisAlignment.start,
236 ■          children: <Widget>[
237 ■            const Text('Contact School',
238 ■              style: NmbTypography.sectionTitle,),
239 ■            const SizedBox(height: 10),
240 ■            _ContactRow(
241 ■              icon: Icons.school_rounded,
242 ■              text: (school?['name'] as String?) ??
243 ■                NmbConstants.schoolName,
244 ■            ),
245 ■            if (school?['phone'] != null)
246 ■              _ContactRow(
247 ■                icon: Icons.call_rounded,
248 ■                text: school!['phone'] as String,
249 ■              ),
250 ■            if (school?['email'] != null)
251 ■              _ContactRow(
252 ■                icon: Icons.mail_rounded,
253 ■                text: school!['email'] as String,
254 ■              ),
255 ■            if (school?['address'] != null)
256 ■              _ContactRow(
257 ■                icon: Icons.location_on_rounded,
258 ■                text: school!['address'] as String,
259 ■              ),
260 ■          ],
261 ■        ),
262 ■      ),
263 ■      const SizedBox(height: 14),
264 ■
265 ■      // ■■ Actions ■■
266 ■      NmbCard(
267 ■        padding: const EdgeInsets.symmetric(
268 ■          horizontal: 14, vertical: 4, ),
269 ■        child: Column(
270 ■          children: <Widget>[
271 ■            ListTile(
272 ■              contentPadding: EdgeInsets.zero,
273 ■              leading: const Icon(Icons.settings_outlined,
274 ■                color: NmbColors.textSecondary, ),
275 ■              title: const Text('Settings'),
276 ■              trailing:
277 ■                const Icon(Icons.chevron_right_rounded),
278 ■              onTap: () =>
279 ■                context.go('/student/profile/settings'),
280 ■            ),
281 ■            const Divider(height: 1),
282 ■            ListTile(
283 ■              contentPadding: EdgeInsets.zero,
284 ■              leading: const Icon(Icons.info_outline_rounded,
285 ■                color: NmbColors.textSecondary, ),
286 ■              title: const Text('About App'),
287 ■              trailing:
288 ■                const Icon(Icons.chevron_right_rounded),
289 ■              onTap: () => showAboutDialog(
290 ■                context: context,

```

```

291 ■         applicationName: NmbConstants.appName,
292 ■         applicationVersion: 'v1.1.1',
293 ■         applicationLegalese:
294 ■             '${NmbConstants.schoolName} – secure '
295 ■             'school bus tracking.',
296 ■     ),
297 ■ ),
298 ■ ],
299 ■ ),
300 ■ ),
301 ■ const SizedBox(height: 20),
302 ■
303 ■ OutlinedButton.icon(
304 ■     style: OutlinedButton.styleFrom(
305 ■         foregroundColor: NmbColors.danger,
306 ■         side: const BorderSide(color: NmbColors.danger),
307 ■     ),
308 ■     onPressed: () => _logout(context, ref),
309 ■     icon: const Icon(Icons.logout_rounded),
310 ■     label: const Text('Logout'),
311 ■ ),
312 ■ const SizedBox(height: 24),
313 ■ ],
314 ■ ),
315 ■ ),
316 ■ ],
317 ■ );
318 ■ },
319 ■ ),
320 ■ );
321 ■ }
322 ■ }
323 ■
324 ■ class _ContactRow extends StatelessWidget {
325 ■     const _ContactRow({required this.icon, required this.text});
326 ■
327 ■     final IconData icon;
328 ■     final String text;
329 ■
330 ■     @override
331 ■     Widget build(BuildContext context) {
332 ■         return Padding(
333 ■             padding: const EdgeInsets.only(bottom: 8),
334 ■             child: Row(
335 ■                 children: <Widget>[
336 ■                     Icon(icon, size: 18, color: NmbColors.textTertiary),
337 ■                     const SizedBox(width: 10),
338 ■                     Expanded(child: Text(text, style: NmbTypography.bodySecondary)),
339 ■                 ],
340 ■             ),
341 ■         );
342 ■     }
343 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:permission_handler/permission_handler.dart';
4 ■
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/utils/result.dart';
11 ■ import '../providers/app_providers.dart';
12 ■ import '../services/auth_service.dart';
13 ■
14 ■ /// Notification preferences + permission helper + password reset.
15 ■ class StudentSettingsScreen extends ConsumerStatefulWidget {
16 ■   const StudentSettingsScreen({super.key});
17 ■
18 ■   @override
19 ■   ConsumerState<StudentSettingsScreen> createState() =>
20 ■     _StudentSettingsScreenState();
21 ■ }
22 ■
23 ■ class _StudentSettingsScreenState
24 ■   extends ConsumerState<StudentSettingsScreen> {
25 ■     bool _tripAlerts = true;
26 ■     bool _approachingAlerts = true;
27 ■     bool _announcementAlerts = true;
28 ■     PermissionStatus? _notifPermission;
29 ■
30 ■     @override
31 ■     void initState() {
32 ■       super.initState();
33 ■       Permission.notification.status.then((PermissionStatus s) {
34 ■         if (mounted) setState(() => _notifPermission = s);
35 ■       });
36 ■     }
37 ■
38 ■     Future<void> _saveSettings() async {
39 ■       final AuthSession? session = ref.read(currentSessionProvider);
40 ■       if (session == null) return;
41 ■       await ref.read(firestoreServiceProvider).saveUserSettings(
42 ■         session.uid,
43 ■         <String, dynamic>{
44 ■           'tripAlerts': _tripAlerts,
45 ■           'approachingAlerts': _approachingAlerts,
46 ■           'announcementAlerts': _announcementAlerts,
47 ■         },
48 ■       );
49 ■       if (mounted) showNmbSnack(context, 'Settings saved', isSuccess: true);
50 ■     }
51 ■
52 ■     Future<void> _requestNotifPermission() async {
53 ■       final PermissionStatus status = await Permission.notification.request();
54 ■       if (!mounted) return;
55 ■       setState(() => _notifPermission = status);
56 ■       if (status.isPermanentlyDenied) {
57 ■         final bool open = await showNmbConfirmDialog(
58 ■           context,

```

```

59 ■         title: 'Enable notifications',
60 ■         message: 'Notifications are turned off for this app. Open system '
61 ■             'settings to enable bus alerts?',
62 ■         confirmLabel: 'Open Settings',
63 ■     );
64 ■     if (open) await openAppSettings();
65 ■ }
66 ■ }
67 ■
68 ■ Future<void> _resetPassword() async {
69 ■     final AuthSession? session = ref.read(currentSessionProvider);
70 ■     if (session == null) return;
71 ■     final Result<void> result = await ref
72 ■         .read(authServiceProvider)
73 ■         .sendPasswordReset(session.email);
74 ■     if (!mounted) return;
75 ■     result.when(
76 ■         ok: (_) => showNmbSnack(
77 ■             context,
78 ■             'Password reset email sent to ${session.email}',
79 ■             isSuccess: true,
80 ■         ),
81 ■         err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
82 ■     );
83 ■ }
84 ■
85 ■ @override
86 ■ Widget build(BuildContext context) {
87 ■     final bool notifDenied =
88 ■         _notifPermission != null && !_notifPermission!.isGranted;
89 ■
90 ■     return Scaffold(
91 ■         appBar: AppBar(title: const Text('Settings')),
92 ■         body: ResponsiveBody(
93 ■             child: Column(
94 ■                 crossAxisAlignment: CrossAxisAlignment.stretch,
95 ■                 children: <Widget>[
96 ■                     if (notifDenied) ...<Widget>[
97 ■                         NmbCard(
98 ■                             color: NmbColors.warningSoft,
99 ■                             child: Column(
100 ■                                 crossAxisAlignment: CrossAxisAlignment.start,
101 ■                                 children: <Widget>[
102 ■                                     const Row(
103 ■                                         children: <Widget>[
104 ■                                             Icon(Icons.notifications_off_rounded,
105 ■                                                 color: NmbColors.warning,),
106 ■                                             SizedBox(width: 10),
107 ■                                             Expanded(
108 ■                                                 child: Text(
109 ■                                                     'Notifications are off – you won\'t receive '
110 ■                                                     'bus alerts.',
111 ■                                                     style: NmbTypography.bodySecondary,
112 ■                                                 ),
113 ■                                             ),
114 ■                                         ],
115 ■                                     ),
116 ■                                     const SizedBox(height: 10),

```

```

117 ■         OutlinedButton(
118 ■             onPressed: _requestNotifPermission,
119 ■             child: const Text('Enable notifications'),
120 ■         ),
121 ■     ],
122 ■ ),
123 ■ ),
124 ■     const SizedBox(height: 14),
125 ■ ],
126 ■ const Text('Notification preferences',
127 ■     style: NmbTypography.sectionTitle,),
128 ■ const SizedBox(height: 10),
129 ■ NmbCard(
130 ■     padding: const EdgeInsets.symmetric(
131 ■         horizontal: 12, vertical: 4,),
132 ■     child: Column(
133 ■         children: <Widget>[
134 ■             SwitchListTile(
135 ■                 title: const Text('Trip started / ended'),
136 ■                 value: _tripAlerts,
137 ■                 onChanged: (bool v) =>
138 ■                     setState(() => _tripAlerts = v),
139 ■             ),
140 ■             SwitchListTile(
141 ■                 title: const Text('Bus approaching my stop'),
142 ■                 value: _approachingAlerts,
143 ■                 onChanged: (bool v) =>
144 ■                     setState(() => _approachingAlerts = v),
145 ■             ),
146 ■             SwitchListTile(
147 ■                 title: const Text('School announcements'),
148 ■                 value: _announcementAlerts,
149 ■                 onChanged: (bool v) =>
150 ■                     setState(() => _announcementAlerts = v),
151 ■             ),
152 ■         ],
153 ■     ),
154 ■ ),
155 ■ const SizedBox(height: 12),
156 ■ FilledButton(
157 ■     onPressed: _saveSettings,
158 ■     child: const Text('Save preferences'),
159 ■ ),
160 ■ const SizedBox(height: 24),
161 ■ const Text('Account', style: NmbTypography.sectionTitle),
162 ■ const SizedBox(height: 10),
163 ■ NmbCard(
164 ■     onTap: _resetPassword,
165 ■     child: const Row(
166 ■         children: <Widget>[
167 ■             Icon(Icons.lock_reset_rounded, color: NmbColors.primary),
168 ■             SizedBox(width: 12),
169 ■             Expanded(
170 ■                 child: Text('Send password reset email',
171 ■                     style: NmbTypography.cardTitle,),
172 ■             ),
173 ■             Icon(Icons.chevron_right_rounded,
174 ■                 color: NmbColors.textTertiary,),
175 ■         ],
176 ■     ),
177 ■ ),
178 ■ const SizedBox(height: 24),
179 ■ ],
180 ■ ),
181 ■ ),
182 ■ );
183 ■ }
184 ■ }

```

9. Driver App

Shell, one-tap trip dashboard, route, profile.

lib/features/driver/driver_shell.dart

74 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter/services.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■ import 'package:go_router/go_router.dart';
5 ■
6 ■ import '../core/widgets/state_views.dart';
7 ■ import '../providers/app_providers.dart';
8 ■
9 ■ /// Driver shell: three tabs only – deliberately minimal so the interface is
10 ■ safe to use around a vehicle.
11 ■ ///
12 ■ /// Back button: sub-page → pop; doosra tab → Trip tab; Trip tab → exit.
13 ■ class DriverShell extends ConsumerWidget {
14 ■   const DriverShell({super.key, required this.shell});
15 ■
16 ■   final StatefulNavigationShell shell;
17 ■
18 ■   void _handleBack(BuildContext context) {
19 ■     final GoRouter router = GoRouter.of(context);
20 ■     if (router.canPop()) {
21 ■       router.pop();
22 ■       return;
23 ■     }
24 ■     if (shell.currentIndex != 0) {
25 ■       shell.goBranch(0);
26 ■       return;
27 ■     }
28 ■     SystemNavigator.pop();
29 ■   }
30 ■
31 ■   @override
32 ■   Widget build(BuildContext context, WidgetRef ref) {
33 ■     final bool online = ref.watch(isOnlineProvider).valueOrNull ?? true;
34 ■
35 ■     return PopScope(
36 ■       canPop: false,
37 ■       onPopInvokedWithResult: (bool didPop, Object? result) {
38 ■         if (!didPop) _handleBack(context);
39 ■       },
40 ■       child: Scaffold(
41 ■         body: Column(
42 ■           children: <Widget>[
43 ■             if (!online) const OfflineBanner(),
44 ■             Expanded(child: shell),
45 ■           ],
46 ■         ),
47 ■         bottomNavigationBar: NavigationBar(
48 ■           selectedIndex: shell.currentIndex,
49 ■           onDestinationSelected: (int index) => shell.goBranch(
50 ■             index,
51 ■             initialLocation: index == shell.currentIndex,
52 ■           ),
53 ■           destinations: const <Widget>[
54 ■             NavigationDestination(
55 ■               icon: Icon(Icons.directions_bus_outlined),
56 ■               selectedIcon: Icon(Icons.directions_bus_rounded),
57 ■               label: 'Trip',
58 ■             ),

```

```

59 ■      NavigationDestination(
60 ■          icon: Icon(Icons.route_outlined),
61 ■          selectedIcon: Icon(Icons.route_rounded),
62 ■          label: 'Route',
63 ■      ),
64 ■      NavigationDestination(
65 ■          icon: Icon(Icons.person_outline_rounded),
66 ■          selectedIcon: Icon(Icons.person_rounded),
67 ■          label: 'Profile',
68 ■      ),
69 ■  ],
70 ■ ),
71 ■ ),
72 ■ );
73 ■ }
74 ■ }

```

lib/features/driver/screens/driver_home_screen.dart

418 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:geolocator/geolocator.dart';
4 ■
5 ■ import '../core/constants/enums.dart';
6 ■ import '../core/theme/nmb_colors.dart';
7 ■ import '../core/theme/nmb_typography.dart';
8 ■ import '../core/utils/formatters.dart';
9 ■ import '../core/utils/result.dart';
10 ■ import '../core/widgets/nmb_card.dart';
11 ■ import '../core/widgets/nmb_dialogs.dart';
12 ■ import '../core/widgets/responsive_scaffold_body.dart';
13 ■ import '../core/widgets/state_views.dart';
14 ■ import '../models/app_user.dart';
15 ■ import '../models/bus.dart';
16 ■ import '../models/bus_route.dart';
17 ■ import '../models/live_location.dart';
18 ■ import '../models/trip.dart';
19 ■ import '../providers/app_providers.dart';
20 ■ import '../providers/data_providers.dart';
21 ■ import '../services/auth_service.dart';
22 ■ import '../services/driver_trip_service.dart';
23 ■
24 ■ /// Driver home: giant Start/End trip control, GPS + connection status, and
25 ■ /// assigned bus info. Everything is one-tap; nothing requires interaction
26 ■ /// while driving.
27 ■ class DriverHomeScreen extends ConsumerStatefulWidget {
28 ■   const DriverHomeScreen({super.key});
29 ■
30 ■   @override
31 ■   ConsumerState<DriverHomeScreen> createState() => _DriverHomeScreenState();
32 ■ }
33 ■
34 ■ class _DriverHomeScreenState extends ConsumerState<DriverHomeScreen> {
35 ■   bool _busy = false;
36 ■   bool _gpsServiceOn = false;
37 ■   bool _resumedCheckDone = false;
38 ■
39 ■   @override
40 ■   void initState() {
41 ■     super.initState();
42 ■     _refreshGpsStatus();
43 ■   }
44 ■
45 ■   Future<void> _refreshGpsStatus() async {
46 ■     final bool on = await Geolocator.isLocationServiceEnabled();
47 ■     if (mounted) setState(() => _gpsServiceOn = on);
48 ■   }
49 ■
50 ■   /// If the app restarted while a trip is still active on the server,
51 ■   /// offer to resume publishing GPS.
52 ■   Future<void> _maybeResume(Trip trip, AuthSession session) async {
53 ■     if (_resumedCheckDone) return;
54 ■     _resumedCheckDone = true;
55 ■     final DriverTripService svc = ref.read(driverTripServiceProvider);
56 ■     if (svc.hasLocalActiveTrip) return;
57 ■     if (trip.driverId != session.uid) return;
58 ■     await svc.resumePublishing(busId: trip.busId, tripId: trip.id);

```

```

59 ■ }
60 ■
61 ■ Future<void> _startTrip({
62 ■   required AuthSession session,
63 ■   required Bus bus,
64 ■   required BusRoute? route,
65 ■ }) async {
66 ■   final TripDirection? direction = await showModalBottomSheet<TripDirection>(
67 ■     context: context,
68 ■     shape: const RoundedRectangleBorder(
69 ■       borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
70 ■     ),
71 ■     builder: (BuildContext ctx) => SafeArea(
72 ■       child: Padding(
73 ■         padding: const EdgeInsets.all(20),
74 ■         child: Column(
75 ■           mainAxisAlignment: MainAxisAlignment.min,
76 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
77 ■           children: <Widget>[
78 ■             const Text('Start which trip?',
79 ■               style: NmbTypography.sectionTitle,
80 ■               textAlign: TextAlign.center,)),
81 ■             const SizedBox(height: 18),
82 ■             FilledButton.icon(
83 ■               icon: const Icon(Icons.wb_sunny_rounded),
84 ■               label: const Text('Morning pickup – towards school'),
85 ■               onPressed: () =>
86 ■                 Navigator.of(ctx).pop(TripDirection.pickup),
87 ■             ),
88 ■             const SizedBox(height: 12),
89 ■             FilledButton.icon(
90 ■               style: FilledButton.styleFrom(
91 ■                 backgroundColor: NmbColors.accentDark,)),
92 ■               icon: const Icon(Icons.home_rounded),
93 ■               label: const Text('Afternoon drop – towards home'),
94 ■               onPressed: () => Navigator.of(ctx).pop(TripDirection.drop),
95 ■             ),
96 ■           ],
97 ■         ),
98 ■       ),
99 ■     ),
100 ■   );
101 ■   if (direction == null || !mounted) return;
102 ■
103 ■   setState(() => _busy = true);
104 ■   final Result<String> result =
105 ■     await ref.read(driverTripServiceProvider).startTrip(
106 ■       driverId: session.uid,
107 ■       busId: bus.id,
108 ■       routeId: route?.id ?? '',
109 ■       direction: direction.name,
110 ■     );
111 ■   if (!mounted) return;
112 ■   setState(() => _busy = false);
113 ■   result.when(
114 ■     ok: (_) => showNmbSnack(
115 ■       context,
116 ■       'Trip started. Location is now shared with your students.',

```

```

117 ■         isSuccess: true,
118 ■     ),
119 ■     err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
120 ■ );
121 ■ _refreshGpsStatus();
122 ■ }
123 ■
124 ■ Future<void> _endTrip() async {
125 ■     final bool confirmed = await showNmbConfirmDialog(
126 ■         context,
127 ■         title: 'End trip?',
128 ■         message: 'Live tracking will stop and students will no longer see '
129 ■             'the bus location.',
130 ■         confirmLabel: 'End trip',
131 ■         destructive: true,
132 ■         icon: Icons.stop_circle_outlined,
133 ■     );
134 ■     if (!confirmed || !mounted) return;
135 ■
136 ■     setState(() => _busy = true);
137 ■     final Result<void> result =
138 ■         await ref.read(driverTripServiceProvider).endTrip();
139 ■     if (!mounted) return;
140 ■     setState(() => _busy = false);
141 ■     result.when(
142 ■         ok: (_) =>
143 ■             showNmbSnack(context, 'Trip ended. Thank you!', isSuccess: true),
144 ■         err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
145 ■     );
146 ■ }
147 ■
148 ■ @override
149 ■ Widget build(BuildContext context) {
150 ■     final AuthSession? session = ref.watch(currentSessionProvider);
151 ■     final AppUser? me = ref.watch(myProfileProvider).valueOrNull;
152 ■     final AsyncValue<Bus?> busAsync = ref.watch(myBusProvider);
153 ■     final BusRoute? route = ref.watch(myRouteProvider).valueOrNull;
154 ■     final Trip? trip = ref.watch(myBusActiveTripProvider).valueOrNull;
155 ■     final LiveLocation? live =
156 ■         ref.watch(myBusLiveLocationProvider).valueOrNull;
157 ■     final bool online = ref.watch(isOnlineProvider).valueOrNull ?? true;
158 ■
159 ■     if (session == null) return const LoadingView();
160 ■
161 ■     if (trip != null && trip.isActive) {
162 ■         _maybeResume(trip, session);
163 ■     }
164 ■
165 ■     return Scaffold(
166 ■         appBar: AppBar(title: const Text('Driver Dashboard')),
167 ■         body: busAsync.when(
168 ■             loading: () => const LoadingView(),
169 ■             error: (Object e, _) => ErrorView(
170 ■                 message: 'Couldn\'t load your bus. Check your connection.',
171 ■                 onRetry: () => ref.invalidate(myBusProvider),
172 ■             ),
173 ■         data: (Bus? bus) {
174 ■             if (bus == null) {

```

```

175 ■         return const EmptyState(
176 ■             icon: Icons.no_transfer_rounded,
177 ■             title: 'No bus assigned',
178 ■             message: 'The school hasn\'t assigned a bus to your account '
179 ■                 'yet. Please contact the transport office.',
180 ■         );
181 ■     }
182 ■
183 ■     final bool onTrip = trip != null && trip.isActive;
184 ■
185 ■     return ResponsiveBody(
186 ■         child: Column(
187 ■             crossAxisAlignment: CrossAxisAlignment.stretch,
188 ■             children: <Widget>[
189 ■                 Text(
190 ■                     Formatters.greetingForNow(
191 ■                         firstName: me?.firstName ?? 'Driver',),
192 ■                     style: NmbTypography.displayTitle,
193 ■                 ),
194 ■                 const SizedBox(height: 16),
195 ■
196 ■                 // Assigned bus
197 ■                 NmbCard(
198 ■                     color: onTrip ? NmbColors.successSoft : NmbColors.surface,
199 ■                     borderColor: onTrip ? NmbColors.success : null,
200 ■                     child: Row(
201 ■                         children: <Widget>[
202 ■                             Container(
203 ■                                 width: 54,
204 ■                                 height: 54,
205 ■                                 decoration: BoxDecoration(
206 ■                                     color: NmbColors.accent,
207 ■                                     borderRadius: BorderRadius.circular(16),
208 ■                                 ),
209 ■                                 child: const Icon(Icons.directions_bus_rounded,
210 ■                                     color: Colors.white, size: 30,),
211 ■                             ),
212 ■                             const SizedBox(width: 14),
213 ■                             Expanded(
214 ■                                 child: Column(
215 ■                                     crossAxisAlignment: CrossAxisAlignment.start,
216 ■                                     children: <Widget>[
217 ■                                         Text(bus.busNumber,
218 ■                                             style: NmbTypography.sectionTitle,),
219 ■                                         Text(bus.plateNumber,
220 ■                                             style: NmbTypography.bodySecondary,),
221 ■                                         if (route != null)
222 ■                                             Text(route.name,
223 ■                                                 style: NmbTypography.caption,),
224 ■                                     ],
225 ■                                 ),
226 ■                             ),
227 ■                             Column(
228 ■                                 crossAxisAlignment: CrossAxisAlignment.end,
229 ■                                 children: <Widget>[
230 ■                                     Text(
231 ■                                         onTrip ? 'TRIP ACTIVE' : 'IDLE',
232 ■                                         style: NmbTypography.caption.copyWith(

```

```

233 ■         color: onTrip
234 ■             ? NmbColors.success
235 ■             : NmbColors.textTertiary,
236 ■         fontWeight: FontWeight.w800,
237 ■     ),
238 ■ ),
239 ■     if (onTrip && trip.startedAt != null)
240 ■         Text(
241 ■             'since '
242 ■             '${Formatters.timeOfDay(trip.startedAt!)}',
243 ■             style: NmbTypography.caption,
244 ■         ),
245 ■     ],
246 ■ ),
247 ■ ],
248 ■ ),
249 ■ ),
250 ■ const SizedBox(height: 14),
251 ■
252 ■ // GPS + connection status
253 ■ Row(
254 ■     children: <Widget>[
255 ■         Expanded(
256 ■             child: _StatusTile(
257 ■                 icon: _gpsServiceOn
258 ■                     ? Icons.gps_fixed_rounded
259 ■                     : Icons.gps_off_rounded,
260 ■                 label: 'GPS',
261 ■                 value: _gpsServiceOn ? 'On' : 'Off',
262 ■                 ok: _gpsServiceOn,
263 ■                 onTap: () async {
264 ■                     await Geolocator.openLocationSettings();
265 ■                     _refreshGpsStatus();
266 ■                 },
267 ■             ),
268 ■         ),
269 ■         const SizedBox(width: 12),
270 ■         Expanded(
271 ■             child: _StatusTile(
272 ■                 icon: online
273 ■                     ? Icons.wifi_rounded
274 ■                     : Icons.wifi_off_rounded,
275 ■                 label: 'Internet',
276 ■                 value: online ? 'Connected' : 'Offline',
277 ■                 ok: online,
278 ■             ),
279 ■         ),
280 ■         const SizedBox(width: 12),
281 ■         Expanded(
282 ■             child: _StatusTile(
283 ■                 icon: onTrip && live != null
284 ■                     ? Icons.podcasts_rounded
285 ■                     : Icons.portable_wifi_off_rounded,
286 ■                 label: 'Sharing',
287 ■                 value: onTrip
288 ■                     ? (live != null ? 'Live' : 'Waiting...')
289 ■                     : 'Stopped',
290 ■                 ok: onTrip && live != null,

```

```

291 ■         ),
292 ■       ),
293 ■     ],
294 ■   ),
295 ■   const SizedBox(height: 14),
296 ■
297 ■   // Current location (while on trip)
298 ■   if (onTrip && live != null)
299 ■     NmbCard(
300 ■       child: Row(
301 ■         children: <Widget>[
302 ■           const Icon(Icons.my_location_rounded,
303 ■             color: NmbColors.primary, ),
304 ■           const SizedBox(width: 12),
305 ■           Expanded(
306 ■             child: Column(
307 ■               crossAxisAlignment: CrossAxisAlignment.start,
308 ■               children: <Widget>[
309 ■                 Text(
310 ■                   'Sharing location - '
311 ■                   '${live.speedKmh.toStringAsFixed(0)} km/h',
312 ■                   style: NmbTypography.cardTitle,
313 ■                 ),
314 ■                 Text(
315 ■                   'Last sent '
316 ■                   '${Formatters.relativeTime(live.updatedAt)}',
317 ■                   style: NmbTypography.bodySecondary,
318 ■                 ),
319 ■               ],
320 ■             ),
321 ■           ),
322 ■         ],
323 ■       ),
324 ■     ),
325 ■     const SizedBox(height: 20),
326 ■
327 ■     // THE control: one giant button.
328 ■     if (_busy)
329 ■       const Center(
330 ■         child: Padding(
331 ■           padding: EdgeInsets.all(12),
332 ■           child: CircularProgressIndicator(),
333 ■         ),
334 ■       )
335 ■     else if (!onTrip)
336 ■       FilledButton.icon(
337 ■         style: FilledButton.styleFrom(
338 ■           minimumSize: const Size.fromHeight(72),
339 ■           backgroundColor: NmbColors.success,
340 ■           textStyle: NmbTypography.button.copyWith(fontSize: 19),
341 ■         ),
342 ■         onPressed: () => _startTrip(
343 ■           session: session,
344 ■           bus: bus,
345 ■           route: route,
346 ■         ),
347 ■         icon: const Icon(Icons.play_arrow_rounded, size: 30),
348 ■         label: const Text('START TRIP'),
349 ■       )

```

```

349 ■         else
350 ■             FilledButton.icon(
351 ■                 style: FilledButton.styleFrom(
352 ■                     minimumSize: const Size.fromHeight(72),
353 ■                     backgroundColor: NmbColors.danger,
354 ■                     textStyle: NmbTypography.button.copyWith(fontSize: 19),
355 ■                 ),
356 ■                 onPressed: _endTrip,
357 ■                 icon: const Icon(Icons.stop_rounded, size: 30),
358 ■                 label: const Text('END TRIP'),
359 ■             ),
360 ■         const SizedBox(height: 12),
361 ■         Text(
362 ■             onTrip
363 ■                 ? 'Keep the phone connected to power. Location is '
364 ■                 'shared automatically – no further action needed.'
365 ■                 : 'Start the trip when you begin the route. Students '
366 ■                 'will see the bus live on their map.',
367 ■             style: NmbTypography.caption,
368 ■             textAlign: TextAlign.center,
369 ■         ),
370 ■         const SizedBox(height: 24),
371 ■     ],
372 ■ ),
373 ■ );
374 ■ },
375 ■ ),
376 ■ );
377 ■ }
378 ■ }
379 ■
380 ■ class _StatusTile extends StatelessWidget {
381 ■     const _StatusTile({
382 ■         required this.icon,
383 ■         required this.label,
384 ■         required this.value,
385 ■         required this.ok,
386 ■         this.onTap,
387 ■     });
388 ■
389 ■     final IconData icon;
390 ■     final String label;
391 ■     final String value;
392 ■     final bool ok;
393 ■     final VoidCallback? onTap;
394 ■
395 ■     @override
396 ■     Widget build(BuildContext context) {
397 ■         return NmbCard(
398 ■             onTap: onTap,
399 ■             padding: const EdgeInsets.symmetric(vertical: 14, horizontal: 8),
400 ■             color: ok ? NmbColors.surface : NmbColors.dangerSoft,
401 ■             child: Column(
402 ■                 children: <Widget>[
403 ■                     Icon(icon,
404 ■                         color: ok ? NmbColors.success : NmbColors.danger, size: 26,),
405 ■                     const SizedBox(height: 6),
406 ■                     Text(label, style: NmbTypography.caption),
407 ■
408 ■                     Text(
409 ■                         value,
410 ■                         style: NmbTypography.caption.copyWith(
411 ■                             fontWeight: FontWeight.w800,
412 ■                             color: ok ? NmbColors.textPrimary : NmbColors.danger,
413 ■                         ),
414 ■                     ),
415 ■                 ],
416 ■             ),
417 ■         );
418 ■     }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../models/bus_route.dart';
10 ■ import '../models/bus_stop.dart';
11 ■ import '../providers/data_providers.dart';
12 ■
13 ■ /// Read-only route/stop list for the driver – big text, glanceable.
14 ■ class DriverRouteScreen extends ConsumerWidget {
15 ■   const DriverRouteScreen({super.key});
16 ■
17 ■   @override
18 ■   Widget build(BuildContext context, WidgetRef ref) {
19 ■     final BusRoute? route = ref.watch(myRouteProvider).valueOrNull;
20 ■     final AsyncValue<List<BusStop>> stopsAsync = ref.watch(myStopsProvider);
21 ■
22 ■     return Scaffold(
23 ■       appBar: AppBar(title: const Text('My Route')),
24 ■       body: stopsAsync.when(
25 ■         loading: () => const LoadingView(),
26 ■         error: (Object e, _) => ErrorView(
27 ■           message: 'Couldn\'t load the route.',
28 ■           onRetry: () => ref.invalidate(myStopsProvider),
29 ■         ),
30 ■         data: (List<BusStop> stops) {
31 ■           if (stops.isEmpty) {
32 ■             return const EmptyState(
33 ■               icon: Icons.route_outlined,
34 ■               title: 'No route published',
35 ■               message: 'The transport office hasn\'t published your route '
36 ■                 'yet.',
37 ■             );
38 ■           }
39 ■           return ResponsiveBody(
40 ■             child: Column(
41 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
42 ■               children: <Widget>[
43 ■                 if (route != null) ...<Widget>[
44 ■                   Text(route.name, style: NmbTypography.screenTitle),
45 ■                   const SizedBox(height: 4),
46 ■                   Text(
47 ■                     '${stops.length} stops • ${route.direction.label}',
48 ■                     style: NmbTypography.bodySecondary,
49 ■                   ),
50 ■                   const SizedBox(height: 16),
51 ■                 ],
52 ■                 for (int i = 0; i < stops.length; i++) ...<Widget>[
53 ■                   NmbCard(
54 ■                     padding: const EdgeInsets.all(16),
55 ■                     child: Row(
56 ■                       children: <Widget>[
57 ■                         Container(
58 ■                           width: 34,

```

```

59 ■          height: 34,
60 ■          decoration: const BoxDecoration(
61 ■            color: NmbColors.primarySoft,
62 ■            shape: BoxShape.circle,
63 ■          ),
64 ■          child: Center(
65 ■            child: Text(
66 ■              '${i + 1}',
67 ■              style: NmbTypography.cardTitle
68 ■                .copyWith(color: NmbColors.primaryDark),
69 ■            ),
70 ■          ),
71 ■        ),
72 ■        const SizedBox(width: 14),
73 ■        Expanded(
74 ■          child: Text(stops[i].name,
75 ■            style: NmbTypography.cardTitle,),
76 ■        ),
77 ■        if (stops[i].scheduledTime != null)
78 ■          Text(
79 ■            stops[i].scheduledTime!,
80 ■            style: NmbTypography.cardTitle
81 ■              .copyWith(color: NmbColors.primary),
82 ■          ),
83 ■      ],
84 ■    ),
85 ■  ),
86 ■  const SizedBox(height: 10),
87 ■ ],
88 ■ const SizedBox(height: 16),
89 ■ ],
90 ■ ),
91 ■ );
92 ■ },
93 ■ ),
94 ■ );
95 ■ }
96 ■ }

```

lib/features/driver/screens/driver_profile_screen.dart

167 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/nmb_dialogs.dart';
8 ■ import '../core/widgets/responsive_scaffold_body.dart';
9 ■ import '../core/widgets/state_views.dart';
10 ■ import '../models/app_user.dart';
11 ■ import '../models/bus.dart';
12 ■ import '../models/trip.dart';
13 ■ import '../providers/app_providers.dart';
14 ■ import '../providers/data_providers.dart';
15 ■ import '../services/auth_service.dart';
16 ■
17 ■ class DriverProfileScreen extends ConsumerWidget {
18 ■   const DriverProfileScreen({super.key});
19 ■
20 ■   Future<void> _logout(BuildContext context, WidgetRef ref) async {
21 ■     final AuthSession? session = ref.read(currentSessionProvider);
22 ■     // Capture EVERYTHING from ref BEFORE any await/pop – after the widget
23 ■     // is disposed, ref must not be touched ("ref after disposed" fix).
24 ■     final notifSvc = ref.read(notificationServiceProvider);
25 ■     final devSvc = ref.read(deviceSessionServiceProvider);
26 ■     final fsSvc = ref.read(firestoreServiceProvider);
27 ■     final authSvc = ref.read(authServiceProvider);
28 ■     final Trip? trip = ref.read(myBusActiveTripProvider).valueOrNull;
29 ■     if (trip != null && trip.isActive) {
30 ■       showNmbSnack(
31 ■         context,
32 ■         'Please end your active trip before logging out.',
33 ■         isError: true,
34 ■       );
35 ■       return;
36 ■     }
37 ■     final bool confirmed = await showNmbConfirmDialog(
38 ■       context,
39 ■       title: 'Log out?',
40 ■       message: 'You\'ll need your driver account credentials to sign '
41 ■         'back in.',
42 ■       confirmLabel: 'Log out',
43 ■       destructive: true,
44 ■       icon: Icons.logout_rounded,
45 ■     );
46 ■     if (!confirmed) return;
47 ■     if (session != null) {
48 ■       try {
49 ■         await notifSvc.unregisterDevice(session.uid);
50 ■       } catch (_) {/* best effort */}
51 ■       try {
52 ■         final String devId = await devSvc.getDeviceId();
53 ■         await fsSvc.clearActiveDevice(
54 ■           uid: session.uid,
55 ■           deviceId: devId,
56 ■         );
57 ■       } catch (_) {/* best effort */}
58 ■     }

```

```

59 ■      await authSvc.signOut();
60 ■    }
61 ■
62 ■    @override
63 ■    Widget build(BuildContext context, WidgetRef ref) {
64 ■      final AsyncValue<AppUser?> profileAsync = ref.watch(myProfileProvider);
65 ■      final Bus? bus = ref.watch(myBusProvider).valueOrNull;
66 ■      final Map<String, dynamic>? school =
67 ■        ref.watch(schoolConfigProvider).valueOrNull;
68 ■
69 ■      return Scaffold(
70 ■        appBar: AppBar(title: const Text('Profile')),
71 ■        body: profileAsync.when(
72 ■          loading: () => const LoadingView(),
73 ■          error: (Object e, _) => ErrorView(
74 ■            message: 'Couldn\'t load your profile.',
75 ■            onRetry: () => ref.invalidate(myProfileProvider),
76 ■          ),
77 ■        data: (AppUser? me) {
78 ■          if (me == null) {
79 ■            return const ErrorView(
80 ■              title: 'Profile unavailable',
81 ■              message: 'Please contact the transport office.',
82 ■            );
83 ■          }
84 ■          return ResponsiveBody(
85 ■            child: Column(
86 ■              crossAxisAlignment: CrossAxisAlignment.stretch,
87 ■              children: <Widget>[
88 ■                NmbCard(
89 ■                  child: Row(
90 ■                    children: <Widget>[
91 ■                      const CircleAvatar(
92 ■                        radius: 30,
93 ■                        backgroundColor: NmbColors.accentSoft,
94 ■                        child: Icon(Icons.badge_rounded,
95 ■                          color: NmbColors.accentDark, size: 30,),
96 ■                      ),
97 ■                      const SizedBox(width: 14),
98 ■                      Expanded(
99 ■                        child: Column(
100 ■                          crossAxisAlignment: CrossAxisAlignment.start,
101 ■                          children: <Widget>[
102 ■                            Text(me.fullName,
103 ■                              style: NmbTypography.sectionTitle,),
104 ■                            Text(me.email,
105 ■                              style: NmbTypography.bodySecondary,),
106 ■                            const Text('School bus driver',
107 ■                              style: NmbTypography.caption,),
108 ■                          ],
109 ■                        ),
110 ■                      ),
111 ■                    ],
112 ■                  ),
113 ■                ),
114 ■                const SizedBox(height: 14),
115 ■                NmbCard(
116 ■                  child: ListTile(

```

```

117 ■         contentPadding: EdgeInsets.zero,
118 ■         leading: const Icon(Icons.directions_bus_rounded,
119 ■             color: NmbColors.primary,),
120 ■         title: const Text('Assigned bus'),
121 ■         trailing: Text(
122 ■             bus?.busNumber ?? 'Not assigned',
123 ■             style: NmbTypography.cardTitle,
124 ■         ),
125 ■     ),
126 ■ ),
127 ■ const SizedBox(height: 14),
128 ■ if (school?['phone'] != null)
129 ■     NmbCard(
130 ■         child: Row(
131 ■             children: <Widget>[
132 ■                 const Icon(Icons.support_agent_rounded,
133 ■                     color: NmbColors.info,),
134 ■                 const SizedBox(width: 12),
135 ■                 Expanded(
136 ■                     child: Column(
137 ■                         crossAxisAlignment: CrossAxisAlignment.start,
138 ■                         children: <Widget>[
139 ■                             const Text('Transport office',
140 ■                                 style: NmbTypography.cardTitle,),
141 ■                             Text(school!['phone'] as String,
142 ■                                 style: NmbTypography.bodySecondary,),
143 ■                         ],
144 ■                     ),
145 ■                 ),
146 ■             ],
147 ■         ),
148 ■     ),
149 ■ const SizedBox(height: 20),
150 ■ OutlinedButton.icon(
151 ■     style: OutlinedButton.styleFrom(
152 ■         foregroundColor: NmbColors.danger,
153 ■         side: const BorderSide(color: NmbColors.danger),
154 ■     ),
155 ■     onPressed: () => _logout(context, ref),
156 ■     icon: const Icon(Icons.logout_rounded),
157 ■     label: const Text('Log out'),
158 ■ ),
159 ■ const SizedBox(height: 24),
160 ■ ],
161 ■ ),
162 ■ );
163 ■ },
164 ■ ),
165 ■ );
166 ■ }
167 ■ }

```

10. Admin App

Shell + dashboard (stat cards, live banner) + all management screens incl. trip history & login-status.

lib/features/admin/admin_shell.dart

178 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter/services.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■ import 'package:go_router/go_router.dart';
5 ■
6 ■ import '../core/theme/nmb_colors.dart';
7 ■ import '../core/theme/nmb_typography.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/state_views.dart';
10 ■ import '../providers/app_providers.dart';
11 ■ import '../services/auth_service.dart';
12 ■
13 ■ /// Admin shell with a navigation drawer (admin has more sections than fit a
14 ■ /// bottom bar, but the design stays friendly, not "corporate dashboard").
15 ■ class AdminShell extends ConsumerWidget {
16 ■   const AdminShell({super.key, required this.child});
17 ■
18 ■   final Widget child;
19 ■
20 ■   static const List<(String, String, IconData)> _items =
21 ■     <(String, String, IconData)>[
22 ■       ('/admin/dashboard', 'Dashboard', Icons.dashboard_rounded),
23 ■       ('/admin/students', 'Students', Icons.school_rounded),
24 ■       ('/admin/drivers', 'Drivers', Icons.badge_rounded),
25 ■       ('/admin/buses', 'Buses', Icons.directions_bus_rounded),
26 ■       ('/admin/routes', 'Routes & Stops', Icons.route_rounded),
27 ■       ('/admin/monitoring', 'Live Monitoring', Icons.radar_rounded),
28 ■       ('/admin/notifications', 'Notifications', Icons.campaign_rounded),
29 ■       ('/admin/trips', 'Trip History', Icons.history_rounded),
30 ■       ('/admin/audit', 'Activity Logs', Icons.receipt_long_rounded),
31 ■       ('/admin/settings', 'Settings', Icons.settings_rounded),
32 ■     ];
33 ■
34 ■   Future<void> _logout(BuildContext context, WidgetRef ref) async {
35 ■     final AuthSession? session = ref.read(currentSessionProvider);
36 ■     // Capture EVERYTHING from ref BEFORE any await/pop – after the widget
37 ■     // is disposed, ref must not be touched ("ref after disposed" fix).
38 ■     final notifSvc = ref.read(notificationServiceProvider);
39 ■     final devSvc = ref.read(deviceSessionServiceProvider);
40 ■     final fsSvc = ref.read(firestoreServiceProvider);
41 ■     final authSvc = ref.read(authServiceProvider);
42 ■     final bool ok = await showNmbConfirmDialog(
43 ■       context,
44 ■       title: 'Log out?',
45 ■       message: 'End your admin session?',
46 ■       confirmLabel: 'Log out',
47 ■       destructive: true,
48 ■       icon: Icons.logout_rounded,
49 ■     );
50 ■     if (!ok) return;
51 ■     if (session != null) {
52 ■       try {
53 ■         await notifSvc.unregisterDevice(session.uid);
54 ■       } catch (_) {/* best effort */}
55 ■       try {
56 ■         final String devId = await devSvc.getDeviceId();
57 ■         await fsSvc.clearActiveDevice(
58 ■           uid: session.uid,

```

```

59 ■         deviceId: deviceId,
60 ■     );
61 ■     } catch (_) {/* best effort */}
62 ■ }
63 ■ await authSvc.signOut();
64 ■ }
65 ■
66 ■ @override
67 ■ Widget build(BuildContext context, WidgetRef ref) {
68 ■     final String location = GoRouterState.of(context).matchedLocation;
69 ■     final bool online = ref.watch(isOnlineProvider).valueOrNull ?? true;
70 ■
71 ■     // Back button: kisi section me ho → Dashboard; Dashboard pe ho → exit.
72 ■     return PopScope(
73 ■         canPop: false,
74 ■         onPopInvokedWithResult: (bool didPop, Object? result) {
75 ■             if (didPop) return;
76 ■             final GoRouter router = GoRouter.of(context);
77 ■             if (router.canPop()) {
78 ■                 router.pop();
79 ■             } else if (location != '/admin/dashboard') {
80 ■                 context.go('/admin/dashboard');
81 ■             } else {
82 ■                 SystemNavigator.pop();
83 ■             }
84 ■         },
85 ■         child: Scaffold(
86 ■             drawer: Drawer(
87 ■                 child: SafeArea(
88 ■                     child: Column(
89 ■                         children: <Widget>[
90 ■                             Padding(
91 ■                                 padding: const EdgeInsets.all(20),
92 ■                                 child: Row(
93 ■                                     children: <Widget>[
94 ■                                         Container(
95 ■                                             width: 46,
96 ■                                             height: 46,
97 ■                                             decoration: BoxDecoration(
98 ■                                                 color: NmbColors.primary,
99 ■                                                 borderRadius: BorderRadius.circular(14),
100 ■                                             ),
101 ■                                             child: const Icon(Icons.directions_bus_rounded,
102 ■                                                 color: Colors.white,)),
103 ■                                     ],
104 ■                                     const SizedBox(width: 12),
105 ■                                     const Expanded(
106 ■                                         child: Column(
107 ■                                             crossAxisAlignment: CrossAxisAlignment.start,
108 ■                                             children: <Widget>[
109 ■                                                 Text('Never Miss Bus',
110 ■                                                     style: NmbTypography.sectionTitle,)),
111 ■                                                 Text('Admin Console',
112 ■                                                     style: NmbTypography.caption,)),
113 ■                                             ],
114 ■                                         ),
115 ■                                     ),
116 ■                                 ],

```

```

117 ■      ),
118 ■    ),
119 ■    const Divider(),
120 ■    Expanded(
121 ■      child: ListView(
122 ■        padding: const EdgeInsets.symmetric(
123 ■          horizontal: 10, vertical: 6, ),
124 ■        children: <Widget>[
125 ■          for (final (String path, String label, IconData icon)
126 ■            in _items)
127 ■            ListTile(
128 ■              shape: RoundedRectangleBorder(
129 ■                borderRadius: BorderRadius.circular(12),
130 ■              ),
131 ■              selected: location.startsWith(path),
132 ■              selectedTileColor: NmbColors.primarySoft,
133 ■              leading: Icon(icon),
134 ■              title: Text(
135 ■                label,
136 ■                style: NmbTypography.body.copyWith(
137 ■                  fontWeight: location.startsWith(path)
138 ■                    ? FontWeight.w700
139 ■                    : FontWeight.w500,
140 ■                ),
141 ■            ),
142 ■            onTap: () {
143 ■              Navigator.of(context).pop();
144 ■              context.go(path);
145 ■            },
146 ■          ),
147 ■        ],
148 ■      ),
149 ■    ),
150 ■    const Divider(),
151 ■    ListTile(
152 ■      leading:
153 ■        const Icon(Icons.logout_rounded, color: NmbColors.danger),
154 ■      title: Text(
155 ■        'Log out',
156 ■        style:
157 ■          NmbTypography.body.copyWith(color: NmbColors.danger),
158 ■      ),
159 ■      onTap: () {
160 ■        Navigator.of(context).pop();
161 ■        _logout(context, ref);
162 ■      },
163 ■    ),
164 ■    const SizedBox(height: 8),
165 ■  ],
166 ■ ),
167 ■ ),
168 ■ ),
169 ■ body: Column(
170 ■   children: <Widget>[
171 ■     if (!online) const OfflineBanner(),
172 ■     Expanded(child: child),
173 ■   ],
174 ■ ),
175 ■ ),
176 ■ );
177 ■ }
178 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■ import 'package:go_router/go_router.dart';
4 ■ import 'package:intl/intl.dart';
5 ■
6 ■ import '../core/constants/nmb_constants.dart';
7 ■ import '../core/theme/nmb_colors.dart';
8 ■ import '../core/theme/nmb_typography.dart';
9 ■ import '../core/utils/formatters.dart';
10 ■ import '../core/widgets/nmb_card.dart';
11 ■ import '../core/widgets/responsive_scaffold_body.dart';
12 ■ import '../models/app_user.dart';
13 ■ import '../models/audit_log.dart';
14 ■ import '../models/bus.dart';
15 ■ import '../models/trip.dart';
16 ■ import '../providers/data_providers.dart';
17 ■
18 ■ /// v1.1.1 Admin Dashboard – design-mock style: greeting + date chip,
19 ■ /// 4 colored stat cards, live-monitoring banner, quick actions,
20 ■ /// recent activity and active trips lists.
21 ■ class AdminDashboardScreen extends ConsumerWidget {
22 ■   const AdminDashboardScreen({super.key});
23 ■
24 ■   @override
25 ■   Widget build(BuildContext context, WidgetRef ref) {
26 ■     final List<AppUser> students =
27 ■       ref.watch(allStudentsProvider).valueOrNull ?? const <AppUser>[];
28 ■     final List<AppUser> drivers =
29 ■       ref.watch(allDriversProvider).valueOrNull ?? const <AppUser>[];
30 ■     final List<Bus> buses =
31 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
32 ■     final List<Trip> activeTrips =
33 ■       ref.watch(activeTripsProvider).valueOrNull ?? const <Trip>[];
34 ■     final List<AuditLog> logs =
35 ■       ref.watch(auditLogsProvider).valueOrNull ?? const <AuditLog>[];
36 ■
37 ■     return Scaffold(
38 ■       appBar: AppBar(title: const Text('Dashboard')),
39 ■       body: ResponsiveBody(
40 ■         child: Column(
41 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
42 ■           children: <Widget>[
43 ■             // ■■ Greeting + date ■■
44 ■             Row(
45 ■               children: <Widget>[
46 ■                 Expanded(
47 ■                   child: Column(
48 ■                     crossAxisAlignment: CrossAxisAlignment.start,
49 ■                     children: <Widget>[
50 ■                       Text(
51 ■                         '$_greetPart()', Admin ■',
52 ■                         style: NmbTypography.displayTitle,
53 ■                       ),
54 ■                       const Text(
55 ■                         NmbConstants.schoolName,
56 ■                         style: NmbTypography.bodySecondary,
57 ■                       ),
58 ■                     ],

```

```

59 ■         ),
60 ■     ),
61 ■     Container(
62 ■         padding: const EdgeInsets.symmetric(
63 ■             horizontal: 14, vertical: 10, ),
64 ■         decoration: BoxDecoration(
65 ■             color: NmbColors.surface,
66 ■             borderRadius: BorderRadius.circular(14),
67 ■             border: Border.all(color: NmbColors.divider),
68 ■         ),
69 ■         child: Column(
70 ■             children: <Widget>[
71 ■                 Text(
72 ■                     DateFormat('d MMM yyyy').format(DateTime.now()),
73 ■                     style: NmbTypography.cardTitle,
74 ■                 ),
75 ■                 Text(
76 ■                     DateFormat('EEEE').format(DateTime.now()),
77 ■                     style: NmbTypography.caption,
78 ■                 ),
79 ■             ],
80 ■         ),
81 ■     ),
82 ■ ],
83 ■ ),
84 ■ const SizedBox(height: 18),
85 ■
86 ■ // ■■ 4 stat cards ■■
87 ■ Row(
88 ■     children: <Widget>[
89 ■         Expanded(
90 ■             child: _StatCard(
91 ■                 label: 'Total Buses',
92 ■                 value: '${buses.length}',
93 ■                 icon: Icons.directions_bus_rounded,
94 ■                 color: NmbColors.primary,
95 ■                 bg: NmbColors.primarySoft,
96 ■                 onTap: () => context.go('/admin/buses'),
97 ■             ),
98 ■         ),
99 ■         const SizedBox(width: 12),
100 ■         Expanded(
101 ■             child: _StatCard(
102 ■                 label: 'Active Trips',
103 ■                 value: '${activeTrips.length}',
104 ■                 icon: Icons.wifi_tethering_rounded,
105 ■                 color: NmbColors.success,
106 ■                 bg: NmbColors.successSoft,
107 ■                 onTap: () => context.go('/admin/monitoring'),
108 ■             ),
109 ■         ),
110 ■     ],
111 ■ ),
112 ■ const SizedBox(height: 12),
113 ■ Row(
114 ■     children: <Widget>[
115 ■         Expanded(
116 ■             child: _StatCard(

```

```

117 ■         label: 'Students',
118 ■         value: '${students.length}',
119 ■         icon: Icons.groups_rounded,
120 ■         color: const Color(0xFF7B61FF),
121 ■         bg: const Color(0xFFEAEAFF),
122 ■         onTap: () => context.go('/admin/students'),
123 ■       ),
124 ■     ),
125 ■     const SizedBox(width: 12),
126 ■     Expanded(
127 ■       child: _StatCard(
128 ■         label: 'Drivers',
129 ■         value: '${drivers.length}',
130 ■         icon: Icons.badge_rounded,
131 ■         color: NmbColors.accentDark,
132 ■         bg: NmbColors.accentSoft,
133 ■         onTap: () => context.go('/admin/drivers'),
134 ■       ),
135 ■     ),
136 ■   ],
137 ■ ),
138 ■ const SizedBox(height: 18),
139 ■
140 ■ // ■■ Live monitoring banner ■■
141 ■ Material(
142 ■   color: Colors.transparent,
143 ■   child: InkWell(
144 ■     borderRadius: BorderRadius.circular(20),
145 ■     onTap: () => context.go('/admin/monitoring'),
146 ■     child: Container(
147 ■       padding: const EdgeInsets.all(18),
148 ■       decoration: BoxDecoration(
149 ■         gradient: const LinearGradient(
150 ■           colors: <Color>[
151 ■             NmbColors.primary,
152 ■             NmbColors.primaryDark,
153 ■           ],
154 ■         ),
155 ■         borderRadius: BorderRadius.circular(20),
156 ■       ),
157 ■       child: Row(
158 ■         children: <Widget>[
159 ■           Container(
160 ■             width: 50,
161 ■             height: 50,
162 ■             decoration: const BoxDecoration(
163 ■               color: Colors.white24,
164 ■               shape: BoxShape.circle,
165 ■             ),
166 ■             child: const Icon(Icons.radar_rounded,
167 ■               color: Colors.white, size: 28,)),
168 ■         ),
169 ■         const SizedBox(width: 14),
170 ■         Expanded(
171 ■           child: Column(
172 ■             crossAxisAlignment: CrossAxisAlignment.start,
173 ■             children: <Widget>[
174 ■               Text('Live Bus Monitoring',

```

```

175 ■                 style: NmbTypography.sectionTitle
176 ■                 .copyWith(color: Colors.white)),),
177 ■             Text(
178 ■                 activeTrips.isEmpty
179 ■                 ? 'No buses on a trip right now'
180 ■                 : '${activeTrips.length} bus'
181 ■                 '${activeTrips.length > 1 ? 'es' : ''}'
182 ■                 ' live right now – view on map',
183 ■                 style: NmbTypography.bodySecondary
184 ■                 .copyWith(color: Colors.white70),
185 ■             ),
186 ■         ],
187 ■     ),
188 ■ ),
189 ■ if (activeTrips.isNotEmpty)
190 ■     Container(
191 ■         padding: const EdgeInsets.symmetric(
192 ■             horizontal: 10, vertical: 5),
193 ■         decoration: BoxDecoration(
194 ■             color: NmbColors.success,
195 ■             borderRadius: BorderRadius.circular(999),
196 ■         ),
197 ■         child: Text(
198 ■             'LIVE',
199 ■             style: NmbTypography.caption.copyWith(
200 ■                 color: Colors.white,
201 ■                 fontWeight: FontWeight.w800,
202 ■             ),
203 ■         ),
204 ■     ),
205 ■ ],
206 ■ ),
207 ■ ),
208 ■ ),
209 ■ ),
210 ■ const SizedBox(height: 20),
211 ■
212 ■ // ■■ Quick Actions ■■
213 ■ const Text('Quick Actions', style: NmbTypography.sectionTitle),
214 ■ const SizedBox(height: 12),
215 ■ Row(
216 ■     children: <Widget>[
217 ■         _ActionTile(
218 ■             icon: Icons.directions_bus_rounded,
219 ■             color: NmbColors.primary,
220 ■             bg: NmbColors.primarySoft,
221 ■             label: 'Manage\nBuses',
222 ■             onTap: () => context.go('/admin/buses'),
223 ■         ),
224 ■         _ActionTile(
225 ■             icon: Icons.badge_rounded,
226 ■             color: NmbColors.accentDark,
227 ■             bg: NmbColors.accentSoft,
228 ■             label: 'Manage\nDrivers',
229 ■             onTap: () => context.go('/admin/drivers'),
230 ■         ),
231 ■         _ActionTile(
232 ■             icon: Icons.route_rounded,

```

```

233 ■         color: const Color(0xFF7B61FF),
234 ■         bg: const Color(0xFFEFEAFF),
235 ■         label: 'Routes &\nStops',
236 ■         onTap: () => context.go('/admin/routes'),
237 ■     ),
238 ■     _ActionTile(
239 ■         icon: Icons.groups_rounded,
240 ■         color: NmbColors.success,
241 ■         bg: NmbColors.successSoft,
242 ■         label: 'Manage\nStudents',
243 ■         onTap: () => context.go('/admin/students'),
244 ■     ),
245 ■ ],
246 ■ ),
247 ■ const SizedBox(height: 20),
248 ■
249 ■ // ■■ Active Trips ■■
250 ■ Row(
251 ■     mainAxisAlignment: MainAxisAlignment.spaceBetween,
252 ■     children: <Widget>[
253 ■         const Text('Active Trips',
254 ■             style: NmbTypography.sectionTitle,),
255 ■         TextButton(
256 ■             onPressed: () => context.go('/admin/monitoring'),
257 ■             child: const Text('View All'),
258 ■         ),
259 ■     ],
260 ■ ),
261 ■ if (activeTrips.isEmpty)
262 ■     const NmbCard(
263 ■         child: Row(
264 ■             children: <Widget>[
265 ■                 Icon(Icons.nightlight_round,
266 ■                     color: NmbColors.textTertiary,),
267 ■                 SizedBox(width: 12),
268 ■                 Expanded(
269 ■                     child: Text(
270 ■                         'No buses are on a trip right now.',
271 ■                         style: NmbTypography.bodySecondary,
272 ■                     ),
273 ■                 ),
274 ■             ],
275 ■         ),
276 ■     )
277 ■ else
278 ■     NmbCard(
279 ■         padding: const EdgeInsets.symmetric(
280 ■             horizontal: 14, vertical: 6,),
281 ■         child: Column(
282 ■             children: <Widget>[
283 ■                 for (final Trip trip in activeTrips.take(4))
284 ■                     _ActiveTripRow(
285 ■                         trip: trip, buses: buses, drivers: drivers,),
286 ■             ],
287 ■         ),
288 ■     ),
289 ■ const SizedBox(height: 20),
290 ■

```

```

291 ■      // ■■ Recent Activity ■■
292 ■      Row(
293 ■        mainAxisAlignment: MainAxisAlignment.spaceBetween,
294 ■        children: <Widget>[
295 ■          const Text('Recent Activity',
296 ■            style: NmbTypography.sectionTitle, ),
297 ■          TextButton(
298 ■            onPressed: () => context.go('/admin/audit'),
299 ■            child: const Text('View All'),
300 ■          ),
301 ■        ],
302 ■      ),
303 ■      if (logs.isEmpty)
304 ■        const NmbCard(
305 ■          child: Text(
306 ■            'No activity yet.',
307 ■            style: NmbTypography.bodySecondary,
308 ■          ),
309 ■        )
310 ■      else
311 ■        NmbCard(
312 ■          padding: const EdgeInsets.symmetric(
313 ■            horizontal: 14, vertical: 6, ),
314 ■          child: Column(
315 ■            children: <Widget>[
316 ■              for (final AuditLog log in logs.take(5))
317 ■                ListTile(
318 ■                  contentPadding: EdgeInsets.zero,
319 ■                  dense: true,
320 ■                  leading: Container(
321 ■                    width: 36,
322 ■                    height: 36,
323 ■                    decoration: BoxDecoration(
324 ■                      color: NmbColors.primarySoft,
325 ■                      borderRadius: BorderRadius.circular(10),
326 ■                    ),
327 ■                    child: Icon(
328 ■                      _logIcon(log.action),
329 ■                      size: 18,
330 ■                      color: NmbColors.primary,
331 ■                    ),
332 ■                  ),
333 ■                  title: Text(
334 ■                    log.action.replaceAll('_', ' '),
335 ■                    style: NmbTypography.cardTitle
336 ■                      .copyWith(fontSize: 14),
337 ■                  ),
338 ■                  trailing: Text(
339 ■                    Formatters.timeOfDay(log.at),
340 ■                    style: NmbTypography.caption,
341 ■                  ),
342 ■                ],
343 ■            ),
344 ■          ),
345 ■        ),
346 ■      const SizedBox(height: 24),
347 ■    ],
348 ■  ),

```

```

349 ■     ),
350 ■   );
351 ■ }
352 ■
353 ■ String _greetPart() {
354 ■   final int h = DateTime.now().hour;
355 ■   if (h < 12) return 'Good Morning';
356 ■   if (h < 17) return 'Good Afternoon';
357 ■   return 'Good Evening';
358 ■ }
359 ■
360 ■ IconData _logIcon(String action) {
361 ■   if (action.contains('TRIP')) return Icons.directions_bus_rounded;
362 ■   if (action.contains('PASSWORD')) return Icons.lock_reset_rounded;
363 ■   if (action.contains('ANNOUNCEMENT')) return Icons.campaign_rounded;
364 ■   if (action.contains('DISABLED')) return Icons.block_rounded;
365 ■   return Icons.person_rounded;
366 ■ }
367 ■ }
368 ■
369 ■ class _StatCard extends StatelessWidget {
370 ■   const _StatCard({
371 ■     required this.label,
372 ■     required this.value,
373 ■     required this.icon,
374 ■     required this.color,
375 ■     required this.bg,
376 ■     required this.onTap,
377 ■   });
378 ■
379 ■   final String label;
380 ■   final String value;
381 ■   final IconData icon;
382 ■   final Color color;
383 ■   final Color bg;
384 ■   final VoidCallback onTap;
385 ■
386 ■   @override
387 ■   Widget build(BuildContext context) {
388 ■     return NmbCard(
389 ■       onTap: onTap,
390 ■       child: Column(
391 ■         crossAxisAlignment: CrossAxisAlignment.start,
392 ■         children: <Widget>[
393 ■           Container(
394 ■             width: 44,
395 ■             height: 44,
396 ■             decoration: BoxDecoration(
397 ■               color: bg,
398 ■               borderRadius: BorderRadius.circular(13),
399 ■             ),
400 ■             child: Icon(icon, color: color, size: 24),
401 ■           ),
402 ■           const SizedBox(height: 12),
403 ■           Text(value, style: NmbTypography.statLarge),
404 ■           Text(label, style: NmbTypography.bodySecondary),
405 ■         ],
406 ■       ),

```

```

407 ■     });
408 ■   }
409 ■ }
410 ■
411 ■ class _ActionTile extends StatelessWidget {
412 ■   const _ActionTile({
413 ■     required this.icon,
414 ■     required this.color,
415 ■     required this.bg,
416 ■     required this.label,
417 ■     required this.onTap,
418 ■   });
419 ■
420 ■   final IconData icon;
421 ■   final Color color;
422 ■   final Color bg;
423 ■   final String label;
424 ■   final VoidCallback onTap;
425 ■
426 ■   @override
427 ■   Widget build(BuildContext context) {
428 ■     return Expanded(
429 ■       child: InkWell(
430 ■         borderRadius: BorderRadius.circular(16),
431 ■         onTap: onTap,
432 ■         child: Column(
433 ■           children: <Widget>[
434 ■             Container(
435 ■               width: 54,
436 ■               height: 54,
437 ■               decoration: BoxDecoration(
438 ■                 color: bg,
439 ■                 borderRadius: BorderRadius.circular(16),
440 ■               ),
441 ■               child: Icon(icon, color: color, size: 26),
442 ■             ),
443 ■             const SizedBox(height: 8),
444 ■             Text(
445 ■               label,
446 ■               textAlign: TextAlign.center,
447 ■               style: NmbTypography.caption
448 ■                 .copyWith(color: NmbColors.textSecondary),
449 ■             ),
450 ■           ],
451 ■         ),
452 ■       ),
453 ■     );
454 ■   }
455 ■ }
456 ■
457 ■ class _ActiveTripRow extends ConsumerWidget {
458 ■   const _ActiveTripRow({
459 ■     required this.trip,
460 ■     required this.buses,
461 ■     required this.drivers,
462 ■   });
463 ■
464 ■   final Trip trip;

```

```

465 ■ final List<Bus> buses;
466 ■ final List<AppUser> drivers;
467 ■
468 ■ @override
469 ■ Widget build(BuildContext context, WidgetRef ref) {
470 ■   Bus? bus;
471 ■   for (final Bus b in buses) {
472 ■     if (b.id == trip.busId) bus = b;
473 ■   }
474 ■   AppUser? driver;
475 ■   for (final AppUser d in drivers) {
476 ■     if (d.uid == trip.driverId) driver = d;
477 ■   }
478 ■
479 ■   return ListTile(
480 ■     contentPadding: EdgeInsets.zero,
481 ■     leading: Container(
482 ■       width: 42,
483 ■       height: 42,
484 ■       decoration: BoxDecoration(
485 ■         color: NmbColors.accentSoft,
486 ■         borderRadius: BorderRadius.circular(12),
487 ■       ),
488 ■       child: const Icon(Icons.directions_bus_rounded,
489 ■         color: NmbColors.accentDark,),
490 ■     ),
491 ■     title: Text(bus?.busNumber ?? trip.busId,
492 ■       style: NmbTypography.cardTitle,),
493 ■     subtitle: Text(
494 ■       '${driver?.fullName ?? 'Driver'} • '
495 ■       '${trip.startedAt != null ? 'Started at ${Formatters.timeOfDay(trip.startedAt!)}' : trip
496 ■         .direction.label}',
497 ■       style: NmbTypography.bodySecondary,
498 ■       maxLines: 1,
499 ■       overflow: TextOverflow.ellipsis,
500 ■     ),
501 ■     trailing: Container(
502 ■       padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
503 ■       decoration: BoxDecoration(
504 ■         color: NmbColors.successSoft,
505 ■         borderRadius: BorderRadius.circular(999),
506 ■       ),
507 ■       child: Text(
508 ■         'Live',
509 ■         style: NmbTypography.caption.copyWith(
510 ■           color: NmbColors.success,
511 ■           fontWeight: FontWeight.w800,
512 ■         ),
513 ■       ),
514 ■       onTap: () => context.go('/admin/monitoring'),
515 ■     );
516 ■   }
517 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/constants/nmb_constants.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/result.dart';
7 ■ import '../core/utils/validators.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../models/app_user.dart';
10 ■ import '../models/bus.dart';
11 ■ import '../models/bus_stop.dart';
12 ■ import '../providers/app_providers.dart';
13 ■ import '../providers/data_providers.dart';
14 ■
15 ■ /// Bottom sheet to provision a new student/driver account.
16 ■ /// Provisioning goes through the admin-only `provisionUser` Cloud Function –
17 ■ /// the app cannot create privileged accounts by itself.
18 ■ class UserEditorSheet extends ConsumerStatefulWidget {
19 ■   const UserEditorSheet({super.key, required this.role});
20 ■
21 ■   final String role; // 'student' | 'driver'
22 ■
23 ■   static Future<void> show(BuildContext context, {required String role}) =>
24 ■     showModalBottomSheet<void>(
25 ■       context: context,
26 ■       isScrollControlled: true,
27 ■       shape: const RoundedRectangleBorder(
28 ■         borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
29 ■       ),
30 ■       builder: (_) => UserEditorSheet(role: role),
31 ■     );
32 ■
33 ■   @override
34 ■   ConsumerState<UserEditorSheet> createState() => _UserEditorSheetState();
35 ■ }
36 ■
37 ■ class _UserEditorSheetState extends ConsumerState<UserEditorSheet> {
38 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
39 ■   final TextEditingController _name = TextEditingController();
40 ■   final TextEditingController _email = TextEditingController();
41 ■   final TextEditingController _password = TextEditingController();
42 ■   final TextEditingController _roll = TextEditingController();
43 ■   String? _selectedClass;
44 ■   String? _selectedSection;
45 ■   final TextEditingController _phone = TextEditingController();
46 ■   String? _busId;
47 ■   String? _stopId;
48 ■   bool _saving = false;
49 ■
50 ■   bool get isStudent => widget.role == 'student';
51 ■
52 ■   @override
53 ■   void dispose() {
54 ■     _name.dispose();
55 ■     _email.dispose();
56 ■     _password.dispose();
57 ■     _roll.dispose();
58 ■     _phone.dispose();

```

```

59 ■     super.dispose();
60 ■   }
61 ■
62 ■   Future<void> _save() async {
63 ■     if (!(_formKey.currentState?.validate() ?? false)) return;
64 ■     setState(() => _saving = true);
65 ■
66 ■     // Students: login-id class+section+roll se banta hai (email nahi
67 ■     // chahiye). Staff: normal email.
68 ■     final String loginEmail = isStudent
69 ■       ? StudentLoginId.email(
70 ■         klass: _selectedClass!,
71 ■         section: _selectedSection!,
72 ■         roll: _roll.text,
73 ■       )
74 ■       : _email.text.trim();
75 ■
76 ■     final Result<String> result =
77 ■       await ref.read(adminServiceProvider).provisionUser(
78 ■         email: loginEmail,
79 ■         fullName: _name.text.trim(),
80 ■         role: widget.role,
81 ■         temporaryPassword: _password.text,
82 ■         busId: _busId,
83 ■         stopId: isStudent ? _stopId : null,
84 ■         classSection: isStudent &&
85 ■           _selectedClass != null &&
86 ■           _selectedSection != null
87 ■           ? SchoolClasses.label(_selectedClass!, _selectedSection!)
88 ■           : null,
89 ■         rollNumber: isStudent ? _roll.text.trim() : null,
90 ■         phone: _phone.text.trim().isEmpty
91 ■           ? _phone.text.trim()
92 ■           : null,
93 ■       );
94 ■     if (!mounted) return;
95 ■     setState(() => _saving = false);
96 ■     result.when(
97 ■       ok: (_) {
98 ■         Navigator.of(context).pop();
99 ■         showNmbSnack(
100 ■           context,
101 ■           '${isStudent ? 'Student' : 'Driver'} account created.',
102 ■           isSuccess: true,
103 ■         );
104 ■       },
105 ■       err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
106 ■     );
107 ■   }
108 ■
109 ■   @override
110 ■   Widget build(BuildContext context) {
111 ■     final List<Bus> buses =
112 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
113 ■     final List<BusStop> stops = _busId == null
114 ■       ? const <BusStop>[]
115 ■       : ref.watch(stopsOfBusProvider(_busId!)).valueOrNull ??
116 ■         const <BusStop>[];

```

```

117 ■
118 ■ return Padding(
119 ■   padding: EdgeInsets.only(
120 ■     bottom: MediaQuery.of(context).viewInsets.bottom,
121 ■   ),
122 ■   child: SafeArea(
123 ■     child: SingleChildScrollView(
124 ■       padding: const EdgeInsets.all(20),
125 ■       child: Form(
126 ■         key: _formKey,
127 ■         child: Column(
128 ■           mainAxisAlignment: MainAxisAlignment.min,
129 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
130 ■           children: <Widget>[
131 ■             Text(
132 ■               isStudent ? 'Add student' : 'Add driver',
133 ■               style: NmbTypography.screenTitle,
134 ■             ),
135 ■             const SizedBox(height: 16),
136 ■             TextFormField(
137 ■               controller: _name,
138 ■               decoration:
139 ■                 const InputDecoration(hintText: 'Full name'),
140 ■               validator: (String? v) =>
141 ■                 Validators.requiredField(v, label: 'Name'),
142 ■             ),
143 ■             const SizedBox(height: 12),
144 ■             // Staff ko email chahiye; students ka login roll number
145 ■             // se hota hai (email nahi).
146 ■             if (!isStudent) ...<Widget>[
147 ■               TextFormField(
148 ■                 controller: _email,
149 ■                 keyboardType: TextInputType.emailAddress,
150 ■                 decoration:
151 ■                   const InputDecoration(hintText: 'Email'),
152 ■                 validator: Validators.email,
153 ■               ),
154 ■               const SizedBox(height: 12),
155 ■             ],
156 ■             TextFormField(
157 ■               controller: _password,
158 ■               decoration: InputDecoration(
159 ■                 hintText: isStudent ? 'Password' : 'Temporary password',
160 ■                 helperText: isStudent
161 ■                   ? 'Share this password with the student/parent.'
162 ■                   : 'User should change it after first sign-in.',
163 ■               ),
164 ■               validator: Validators strongPassword,
165 ■             ),
166 ■             const SizedBox(height: 12),
167 ■             if (isStudent) ...<Widget>[
168 ■               // Class & section – dropdowns, no typing needed.
169 ■               Row(
170 ■                 children: <Widget>[
171 ■                   Expanded(
172 ■                     flex: 3,
173 ■                     child: DropdownButtonFormField<String>(
174 ■                       value: _selectedClass,

```

```

175 ■         decoration:
176 ■             const InputDecoration(hintText: 'Class'),
177 ■         items: <DropDownMenuItem<String>>[
178 ■             for (final String c in SchoolClasses.classes)
179 ■                 DropDownMenuItem<String>(
180 ■                     value: c,
181 ■                     child: Text('Class $c'),
182 ■                 ),
183 ■     ],
184 ■     onChanged: (String? v) =>
185 ■         setState(() => _selectedClass = v),
186 ■     validator: (String? v) =>
187 ■         v == null ? 'Select class' : null,
188 ■ ),
189 ■ ),
190 ■ const SizedBox(width: 12),
191 ■ Expanded(
192 ■     flex: 2,
193 ■     child: DropDownButtonFormField<String>(
194 ■         value: _selectedSection,
195 ■         decoration:
196 ■             const InputDecoration(hintText: 'Section'),
197 ■         items: <DropDownMenuItem<String>>[
198 ■             for (final String s in SchoolClasses.sections)
199 ■                 DropDownMenuItem<String>(
200 ■                     value: s,
201 ■                     child: Text(s),
202 ■                 ),
203 ■         ],
204 ■         onChanged: (String? v) =>
205 ■             setState(() => _selectedSection = v),
206 ■         validator: (String? v) =>
207 ■             v == null ? 'Select' : null,
208 ■     ),
209 ■ ),
210 ■ ],
211 ■ ),
212 ■ const SizedBox(height: 12),
213 ■ TextFormField(
214 ■     controller: _roll,
215 ■     keyboardType: TextInputType.number,
216 ■     decoration: const InputDecoration(
217 ■         hintText: 'Roll number',
218 ■         helperText:
219 ■             'Student will log in with: Class + Section + '
220 ■             'Roll + Password',
221 ■     ),
222 ■     validator: (String? v) {
223 ■         final String r = (v ?? '').trim();
224 ■         if (r.isEmpty) return 'Enter roll number';
225 ■         if (int.tryParse(r) == null) return 'Numbers only';
226 ■         return null;
227 ■     },
228 ■ ),
229 ■ const SizedBox(height: 12),
230 ■ TextFormField(
231 ■     controller: _phone,
232 ■     keyboardType: TextInputType.phone,

```

```

233 ■         decoration: const InputDecoration(
234 ■             hintText: 'Parent mobile number (optional)',
235 ■             helperText: 'For school records',
236 ■         ),
237 ■     ),
238 ■     const SizedBox(height: 12),
239 ■ ] else ...<Widget>[
240 ■     TextFormField(
241 ■         controller: _phone,
242 ■         keyboardType: TextInputType.phone,
243 ■         decoration:
244 ■             const InputDecoration(hintText: 'Phone number'),
245 ■     ),
246 ■     const SizedBox(height: 12),
247 ■ ],
248 ■ DropdownButtonFormField<String>({
249 ■     value: _busId,
250 ■     decoration:
251 ■         const InputDecoration(hintText: 'Assign to bus'),
252 ■     items: <DropdownMenuItem<String>>[
253 ■         const DropdownMenuItem<String>({
254 ■             value: null,
255 ■             child: Text('No bus (assign later)'),
256 ■         },
257 ■         for (final Bus b in buses)
258 ■             DropdownMenuItem<String>({
259 ■                 value: b.id,
260 ■                 child: Text('${b.busNumber} – ${b.plateNumber}'),
261 ■             },
262 ■         ],
263 ■     onChanged: (String? v) => setState(() {
264 ■         _busId = v;
265 ■         _stopId = null;
266 ■     })),
267 ■ ),
268 ■ if (isStudent && _busId != null) ...<Widget>[
269 ■     const SizedBox(height: 12),
270 ■     DropdownButtonFormField<String>({
271 ■         value: _stopId,
272 ■         decoration: const InputDecoration(
273 ■             hintText: 'Assigned bus stop',
274 ■         ),
275 ■         items: <DropdownMenuItem<String>>[
276 ■             const DropdownMenuItem<String>({
277 ■                 value: null,
278 ■                 child: Text('No stop (assign later)'),
279 ■             },
280 ■             for (final BusStop s in stops)
281 ■                 DropdownMenuItem<String>({
282 ■                     value: s.id,
283 ■                     child: Text(s.name),
284 ■                 },
285 ■             ],
286 ■         onChanged: (String? v) => setState(() => _stopId = v),
287 ■     ),
288 ■ ],
289 ■ const SizedBox(height: 18),
290 ■ FilledButton(
291 ■     onPressed: _saving ? null : _save,

```

```

291 ■         child: _saving
292 ■         ? const SizedBox(
293 ■             width: 22,
294 ■             height: 22,
295 ■             child: CircularProgressIndicator(
296 ■                 strokeWidth: 2.4,
297 ■                 color: Colors.white,
298 ■             ),
299 ■         ),
300 ■         : const Text('Create account'),
301 ■     ),
302 ■     const SizedBox(height: 8),
303 ■ ],
304 ■ ),
305 ■ ),
306 ■ ),
307 ■ ),
308 ■ );
309 ■ }
310 ■ }
311 ■
312 ■ /// Bottom sheet to reassign an existing user's bus/stop or toggle access.
313 ■ /// All changes go through admin-only Cloud Functions (claims + audit log).
314 ■ class UserManageSheet extends ConsumerStatefulWidget {
315 ■     const UserManageSheet({super.key, required this.user});
316 ■
317 ■     final AppUser user;
318 ■
319 ■     static Future<void> show(BuildContext context, AppUser user) =>
320 ■         showModalBottomSheet<void>(
321 ■             context: context,
322 ■             isScrollControlled: true,
323 ■             shape: const RoundedRectangleBorder(
324 ■                 borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
325 ■             ),
326 ■             builder: (_) => UserManageSheet(user: user),
327 ■         );
328 ■
329 ■     @override
330 ■     ConsumerState<UserManageSheet> createState() => _UserManageSheetState();
331 ■ }
332 ■
333 ■ class _UserManageSheetState extends ConsumerState<UserManageSheet> {
334 ■     late String? _busId = widget.user.busId;
335 ■     late String? _stopId = widget.user.stopId;
336 ■     late final TextEditingController _editName =
337 ■         TextEditingController(text: widget.user.fullName);
338 ■     late final TextEditingController _editPhone =
339 ■         TextEditingController(text: widget.user.phone ?? '');
340 ■     bool _saving = false;
341 ■
342 ■     bool get isStudent => widget.user.role.name == 'student';
343 ■
344 ■     @override
345 ■     void dispose() {
346 ■         _editName.dispose();
347 ■         _editPhone.dispose();
348 ■         super.dispose();

```

```

349 ■ }
350 ■
351 ■ Future<void> _saveProfileEdits() async {
352 ■   final String name = _editName.text.trim();
353 ■   if (name.isEmpty) {
354 ■     showNmbSnack(context, 'Name cannot be empty.', isError: true);
355 ■     return;
356 ■   }
357 ■   setState(() => _saving = true);
358 ■   final Result<void> result =
359 ■     await ref.read(adminServiceProvider).updateUserProfile(
360 ■       uid: widget.user.uid,
361 ■       data: <String, dynamic>{
362 ■         'fullName': name,
363 ■         'phone':
364 ■           _editPhone.text.trim().isEmpty ? null : _editPhone.text.trim(),
365 ■       },
366 ■     );
367 ■   if (!mounted) return;
368 ■   setState(() => _saving = false);
369 ■   result.when(
370 ■     ok: (_) => showNmbSnack(context, 'Profile updated.', isSuccess: true),
371 ■     err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
372 ■   );
373 ■ }
374 ■
375 ■ Future<void> _forceLogout() async {
376 ■   final bool ok = await showNmbConfirmDialog(
377 ■     context,
378 ■     title: 'Force logout?',
379 ■     message: '${widget.user.fullName} will be signed out of their device '
380 ■       'immediately.',
381 ■     confirmLabel: 'Force logout',
382 ■     destructive: true,
383 ■     icon: Icons.phonelink_erase_rounded,
384 ■   );
385 ■   if (!ok || !mounted) return;
386 ■   setState(() => _saving = true);
387 ■   final Result<void> result =
388 ■     await ref.read(adminServiceProvider).forceLogout(widget.user.uid);
389 ■   if (!mounted) return;
390 ■   setState(() => _saving = false);
391 ■   result.when(
392 ■     ok: (_) {
393 ■       Navigator.of(context).pop();
394 ■       showNmbSnack(context, 'User signed out.', isSuccess: true);
395 ■     },
396 ■     err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
397 ■   );
398 ■ }
399 ■
400 ■ Future<void> _resetPassword() async {
401 ■   final TextEditingController pw = TextEditingController();
402 ■   final bool? go = await showDialog<bool>(
403 ■     context: context,
404 ■     builder: (BuildContext ctx) => AlertDialog(
405 ■       title: const Text('Set a new password'),
406 ■       content: TextField(

```

```

407 ■      controller: pw,
408 ■      decoration: const InputDecoration(
409 ■        hintText: 'New password (min 10, capital + number)',
410 ■      ),
411 ■    ),
412 ■    actions: <Widget>[
413 ■      TextButton(
414 ■        onPressed: () => Navigator.of(ctx).pop(false),
415 ■        child: const Text('Cancel'),
416 ■      ),
417 ■      FilledButton(
418 ■        onPressed: () => Navigator.of(ctx).pop(true),
419 ■        child: const Text('Reset'),
420 ■      ),
421 ■    ],
422 ■  ),
423 ■ );
424 ■ if (go != true || !mounted) return;
425 ■ if (Validators strongPassword(pw.text) != null) {
426 ■   showNmbSnack(context,
427 ■     'Password too weak – min 10 chars, 1 capital, 1 number.',
428 ■     isError: true,);
429 ■   return;
430 ■ }
431 ■ setState(() => _saving = true);
432 ■ final Result<void> result =
433 ■   await ref.read(adminServiceProvider).resetStudentPassword(
434 ■     student: widget.user,
435 ■     newPassword: pw.text,
436 ■   );
437 ■ if (!mounted) return;
438 ■ setState(() => _saving = false);
439 ■ result.when(
440 ■   ok: (_) {
441 ■     Navigator.of(context).pop();
442 ■     showNmbSnack(
443 ■       context,
444 ■       'Password reset! The student can now log in with the new '
445 ■       'password (same class + roll).',
446 ■       isSuccess: true,
447 ■     );
448 ■   },
449 ■   err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
450 ■ );
451 ■ }
452 ■
453 ■ Future<void> _saveAssignment() async {
454 ■   setState(() => _saving = true);
455 ■   final Result<void> result = isStudent
456 ■     ? await ref.read(adminServiceProvider).assignUserToBus(
457 ■       uid: widget.user.uid,
458 ■       busId: _busId,
459 ■       stopId: _stopId,
460 ■     )
461 ■     : await ref.read(adminServiceProvider).assignDriverToBus(
462 ■       driverUid: widget.user.uid,
463 ■       busId: _busId,
464 ■     );

```

```

465 ■   if (!mounted) return;
466 ■   setState(() => _saving = false);
467 ■   result.when(
468 ■     ok: (_) {
469 ■       Navigator.of(context).pop();
470 ■       showNmbSnackBar(context, 'Assignment updated.', isSuccess: true);
471 ■     },
472 ■     err: (AppFailure f) => showNmbSnackBar(context, f.message, isError: true),
473 ■   );
474 ■ }
475 ■
476 ■ Future<void> _toggleActive() async {
477 ■   final bool disable = widget.user.isActive;
478 ■   final bool confirmed = await showNmbConfirmDialog(
479 ■     context,
480 ■     title: disable ? 'Disable account?' : 'Enable account?',
481 ■     message: disable
482 ■       ? '${widget.user.fullName} will be signed out and blocked from '
483 ■         'the app until re-enabled.'
484 ■       : '${widget.user.fullName} will regain access to the app.',
485 ■     confirmLabel: disable ? 'Disable' : 'Enable',
486 ■     destructive: disable,
487 ■     icon: disable ? Icons.block_rounded : Icons.check_circle_outline,
488 ■   );
489 ■   if (!confirmed || !mounted) return;
490 ■   setState(() => _saving = true);
491 ■   final Result<void> result =
492 ■     await ref.read(adminServiceProvider).setAccountActive(
493 ■       uid: widget.user.uid,
494 ■       active: !widget.user.isActive,
495 ■     );
496 ■   if (!mounted) return;
497 ■   setState(() => _saving = false);
498 ■   result.when(
499 ■     ok: (_) {
500 ■       Navigator.of(context).pop();
501 ■       showNmbSnackBar(context, 'Account updated.', isSuccess: true);
502 ■     },
503 ■     err: (AppFailure f) => showNmbSnackBar(context, f.message, isError: true),
504 ■   );
505 ■ }
506 ■
507 ■ @override
508 ■ Widget build(BuildContext context) {
509 ■   final List<Bus> buses =
510 ■     ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
511 ■   final List<BusStop> stops = _busId == null
512 ■     ? const <BusStop>[]
513 ■     : ref.watch(stopsOfBusProvider(_busId!)).valueOrNull ??
514 ■       const <BusStop>[];
515 ■
516 ■   return SafeArea(
517 ■     child: SingleChildScrollView(
518 ■       padding: const EdgeInsets.all(20),
519 ■       child: Column(
520 ■         mainAxisAlignment: MainAxisAlignment.min,
521 ■         crossAxisAlignment: CrossAxisAlignment.stretch,
522 ■         children: <Widget>[

```

```

523 ■      Text(widget.user.fullName, style: NmbTypography.screenTitle),
524 ■      Text(
525 ■        isStudent
526 ■          ? 'Class ${widget.user.classSection ?? '?'} • '
527 ■          : 'Roll ${widget.user.rollNumber ?? '?'}'
528 ■        : widget.user.email,
529 ■        style: NmbTypography.bodySecondary,
530 ■      ),
531 ■      const SizedBox(height: 18),
532 ■
533 ■      // ■■ Profile edit ■■
534 ■      TextField(
535 ■        controller: _editName,
536 ■        decoration: const InputDecoration(hintText: 'Full name'),
537 ■      ),
538 ■      const SizedBox(height: 12),
539 ■      TextField(
540 ■        controller: _editPhone,
541 ■        keyboardType: TextInputType.phone,
542 ■        decoration: InputDecoration(
543 ■          hintText: isStudent
544 ■            ? 'Parent mobile number'
545 ■            : 'Phone number',
546 ■        ),
547 ■      ),
548 ■      const SizedBox(height: 12),
549 ■      OutlinedButton.icon(
550 ■        onPressed: _saving ? null : _saveProfileEdits,
551 ■        icon: const Icon(Icons.save_outlined),
552 ■        label: const Text('Save profile changes'),
553 ■      ),
554 ■      const SizedBox(height: 18),
555 ■      const Divider(),
556 ■      const SizedBox(height: 10),
557 ■
558 ■      DropdownButtonFormField<String>(
559 ■        value: _busId,
560 ■        decoration: const InputDecoration(hintText: 'Assigned bus'),
561 ■        items: <DropdownMenuItem<String>>[
562 ■          const DropdownMenuItem<String>(
563 ■            value: null,
564 ■            child: Text('No bus'),
565 ■          ),
566 ■          for (final Bus b in buses)
567 ■            DropdownMenuItem<String>(
568 ■              value: b.id,
569 ■              child: Text('${b.busNumber} – ${b.plateNumber}'),
570 ■            ),
571 ■        ],
572 ■        onChanged: (String? v) => setState(() {
573 ■          _busId = v;
574 ■          _stopId = null;
575 ■        })),
576 ■      ),
577 ■      if (isStudent && _busId != null) ...<Widget>[
578 ■        const SizedBox(height: 12),
579 ■        DropdownButtonFormField<String>(
580 ■          value: stops.any((BusStop s) => s.id == _stopId)

```

```

581 ■         ? _stopId
582 ■         : null,
583 ■     decoration:
584 ■         const InputDecoration(hintText: 'Assigned stop'),
585 ■     items: <DropDownMenuItem<String>>[
586 ■         const DropDownMenuItem<String>(
587 ■             value: null,
588 ■             child: Text('No stop'),
589 ■         ),
590 ■         for (final BusStop s in stops)
591 ■             DropDownMenuItem<String>(
592 ■                 value: s.id,
593 ■                 child: Text(s.name),
594 ■             ),
595 ■     ],
596 ■     onChanged: (String? v) => setState(() => _stopId = v),
597 ■ ),
598 ■ ],
599 ■ const SizedBox(height: 18),
600 ■ FilledButton(
601 ■     onPressed: _saving ? null : _saveAssignment,
602 ■     child: const Text('Save assignment'),
603 ■ ),
604 ■ const SizedBox(height: 10),
605 ■ // ■■ Device / login status (single-device security) ■■
606 ■ const SizedBox(height: 10),
607 ■ Builder(
608 ■     builder: (BuildContext ctx) {
609 ■         final Map<String, dynamic>? dev = widget.user.activeDevice;
610 ■         final String? devId = dev?['id'] as String?;
611 ■         final bool loggedIn = devId != null &&
612 ■             devId.isNotEmpty &&
613 ■             devId != 'REVOKED';
614 ■         return Container(
615 ■             padding: const EdgeInsets.all(12),
616 ■             decoration: BoxDecoration(
617 ■                 color: loggedIn
618 ■                     ? const Color(0xFFE2F3E7)
619 ■                     : const Color(0xFFF1F3F7),
620 ■                 borderRadius: BorderRadius.circular(12),
621 ■             ),
622 ■             child: Row(
623 ■                 children: <Widget>[
624 ■                     Icon(
625 ■                         loggedIn
626 ■                             ? Icons.phone_android_rounded
627 ■                             : Icons.phonelink_erase_rounded,
628 ■                         color: loggedIn
629 ■                             ? const Color(0xFF1E8E3E)
630 ■                             : const Color(0xFF8A94A6),
631 ■                     ),
632 ■                     const SizedBox(width: 10),
633 ■                     Expanded(
634 ■                         child: Column(
635 ■                             crossAxisAlignment: CrossAxisAlignment.start,
636 ■                             children: <Widget>[
637 ■                                 Text(
638 ■                                     loggedIn

```

```

639 ■         ? 'Logged in on: '
640 ■         '${dev!['name'] ?? 'a device'}'
641 ■         : 'Not logged in on any device',
642 ■         style: NmbTypography.cardTitle
643 ■         .copyWith(fontSize: 14),
644 ■     ),
645 ■     if (loggedIn && dev!['at'] != null)
646 ■     Text(
647 ■         'Since ${DateTime.fromMillisecondsSinceEpoch(dev['at'] as int).t
648 ■         oString().substring(0, 16)}',
649 ■         style: NmbTypography.caption,
650 ■     ),
651 ■ ),
652 ■ ),
653 ■ if (loggedIn)
654 ■     TextButton(
655 ■         onPressed: _saving ? null : _forceLogout,
656 ■         child: const Text('Force logout'),
657 ■     ),
658 ■ ],
659 ■ ),
660 ■ );
661 ■ },
662 ■ ),
663 ■ if (isStudent) ...<Widget>[
664 ■     const SizedBox(height: 10),
665 ■     OutlinedButton.icon(
666 ■         onPressed: _saving ? null : _resetPassword,
667 ■         icon: const Icon(Icons.lock_reset_rounded),
668 ■         label: const Text('Reset student password'),
669 ■     ),
670 ■ ],
671 ■ const SizedBox(height: 10),
672 ■ OutlinedButton.icon(
673 ■     onPressed: _saving ? null : _toggleActive,
674 ■     icon: Icon(
675 ■         widget.user.isActive
676 ■         ? Icons.block_rounded
677 ■         : Icons.check_circle_outline_rounded,
678 ■     ),
679 ■     label: Text(
680 ■         widget.user.isActive
681 ■         ? 'Disable account'
682 ■         : 'Enable account',
683 ■     ),
684 ■ ),
685 ■ const SizedBox(height: 8),
686 ■ ],
687 ■ ),
688 ■ ),
689 ■ );
690 ■ }
691 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/skeleton.dart';
9 ■ import '../core/widgets/state_views.dart';
10 ■ import '../core/widgets/status_pill.dart';
11 ■ import '../models/app_user.dart';
12 ■ import '../models/bus.dart';
13 ■ import '../providers/data_providers.dart';
14 ■ import '../widgets/user_editor_sheet.dart';
15 ■
16 ■ /// Manage students: list, search, add, and per-student detail sheet
17 ■ /// (assign bus/stop, enable/disable). All mutations go via Cloud Functions.
18 ■ class AdminStudentsScreen extends ConsumerStatefulWidget {
19 ■   const AdminStudentsScreen({super.key});
20 ■
21 ■   @override
22 ■   ConsumerState<AdminStudentsScreen> createState() =>
23 ■     _AdminStudentsScreenState();
24 ■ }
25 ■
26 ■ class _AdminStudentsScreenState extends ConsumerState<AdminStudentsScreen> {
27 ■   String _query = '';
28 ■   String? _classFilter; // null = All classes
29 ■
30 ■   @override
31 ■   Widget build(BuildContext context) {
32 ■     final AsyncValue<List<AppUser>> studentsAsync =
33 ■       ref.watch(allStudentsProvider);
34 ■     final List<Bus> buses =
35 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
36 ■
37 ■     return Scaffold(
38 ■       appBar: AppBar(title: const Text('Students')),
39 ■       floatingActionButton: FloatingActionButton.extended(
40 ■         onPressed: () => UserEditorSheet.show(context, role: 'student'),
41 ■         icon: const Icon(Icons.person_add_alt_1_rounded),
42 ■         label: const Text('Add student'),
43 ■       ),
44 ■       body: studentsAsync.when(
45 ■         loading: () => const ListSkeleton(),
46 ■         error: (Object e, _) => ErrorView(
47 ■           message: 'Couldn\'t load students.',
48 ■           onRetry: () => ref.invalidate(allStudentsProvider),
49 ■         ),
50 ■         data: (List<AppUser> students) {
51 ■           // Jo classes actually use ho rahi hain, unke filter chips banao.
52 ■           final List<String> presentClasses = students
53 ■             .map((AppUser s) => s.classSection ?? '')
54 ■             .where((String c) => c.isNotEmpty)
55 ■             .toSet()
56 ■             .toList()
57 ■             ..sort();
58 ■

```

```

59 ■      final List<AppUser> filtered = students.where((AppUser s) {
60 ■          final bool matchesClass =
61 ■              _classFilter == null || s.classSection == _classFilter;
62 ■          final bool matchesQuery = _query.isEmpty ||
63 ■              s.fullName.toLowerCase().contains(_query.toLowerCase()) ||
64 ■              s.email.toLowerCase().contains(_query.toLowerCase());
65 ■          return matchesClass && matchesQuery;
66 ■      }).toList();
67 ■
68 ■      return ResponsiveBody(
69 ■          child: Column(
70 ■              crossAxisAlignment: CrossAxisAlignment.stretch,
71 ■              children: <Widget>[
72 ■                  TextField(
73 ■                      decoration: const InputDecoration(
74 ■                          hintText: 'Search by name or email...',
75 ■                          prefixIcon: Icon(Icons.search_rounded),
76 ■                      ),
77 ■                      onChanged: (String v) => setState(() => _query = v),
78 ■                  ),
79 ■                  const SizedBox(height: 12),
80 ■
81 ■                  // Class-wise filter chips: "All" + jo classes maujood hain.
82 ■                  if (presentClasses.isNotEmpty)
83 ■                      SingleChildScrollView(
84 ■                          scrollDirection: Axis.horizontal,
85 ■                          child: Row(
86 ■                              children: <Widget>[
87 ■                                  Padding(
88 ■                                      padding: const EdgeInsets.only(right: 8),
89 ■                                      child: ChoiceChip(
90 ■                                          label: Text('All (${students.length})'),
91 ■                                          selected: _classFilter == null,
92 ■                                          onSelected: (_) =>
93 ■                                              setState(() => _classFilter = null),
94 ■                                      ),
95 ■                                  ),
96 ■                                  for (final String c in presentClasses)
97 ■                                      Padding(
98 ■                                          padding: const EdgeInsets.only(right: 8),
99 ■                                          child: ChoiceChip(
100 ■                                              label: Text(
101 ■                                                  'Class $c '
102 ■                                                  '(${students.where((AppUser s) => s.classSection == c).length})'
103 ■                                              ),
104 ■                                              selected: _classFilter == c,
105 ■                                              onSelected: (_) =>
106 ■                                                  setState(() => _classFilter = c),
107 ■                                          ),
108 ■                                      ),
109 ■                                  ],
110 ■                              ),
111 ■                          ),
112 ■                      const SizedBox(height: 14),
113 ■                      if (filtered.isEmpty)
114 ■                          const Padding(
115 ■                              padding: EdgeInsets.only(top: 60),

```

```

116 ■         child: EmptyState(
117 ■             icon: Icons.school_outlined,
118 ■             title: 'No students found',
119 ■             message:
120 ■                 'Add students with the button below, or adjust '
121 ■                 'your search.',
122 ■         ),
123 ■     )
124 ■     else
125 ■         for (final AppUser s in filtered) ...<Widget>[
126 ■             _StudentTile(student: s, buses: buses),
127 ■             const SizedBox(height: 10),
128 ■         ],
129 ■         const SizedBox(height: 90),
130 ■     ],
131 ■ ),
132 ■ );
133 ■ },
134 ■ ),
135 ■ );
136 ■ }
137 ■ }
138 ■
139 ■ bool _isLoggedIn(AppUser u) {
140 ■     final String? id = u.activeDevice?['id'] as String?;
141 ■     return id != null && id.isNotEmpty && id != 'REVOKED';
142 ■ }
143 ■
144 ■ class _StudentTile extends StatelessWidget {
145 ■     const _StudentTile({required this.student, required this.buses});
146 ■
147 ■     final AppUser student;
148 ■     final List<Bus> buses;
149 ■
150 ■     @override
151 ■     Widget build(BuildContext context) {
152 ■         String busLabel = 'No bus';
153 ■         for (final Bus b in buses) {
154 ■             if (b.id == student.busId) busLabel = b.busNumber;
155 ■         }
156 ■         return NmbCard(
157 ■             onTap: () => UserManageSheet.show(context, student),
158 ■             child: Row(
159 ■                 children: <Widget>[
160 ■                     CircleAvatar(
161 ■                         backgroundColor: NmbColors.primarySoft,
162 ■                         child: Text(
163 ■                             student.firstName.isNotEmpty
164 ■                                 ? student.firstName[0].toUpperCase()
165 ■                                 : '?',
166 ■                             style: NmbTypography.cardTitle
167 ■                                 .copyWith(color: NmbColors.primary),
168 ■                         ),
169 ■                     ),
170 ■                     const SizedBox(width: 12),
171 ■                     Expanded(
172 ■                         child: Column(
173 ■                             crossAxisAlignment: CrossAxisAlignment.start,

```

```

174 ■      children: <Widget>[
175 ■          Text(student.fullName, style: NmbTypography.cardTitle),
176 ■          Text(
177 ■              <String>[
178 ■                  if (student.classSection != null)
179 ■                      'Class ${student.classSection}',
180 ■                      busLabel,
181 ■                  ].join(' • '),
182 ■                  style: NmbTypography.bodySecondary,
183 ■              ),
184 ■          ],
185 ■      ),
186 ■  ),
187 ■  if (!student.isActive)
188 ■      const StatusPill(
189 ■          label: 'DISABLED',
190 ■          color: NmbColors.danger,
191 ■          background: NmbColors.dangerSoft,
192 ■      )
193 ■  else if (_isLoggedIn(student))
194 ■      const StatusPill(
195 ■          label: 'LOGGED IN',
196 ■          color: NmbColors.success,
197 ■          background: NmbColors.successSoft,
198 ■      )
199 ■  else
200 ■      const StatusPill(
201 ■          label: 'NOT LOGGED IN',
202 ■          color: NmbColors.textTertiary,
203 ■          background: NmbColors.divider,
204 ■      ),
205 ■      const Icon(Icons.chevron_right_rounded,
206 ■          color: NmbColors.textTertiary, ),
207 ■  ],
208 ■  ),
209 ■  );
210 ■ }
211 ■ }

```

lib/features/admin/screens/admin_drivers_screen.dart

125 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/widgets/nmb_card.dart';
7 ■ import '../core/widgets/responsive_scaffold_body.dart';
8 ■ import '../core/widgets/state_views.dart';
9 ■ import '../core/widgets/status_pill.dart';
10 ■ import '../models/app_user.dart';
11 ■ import '../models/bus.dart';
12 ■ import '../providers/data_providers.dart';
13 ■ import '../widgets/user_editor_sheet.dart';
14 ■
15 ■ /// Manage drivers: list, add, assign to bus, enable/disable.
16 ■ class AdminDriversScreen extends ConsumerWidget {
17 ■   const AdminDriversScreen({super.key});
18 ■
19 ■   @override
20 ■   Widget build(BuildContext context, WidgetRef ref) {
21 ■     final AsyncValue<List<AppUser>> driversAsync =
22 ■       ref.watch(allDriversProvider);
23 ■     final List<Bus> buses =
24 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
25 ■
26 ■     return Scaffold(
27 ■       appBar: AppBar(title: const Text('Drivers')),
28 ■       floatingActionButton: FloatingActionButton.extended(
29 ■         onPressed: () => UserEditorSheet.show(context, role: 'driver'),
30 ■         icon: const Icon(Icons.person_add_alt_1_rounded),
31 ■         label: const Text('Add driver'),
32 ■       ),
33 ■       body: driversAsync.when(
34 ■         loading: () => const LoadingView(),
35 ■         error: (Object e, _) => ErrorView(
36 ■           message: 'Couldn\'t load drivers.',
37 ■           onRetry: () => ref.invalidate(allDriversProvider),
38 ■         ),
39 ■         data: (List<AppUser> drivers) {
40 ■           if (drivers.isEmpty) {
41 ■             return const EmptyState(
42 ■               icon: Icons.badge_outlined,
43 ■               title: 'No drivers yet',
44 ■               message: 'Add your first driver with the button below.',
45 ■             );
46 ■           }
47 ■           return ResponsiveBody(
48 ■             child: Column(
49 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
50 ■               children: <Widget>[
51 ■                 for (final AppUser d in drivers) ...<Widget>[
52 ■                   _DriverTile(driver: d, buses: buses),
53 ■                   const SizedBox(height: 10),
54 ■                 ],
55 ■                 const SizedBox(height: 90),
56 ■               ],
57 ■             ),
58 ■           );

```

```

59 ■     },
60 ■     ),
61 ■   );
62 ■ }
63 ■ }
64 ■
65 ■ bool _isLoggedIn(AppUser u) {
66 ■   final String? id = u.activeDevice?['id'] as String?;
67 ■   return id != null && id.isNotEmpty && id != 'REVOKED';
68 ■ }
69 ■
70 ■ class _DriverTile extends StatelessWidget {
71 ■   const _DriverTile({required this.driver, required this.buses});
72 ■
73 ■   final AppUser driver;
74 ■   final List<Bus> buses;
75 ■
76 ■   @override
77 ■   Widget build(BuildContext context) {
78 ■     String busLabel = 'No bus assigned';
79 ■     for (final Bus b in buses) {
80 ■       if (b.id == driver.busId) busLabel = b.busNumber;
81 ■     }
82 ■     return NmbCard(
83 ■       onTap: () => UserManageSheet.show(context, driver),
84 ■       child: Row(
85 ■         children: <Widget>[
86 ■           const CircleAvatar(
87 ■             backgroundColor: NmbColors.accentSoft,
88 ■             child:
89 ■               Icon(Icons.badge_rounded, color: NmbColors.accentDark),
90 ■           ),
91 ■           const SizedBox(width: 12),
92 ■           Expanded(
93 ■             child: Column(
94 ■               crossAxisAlignment: CrossAxisAlignment.start,
95 ■               children: <Widget>[
96 ■                 Text(driver.fullName, style: NmbTypography.cardTitle),
97 ■                 Text(busLabel, style: NmbTypography.bodySecondary),
98 ■               ],
99 ■             ),
100 ■           ),
101 ■           if (!driver.isActive)
102 ■             const StatusPill(
103 ■               label: 'DISABLED',
104 ■               color: NmbColors.danger,
105 ■               background: NmbColors.dangerSoft,
106 ■             )
107 ■           else if (_isLoggedIn(driver))
108 ■             const StatusPill(
109 ■               label: 'LOGGED IN',
110 ■               color: NmbColors.success,
111 ■               background: NmbColors.successSoft,
112 ■             )
113 ■           else
114 ■             const StatusPill(
115 ■               label: 'NOT LOGGED IN',
116 ■               color: NmbColors.textTertiary,
117 ■               background: NmbColors.divider,
118 ■             ),
119 ■           const Icon(Icons.chevron_right_rounded,
120 ■             color: NmbColors.textTertiary, ),
121 ■         ],
122 ■       ),
123 ■     );
124 ■   }
125 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/widgets/state_views.dart';
11 ■ import '../models/app_user.dart';
12 ■ import '../models/bus.dart';
13 ■ import '../providers/app_providers.dart';
14 ■ import '../providers/data_providers.dart';
15 ■
16 ■ /// Manage buses: create/edit fleet. Structure: each bus gets a driver, a
17 ■ /// route, stops and students (assigned from their own screens).
18 ■ class AdminBusesScreen extends ConsumerWidget {
19 ■   const AdminBusesScreen({super.key});
20 ■
21 ■   @override
22 ■   Widget build(BuildContext context, WidgetRef ref) {
23 ■     final AsyncValue<List<Bus>> busesAsync = ref.watch(allBusesProvider);
24 ■     final List<AppUser> drivers =
25 ■       ref.watch(allDriversProvider).valueOrNull ?? const <AppUser>[];
26 ■     final List<AppUser> students =
27 ■       ref.watch(allStudentsProvider).valueOrNull ?? const <AppUser>[];
28 ■
29 ■     return Scaffold(
30 ■       appBar: AppBar(title: const Text('Buses')),
31 ■       floatingActionButton: FloatingActionButton.extended(
32 ■         onPressed: () => _BusEditorSheet.show(context),
33 ■         icon: const Icon(Icons.add_rounded),
34 ■         label: const Text('Add bus'),
35 ■       ),
36 ■       body: busesAsync.when(
37 ■         loading: () => const LoadingView(),
38 ■         error: (Object e, _) => ErrorView(
39 ■           message: 'Couldn\'t load buses.',
40 ■           onRetry: () => ref.invalidate(allBusesProvider),
41 ■         ),
42 ■         data: (List<Bus> buses) {
43 ■           if (buses.isEmpty) {
44 ■             return const EmptyState(
45 ■               icon: Icons.directions_bus_outlined,
46 ■               title: 'No buses yet',
47 ■               message: 'Create your first bus to start building the '
48 ■                 'transport structure.',
49 ■             );
50 ■           }
51 ■           return ResponsiveBody(
52 ■             child: Column(
53 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
54 ■               children: <Widget>[
55 ■                 for (final Bus bus in buses) ...<Widget>[
56 ■                   _BusTile(
57 ■                     bus: bus,
58 ■                     driverName: _driverName(bus, drivers),

```

```

59 ■         studentCount: students
60 ■         .where((AppUser s) => s.busId == bus.id)
61 ■         .length,
62 ■     ),
63 ■     const SizedBox(height: 10),
64 ■ ],
65 ■     const SizedBox(height: 90),
66 ■ ],
67 ■ ),
68 ■ );
69 ■ },
70 ■ ),
71 ■ );
72 ■ }
73 ■
74 ■ String _driverName(Bus bus, List<AppUser> drivers) {
75 ■     for (final AppUser d in drivers) {
76 ■         if (d.busId == bus.id) return d.fullName;
77 ■     }
78 ■     return 'No driver';
79 ■ }
80 ■ }
81 ■
82 ■ class _BusTile extends ConsumerWidget {
83 ■     const _BusTile({
84 ■         required this.bus,
85 ■         required this.driverName,
86 ■         required this.studentCount,
87 ■     });
88 ■
89 ■     final Bus bus;
90 ■     final String driverName;
91 ■     final int studentCount;
92 ■
93 ■     @override
94 ■     Widget build(BuildContext context, WidgetRef ref) {
95 ■         return NmbCard(
96 ■             onTap: () => _BusEditorSheet.show(context, bus: bus),
97 ■             child: Row(
98 ■                 children: <Widget>[
99 ■                     Container(
100 ■                         width: 48,
101 ■                         height: 48,
102 ■                         decoration: BoxDecoration(
103 ■                             color: bus.isActive
104 ■                                 ? NmbColors.accentSoft
105 ■                                 : NmbColors.divider,
106 ■                             borderRadius: BorderRadius.circular(14),
107 ■                         ),
108 ■                         child: Icon(
109 ■                             Icons.directions_bus_rounded,
110 ■                             color: bus.isActive
111 ■                                 ? NmbColors.accentDark
112 ■                                 : NmbColors.textTertiary,
113 ■                         ),
114 ■                     ),
115 ■                     const SizedBox(width: 12),
116 ■                     Expanded(

```

```

117 ■         child: Column(
118 ■             crossAxisAlignment: CrossAxisAlignment.start,
119 ■             children: <Widget>[
120 ■                 Text('${bus.busNumber} • ${bus.plateNumber}',
121 ■                     style: NmbTypography.cardTitle,),
122 ■                 Text(
123 ■                     '${driverName} • $studentCount students • '
124 ■                     '${bus.capacity} seats',
125 ■                     style: NmbTypography.bodySecondary,
126 ■                 ),
127 ■             ],
128 ■         ),
129 ■     ),
130 ■     const Icon(Icons.chevron_right_rounded,
131 ■         color: NmbColors.textTertiary,),
132 ■ ],
133 ■ ),
134 ■ );
135 ■ }
136 ■ }
137 ■
138 ■ class _BusEditorSheet extends ConsumerStatefulWidget {
139 ■     const _BusEditorSheet({this.bus});
140 ■
141 ■     final Bus? bus;
142 ■
143 ■     static Future<void> show(BuildContext context, {Bus? bus}) =>
144 ■         showModalBottomSheet<void>(
145 ■             context: context,
146 ■             isScrollControlled: true,
147 ■             shape: const RoundedRectangleBorder(
148 ■                 borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
149 ■             ),
150 ■             builder: (_) => _BusEditorSheet(bus: bus),
151 ■         );
152 ■
153 ■     @override
154 ■     ConsumerState<_BusEditorSheet> createState() => _BusEditorSheetState();
155 ■ }
156 ■
157 ■ class _BusEditorSheetState extends ConsumerState<_BusEditorSheet> {
158 ■     final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
159 ■     late final TextEditingController _number =
160 ■         TextEditingController(text: widget.bus?.busNumber);
161 ■     late final TextEditingController _plate =
162 ■         TextEditingController(text: widget.bus?.plateNumber);
163 ■     late final TextEditingController _capacity = TextEditingController(
164 ■         text: widget.bus == null ? '' : '${widget.bus!.capacity}',);
165 ■     late bool _isActive = widget.bus?.isActive ?? true;
166 ■     bool _saving = false;
167 ■
168 ■     @override
169 ■     void dispose() {
170 ■         _number.dispose();
171 ■         _plate.dispose();
172 ■         _capacity.dispose();
173 ■         super.dispose();
174 ■     }

```

```

175 ■
176 ■ Future<void> _save() async {
177 ■   if (!(_formKey.currentState?.validate() ?? false)) return;
178 ■   setState(() => _saving = true);
179 ■   try {
180 ■     final Map<String, dynamic> data = <String, dynamic>{
181 ■       'busNumber': _number.text.trim(),
182 ■       'plateNumber': _plate.text.trim(),
183 ■       'capacity': int.tryParse(_capacity.text.trim()) ?? 0,
184 ■       'isActive': _isActive,
185 ■     };
186 ■     if (widget.bus == null) {
187 ■       await ref.read(firestoreServiceProvider).createBus(
188 ■         Bus(
189 ■           id: '',
190 ■           busNumber: data['busNumber'] as String,
191 ■           plateNumber: data['plateNumber'] as String,
192 ■           capacity: data['capacity'] as int,
193 ■           isActive: _isActive,
194 ■         ),
195 ■       );
196 ■     } else {
197 ■       await ref
198 ■         .read(firestoreServiceProvider)
199 ■         .updateBus(widget.bus!.id, data);
200 ■     }
201 ■     if (!mounted) return;
202 ■     Navigator.of(context).pop();
203 ■     showNmbSnack(context, 'Bus saved.', isSuccess: true);
204 ■   } catch (_) {
205 ■     if (!mounted) return;
206 ■     showNmbSnack(context, 'Couldn\'t save the bus. Try again.',
207 ■       isError: true,);
208 ■   } finally {
209 ■     if (mounted) setState(() => _saving = false);
210 ■   }
211 ■ }
212 ■
213 ■ @override
214 ■ Widget build(BuildContext context) {
215 ■   return Padding(
216 ■     padding:
217 ■       EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
218 ■     child: SafeArea(
219 ■       child: SingleChildScrollView(
220 ■         padding: const EdgeInsets.all(20),
221 ■         child: Form(
222 ■           key: _formKey,
223 ■           child: Column(
224 ■             mainAxisAlignment: MainAxisAlignment.min,
225 ■             crossAxisAlignment: CrossAxisAlignment.stretch,
226 ■             children: <Widget>[
227 ■               Text(
228 ■                 widget.bus == null ? 'Add bus' : 'Edit bus',
229 ■                 style: NmbTypography.screenTitle,
230 ■               ),
231 ■               const SizedBox(height: 16),
232 ■               TextFormField(

```

```

233 ■         controller: _number,
234 ■         decoration: const InputDecoration(
235 ■             hintText: 'Bus number (e.g. Bus 3)',),
236 ■         validator: (String? v) =>
237 ■             Validators.requiredField(v, label: 'Bus number'),
238 ■     ),
239 ■     const SizedBox(height: 12),
240 ■     TextFormField(
241 ■         controller: _plate,
242 ■         decoration: const InputDecoration(
243 ■             hintText: 'Plate number (e.g. UK07 PA 1234)',),
244 ■         validator: (String? v) =>
245 ■             Validators.requiredField(v, label: 'Plate number'),
246 ■     ),
247 ■     const SizedBox(height: 12),
248 ■     TextFormField(
249 ■         controller: _capacity,
250 ■         keyboardType: TextInputType.number,
251 ■         decoration:
252 ■             const InputDecoration(hintText: 'Seat capacity'),
253 ■     ),
254 ■     SwitchListTile(
255 ■         contentPadding: EdgeInsets.zero,
256 ■         title: const Text('Bus is active'),
257 ■         value: _isActive,
258 ■         onChanged: (bool v) => setState(() => _isActive = v),
259 ■     ),
260 ■     const SizedBox(height: 10),
261 ■     FilledButton(
262 ■         onPressed: _saving ? null : _save,
263 ■         child: Text(widget.bus == null ? 'Create bus' : 'Save'),
264 ■     ),
265 ■     const SizedBox(height: 8),
266 ■ ],
267 ■ ),
268 ■ ),
269 ■ ),
270 ■ ),
271 ■ );
272 ■ }
273 ■ }

```

lib/features/admin/screens/admin_routes_stops_screen.dart

388 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_colors.dart';
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/widgets/state_views.dart';
11 ■ import '../models/bus.dart';
12 ■ import '../models/bus_stop.dart';
13 ■ import '../models/geo_point_data.dart';
14 ■ import '../providers/app_providers.dart';
15 ■ import '../providers/data_providers.dart';
16 ■
17 ■ /// Routes & Stops manager: pick a bus, then build its ordered stop list.
18 ■ /// School → Stop 1 → Stop 2 → ... exactly as the transport structure defines.
19 ■ class AdminRoutesStopsScreen extends ConsumerStatefulWidget {
20 ■   const AdminRoutesStopsScreen({super.key});
21 ■
22 ■   @override
23 ■   ConsumerState<AdminRoutesStopsScreen> createState() =>
24 ■     _AdminRoutesStopsScreenState();
25 ■ }
26 ■
27 ■ class _AdminRoutesStopsScreenState
28 ■   extends ConsumerState<AdminRoutesStopsScreen> {
29 ■   String? _busId;
30 ■
31 ■   /// Stop ko list mein upar/neeche karo aur naya order save karo.
32 ■   Future<void> _moveStop(List<BusStop> stops, int from, int to) async {
33 ■     final List<String> ids =
34 ■       stops.map((BusStop s) => s.id).toList(growable: true);
35 ■     final String moved = ids.removeAt(from);
36 ■     ids.insert(to, moved);
37 ■     try {
38 ■       await ref.read(firestoreServiceProvider).reorderStops(ids);
39 ■     } catch (_) {
40 ■       if (mounted) {
41 ■         showNmbSnack(context, 'Couldn\'t save the order, try again.',
42 ■           isError: true,);
43 ■       }
44 ■     }
45 ■   }
46 ■
47 ■   Future<void> _deleteStop(BusStop stop) async {
48 ■     final bool ok = await showNmbConfirmDialog(
49 ■       context,
50 ■       title: 'Delete stop?',
51 ■       message:
52 ■         '"${stop.name}" will be removed from the route. Students assigned '
53 ■         'to this stop will need a new stop.',
54 ■       confirmLabel: 'Delete',
55 ■       destructive: true,
56 ■       icon: Icons.delete_outline_rounded,
57 ■     );
58 ■     if (!ok || !mounted) return;

```

```

59 ■   try {
60 ■       await ref.read(firestoreServiceProvider).deactivateStop(stop.id);
61 ■       if (mounted) {
62 ■         showNmbSnack(context, 'Stop deleted.', isSuccess: true);
63 ■       }
64 ■     } catch (_) {
65 ■       if (mounted) {
66 ■         showNmbSnack(context, 'Couldn\'t delete the stop, try again.',
67 ■           isError: true,);
68 ■       }
69 ■     }
70 ■   }
71 ■
72 ■   @override
73 ■   Widget build(BuildContext context) {
74 ■     final List<Bus> buses =
75 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
76 ■     final List<BusStop> stops = _busId == null
77 ■       ? const <BusStop>[]
78 ■       : ref.watch(stopsOfBusProvider(_busId!)).valueOrNull ??
79 ■         const <BusStop>[];
80 ■
81 ■     return Scaffold(
82 ■       appBar: AppBar(title: const Text('Routes & Stops')),
83 ■       floatingActionButton: _busId == null
84 ■         ? null
85 ■         : FloatingActionButton.extended(
86 ■           onPressed: () => _StopEditorSheet.show(
87 ■             context,
88 ■             busId: _busId!,
89 ■             nextOrder: stops.length + 1,
90 ■           ),
91 ■           icon: const Icon(Icons.add_location_alt_rounded),
92 ■           label: const Text('Add stop'),
93 ■         ),
94 ■       body: ResponsiveBody(
95 ■         child: Column(
96 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
97 ■           children: <Widget>[
98 ■             DropdownButtonFormField<String>(
99 ■               value: _busId,
100 ■               decoration: const InputDecoration(
101 ■                 hintText: 'Select a bus to manage its route',
102 ■                 prefixIcon: Icon(Icons.directions_bus_rounded),
103 ■               ),
104 ■               items: <DropdownMenuItem<String>>[
105 ■                 for (final Bus b in buses)
106 ■                   DropdownMenuItem<String>(
107 ■                     value: b.id,
108 ■                     child: Text('${b.busNumber} – ${b.plateNumber}'),
109 ■                   ),
110 ■               ],
111 ■               onChanged: (String? v) => setState(() => _busId = v),
112 ■             ),
113 ■             const SizedBox(height: 16),
114 ■             if (_busId == null)
115 ■               const Padding(
116 ■                 padding: EdgeInsets.only(top: 40),

```

```

117 ■         child: EmptyState(
118 ■             icon: Icons.route_outlined,
119 ■             title: 'Pick a bus',
120 ■             message:
121 ■                 'Choose a bus above to view and edit its route and '
122 ■                 'stops in order.',
123 ■         ),
124 ■     ),
125 ■     else if (stops.isEmpty)
126 ■         const Padding(
127 ■             padding: EdgeInsets.only(top: 40),
128 ■             child: EmptyState(
129 ■                 icon: Icons.add_location_alt_outlined,
130 ■                 title: 'No stops yet',
131 ■                 message:
132 ■                     'Add the first stop for this route. Stops appear to '
133 ■                     'students in the order you create them.',
134 ■             ),
135 ■         )
136 ■     else ...<Widget>[
137 ■         const Text('Route stops (in order)',
138 ■             style: NmbTypography.sectionTitle,),
139 ■         const SizedBox(height: 10),
140 ■         for (int i = 0; i < stops.length; i++) ...<Widget>[
141 ■             NmbCard(
142 ■                 onTap: () => _StopEditorSheet.show(
143 ■                     context,
144 ■                     busId: _busId!,
145 ■                     stop: stops[i],
146 ■                     nextOrder: stops[i].order,
147 ■                 ),
148 ■                 padding: const EdgeInsets.all(14),
149 ■                 child: Row(
150 ■                     children: <Widget>[
151 ■                         Container(
152 ■                             width: 30,
153 ■                             height: 30,
154 ■                             decoration: const BoxDecoration(
155 ■                                 color: NmbColors.primarySoft,
156 ■                                 shape: BoxShape.circle,
157 ■                             ),
158 ■                             child: Center(
159 ■                                 child: Text(
160 ■                                     '${i + 1}',
161 ■                                     style: NmbTypography.caption.copyWith(
162 ■                                         color: NmbColors.primaryDark,
163 ■                                         fontWeight: FontWeight.w800,
164 ■                                     ),
165 ■                                 ),
166 ■                             ),
167 ■                         ),
168 ■                         const SizedBox(width: 12),
169 ■                         Expanded(
170 ■                             child: Column(
171 ■                                 crossAxisAlignment: CrossAxisAlignment.start,
172 ■                                 children: <Widget>[
173 ■                                     Text(stops[i].name,
174 ■                                         style: NmbTypography.cardTitle,),

```

```

175 ■          Text(
176 ■            '${stops[i].location.lat.toStringAsFixed(5)}, '
177 ■            '${stops[i].location.lng.toStringAsFixed(5)}'
178 ■            '${stops[i].scheduledTime != null ? ' • ${stops[i].scheduledTime}'
179 ■            : ''}',
180 ■            style: NmbTypography.caption,
181 ■          ),
182 ■        ],
183 ■      ),
184 ■      // Upar / neeche reorder
185 ■      if (i > 0)
186 ■        IconButton(
187 ■          tooltip: 'Move up',
188 ■          visualDensity: VisualDensity.compact,
189 ■          icon: const Icon(Icons.arrow_upward_rounded,
190 ■            size: 20, color: NmbColors.textSecondary,),
191 ■          onPressed: () => _moveStop(stops, i, i - 1),
192 ■        ),
193 ■      if (i < stops.length - 1)
194 ■        IconButton(
195 ■          tooltip: 'Move down',
196 ■          visualDensity: VisualDensity.compact,
197 ■          icon: const Icon(Icons.arrow_downward_rounded,
198 ■            size: 20, color: NmbColors.textSecondary,),
199 ■          onPressed: () => _moveStop(stops, i, i + 1),
200 ■        ),
201 ■        IconButton(
202 ■          tooltip: 'Delete stop',
203 ■          visualDensity: VisualDensity.compact,
204 ■          icon: const Icon(Icons.delete_outline_rounded,
205 ■            size: 20, color: NmbColors.danger,),
206 ■          onPressed: () => _deleteStop(stops[i]),
207 ■        ),
208 ■        const Icon(Icons.edit_outlined,
209 ■          size: 18, color: NmbColors.textTertiary,),
210 ■      ],
211 ■    ),
212 ■  ),
213 ■  const SizedBox(height: 8),
214 ■ ],
215 ■ ],
216 ■ const SizedBox(height: 90),
217 ■ ],
218 ■ ),
219 ■ ),
220 ■ );
221 ■ }
222 ■ }
223 ■
224 ■ class _StopEditorSheet extends ConsumerStatefulWidget {
225 ■   const _StopEditorSheet({
226 ■     required this.busId,
227 ■     required this.nextOrder,
228 ■     this.stop,
229 ■   });
230 ■
231 ■   final String busId;

```

```

232 ■ final int nextOrder;
233 ■ final BusStop? stop;
234 ■
235 ■ static Future<void> show(
236 ■     BuildContext context, {
237 ■         required String busId,
238 ■         required int nextOrder,
239 ■         BusStop? stop,
240 ■     }) =>
241 ■         showModalBottomSheet<void>(
242 ■             context: context,
243 ■             isScrollControlled: true,
244 ■             shape: const RoundedRectangleBorder(
245 ■                 borderRadius: BorderRadius.vertical(top: Radius.circular(24)),
246 ■             ),
247 ■             builder: (_) => _StopEditorSheet(
248 ■                 busId: busId,
249 ■                 nextOrder: nextOrder,
250 ■                 stop: stop,
251 ■             ),
252 ■         );
253 ■
254 ■ @override
255 ■ ConsumerState<_StopEditorSheet> createState() => _StopEditorSheetState();
256 ■ }
257 ■
258 ■ class _StopEditorSheetState extends ConsumerState<_StopEditorSheet> {
259 ■     final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
260 ■     late final TextEditingController _name =
261 ■         TextEditingController(text: widget.stop?.name);
262 ■     late final TextEditingController _lat = TextEditingController(
263 ■         text: widget.stop == null ? '' : '${widget.stop!.location.lat}',);
264 ■     late final TextEditingController _lng = TextEditingController(
265 ■         text: widget.stop == null ? '' : '${widget.stop!.location.lng}',);
266 ■     late final TextEditingController _time =
267 ■         TextEditingController(text: widget.stop?.scheduledTime);
268 ■     bool _saving = false;
269 ■
270 ■     @override
271 ■     void dispose() {
272 ■         _name.dispose();
273 ■         _lat.dispose();
274 ■         _lng.dispose();
275 ■         _time.dispose();
276 ■         super.dispose();
277 ■     }
278 ■
279 ■     Future<void> _save() async {
280 ■         if (!(_formKey.currentState?.validate() ?? false)) return;
281 ■         setState(() => _saving = true);
282 ■         try {
283 ■             final BusStop stop = BusStop(
284 ■                 id: widget.stop?.id ?? '',
285 ■                 name: _name.text.trim(),
286 ■                 routeId: widget.stop?.routeId ?? '',
287 ■                 busId: widget.busId,
288 ■                 location: GeoPointData(
289 ■                     lat: double.parse(_lat.text.trim()),

```

```

290 ■         lng: double.parse(_lng.text.trim()),
291 ■     ),
292 ■     order: widget.nextOrder,
293 ■     scheduledTime:
294 ■         _time.text.trim().isEmpty ? null : _time.text.trim(),
295 ■ );
296 ■ if (widget.stop == null) {
297 ■     await ref.read(firestoreServiceProvider).createStop(stop);
298 ■ } else {
299 ■     await ref
300 ■         .read(firestoreServiceProvider)
301 ■         .updateStop(widget.stop!.id, stop.toMap());
302 ■ }
303 ■ if (!mounted) return;
304 ■ Navigator.of(context).pop();
305 ■ showNmbSnack(context, 'Stop saved.', isSuccess: true);
306 ■ } catch (_) {
307 ■     if (!mounted) return;
308 ■     showNmbSnack(context, 'Couldn\'t save the stop.', isError: true);
309 ■ } finally {
310 ■     if (mounted) setState(() => _saving = false);
311 ■ }
312 ■ }
313 ■
314 ■ @override
315 ■ Widget build(BuildContext context) {
316 ■     return Padding(
317 ■         padding:
318 ■             EdgeInsets.only(bottom: MediaQuery.of(context).viewInsets.bottom),
319 ■         child: SafeArea(
320 ■             child: SingleChildScrollView(
321 ■                 padding: const EdgeInsets.all(20),
322 ■                 child: Form(
323 ■                     key: _formKey,
324 ■                     child: Column(
325 ■                         mainAxisAlignment: MainAxisAlignment.min,
326 ■                         crossAxisAlignment: CrossAxisAlignment.stretch,
327 ■                         children: <Widget>[
328 ■                             Text(
329 ■                                 widget.stop == null ? 'Add stop' : 'Edit stop',
330 ■                                 style: NmbTypography.screenTitle,
331 ■                             ),
332 ■                             const SizedBox(height: 16),
333 ■                             TextFormField(
334 ■                                 controller: _name,
335 ■                                 decoration: const InputDecoration(
336 ■                                     hintText: 'Stop name (e.g. Rajpur Road Gate 2)',
337 ■                                     validator: (String? v) =>
338 ■                                         Validators.requiredField(v, label: 'Stop name'),
339 ■                                 ),
340 ■                             const SizedBox(height: 12),
341 ■                             Row(
342 ■                                 children: <Widget>[
343 ■                                     Expanded(
344 ■                                         child: TextFormField(
345 ■                                             controller: _lat,
346 ■                                             keyboardType:
347 ■                                                 const TextInputType.numberWithOptions(

```

```

348 ■         decimal: true, signed: true,)),
349 ■         decoration:
350 ■           const InputDecoration(hintText: 'Latitude'),
351 ■         validator: Validators.latitude,
352 ■       ),
353 ■     ),
354 ■     const SizedBox(width: 12),
355 ■     Expanded(
356 ■       child: TextFormField(
357 ■         controller: _lng,
358 ■         keyboardType:
359 ■           const TextInputType.numberWithOptions(
360 ■             decimal: true, signed: true,)),
361 ■         decoration:
362 ■           const InputDecoration(hintText: 'Longitude'),
363 ■         validator: Validators.longitude,
364 ■       ),
365 ■     ),
366 ■   ],
367 ■ ),
368 ■ const SizedBox(height: 12),
369 ■ TextFormField(
370 ■   controller: _time,
371 ■   decoration: const InputDecoration(
372 ■     hintText: 'Scheduled time (e.g. 07:15) – optional',),
373 ■ ),
374 ■ const SizedBox(height: 18),
375 ■ FilledButton(
376 ■   onPressed: _saving ? null : _save,
377 ■   child:
378 ■     Text(widget.stop == null ? 'Create stop' : 'Save'),
379 ■ ),
380 ■ const SizedBox(height: 8),
381 ■ ],
382 ■ ),
383 ■ ),
384 ■ ),
385 ■ ),
386 ■ );
387 ■ }
388 ■ }

```

lib/features/admin/screens/admin_monitoring_screen.dart

220 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_map/flutter_map.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■ import 'package:latlong2/latlong.dart';
5 ■
6 ■ import '../core/constants/enums.dart';
7 ■ import '../core/theme/nmb_colors.dart';
8 ■ import '../core/theme/nmb_typography.dart';
9 ■ import '../core/utils/formatters.dart';
10 ■ import '../core/widgets/nmb_card.dart';
11 ■ import '../core/widgets/state_views.dart';
12 ■ import '../core/widgets/status_pill.dart';
13 ■ import '../models/bus.dart';
14 ■ import '../models/live_location.dart';
15 ■ import '../models/trip.dart';
16 ■ import '../providers/data_providers.dart';
17 ■ import '../shared/bus_marker_widget.dart';
18 ■
19 ■ /// Live fleet monitoring on OpenStreetMap: all active buses on one map +
20 ■ /// trip status list. Admin-only (route fence + backend rules).
21 ■ class AdminMonitoringScreen extends ConsumerStatefulWidget {
22 ■   const AdminMonitoringScreen({super.key});
23 ■
24 ■   @override
25 ■   ConsumerState<AdminMonitoringScreen> createState() =>
26 ■     _AdminMonitoringScreenState();
27 ■ }
28 ■
29 ■ class _AdminMonitoringScreenState
30 ■   extends ConsumerState<AdminMonitoringScreen> {
31 ■   final MapController _controller = MapController();
32 ■   bool _mapReady = false;
33 ■
34 ■   @override
35 ■   Widget build(BuildContext context) {
36 ■     final AsyncValue<List<Trip>> tripsAsync = ref.watch(activeTripsProvider);
37 ■     final List<Bus> buses =
38 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
39 ■
40 ■     return Scaffold(
41 ■       appBar: AppBar(title: const Text('Live Monitoring')),
42 ■       body: tripsAsync.when(
43 ■         loading: () => const LoadingView(),
44 ■         error: (Object e, _) => ErrorView(
45 ■           message: 'Couldn\'t load active trips.',
46 ■           onRetry: () => ref.invalidate(activeTripsProvider),
47 ■         ),
48 ■         data: (List<Trip> trips) {
49 ■           if (trips.isEmpty) {
50 ■             return const EmptyState(
51 ■               icon: Icons.radar_rounded,
52 ■               title: 'No active trips',
53 ■               message: 'When drivers start trips, their buses appear here '
54 ■                 'in real time.',
55 ■             );
56 ■           }
57 ■
58 ■           final List<Marker> markers = <Marker>[];

```

```

59 ■      LatLng? firstPos;
60 ■      for (final Trip trip in trips) {
61 ■          final LiveLocation? live =
62 ■              ref.watch(busLiveLocationProvider(trip.busId)).valueOrNull;
63 ■          if (live == null) continue;
64 ■          final LatLng pos = LatLng(live.position.lat, live.position.lng);
65 ■          firstPos ??= pos;
66 ■          Bus? bus;
67 ■          for (final Bus b in buses) {
68 ■              if (b.id == trip.busId) bus = b;
69 ■          }
70 ■          markers.add(
71 ■              Marker(
72 ■                  point: pos,
73 ■                  width: 60,
74 ■                  height: 74,
75 ■                  child: Column(
76 ■                      children: <Widget>[
77 ■                          const BusMarkerWidget(size: 48),
78 ■                          const SizedBox(height: 2),
79 ■                          Container(
80 ■                              padding: const EdgeInsets.symmetric(
81 ■                                  horizontal: 6, vertical: 1,),
82 ■                              decoration: BoxDecoration(
83 ■                                  color: NmbColors.primaryDark,
84 ■                                  borderRadius: BorderRadius.circular(8),
85 ■                              ),
86 ■                              child: Text(
87 ■                                  bus?.busNumber ?? trip.busId,
88 ■                                  style: const TextStyle(
89 ■                                      fontSize: 10,
90 ■                                      fontWeight: FontWeight.w700,
91 ■                                      color: Colors.white,
92 ■                                  ),
93 ■                                  maxLines: 1,
94 ■                                  overflow: TextOverflow.ellipsis,
95 ■                              ),
96 ■                          ),
97 ■                      ],
98 ■                  ),
99 ■              ),
100 ■          );
101 ■      }
102 ■
103 ■      return Column(
104 ■          children: <Widget>[
105 ■              Expanded(
106 ■                  flex: 3,
107 ■                  child: FlutterMap(
108 ■                      mapController: _controller,
109 ■                      options: MapOptions(
110 ■                          // Default center: Meerut, India (jab tak koi bus live nahi)
111 ■                          initialCenter:
112 ■                              firstPos ?? const LatLng(28.9845, 77.7064),
113 ■                          initialZoom: 12,
114 ■                          onMapReady: () => _mapReady = true,
115 ■                      ),
116 ■                      children: <Widget>[

```

```

117 ■         TileLayer(
118 ■             urlTemplate:
119 ■                 'https://tile.openstreetmap.org/{z}/{x}/{y}.png',
120 ■             userAgentPackageName: 'com.srbs.never_miss_bus',
121 ■         ),
122 ■         MarkerLayer(markers: markers),
123 ■         const Align(
124 ■             alignment: Alignment.bottomLeft,
125 ■             child: Padding(
126 ■                 padding: EdgeInsets.all(4),
127 ■                 child: Text(
128 ■                     '© OpenStreetMap contributors',
129 ■                     style: TextStyle(
130 ■                         fontSize: 10,
131 ■                         color: NmbColors.textSecondary,
132 ■                     ),
133 ■                 ),
134 ■             ),
135 ■         ),
136 ■     ],
137 ■ ),
138 ■ ),
139 ■ Expanded(
140 ■     flex: 2,
141 ■     child: ListView(
142 ■         padding: const EdgeInsets.all(16),
143 ■         children: <Widget>[
144 ■             for (final Trip trip in trips) ...<Widget>[
145 ■                 _TripStatusTile(
146 ■                     trip: trip,
147 ■                     buses: buses,
148 ■                     onFocus: (LatLng pos) {
149 ■                         if (_mapReady) _controller.move(pos, 15);
150 ■                     },
151 ■                 ),
152 ■                 const SizedBox(height: 10),
153 ■             ],
154 ■         ],
155 ■     ),
156 ■ ),
157 ■ ],
158 ■ );
159 ■ },
160 ■ ),
161 ■ );
162 ■ }
163 ■ }
164 ■
165 ■ class _TripStatusTile extends ConsumerWidget {
166 ■     const _TripStatusTile({
167 ■         required this.trip,
168 ■         required this.buses,
169 ■         required this.onFocus,
170 ■     });
171 ■
172 ■     final Trip trip;
173 ■     final List<Bus> buses;
174 ■     final void Function(LatLng) onFocus;

```

```

175 ■
176 ■ @override
177 ■ Widget build(BuildContext context, WidgetRef ref) {
178 ■   final LiveLocation? live =
179 ■     ref.watch(busLiveLocationProvider(trip.busId)).valueOrNull;
180 ■   ref.watch(freshnessTickProvider);
181 ■   final LocationFreshness freshness =
182 ■     live?.freshness() ?? LocationFreshness.unavailable;
183 ■
184 ■   Bus? bus;
185 ■   for (final Bus b in buses) {
186 ■     if (b.id == trip.busId) bus = b;
187 ■   }
188 ■
189 ■   return NmbCard(
190 ■     onTap: live == null
191 ■       ? null
192 ■       : () => onFocus(LatLng(live.position.lat, live.position.lng)),
193 ■     padding: const EdgeInsets.all(14),
194 ■     child: Row(
195 ■       children: <Widget>[
196 ■         const Icon(Icons.directions_bus_rounded, color: NmbColors.primary),
197 ■         const SizedBox(width: 12),
198 ■         Expanded(
199 ■           child: Column(
200 ■             crossAxisAlignment: CrossAxisAlignment.start,
201 ■             children: <Widget>[
202 ■               Text(bus?.busNumber ?? trip.busId,
203 ■                 style: NmbTypography.cardTitle,),
204 ■               Text(
205 ■                 live == null
206 ■                   ? '${trip.direction.label} • waiting for GPS...'
207 ■                   : '${trip.direction.label} • '
208 ■                     '${live.speedKmh.toStringAsFixed(0)} km/h • '
209 ■                     '${Formatters.relativeTime(live.updatedAt)}',
210 ■                 style: NmbTypography.bodySecondary,
211 ■               ),
212 ■             ],
213 ■           ),
214 ■         ),
215 ■         StatusPill.freshness(freshness),
216 ■       ],
217 ■     ),
218 ■   );
219 ■ }
220 ■ }

```

lib/features/admin/screens/admin_notifications_screen.dart

179 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/theme/nmb_typography.dart';
5 ■ import '../core/utils/result.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../models/bus.dart';
11 ■ import '../providers/app_providers.dart';
12 ■ import '../providers/data_providers.dart';
13 ■
14 ■ /// Compose an announcement to everyone or to one bus's students.
15 ■ /// Delivery is server-side (Cloud Function): audiences are resolved from
16 ■ /// the database, so a client can never widen the recipient list.
17 ■ class AdminNotificationsScreen extends ConsumerStatefulWidget {
18 ■   const AdminNotificationsScreen({super.key});
19 ■
20 ■   @override
21 ■   ConsumerState<AdminNotificationsScreen> createState() =>
22 ■     _AdminNotificationsScreenState();
23 ■ }
24 ■
25 ■ class _AdminNotificationsScreenState
26 ■   extends ConsumerState<AdminNotificationsScreen> {
27 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
28 ■   final TextEditingController _title = TextEditingController();
29 ■   final TextEditingController _body = TextEditingController();
30 ■   String _scope = 'all';
31 ■   String? _busId;
32 ■   bool _sending = false;
33 ■
34 ■   @override
35 ■   void dispose() {
36 ■     _title.dispose();
37 ■     _body.dispose();
38 ■     super.dispose();
39 ■   }
40 ■
41 ■   Future<void> _send() async {
42 ■     if (!(_formKey.currentState?.validate() ?? false)) return;
43 ■     if (_scope == 'bus' && _busId == null) {
44 ■       showNmbSnackBar(context, 'Choose a bus for a bus-specific announcement.',
45 ■         isError: true,);
46 ■       return;
47 ■     }
48 ■     final bool confirmed = await showNmbConfirmDialog(
49 ■       context,
50 ■       title: 'Send announcement?',
51 ■       message: _scope == 'all'
52 ■         ? 'This will notify every active user.'
53 ■         : 'This will notify all students and the driver of the selected '
54 ■           'bus.',
55 ■       confirmLabel: 'Send',
56 ■       icon: Icons.campaign_rounded,
57 ■     );
58 ■     if (!confirmed || !mounted) return;

```

```

59 ■
60 ■     setState(() => _sending = true);
61 ■     final Result<void> result =
62 ■         await ref.read(adminServiceProvider).sendAnnouncement(
63 ■             title: _title.text.trim(),
64 ■             body: _body.text.trim(),
65 ■             scope: _scope,
66 ■             busId: _scope == 'bus' ? _busId : null,
67 ■         );
68 ■     if (!mounted) return;
69 ■     setState(() => _sending = false);
70 ■     result.when(
71 ■         ok: (_) {
72 ■             _title.clear();
73 ■             _body.clear();
74 ■             showNmbSnack(context, 'Announcement sent.', isSuccess: true);
75 ■         },
76 ■         err: (AppFailure f) => showNmbSnack(context, f.message, isError: true),
77 ■     );
78 ■ }
79 ■
80 ■ @override
81 ■ Widget build(BuildContext context) {
82 ■     final List<Bus> buses =
83 ■         ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
84 ■
85 ■     return Scaffold(
86 ■         appBar: AppBar(title: const Text('Notifications')),
87 ■         body: ResponsiveBody(
88 ■             child: Form(
89 ■                 key: _formKey,
90 ■                 child: Column(
91 ■                     crossAxisAlignment: CrossAxisAlignment.stretch,
92 ■                     children: <Widget>[
93 ■                         const Text('Send an announcement',
94 ■                             style: NmbTypography.sectionTitle,),
95 ■                         const SizedBox(height: 12),
96 ■                         NmbCard(
97 ■                             child: Column(
98 ■                                 crossAxisAlignment: CrossAxisAlignment.stretch,
99 ■                                 children: <Widget>[
100 ■                                     TextFormField(
101 ■                                         controller: _title,
102 ■                                         decoration:
103 ■                                             const InputDecoration(hintText: 'Title'),
104 ■                                         maxLength: 80,
105 ■                                         validator: (String? v) =>
106 ■                                             Validators.requiredField(v, label: 'Title'),
107 ■                                     ),
108 ■                                     const SizedBox(height: 8),
109 ■                                     TextFormField(
110 ■                                         controller: _body,
111 ■                                         decoration:
112 ■                                             const InputDecoration(hintText: 'Message'),
113 ■                                         maxLines: 4,
114 ■                                         maxLength: 400,
115 ■                                         validator: (String? v) =>
116 ■                                             Validators.requiredField(v, label: 'Message'),

```

```

117 ■      ),
118 ■      const SizedBox(height: 12),
119 ■      SegmentedButton<String>(  

120 ■        segments: const <ButtonSegment<String>>[  

121 ■          ButtonSegment<String>(  

122 ■            value: 'all',  

123 ■            label: Text('Everyone'),  

124 ■            icon: Icon(Icons.groups_rounded),  

125 ■          ),  

126 ■          ButtonSegment<String>(  

127 ■            value: 'bus',  

128 ■            label: Text('One bus'),  

129 ■            icon: Icon(Icons.directions_bus_rounded),  

130 ■          ),  

131 ■        ],  

132 ■        selected: <String>{_scope},  

133 ■        onSelectionChanged: (Set<String> s) =>  

134 ■          setState(() => _scope = s.first),  

135 ■      ),  

136 ■      if (_scope == 'bus') ...<Widget>[  

137 ■        const SizedBox(height: 12),  

138 ■        DropdownButtonFormField<String>(  

139 ■          value: _busId,  

140 ■          decoration:  

141 ■            const InputDecoration(hintText: 'Select bus'),  

142 ■          items: <DropDownMenuItem<String>>[  

143 ■            for (final Bus b in buses)  

144 ■              DropDownMenuItem<String>(  

145 ■                value: b.id,  

146 ■                child: Text(b.busNumber),  

147 ■              ),  

148 ■          ],  

149 ■          onChanged: (String? v) =>  

150 ■            setState(() => _busId = v),  

151 ■        ),  

152 ■      ],  

153 ■      const SizedBox(height: 16),  

154 ■      FilledButton.icon(  

155 ■        onPressed: _sending ? null : _send,  

156 ■        icon: const Icon(Icons.send_rounded),  

157 ■        label: Text(  

158 ■          _sending ? 'Sending...' : 'Send announcement',  

159 ■        ),  

160 ■      ],  

161 ■    ),  

162 ■  ),  

163 ■  const SizedBox(height: 16),  

164 ■  const NmbCard(  

165 ■    child: Text(  

166 ■      'Automatic notifications (trip started, bus approaching, '  

167 ■      'bus reached stop/school, tracking unavailable) are sent '  

168 ■      'by the system and don\'t need manual action.',  

169 ■      style: NmbTypography.bodySecondary,  

170 ■    ),  

171 ■  ),  

172 ■  const SizedBox(height: 24),  

173 ■ ],  

174 ■ ),  

175 ■ ),  

176 ■ ),  

177 ■ );  

178 ■ }  

179 ■ }

```

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../core/constants/enums.dart';
5 ■ import '../core/theme/nmb_colors.dart';
6 ■ import '../core/theme/nmb_typography.dart';
7 ■ import '../core/utils/formatters.dart';
8 ■ import '../core/widgets/nmb_card.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../core/widgets/skeleton.dart';
11 ■ import '../core/widgets/state_views.dart';
12 ■ import '../models/bus.dart';
13 ■ import '../models/trip.dart';
14 ■ import '../providers/data_providers.dart';
15 ■
16 ■ /// Trip History: kaunsi bus kab chali, kab khatam hui – poora record.
17 ■ class AdminTripHistoryScreen extends ConsumerWidget {
18 ■   const AdminTripHistoryScreen({super.key});
19 ■
20 ■   @override
21 ■   Widget build(BuildContext context, WidgetRef ref) {
22 ■     final AsyncValue<List<Trip>> tripsAsync = ref.watch(recentTripsProvider);
23 ■     final List<Bus> buses =
24 ■       ref.watch(allBusesProvider).valueOrNull ?? const <Bus>[];
25 ■
26 ■     return Scaffold(
27 ■       appBar: AppBar(title: const Text('Trip History')),
28 ■       body: tripsAsync.when(
29 ■         loading: () => const ListSkeleton(),
30 ■         error: (Object e, _) => ErrorView(
31 ■           message: 'Couldn\'t load trip history.',
32 ■           onRetry: () => ref.invalidate(recentTripsProvider),
33 ■         ),
34 ■         data: (List<Trip> trips) {
35 ■           if (trips.isEmpty) {
36 ■             return const EmptyState(
37 ■               icon: Icons.history_rounded,
38 ■               title: 'No trips yet',
39 ■               message:
40 ■                 'Trip records will appear here once drivers start trips.',
41 ■             );
42 ■           }
43 ■           return ResponsiveBody(
44 ■             child: Column(
45 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
46 ■               children: <Widget>[
47 ■                 for (final Trip trip in trips) ...<Widget>[
48 ■                   _TripTile(trip: trip, buses: buses),
49 ■                   const SizedBox(height: 10),
50 ■                 ],
51 ■                 const SizedBox(height: 16),
52 ■               ],
53 ■             ),
54 ■           );
55 ■         },
56 ■       ),
57 ■     );
58 ■   }

```

```

59 ■ }
60 ■
61 ■ class _TripTile extends StatelessWidget {
62 ■   const _TripTile({required this.trip, required this.buses});
63 ■
64 ■   final Trip trip;
65 ■   final List<Bus> buses;
66 ■
67 ■   @override
68 ■   Widget build(BuildContext context) {
69 ■     Bus? bus;
70 ■     for (final Bus b in buses) {
71 ■       if (b.id == trip.busId) bus = b;
72 ■     }
73 ■     final (String label, Color color, Color bg) = switch (trip.status) {
74 ■       TripStatus.active => ('RUNNING', NmbColors.success, NmbColors.successSoft),
75 ■       TripStatus.completed => ('COMPLETED', NmbColors.info, NmbColors.infoSoft),
76 ■       TripStatus.autoClosed => (
77 ■         'AUTO-CLOSED',
78 ■         NmbColors.warning,
79 ■         NmbColors.warningSoft
80 ■       ),
81 ■     };
82 ■
83 ■     String duration = '';
84 ■     if (trip.startedAt != null && trip.endedAt != null) {
85 ■       final int mins = trip.endedAt!.difference(trip.startedAt!).inMinutes;
86 ■       duration = ' • $mins min';
87 ■     }
88 ■
89 ■     return NmbCard(
90 ■       padding: const EdgeInsets.all(14),
91 ■       child: Row(
92 ■         children: <Widget>[
93 ■           Container(
94 ■             width: 42,
95 ■             height: 42,
96 ■             decoration: BoxDecoration(
97 ■               color: bg,
98 ■               borderRadius: BorderRadius.circular(12),
99 ■             ),
100 ■            child: Icon(Icons.directions_bus_rounded, color: color),
101 ■          ),
102 ■          const SizedBox(width: 12),
103 ■          Expanded(
104 ■            child: Column(
105 ■              crossAxisAlignment: CrossAxisAlignment.start,
106 ■              children: <Widget>[
107 ■                Text(
108 ■                  '${bus?.busNumber ?? trip.busId} • '
109 ■                  '${trip.direction.label}',
110 ■                  style: NmbTypography.cardTitle,
111 ■                ),
112 ■                Text(
113 ■                  trip.startedAt != null
114 ■                    ? '${Formatters.dateTime(trip.startedAt!)}$duration'
115 ■                    : '—',
116 ■                  style: NmbTypography.bodySecondary,
117 ■                ),
118 ■              ],
119 ■            ),
120 ■          ),
121 ■          Container(
122 ■            padding:
123 ■              const EdgeInsets.symmetric(horizontal: 8, vertical: 4),
124 ■            decoration: BoxDecoration(
125 ■              color: bg,
126 ■              borderRadius: BorderRadius.circular(999),
127 ■            ),
128 ■            child: Text(
129 ■              label,
130 ■              style: NmbTypography.caption.copyWith(
131 ■                color: color,
132 ■                fontWeight: FontWeight.w800,
133 ■              ),
134 ■            ),
135 ■          ),
136 ■        ],
137 ■      ),
138 ■    );
139 ■  }
140 ■ }

```

lib/features/admin/screens/admin_audit_logs_screen.dart

105 lines

```

1 ■ import 'package:flutter/material.dart';
2 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
3 ■
4 ■ import '../../../core/theme/nmb_colors.dart';
5 ■ import '../../../core/theme/nmb_typography.dart';
6 ■ import '../../../core/utils/formatters.dart';
7 ■ import '../../../core/widgets/nmb_card.dart';
8 ■ import '../../../core/widgets/responsive_scaffold_body.dart';
9 ■ import '../../../core/widgets/state_views.dart';
10 ■ import '../../../models/audit_log.dart';
11 ■ import '../../../providers/data_providers.dart';
12 ■
13 ■ /// Read-only audit trail. Entries are written exclusively by Cloud
14 ■ /// Functions – neither admins nor anyone else can edit or delete them
15 ■ /// from the client.
16 ■ class AdminAuditLogsScreen extends ConsumerWidget {
17 ■   const AdminAuditLogsScreen({super.key});
18 ■
19 ■   static const Map<String, IconData> _icons = <String, IconData>{
20 ■     'USER_PROVISIONED': Icons.person_add_alt_1_rounded,
21 ■     'STUDENT_BUS_REASSIGNED': Icons.swap_horiz_rounded,
22 ■     'DRIVER_BUS_REASSIGNED': Icons.swap_horiz_rounded,
23 ■     'ACCOUNT_DISABLED': Icons.block_rounded,
24 ■     'ACCOUNT_ENABLED': Icons.check_circle_outline_rounded,
25 ■     'TRIP_STARTED': Icons.play_circle_outline_rounded,
26 ■     'TRIP_ENDED': Icons.stop_circle_outlined,
27 ■     'TRIP_AUTO_CLOSED': Icons.timer_off_outlined,
28 ■     'ANNOUNCEMENT_SENT': Icons.campaign_rounded,
29 ■   };
30 ■
31 ■   @override
32 ■   Widget build(BuildContext context, WidgetRef ref) {
33 ■     final AsyncValue<List<AuditLog>> logsAsync = ref.watch(auditLogsProvider);
34 ■
35 ■     return Scaffold(
36 ■       appBar: AppBar(title: const Text('Activity Logs')),
37 ■       body: logsAsync.when(
38 ■         loading: () => const LoadingView(),
39 ■         error: (Object e, _) => ErrorView(
40 ■           message: 'Couldn\'t load activity logs.',
41 ■           onRetry: () => ref.invalidate(auditLogsProvider),
42 ■         ),
43 ■         data: (List<AuditLog> logs) {
44 ■           if (logs.isEmpty) {
45 ■             return const EmptyState(
46 ■               icon: Icons.receipt_long_outlined,
47 ■               title: 'No activity yet',
48 ■               message: 'Administrative actions and trip events will be '
49 ■                 'recorded here.',
50 ■             );
51 ■           }
52 ■           return ResponsiveBody(
53 ■             child: Column(
54 ■               crossAxisAlignment: CrossAxisAlignment.stretch,
55 ■               children: <Widget>[
56 ■                 for (final AuditLog log in logs) ...<Widget>[
57 ■                   NmbCard(
58 ■                     padding: const EdgeInsets.all(14),

```

```

59 ■      child: Row(
60 ■        crossAxisAlignment: CrossAxisAlignment.start,
61 ■        children: <Widget>[
62 ■          Icon(
63 ■            _icons[log.action] ?? Icons.info_outline_rounded,
64 ■            size: 22,
65 ■            color: NmbColors.primary,
66 ■          ),
67 ■          const SizedBox(width: 12),
68 ■          Expanded(
69 ■            child: Column(
70 ■              crossAxisAlignment: CrossAxisAlignment.start,
71 ■              children: <Widget>[
72 ■                Text(
73 ■                  log.action.replaceAll('_', ' '),
74 ■                  style: NmbTypography.cardTitle,
75 ■                ),
76 ■                Text(
77 ■                  '${log.targetType} ${log.targetId}'
78 ■                  '${log.details.isEmpty ? '' : ' • ${log.details}'}',
79 ■                  style: NmbTypography.bodySecondary,
80 ■                  maxLines: 2,
81 ■                  overflow: TextOverflow.ellipsis,
82 ■                ),
83 ■                const SizedBox(height: 4),
84 ■                Text(
85 ■                  'by ${log.actorRole} • '
86 ■                  '${Formatters.dateTime(log.at)}',
87 ■                  style: NmbTypography.caption,
88 ■                ),
89 ■              ],
90 ■            ),
91 ■          ),
92 ■        ],
93 ■      ),
94 ■    ),
95 ■    const SizedBox(height: 8),
96 ■  ],
97 ■  const SizedBox(height: 16),
98 ■ ],
99 ■ ),
100 ■ );
101 ■ },
102 ■ ),
103 ■ );
104 ■ }
105 ■ }

```

```

1 ■ import 'package:cloud_firestore/cloud_firestore.dart';
2 ■ import 'package:flutter/material.dart';
3 ■ import 'package:flutter_riverpod/flutter_riverpod.dart';
4 ■
5 ■ import '../core/theme/nmb_typography.dart';
6 ■ import '../core/utils/validators.dart';
7 ■ import '../core/widgets/nmb_card.dart';
8 ■ import '../core/widgets/nmb_dialogs.dart';
9 ■ import '../core/widgets/responsive_scaffold_body.dart';
10 ■ import '../providers/data_providers.dart';
11 ■
12 ■ /// School profile settings (contact card shown to students) – admin only.
13 ■ class AdminSettingsScreen extends ConsumerStatefulWidget {
14 ■   const AdminSettingsScreen({super.key});
15 ■
16 ■   @override
17 ■   ConsumerState<AdminSettingsScreen> createState() =>
18 ■     _AdminSettingsScreenState();
19 ■ }
20 ■
21 ■ class _AdminSettingsScreenState extends ConsumerState<AdminSettingsScreen> {
22 ■   final GlobalKey<FormState> _formKey = GlobalKey<FormState>();
23 ■   final TextEditingController _name = TextEditingController();
24 ■   final TextEditingController _phone = TextEditingController();
25 ■   final TextEditingController _email = TextEditingController();
26 ■   final TextEditingController _address = TextEditingController();
27 ■   final TextEditingController _lat = TextEditingController();
28 ■   final TextEditingController _lng = TextEditingController();
29 ■   bool _loaded = false;
30 ■   bool _saving = false;
31 ■
32 ■   @override
33 ■   void dispose() {
34 ■     _name.dispose();
35 ■     _phone.dispose();
36 ■     _email.dispose();
37 ■     _address.dispose();
38 ■     _lat.dispose();
39 ■     _lng.dispose();
40 ■     super.dispose();
41 ■   }
42 ■
43 ■   void _fill(Map<String, dynamic>? config) {
44 ■     if (_loaded || config == null) return;
45 ■     _loaded = true;
46 ■     _name.text = (config['name'] as String?) ?? '';
47 ■     _phone.text = (config['phone'] as String?) ?? '';
48 ■     _email.text = (config['email'] as String?) ?? '';
49 ■     _address.text = (config['address'] as String?) ?? '';
50 ■     final Map<String, dynamic>? loc =
51 ■       (config['location'] as Map?)?.cast<String, dynamic>();
52 ■     if (loc != null) {
53 ■       _lat.text = '${loc['lat']}';
54 ■       _lng.text = '${loc['lng']}';
55 ■     }
56 ■   }
57 ■
58 ■   Future<void> _save() async {

```

```

59 ■   if (!(_formKey.currentState?.validate() ?? false)) return;
60 ■   setState(() => _saving = true);
61 ■   try {
62 ■       await FirebaseFirestore.instance
63 ■         .collection('config')
64 ■         .doc('school')
65 ■         .set(<String, dynamic>{
66 ■           'name': _name.text.trim(),
67 ■           'phone': _phone.text.trim(),
68 ■           'email': _email.text.trim(),
69 ■           'address': _address.text.trim(),
70 ■           if (_lat.text.trim().isNotEmpty && _lng.text.trim().isNotEmpty)
71 ■             'location': <String, double>{
72 ■               'lat': double.parse(_lat.text.trim()),
73 ■               'lng': double.parse(_lng.text.trim()),
74 ■             },
75 ■         }, SetOptions(merge: true),);
76 ■     if (!mounted) return;
77 ■     showNmbSnack(context, 'School profile saved.', isSuccess: true);
78 ■   } catch (_) {
79 ■     if (!mounted) return;
80 ■     showNmbSnack(context, 'Couldn\'t save. Try again.', isError: true);
81 ■   } finally {
82 ■     if (mounted) setState(() => _saving = false);
83 ■   }
84 ■ }
85 ■
86 ■ @override
87 ■ Widget build(BuildContext context) {
88 ■   _fill(ref.watch(schoolConfigProvider).valueOrNull);
89 ■
90 ■   return Scaffold(
91 ■     appBar: AppBar(title: const Text('Admin Settings')),
92 ■     body: ResponsiveBody(
93 ■       child: Form(
94 ■         key: _formKey,
95 ■         child: Column(
96 ■           crossAxisAlignment: CrossAxisAlignment.stretch,
97 ■           children: <Widget>[
98 ■             const Text('School profile', style: NmbTypography.sectionTitle),
99 ■             const SizedBox(height: 6),
100 ■             const Text(
101 ■               'Shown to students and drivers as the school contact card.',
102 ■               style: NmbTypography.bodySecondary,
103 ■             ),
104 ■             const SizedBox(height: 14),
105 ■             NmbCard(
106 ■               child: Column(
107 ■                 children: <Widget>[
108 ■                   TextFormField(
109 ■                     controller: _name,
110 ■                     decoration:
111 ■                       const InputDecoration(hintText: 'School name'),
112 ■                     validator: (String? v) =>
113 ■                       Validators.requiredField(v, label: 'School name'),
114 ■                   ),
115 ■                   const SizedBox(height: 12),
116 ■                   TextFormField(

```

```

117 ■         controller: _phone,
118 ■         keyboardType: TextInputType.phone,
119 ■         decoration:
120 ■             const InputDecoration(hintText: 'Office phone'),
121 ■     ),
122 ■     const SizedBox(height: 12),
123 ■     TextFormField(
124 ■         controller: _email,
125 ■         keyboardType: TextInputType.emailAddress,
126 ■         decoration:
127 ■             const InputDecoration(hintText: 'Office email'),
128 ■     ),
129 ■     const SizedBox(height: 12),
130 ■     TextFormField(
131 ■         controller: _address,
132 ■         decoration:
133 ■             const InputDecoration(hintText: 'Address'),
134 ■     ),
135 ■     const SizedBox(height: 12),
136 ■     Row(
137 ■         children: <Widget>[
138 ■             Expanded(
139 ■                 child: TextFormField(
140 ■                     controller: _lat,
141 ■                     keyboardType:
142 ■                         const TextInputType.numberWithOptions(
143 ■                             decimal: true, signed: true,),
144 ■                     decoration: const InputDecoration(
145 ■                         hintText: 'School latitude',),
146 ■                 ),
147 ■             ),
148 ■             const SizedBox(width: 12),
149 ■             Expanded(
150 ■                 child: TextFormField(
151 ■                     controller: _lng,
152 ■                     keyboardType:
153 ■                         const TextInputType.numberWithOptions(
154 ■                             decimal: true, signed: true,),
155 ■                     decoration: const InputDecoration(
156 ■                         hintText: 'School longitude',),
157 ■                 ),
158 ■             ),
159 ■         ],
160 ■     ),
161 ■ ],
162 ■ ),
163 ■ ),
164 ■ const SizedBox(height: 16),
165 ■ FilledButton(
166 ■     onPressed: _saving ? null : _save,
167 ■     child: Text(_saving ? 'Saving...' : 'Save school profile'),
168 ■ ),
169 ■ const SizedBox(height: 24),
170 ■ ],
171 ■ ),
172 ■ ),
173 ■ ),
174 ■ );

```

```

175 ■ }
176 ■ }

```


[illegible]

24 lines

54 lines

```

1 ■ {
2 ■   "indexes": [
3 ■     {
4 ■       "collectionGroup": "users",
5 ■       "queryScope": "COLLECTION",
6 ■       "fields": [
7 ■         { "fieldPath": "role", "order": "ASCENDING" },
8 ■         { "fieldPath": "fullName", "order": "ASCENDING" }
9 ■       ]
10 ■    },
11 ■    {
12 ■      "collectionGroup": "users",
13 ■      "queryScope": "COLLECTION",
14 ■      "fields": [
15 ■        { "fieldPath": "role", "order": "ASCENDING" },
16 ■        { "fieldPath": "busId", "order": "ASCENDING" }
17 ■      ]
18 ■    },
19 ■    {
20 ■      "collectionGroup": "users",
21 ■      "queryScope": "COLLECTION",
22 ■      "fields": [
23 ■        { "fieldPath": "role", "order": "ASCENDING" },
24 ■        { "fieldPath": "stopId", "order": "ASCENDING" }
25 ■      ]
26 ■    },
27 ■    {
28 ■      "collectionGroup": "stops",
29 ■      "queryScope": "COLLECTION",
30 ■      "fields": [
31 ■        { "fieldPath": "busId", "order": "ASCENDING" },
32 ■        { "fieldPath": "isActive", "order": "ASCENDING" },
33 ■        { "fieldPath": "order", "order": "ASCENDING" }
34 ■      ]
35 ■    },
36 ■    {
37 ■      "collectionGroup": "trips",
38 ■      "queryScope": "COLLECTION",
39 ■      "fields": [
40 ■        { "fieldPath": "busId", "order": "ASCENDING" },
41 ■        { "fieldPath": "status", "order": "ASCENDING" }
42 ■      ]
43 ■    },
44 ■    {
45 ■      "collectionGroup": "routes",
46 ■      "queryScope": "COLLECTION",
47 ■      "fields": [
48 ■        { "fieldPath": "busId", "order": "ASCENDING" },
49 ■        { "fieldPath": "isActive", "order": "ASCENDING" }
50 ■      ]
51 ■    }
52 ■  ],
53 ■  "fieldOverrides": []
54 ■ }

```

firebase/firebase.json

20 lines

```

1 ■ {
2 ■   "firestore": {
3 ■     "rules": "firestore.rules",
4 ■     "indexes": "firestore.indexes.json"
5 ■   },
6 ■   "database": {
7 ■     "rules": "database.rules.json"
8 ■   },
9 ■   "functions": {
10 ■    "source": "functions",
11 ■    "runtime": "nodejs20"
12 ■  },
13 ■  "emulators": {
14 ■    "auth": { "port": 9099 },
15 ■    "firestore": { "port": 8080 },
16 ■    "database": { "port": 9000 },
17 ■    "functions": { "port": 5001 },
18 ■    "ui": { "enabled": true }
19 ■  }
20 ■ }

```

Android manifest/gradle (desugaring, blue native splash) + iOS AppDelegate.

60 lines

```
58 ■         </intent>
59 ■     </queries>
60 ■ </manifest>
```

77 lines

```

1 ■ plugins {
2 ■     id "com.android.application"
3 ■     id "kotlin-android"
4 ■     // The Flutter Gradle Plugin must be applied after the Android and Kotlin Gradle plugins.
5 ■     id "dev.flutter.flutter-gradle-plugin"
6 ■ }
7 ■
8 ■ // ■■■■ FIREBASE – auto-applies once android/app/google-services.json exists
9 ■ // (that file is generated by `flutterfire configure`). Until then the build
10 ■ // still works, so you can build before AND after Firebase setup.
11 ■ if (file("google-services.json").exists()) {
12 ■     apply plugin: "com.google.gms.google-services"
13 ■ } else {
14 ■     logger.warn("NOTE: google-services.json not found – Firebase disabled for this build. Run `f
15 ■     ■ lutterfire configure` (see docs/FIREBASE_SETUP_HINDI.md).")
16 ■ }
17 ■
18 ■ // Reads the Google Maps key from android/local.properties (never committed).
19 ■ // (Maps: OpenStreetMap – no key needed)
20 ■ def localProperties = new Properties()
21 ■ def localPropertiesFile = rootProject.file("local.properties")
22 ■ if (localPropertiesFile.exists()) {
23 ■     localPropertiesFile.withReader("UTF-8") { reader ->
24 ■         localProperties.load(reader)
25 ■     }
26 ■ }
27 ■
28 ■ android {
29 ■     namespace = "com.srbs.never_miss_bus"
30 ■     compileSdk = 35
31 ■     ndkVersion = flutter.ndkVersion
32 ■
33 ■     compileOptions {
34 ■         sourceCompatibility = JavaVersion.VERSION_1_8
35 ■         targetCompatibility = JavaVersion.VERSION_1_8
36 ■         // REQUIRED by flutter_local_notifications v17+ – without this the
37 ■         // app crashes instantly in release mode (black screen).
38 ■         coreLibraryDesugaringEnabled = true
39 ■     }
40 ■
41 ■     kotlinOptions {
42 ■         jvmTarget = JavaVersion.VERSION_1_8
43 ■     }
44 ■
45 ■     defaultConfig {
46 ■         applicationId = "com.srbs.never_miss_bus"
47 ■         // firebase_auth / cloud_firestore need at least 23; maps needs 21+.
48 ■         minSdk = 23
49 ■         targetSdk = flutter.targetSdkVersion
50 ■         versionCode = flutter.versionCode
51 ■         versionName = flutter.versionName
52 ■         multiDexEnabled = true
53 ■     }
54 ■
55 ■     buildTypes {
56 ■         release {
57 ■             // ■■■■ PRODUCTION SIGNING – create android/key.properties and a
58 ■
59 ■             // proper signing config before publishing. Debug signing is used
60 ■             // here only so `flutter run --release` works during development.
61 ■             signingConfig = signingConfigs.debug
62 ■             // NOTE: R8 minification disabled for this CI build environment
63 ■             // (2 GB RAM). Re-enable minifyEnabled/shrinkResources for the
64 ■             // final Play Store build on a normal dev machine.
65 ■             minifyEnabled = false
66 ■             shrinkResources = false
67 ■         }
68 ■     }
69 ■
70 ■     flutter {
71 ■         source = "../.."
72 ■     }
73 ■
74 ■     dependencies {
75 ■         // Java 8+ API desugaring (required by flutter_local_notifications v17+)
76 ■         coreLibraryDesugaring "com.android.tools:desugar_jdk_libs:2.1.2"
77 ■     }

```

```
1 ■ <?xml version="1.0" encoding="utf-8"?>
2 ■ <!-- Native splash: solid brand blue so there is no white flash before
3 ■     Flutter's animated splash takes over. -->
4 ■ <layer-list xmlns:android="http://schemas.android.com/apk/res/android">
5 ■     <item android:drawable="@color/nmb_primary_dark" />
6 ■ </layer-list>
```

ios/Runner/AppDelegate.swift

14 lines

```
1 ■ import Flutter
2 ■ import UIKit
3 ■
4 ■ @main
5 ■ @objc class AppDelegate: FlutterAppDelegate {
6 ■     override func application(
7 ■         _ application: UIApplication,
8 ■         didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
9 ■     ) -> Bool {
10 ■         // Maps: OpenStreetMap via flutter_map – no Google Maps key needed.
11 ■         GeneratedPluginRegistrant.register(with: self)
12 ■         return super.application(application, didFinishLaunchingWithOptions: launchOptions)
13 ■     }
14 ■ }
```

13. Tests

Unit tests: validators, freshness honesty, ETA guardrails.

test/unit_test.dart

161 lines

```

1 ■ import 'package:flutter_test/flutter_test.dart';
2 ■ import 'package:never_miss_bus/core/constants/enums.dart';
3 ■ import 'package:never_miss_bus/core/utils/formatters.dart';
4 ■ import 'package:never_miss_bus/core/utils/validators.dart';
5 ■ import 'package:never_miss_bus/models/bus_stop.dart';
6 ■ import 'package:never_miss_bus/models/geo_point_data.dart';
7 ■ import 'package:never_miss_bus/models/live_location.dart';
8 ■ import 'package:never_miss_bus/services/eta_service.dart';
9 ■
10 ■ void main() {
11 ■   group('Validators', () {
12 ■     test('accepts valid email', () {
13 ■       expect(Validators.email('student@srbs.edu.in'), isNull);
14 ■     });
15 ■     test('rejects invalid email', () {
16 ■       expect(Validators.email('not-an-email'), isNotNull);
17 ■       expect(Validators.email(''), isNotNull);
18 ■     });
19 ■     test('password length enforced', () {
20 ■       expect(Validators.password('short'), isNotNull);
21 ■       expect(Validators.password('longenough1'), isNull);
22 ■     });
23 ■     test('strong password requires upper + digit + length', () {
24 ■       expect(Validators.strongPassword('weakpassword'), isNotNull);
25 ■       expect(Validators.strongPassword('Weakpassword'), isNotNull);
26 ■       expect(Validators.strongPassword('Str0ngPassword'), isNull);
27 ■     });
28 ■     test('lat/lng bounds', () {
29 ■       expect(Validators.latitude('91'), isNotNull);
30 ■       expect(Validators.latitude('30.32'), isNull);
31 ■       expect(Validators.longitude('-181'), isNotNull);
32 ■       expect(Validators.longitude('78.03'), isNull);
33 ■     });
34 ■   });
35 ■
36 ■   group('GeoPointData', () {
37 ■     test('haversine distance is sane (Dehradun ~1km check)', () {
38 ■       const GeoPointData a = GeoPointData(lat: 30.3165, lng: 78.0322);
39 ■       const GeoPointData b = GeoPointData(lat: 30.3255, lng: 78.0322);
40 ■       final double d = a.distanceTo(b);
41 ■       expect(d, greaterThan(900));
42 ■       expect(d, lessThan(1100));
43 ■     });
44 ■   });
45 ■
46 ■   group('LiveLocation freshness (honest status)', () {
47 ■     LiveLocation fixAgedSeconds(int seconds) => LiveLocation(
48 ■       position: const GeoPointData(lat: 30, lng: 78),
49 ■       heading: 0,
50 ■       speedKmh: 20,
51 ■       accuracy: 5,
52 ■       tripId: 't1',
53 ■       updatedAt: DateTime.now().subtract(Duration(seconds: seconds)),
54 ■     );
55 ■
56 ■     test('fresh fix is LIVE', () {
57 ■       expect(fixAgedSeconds(5).freshness(), LocationFreshness.live);
58 ■     });

```

```

59 ■ test('45s old fix is STALE, never live', () {
60 ■     expect(fixAgedSeconds(45).freshness(), LocationFreshness.stale);
61 ■ });
62 ■ test('4min old fix is UNAVAILABLE', () {
63 ■     expect(fixAgedSeconds(240).freshness(), LocationFreshness.unavailable);
64 ■ });
65 ■ });
66 ■
67 ■ group('EtaService (never over-promises)', () {
68 ■     final List<BusStop> stops = <BusStop>[
69 ■         const BusStop(
70 ■             id: 's1',
71 ■             name: 'Stop 1',
72 ■             routeId: 'r',
73 ■             busId: 'b',
74 ■             location: GeoPointData(lat: 30.30, lng: 78.03),
75 ■             order: 1,
76 ■         ),
77 ■         const BusStop(
78 ■             id: 's2',
79 ■             name: 'Stop 2',
80 ■             routeId: 'r',
81 ■             busId: 'b',
82 ■             location: GeoPointData(lat: 30.32, lng: 78.03),
83 ■             order: 2,
84 ■         ),
85 ■     ];
86 ■
87 ■     test('no live data → unavailable', () {
88 ■         final EtaResult r = EtaService().etaToStop(
89 ■             live: null,
90 ■             targetStop: stops[1],
91 ■             orderedStops: stops,
92 ■         );
93 ■         expect(r, isA<EtaUnavailable>());
94 ■     });
95 ■
96 ■     test('stale data → unavailable (no fake ETA)', () {
97 ■         final LiveLocation stale = LiveLocation(
98 ■             position: const GeoPointData(lat: 30.29, lng: 78.03),
99 ■             heading: 0,
100 ■             speedKmh: 25,
101 ■             accuracy: 5,
102 ■             tripId: 't',
103 ■             updatedAt: DateTime.now().subtract(const Duration(minutes: 10)),
104 ■         );
105 ■         final EtaResult r = EtaService().etaToStop(
106 ■             live: stale,
107 ■             targetStop: stops[1],
108 ■             orderedStops: stops,
109 ■         );
110 ■         expect(r, isA<EtaUnavailable>());
111 ■     });
112 ■
113 ■     test('fresh data → approximate ETA with floored speed', () {
114 ■         final LiveLocation live = LiveLocation(
115 ■             position: const GeoPointData(lat: 30.29, lng: 78.03),
116 ■             heading: 0,

```

```

117 ■      speedKmh: 0, // stationary: min-speed floor must prevent ∞
118 ■      accuracy: 5,
119 ■      tripId: 't',
120 ■      updatedAt: DateTime.now(),
121 ■    );
122 ■    final EtaResult r = EtaService().etaToStop(
123 ■      live: live,
124 ■      targetStop: stops[1],
125 ■      orderedStops: stops,
126 ■    );
127 ■    expect(r, isA<EtaAvailable>());
128 ■    final EtaAvailable eta = r as EtaAvailable;
129 ■    expect(eta.duration.inMinutes, lessThan(60));
130 ■    expect(eta.duration.inSeconds, greaterThan(0));
131 ■  });
132 ■ };
133 ■
134 ■ group('Formatters', () {
135 ■   test('ETA wording is always approximate', () {
136 ■     expect(
137 ■       Formatters.eta(const Duration(minutes: 8)),
138 ■       contains('approximately'),
139 ■     );
140 ■     expect(Formatters.etaShort(const Duration(minutes: 8)), '~8 min');
141 ■   });
142 ■   test('relative time', () {
143 ■     expect(Formatters.relativeTime(DateTime.now()), 'just now');
144 ■     expect(
145 ■       Formatters.relativeTime(
146 ■         DateTime.now().subtract(const Duration(minutes: 5)),
147 ■       ),
148 ■       '5 minutes ago',
149 ■     );
150 ■   });
151 ■ });
152 ■
153 ■ group('Role parsing (no privilege from bad data)', () {
154 ■   test('unknown role parses to null, never defaults to admin', () {
155 ■     expect(UserRole.tryParse('superadmin'), isNull);
156 ■     expect(UserRole.tryParse(''), isNull);
157 ■     expect(UserRole.tryParse(null), isNull);
158 ■     expect(UserRole.tryParse('admin'), UserRole.admin);
159 ■   });
160 ■ });
161 ■ }

```

test/widget_test.dart

2 lines

```

1 ■ // Placeholder replaced by real tests in test/unit_test.dart.
2 ■ void main() {}

```